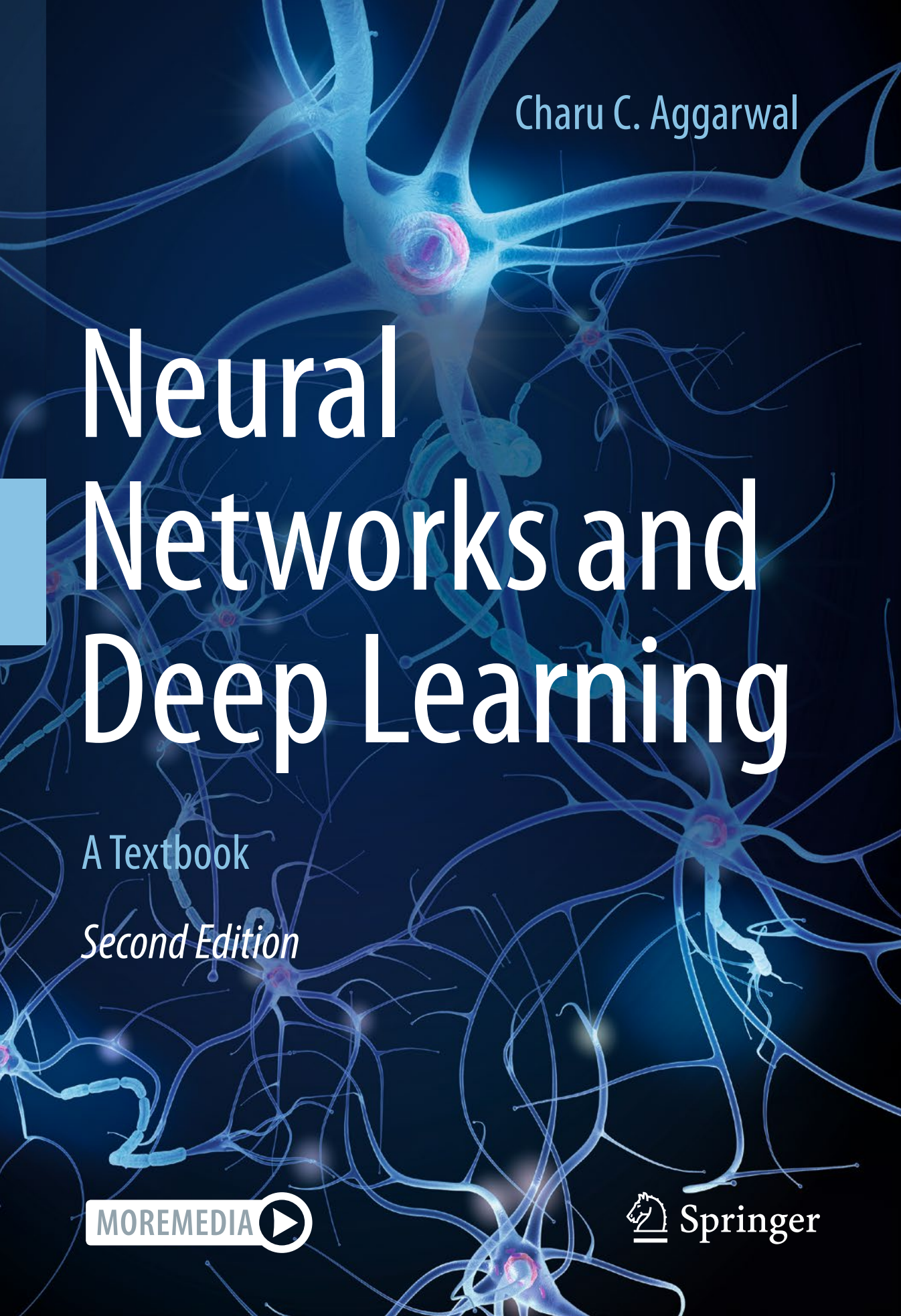

Charu C. Aggarwal
Neural Networks and Deep Learning
A Textbook
Second Edition
MOREMEDIA
Springer

Neural Networks and Deep Learning

Charu C. Aggarwal

# Neural Networks and Deep Learning

## A Textbook

Second Edition

Springer

Charu C. Aggarwal
IBM T. J. Watson Research Center
International Business Machines
Yorktown Heights, NY, USA

ISBN 978-3-031-29641-3 ISBN 978-3-031-29642-0 (eBook)
https://doi.org/10.1007/978-3-031-29642-0



This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

To my wife Lata, my daughter Sayani,
and my late parents Dr. Prem Sarup and Mrs. Pushplata Aggarwal.

# Preface

"Any A.I. smart enough to pass a Turing test is smart enough to know to fail it."–***Ian McDonald

Neural networks were developed to simulate the human nervous system for machine learning tasks by treating the computational units in a learning model in a manner similar to human neurons. The grand vision of neural networks is to create artificial intelligence by building machines whose architecture simulates the computations in the human nervous system. Although the biological model of neural networks is an exciting one and evokes comparisons with science fiction, neural networks have a much simpler and mundane mathematical basis than a complex biological system. The neural network abstraction can be viewed as a modular approach of enabling learning algorithms that are based on continuous optimization on a computational graph of mathematical dependencies between the input and output. These ideas are strikingly similar to classical optimization methods in control theory, which historically preceded the development of neural network algorithms.

Neural networks were developed soon after the advent of computers in the fifties and sixties. Rosenblatt's perceptron algorithm was seen as a fundamental cornerstone of neural networks, which caused an initial period of euphoria — it was soon followed by disappointment as the initial successes were somewhat limited. Eventually, at the turn of the century, greater data availability and increasing computational power lead to increased successes of neural networks, and this area was reborn under the new label of "deep learning." Although we are still far from the day that artificial intelligence (AI) is close to human performance, there are specific domains like image recognition, self-driving cars, and game playing, where AI has matched or exceeded human performance. It is also hard to predict what AI might be able to do in the future. For example, few computer vision experts would have thought two decades ago that any automated system could ever perform an intuitive task like categorizing an image more accurately than a human. The large amounts of data available in recent years together with increased computational power have enabled experimentation with more sophisticated and deep neural architectures than was previously possible. The resulting success has changed the broader perception of the potential of deep learning. This book discusses neural networks from this modern perspective. The chapters of the book are organized as follows:

1. *The basics of neural networks:* Chapters 1, 2, and 3 discuss the basics of neural network design and the backpropagation algorithm. Many traditional machine learning models

can be understood as special cases of neural learning. Understanding the relationship between traditional machine learning and neural networks is the first step to understanding the latter. The simulation of various machine learning models with neural networks is provided in Chapter 3. This will give the analyst a feel of how neural networks push the envelope of traditional machine learning algorithms.

2. *Fundamentals of neural networks:* Although Chapters 1, 2, and 3 provide an overview of the training methods for neural networks, a more detailed understanding of the training challenges is provided in Chapters 4 and 5. Chapters 6 and 7 present radial-basis function (RBF) networks and restricted Boltzmann machines.

3. *Advanced topics in neural networks:* A lot of the recent success of deep learning is a result of the specialized architectures for various domains, such as recurrent neural networks and convolutional neural networks. Chapters 8 and 9 discuss recurrent and convolutional neural networks. Graph neural networks are discussed in Chapter 10. Several advanced topics like deep reinforcement learning, attention mechanisms, neural Turing mechanisms, and generative adversarial networks are discussed in Chapters 11 and 12.

We have included some "forgotten" architectures like RBF networks and Kohonen self-organizing maps because of their potential in many applications. An application-centric view is highlighted throughout the book in order to give the reader a feel for the technology.

## What Is New in The Second Edition

The second edition has focused on improving the presentations in the first edition and also on adding new material. Significant changes have been made to almost all the chapters to improve presentation and add new material where needed. In some cases, the material in different chapters has been reorganized and reordered in order to improve exposition. The discussion of training challenges with depth has been separated from the backpropagation chapter, so that both topics could be discussed in greater detail. Second-order methods have been explained with greater clarity and examples. Chapter 10 on graph neural networks is entirely new. The discussion on GRUs in Chapter 8 has been greatly enhanced. New convolutional architectures using attention, such as Squeeze-and-Excitation Networks, are discussed in Chapter 9. The chapter on reinforcement learning has been significantly reorganized in order to provide a clearer exposition. The Monte Carlo sampling approach for reinforcement learning is discussed in greater detail in its own dedicated section. Chapter 12 contains expanded discussions on attention mechanisms for graphs and computer vision, transformers, pre-trained language models, and adversarial learning.

## Notations

Throughout this book, a vector or a multidimensional data point is annotated with a bar, such as $\overline{X}$ or $\overline{y}$. A vector or multidimensional point may be denoted by either small letters or capital letters, as long as it has a bar. Vector dot products are denoted by centered dots, such as $\overline{X} \cdot \overline{Y}$. A matrix is denoted in capital letters without a bar, such as $R$. Throughout the book, the $n \times d$ matrix corresponding to the training data set is denoted by $D$, with $n$ documents and $d$ dimensions. The individual data points in $D$ are therefore $d$-dimensional

row vectors. On the other hand, vectors with one component for each data point are usually $n$-dimensional column vectors. An example is the $n$-dimensional column vector $\overline{y}$ of class variables. An observed value $y_i$ is distinguished from a predicted value $\hat{y}_i$ by a circumflex at the top of the variable.

Yorktown Heights, NY, USA Charu C. Aggarwal

# Acknowledgments

## Acknowledgements for the First Edition

I would like to thank my family for their love and support during the busy time spent in writing this book. I would also like to thank my manager Nagui Halim for his support during the writing of this book.

Several figures in this book have been provided by the courtesy of various individuals and institutions. The Smithsonian Institution made the image of the Mark I perceptron (cf. Figure 1.5) available at no cost. Saket Sathe provided the outputs in Chapter 8 for the tiny Shakespeare data set, based on code available/described in [243, 604]. Andrew Zisserman provided Figures 9.13 and 9.17 in the section on convolutional visualizations. Another visualization of the feature maps in the convolution network (cf. Figure 9.16) was provided by Matthew Zeiler. NVIDIA provided Figure 11.10 on the convolutional neural network for self-driving cars in Chapter 11, and Sergey Levine provided the image on self-learning robots (cf. Figure 11.9) in the same chapter. Alec Radford provided Figure 12.11, which appears in Chapter 12. Alex Krizhevsky provided Figure 9.9(b) containing *AlexNet*.

This book has benefitted from significant feedback and several collaborations that I have had with numerous colleagues over the years. I would like to thank Quoc Le, Saket Sathe, Karthik Subbian, Jiliang Tang, and Suhang Wang for their feedback on various portions of this book. Shuai Zheng provided feedback on the section on regularized autoencoders in Chapter 5. I received feedback on the sections on autoencoders from Lei Cai and Hao Yuan. Feedback on the chapter on convolutional neural networks was provided by Hongyang Gao, Shuiwang Ji, and Zhengyang Wang. Shuiwang Ji, Lei Cai, Zhengyang Wang and Hao Yuan also reviewed the Chapters 4 and 8, and suggested several edits. They also suggested the ideas of using Figures 9.6 and 9.7 for elucidating the convolution/deconvolution operations.

For their collaborations, I would like to thank Tarek F. Abdelzaher, Jinghui Chen, Jing Gao, Quanquan Gu, Manish Gupta, Jiawei Han, Alexander Hinneburg, Thomas Huang, Nan Li, Huan Liu, Ruoming Jin, Daniel Keim, Arijit Khan, Latifur Khan, Mohammad M. Masud, Jian Pei, Magda Procopiuc, Guojun Qi, Chandan Reddy, Saket Sathe, Jaideep Srivastava, Karthik Subbian, Yizhou Sun, Jiliang Tang, Min-Hsuan Tsai, Haixun Wang, Jianyong Wang, Min Wang, Suhang Wang, Joel Wolf, Xifeng Yan, Mohammed Zaki, ChengXiang Zhai, and Peixiang Zhao. I would also like to thank my advisor James B. Orlin for his guidance during my early years as a researcher.

I would like to thank Lata Aggarwal for helping me with some of the figures created using PowerPoint graphics in this book. My daughter, Sayani, was helpful in incorporating special effects (e.g., image color, contrast, and blurring) in several JPEG images used at various places in this book.

## Acknowledgements for the Second Edition

The chapter on graph neural networks benefited from collaborations with Jiliang Tang, Lingfei Wu, and Suhang Wang. Shuiwang Ji, Meng Liu, and Hongyang Gao provided detailed comments on the chapter on graph neural networks as well as other parts of the book. Zhengyang Wang and Shuiwang Ji provided feedback on the new section on transformer networks. Yao Ma and Jiliang Tang provided the permission to use Figure 10.3. Yao Ma also provided detailed comments on the chapter on graph neural networks. Sharmishtha Dutta, a PhD student at RPI, proofread Chapter 10, and also pointed out the sections that needed further clarity. My manager, Horst Samulowitz, provided support during the writing of the second edition of this book.

# Contents


**1 An Introduction to Neural Networks** **1**
- 1.1 Introduction . . . 1
- 1.2 Single Computational Layer: The Perceptron . . . 5
  - 1.2.1 Use of Bias . . . 8
  - 1.2.2 What Objective Function Is the Perceptron Optimizing? . . . 8
- 1.3 The Base Components of Neural Architectures . . . 10
  - 1.3.1 Choice of Activation Function . . . 10
  - 1.3.2 Softmax Activation Function . . . 12
  - 1.3.3 Common Loss Functions . . . 13
- 1.4 Multilayer Neural Networks . . . 13
  - 1.4.1 The Multilayer Network as a Computational Graph . . . 15
- 1.5 The Importance of Nonlinearity . . . 17
  - 1.5.1 Nonlinear Activations in Action . . . 18
- 1.6 Advanced Architectures and Structured Data . . . 20
- 1.7 Two Notable Benchmarks . . . 21
  - 1.7.1 The MNIST Database of Handwritten Digits . . . 21
  - 1.7.2 The ImageNet Database . . . 22
- 1.8 Summary . . . 23
- 1.9 Bibliographic Notes and Software Resources . . . 23
- 1.10 Exercises . . . 25

**2 The Backpropagation Algorithm** **29**
- 2.1 Introduction . . . 29
- 2.2 The Computational Graph Abstraction . . . 30
  - 2.2.1 Computational Graphs Create Complex Functions . . . 31
- 2.3 Backpropagation in Computational Graphs . . . 33
  - 2.3.1 Computing Node-to-Node Derivatives with the Chain Rule . . . 34
  - 2.3.2 Dynamic Programming for Computing Node-to-Node Derivatives . . . 38
  - 2.3.3 Converting Node-to-Node Derivatives into Loss-to-Weight Derivatives . . . 42

2.4 Backpropagation in Neural Networks . . . 44
2.4.1 Some Useful Derivatives of Activation Functions . . . 46
2.4.2 Examples of Updates for Various Activations . . . 48
2.5 The Vector-Centric View of Backpropagation . . . 50
2.5.1 Derivatives with Respect to Vectors . . . 51
2.5.2 Vector-Centric Chain Rule . . . 51
2.5.3 A Decoupled View of Vector-Centric Backpropagation . . . 52
2.5.4 Vector-Centric Backpropagation with Non-Layered Architectures . . . 57
2.6 The Not-So-Unimportant Details . . . 58
2.6.1 Mini-Batch Stochastic Gradient Descent . . . 58
2.6.2 Learning Rate Decay . . . 60
2.6.3 Checking the Correctness of Gradient Computation . . . 60
2.6.4 Regularization . . . 61
2.6.5 Loss Functions on Hidden Nodes . . . 61
2.6.6 Backpropagation Tricks for Handling Shared Weights . . . 62
2.7 Tuning and Preprocessing . . . 62
2.7.1 Tuning Hyperparameters . . . 63
2.7.2 Feature Preprocessing . . . 64
2.7.3 Initialization . . . 66
2.8 Backpropagation Is Interpretable . . . 67
2.9 Summary . . . 67
2.10 Bibliographic Notes and Software Resources . . . 68
2.11 Exercises . . . 68

**3 Machine Learning with Shallow Neural Networks 73**
3.1 Introduction . . . 73
3.2 Neural Architectures for Binary Classification Models . . . 75
3.2.1 Revisiting the Perceptron . . . 75
3.2.2 Least-Squares Regression . . . 76
3.2.2.1 Widrow-Hoff Learning . . . 78
3.2.2.2 Closed Form Solutions . . . 79
3.2.3 Support Vector Machines . . . 79
3.2.4 Logistic Regression . . . 81
3.2.5 Comparison of Different Models . . . 82
3.3 Neural Architectures for Multiclass Models . . . 84
3.3.1 Multiclass Perceptron . . . 84
3.3.2 Weston-Watkins SVM . . . 85
3.3.3 Multinomial Logistic Regression (Softmax Classifier) . . . 86
3.4 Unsupervised Learning with Autoencoders . . . 88
3.4.1 Linear Autoencoder with a Single Hidden Layer . . . 89
3.4.1.1 Connections with Singular Value Decomposition . . . 91
3.4.1.2 Sharing Weights in the Encoder and Decoder . . . 91
3.4.2 Nonlinear Activation Functions and Depth . . . 92
3.4.3 Application to Visualization . . . 93
3.4.4 Application to Outlier Detection . . . 95
3.4.5 Application to Multimodal Embeddings . . . 95
3.4.6 Benefits of Autoencoders . . . 96
3.5 Recommender Systems . . . 96

3.6 Text Embedding with Word2vec . . . 99
3.6.1 Neural Embedding with Continuous Bag of Words . . . 100
3.6.2 Neural Embedding with Skip-Gram Model . . . 103
3.6.3 Word2vec (SGNS) is Logistic Matrix Factorization . . . 107
3.7 Simple Neural Architectures for Graph Embeddings . . . 110
3.7.1 Handling Arbitrary Edge Counts . . . 111
3.7.2 Beyond One-Hop Structural Models . . . 112
3.7.3 Multinomial Model . . . 112
3.8 Summary . . . 113
3.9 Bibliographic Notes and Software Resources . . . 113
3.10 Exercises . . . 114

**4 Deep Learning: Principles and Training Algorithms 119**
4.1 Introduction . . . 119
4.2 Why Is Depth Beneficial? . . . 120
4.2.1 Hierarchical Feature Engineering: How Depth Reveals Rich Structure . . . 120
4.3 Why Is Training Deep Networks Hard? . . . 122
4.3.1 Geometric Understanding of the Effect of Gradient Ratios . . . 122
4.3.2 The Vanishing and Exploding Gradient Problems . . . 124
4.3.3 Cliffs and Valleys . . . 126
4.3.4 Convergence Problems with Depth . . . 127
4.3.5 Local Minima . . . 127
4.4 Depth-Friendly Neural Architectures . . . 129
4.4.1 Activation Function Choice . . . 129
4.4.2 Dying Neurons and "Brain Damage" . . . 130
4.4.2.1 Leaky ReLU . . . 130
4.4.2.2 Maxout Networks . . . 131
4.4.3 Using Skip Connections . . . 131
4.5 Depth-Friendly Gradient-Descent Strategies . . . 132
4.5.1 Importance of Preprocessing and Initialization . . . 132
4.5.2 Momentum-Based Learning . . . 133
4.5.3 Nesterov Momentum . . . 134
4.5.4 Parameter-Specific Learning Rates . . . 135
4.5.4.1 AdaGrad . . . 136
4.5.4.2 RMSProp . . . 136
4.5.4.3 AdaDelta . . . 137
4.5.5 Combining Parameter-Specific Learning and Momentum . . . 138
4.5.5.1 RMSProp with Nesterov Momentum . . . 138
4.5.5.2 Adam . . . 138
4.5.6 Gradient Clipping . . . 139
4.5.7 Polyak Averaging . . . 139
4.6 Second-Order Derivatives: The Newton Method . . . 140
4.6.1 Example: Newton Method in the Quadratic Bowl . . . 142
4.6.2 Example: Newton Method in a Non-Quadratic Function . . . 142
4.6.3 The Saddle-Point Problem with Second-Order Methods . . . 143
4.7 Fast Approximations of Newton Method . . . 145
4.7.1 Conjugate Gradient Method . . . 145
4.7.2 Quasi-Newton Methods and BFGS . . . 148

4.8 Batch Normalization . . . 150
4.9 Practical Tricks for Acceleration and Compression . . . 153
4.9.1 GPU Acceleration . . . 154
4.9.2 Parallel and Distributed Implementations . . . 156
4.9.3 Algorithmic Tricks for Model Compression . . . 157
4.10 Summary . . . 160
4.11 Bibliographic Notes and Software Resources . . . 160
4.12 Exercises . . . 162

**5 Teaching Deep Learners to Generalize 165**
5.1 Introduction . . . 165
5.1.1 Example: Linear Regression . . . 166
5.1.2 Example: Polynomial Regression . . . 167
5.2 The Bias-Variance Trade-Off . . . 171
5.3 Generalization Issues in Model Tuning and Evaluation . . . 174
5.3.1 Evaluating with Hold-Out and Cross-Validation . . . 176
5.3.2 Issues with Training at Scale . . . 177
5.3.3 How to Detect Need to Collect More Data . . . 178
5.4 Penalty-Based Regularization . . . 178
5.4.1 Connections with Noise Injection . . . 179
5.4.2 $L_1$-Regularization . . . 180
5.4.3 $L_1$- or $L_2$-Regularization? . . . 181
5.4.4 Penalizing Hidden Units: Learning Sparse Representations . . . 181
5.5 Ensemble Methods . . . 182
5.5.1 Bagging and Subsampling . . . 182
5.5.2 Parametric Model Selection and Averaging . . . 184
5.5.3 Randomized Connection Dropping . . . 184
5.5.4 Dropout . . . 185
5.5.5 Data Perturbation Ensembles . . . 187
5.6 Early Stopping . . . 188
5.6.1 Understanding Early Stopping from the Variance Perspective . . . 189
5.7 Unsupervised Pretraining . . . 189
5.7.1 Variations of Unsupervised Pretraining . . . 192
5.7.2 What About Supervised Pretraining? . . . 193
5.8 Continuation and Curriculum Learning . . . 194
5.9 Parameter Sharing . . . 196
5.10 Regularization in Unsupervised Applications . . . 197
5.10.1 When the Hidden Layer is Broader than the Input Layer . . . 197
5.10.1.1 Sparse Feature Learning . . . 198
5.10.2 Noise Injection: De-noising Autoencoders . . . 198
5.10.3 Gradient-Based Penalization: Contractive Autoencoders . . . 199
5.10.4 Hidden Probabilistic Structure: Variational Autoencoders . . . 203
5.10.4.1 Reconstruction and Generative Sampling . . . 206
5.10.4.2 Conditional Variational Autoencoders . . . 208
5.10.4.3 Relationship with Generative Adversarial Networks . . . 208
5.11 Summary . . . 209
5.12 Bibliographic Notes and Software Resources . . . 210
5.13 Exercises . . . 211

**6 Radial Basis Function Networks** **215**
6.1 Introduction . . . 215
6.2 Training an RBF Network . . . 218
6.2.1 Training the Hidden Layer . . . 218
6.2.2 Training the Output Layer . . . 220
6.2.3 Iterative Construction of Hidden Layer . . . 221
6.2.4 Fully Supervised Learning of Hidden Layer . . . 222
6.3 Variations and Special Cases of RBF Networks . . . 223
6.3.1 Classification with Perceptron Criterion . . . 224
6.3.2 Classification with Hinge Loss . . . 224
6.3.3 Example of Linear Separability Promoted by RBF . . . 224
6.3.4 Application to Interpolation . . . 226
6.4 Relationship with Kernel Methods . . . 227
6.4.1 Kernel Regression Is a Special Case of RBF Networks . . . 227
6.4.2 Kernel SVM Is a Special Case of RBF Networks . . . 228
6.5 Summary . . . 229
6.6 Bibliographic Notes and Software Resources . . . 229
6.7 Exercises . . . 229

**7 Restricted Boltzmann Machines** **231**
7.1 Introduction . . . 231
7.2 Hopfield Networks . . . 232
7.2.1 Training a Hopfield Network . . . 235
7.2.2 Building a Toy Recommender and Its Limitations . . . 236
7.2.3 Increasing the Expressive Power of the Hopfield Network . . . 237
7.3 The Boltzmann Machine . . . 238
7.3.1 How a Boltzmann Machine Generates Data . . . 240
7.3.2 Learning the Weights of a Boltzmann Machine . . . 240
7.4 Restricted Boltzmann Machines . . . 242
7.4.1 Training the RBM . . . 244
7.4.2 Contrastive Divergence Algorithm . . . 245
7.5 Applications of Restricted Boltzmann Machines . . . 247
7.5.1 Dimensionality Reduction and Data Reconstruction . . . 247
7.5.2 RBMs for Collaborative Filtering . . . 249
7.5.3 Using RBMs for Classification . . . 252
7.5.4 Topic Models with RBMs . . . 254
7.5.5 RBMs for Machine Learning with Multimodal Data . . . 256
7.6 Using RBMs beyond Binary Data Types . . . 258
7.7 Stacking Restricted Boltzmann Machines . . . 258
7.7.1 Unsupervised Learning . . . 261
7.7.2 Supervised Learning . . . 261
7.7.3 Deep Boltzmann Machines and Deep Belief Networks . . . 261
7.8 Summary . . . 262
7.9 Bibliographic Notes and Software Resources . . . 262
7.10 Exercises . . . 264

**8 Recurrent Neural Networks** **265**
8.1 Introduction . . . 265
8.2 The Architecture of Recurrent Neural Networks . . . 267
8.2.1 Language Modeling Example of RNN . . . 270
8.2.2 Backpropagation Through Time . . . 273
8.2.3 Bidirectional Recurrent Networks . . . 275
8.2.4 Multilayer Recurrent Networks . . . 277
8.3 The Challenges of Training Recurrent Networks . . . 278
8.3.1 Layer Normalization . . . 281
8.4 Echo-State Networks . . . 282
8.5 Long Short-Term Memory (LSTM) . . . 285
8.6 Gated Recurrent Units (GRUs) . . . 287
8.7 Applications of Recurrent Neural Networks . . . 289
8.7.1 Contextualized Word Embeddings with ELMo . . . 290
8.7.2 Application to Automatic Image Captioning . . . 291
8.7.3 Sequence-to-Sequence Learning and Machine Translation . . . 292
8.7.4 Application to Sentence-Level Classification . . . 295
8.7.5 Token-Level Classification with Linguistic Features . . . 296
8.7.6 Time-Series Forecasting and Prediction . . . 297
8.7.7 Temporal Recommender Systems . . . 299
8.7.8 Secondary Protein Structure Prediction . . . 301
8.7.9 End-to-End Speech Recognition . . . 301
8.7.10 Handwriting Recognition . . . 301
8.8 Summary . . . 302
8.9 Bibliographic Notes and Software Resources . . . 302
8.10 Exercises . . . 303

**9 Convolutional Neural Networks** **305**
9.1 Introduction . . . 305
9.1.1 Historical Perspective and Biological Inspiration . . . 305
9.1.2 Broader Observations about Convolutional Neural Networks . . . 306
9.2 The Basic Structure of a Convolutional Network . . . 307
9.2.1 Padding . . . 312
9.2.2 Strides . . . 313
9.2.3 The ReLU Layer . . . 315
9.2.4 Pooling . . . 315
9.2.5 Fully Connected Layers . . . 317
9.2.6 The Interleaving between Layers . . . 317
9.2.7 Hierarchical Feature Engineering . . . 320
9.3 Training a Convolutional Network . . . 321
9.3.1 Backpropagating Through Convolutions . . . 321
9.3.2 Backpropagation as Convolution with Inverted/Transposed Filter . . . 322
9.3.3 Convolution/Backpropagation as Matrix Multiplications . . . 324
9.3.4 Data Augmentation . . . 326
9.4 Case Studies of Convolutional Architectures . . . 326
9.4.1 AlexNet . . . 327
9.4.2 ZFNet . . . 329
9.4.3 VGG . . . 330

9.4.4 GoogLeNet . . . 333
9.4.5 ResNet . . . 335
9.4.6 Squeeze-and-Excitation Networks (SENets) . . . 338
9.4.7 The Effects of Depth . . . 339
9.4.8 Pretrained Models . . . 340
9.5 Visualization and Unsupervised Learning . . . 341
9.5.1 Visualizing the Features of a Trained Network . . . 341
9.5.2 Convolutional Autoencoders . . . 347
9.6 Applications of Convolutional Networks . . . 351
9.6.1 Content-Based Image Retrieval . . . 352
9.6.2 Object Localization . . . 352
9.6.3 Object Detection . . . 354
9.6.4 Natural Language and Sequence Learning with TextCNN . . . 355
9.6.5 Video Classification . . . 355
9.7 Summary . . . 356
9.8 Bibliographic Notes and Software Resources . . . 356
9.9 Exercises . . . 359

**10 Graph Neural Networks** **361**
10.1 Introduction . . . 361
10.2 Node Embeddings with Conventional Architectures . . . 362
10.2.1 Adjacency Matrix Representation and Feature Engineering . . . 364
10.3 Graph Neural Networks: The General Framework . . . 364
10.3.1 The Neighborhood Function . . . 368
10.3.2 Graph Convolution Function . . . 368
10.3.3 GraphSAGE . . . 369
10.3.4 Handling Edge Weights . . . 371
10.3.5 Handling New Vertices . . . 371
10.3.6 Handling Relational Networks . . . 372
10.3.7 Directed Graphs . . . 373
10.3.8 Gated Graph Neural Networks . . . 373
10.3.9 Comparison with Image Convolutional Networks . . . 374
10.4 Backpropagation in Graph Neural Networks . . . 375
10.5 Beyond Nodes: Generating Graph-Level Models . . . 377
10.6 Applications of Graph Neural Networks . . . 382
10.7 Summary . . . 384
10.8 Bibliographic Notes and Software Resources . . . 384
10.9 Exercises . . . 385

**11 Deep Reinforcement Learning** **389**
11.1 Introduction . . . 389
11.2 Stateless Algorithms: Multi-Armed Bandits . . . 391
11.3 The Basic Framework of Reinforcement Learning . . . 393
11.4 Monte Carlo Sampling . . . 395
11.4.1 Monte Carlo Sampling Algorithm . . . 395
11.4.2 Monte Carlo Rollouts with Function Approximators . . . 396

11.5 Bootstrapping for Value Function Learning . . . 398
11.5.1 Q-Learning . . . 399
11.5.2 Deep Learning Models as Function Approximators . . . 400
11.5.3 Example: Neural Network Specifics for Video Game Setting . . . 403
11.5.4 On-Policy versus Off-Policy Methods: SARSA . . . 404
11.5.5 Modeling States versus State-Action Pairs . . . 405
11.6 Policy Gradient Methods . . . 407
11.6.1 Finite Difference Methods . . . 408
11.6.2 Likelihood Ratio Methods . . . 409
11.6.3 Actor-Critic Methods . . . 411
11.6.4 Continuous Action Spaces . . . 413
11.7 Monte Carlo Tree Search . . . 413
11.8 Case Studies . . . 415
11.8.1 AlphaGo and AlphaZero for Go and Chess . . . 415
11.8.2 Self-Learning Robots . . . 420
11.8.2.1 Deep Learning of Locomotion Skills . . . 420
11.8.2.2 Deep Learning of Visuomotor Skills . . . 422
11.8.3 Building Conversational Systems: Deep Learning for Chatbots . . . 423
11.8.4 Self-Driving Cars . . . 425
11.8.5 Neural Architecture Search with Reinforcement Learning . . . 428
11.9 Practical Challenges Associated with Safety . . . 429
11.10 Summary . . . 429
11.11 Bibliographic Notes and Software Resources . . . 430
11.12 Exercises . . . 432

**12 Advanced Topics in Deep Learning 435**
12.1 Introduction . . . 435
12.2 Attention Mechanisms . . . 436
12.2.1 Recurrent Models of Visual Attention . . . 437
12.2.2 Attention Mechanisms for Image Captioning . . . 439
12.2.3 Soft Image Attention with Spatial Transformer . . . 440
12.2.4 Attention Mechanisms for Machine Translation . . . 442
12.2.5 Transformer Networks . . . 446
12.2.5.1 How Self Attention Helps . . . 446
12.2.5.2 The Self-Attention Module . . . 447
12.2.5.3 Incorporating Positional Information . . . 449
12.2.5.4 The Sequence-to-Sequence Transformer . . . 450
12.2.5.5 Multihead Attention . . . 450
12.2.6 Transformer-Based Pre-trained Language Models . . . 451
12.2.6.1 GPT-n . . . 452
12.2.6.2 BERT . . . 454
12.2.6.3 T5 . . . 455
12.2.7 Vision Transformer (ViT) . . . 457
12.2.8 Attention Mechanisms in Graphs . . . 458
12.3 Neural Turing Machines . . . 459
12.4 Adversarial Deep Learning . . . 463
12.5 Generative Adversarial Networks (GANs) . . . 467
12.5.1 Training a Generative Adversarial Network . . . 468
12.5.2 Comparison with Variational Autoencoder . . . 470

12.5.3 Using GANs for Generating Image Data . . . . . 470
12.5.4 Conditional Generative Adversarial Networks . . . . . 471
12.6 Competitive Learning . . . . . 476
12.6.1 Vector Quantization . . . . . 477
12.6.2 Kohonen Self-Organizing Map . . . . . 478
12.7 Limitations of Neural Networks . . . . . 480
12.7.1 An Aspirational Goal: Few Shot Learning . . . . . 481
12.7.2 An Aspirational Goal: Energy-Efficient Learning . . . . . 482
12.8 Summary . . . . . 483
12.9 Bibliographic Notes and Software Resources . . . . . 483
12.10 Exercises . . . . . 485

**Bibliography** **487**

**Index** **525**

# Author Biography

**Charu C. Aggarwal** is a Distinguished Research Staff Member (DRSM) at the IBM T. J. Watson Research Center in Yorktown Heights, New York. He completed his undergraduate degree in Computer Science from the Indian Institute of Technology at Kanpur in 1993 and his Ph.D. from the Massachusetts Institute of Technology in 1996.

He has worked extensively in the field of data mining. He has published more than 400 papers in refereed conferences and journals and authored over 80 patents. He is the author or editor of 20 books, including textbooks on data mining, recommender systems, and outlier analysis. Because of the commercial value of his patents, he has thrice been designated a Master Inventor at IBM. He is a recipient of an IBM Corporate Award (2003) for his work on bio-terrorist threat detection in data streams, a recipient of the IBM Outstanding Innovation Award (2008) for his scientific contributions to privacy technology, and a recipient of two IBM Outstanding Technical Achievement Awards (2009, 2015) for his work on data streams/high-dimensional data. He received the EDBT 2014 Test of Time Award for his work on condensation-based privacy-preserving data mining. He is a recipient of the IEEE ICDM Research Contributions Award (2015) and ACM SIGKDD Innovation Award, which are the two most prestigious awards for influential research contributions in the field of data mining. He is also a recipient of the W. Wallace McDowell Award, which is the highest award given solely by the IEEE Computer Society across the field of Computer Science.

He has served as the general co-chair of the IEEE Big Data Conference (2014) and as the program co-chair of the ACM CIKM Conference (2015), the IEEE ICDM Conference (2015), and the ACM KDD Conference (2016). He served as an associate editor of the IEEE Transactions on Knowledge and Data Engineering from 2004 to 2008. He is an associate editor of the IEEE Transactions on Big Data, an action editor of the Data Mining and Knowledge Discovery Journal, and an associate editor of the Knowledge and Information Systems Journal. He has served or currently serves as the editor-in-chief of the ACM Transactions on Knowledge Discovery from Data as well as the ACM SIGKDD Explorations. He is also an editor-in-chief of ACM Books. He serves on the advisory board of the Lecture Notes on Social Networks, a publication by Springer. He has served as the vice-president of

the SIAM Activity Group on Data Mining and is a member of the SIAM industry committee. He received the ACM SIGKDD Service Award for the aforem3entioned contributions to running conferences and journals— this honor is the most prestigious award for services to the field of data mining. He is a fellow of the SIAM, ACM, and the IEEE, for "contributions to knowledge discovery and data mining algorithms."


# Chapter 1

# An Introduction to Neural Networks

"Thou shalt not make a machine to counterfeit a human mind."–Frank Herbert

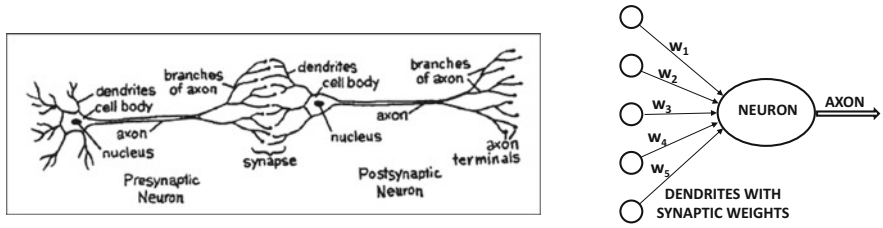
## 1.1 Introduction

Artificial neural networks are popular machine learning techniques that simulate the mechanism of learning in biological organisms. The human nervous system contains cells, which are referred to as *neurons*. The neurons are connected to one another with the use of *axons* and *dendrites*, and the connecting regions between axons and dendrites are referred to as *synapses*. These connections are illustrated in Figure 1.1(a). The strengths of synaptic connections often change in response to external stimuli. This change is how learning takes place in living organisms.

This biological mechanism is simulated in *artificial* neural networks, which contain computation units that are referred to as neurons. Throughout this book, we will use the term "neural networks" to refer to artificial neural networks rather than biological ones. The computational units are connected to one another through weights, which serve the same role as the strengths of synaptic connections in biological organisms. Each input to a neuron is scaled with a weight, which affects the function computed at that unit. This architecture is illustrated in Figure 1.1(b). An artificial neural network computes a function of the inputs by propagating the computed values from the input neurons to the output neuron(s) and using the weights as intermediate parameters. Learning occurs by changing the weights connecting the neurons. Just as external stimuli are needed for learning in biological organisms, the external stimulus in artificial neural networks is provided by the *training data* containing examples of input-output pairs of the function to be learned. For example, the training data might contain pixel representations of images (input) and their annotated labels (e.g., carrot, banana) as the output. These training data pairs are fed into the neural network by using the input representations to make predictions about the output labels. The training data provides feedback to the correctness of the weights in the neural network depending on how well the predicted output (e.g., probability of carrot) for a particular input matches



C. C. Aggarwal, *Neural Networks and Deep Learning*,
https://doi.org/10.1007/978-3-031-29642-0_1




(a) Biological neural network (b) Artificial neural network

Figure 1.1: The synaptic connections between neurons. The image in (a) is from "*The Brain: Understanding Neurobiology Through the Study of Addiction* [621]." Copyright ©2000 by BSCS & Videodiscovery. All rights reserved. Used with permission.

the annotated output label in the training data. One can view the errors made by the neural network in the computation of a function as a kind of unpleasant feedback in a biological organism, leading to an adjustment in the synaptic strengths. Similarly, the weights between neurons are adjusted in a neural network in response to prediction errors. The goal of changing the weights is to modify the computed function to make the predictions more correct in future iterations. Therefore, the weights are changed carefully in a mathematically justified way so as to reduce the error in computation on that example. By successively adjusting the weights between neurons over many input-output pairs, the function computed by the neural network is refined over time so that it provides more accurate predictions. Therefore, if the neural network is trained with many different images of bananas, it will eventually be able to properly recognize a banana in an image it has not seen before. This ability to accurately compute functions of unseen inputs by training over a finite set of input-output pairs is referred to as *model generalization*. The primary usefulness of all machine learning models is gained from their ability to generalize their learning from seen training data to unseen examples.

The biological comparison is often criticized as a very poor caricature of the workings of the human brain; nevertheless, the principles of neuroscience have often been useful in designing neural network architectures. A different view is that neural networks are built as higher-level abstractions of the classical models that are commonly used in machine learning. In fact, the most basic units of computation in the neural network are inspired by traditional machine learning algorithms like *least-squares regression* and *logistic regression*. Neural networks gain their power by putting together many such basic units, and learning the weights of the different units jointly in order to minimize the prediction error. From this point of view, a neural network can be viewed as a *computational graph* of elementary units in which greater power is gained by connecting them in particular ways. When a neural network is used in its most basic form, without hooking together multiple units, the learning algorithms often reduce to classical machine learning models (see Chapter 3). The real power of a neural model over classical methods is unleashed when these elementary computational units are combined. By combining multiple units, one is increasing the power of the model to learn more complicated functions of the data than are inherent in the elementary models of basic machine learning. The way in which these units are combined also plays a role in the power of the architecture, and requires some understanding and insight from the analyst.

## Humans versus Computers: Stretching the Limits of Artificial Intelligence

Humans and computers are inherently suited to different types of tasks. For example, computing the cube root of a large number is very easy for a computer, but it is extremely difficult for humans. On the other hand, a task such as recognizing the objects in an image is a simple matter for a human, but has traditionally been very difficult for an automated learning algorithm. It is only in recent years that deep learning has shown an accuracy on some of these tasks that exceeds that of a human. In fact, the recent results by deep learning algorithms that surpass human performance [194] in (some narrow tasks on) image recognition would not have been considered likely by most computer vision experts as recently as 20 years ago.

Many neural networks that have shown such extraordinary performance by connecting computational units in careful ways with a *deep architecture*. This performance mirrors the fact that biological neural networks gain much of their power from depth and careful connectivity as well. Unfortunately, biological networks are connected in ways we do not fully understand. In the few cases that the biological structure is understood at some level, significant breakthroughs have been achieved by designing artificial neural networks along those lines. A classical example of this type of architecture is the use of the *convolutional neural network* for image recognition. This architecture was inspired by Hubel and Wiesel's experiments [223] in 1959 on the organization of the neurons in the cat's visual cortex. The precursor to the convolutional neural network was the *neocognitron* [132], which was directly based on these results.

The human neuronal connection structure has evolved over millions of years to optimize survival-driven performance; survival is closely related to our ability to merge sensation and intuition in a way that is currently not possible with machines. Biological neuroscience [242] is a field that is still very much in its infancy, and only a limited amount is known about how the brain truly works. Therefore, it is fair to suggest that the biologically inspired success of convolutional neural networks might be replicated in other settings, as we learn more about how the human brain works [186].

A large part of the recent success of neural networks is explained by the fact that the increased data availability and computational power of modern computers has outgrown the limits of traditional machine learning algorithms, which fail to take full advantage of what is now possible. This situation is illustrated in Figure 1.2. The performance of traditional machine learning remains better at times for smaller data sets because of more

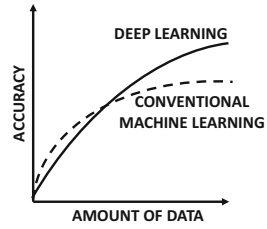



Figure 1.2: An illustrative comparison of the accuracy of a typical machine learning algorithm with that of a large neural network. Recent years have seen an increase in data availability and computational power, which has led to a "Cambrian explosion" in deep learning technology.

choices, greater ease of model interpretation, and the tendency to hand-craft interpretable features that incorporate domain-specific insights. With limited data, the best of a very wide diversity of models in machine learning will usually perform better than a single class of models (like neural networks). This is one reason why the potential of neural networks was not realized in the early years.

The "big data" era has been enabled by the advances in data collection technology; virtually everything we do today, including purchasing an item, using the phone, or clicking on a site, is collected and stored somewhere. Furthermore, the development of powerful Graphics Processor Units (GPUs) has enabled increasingly efficient processing on such large data sets. These advances largely explain the recent success of deep learning using algorithms that are only slightly adjusted from the versions that were available two decades back. Furthermore, these recent adjustments to the algorithms have been enabled by increased speed of computation, because reduced run-times enable efficient testing (and subsequent algorithmic adjustment). If it requires a month to test an algorithm, at most twelve variations can be tested in an year on a single hardware platform. This situation has historically constrained the intensive experimentation required for tweaking neural-network learning algorithms. The rapid advances associated with the three pillars of improved data, computation, and experimentation have resulted in an increasingly optimistic outlook about the future of deep learning. By the end of this century, it is expected that computers will have the power to train neural networks with as many neurons as the human brain. Although it is hard to predict what the true capabilities of artificial intelligence will be by then, our experience with computer vision should prepare us to expect the unexpected.

### Chapter Organization

This chapter is organized as follows. The next section introduces single-layer and multilayer networks. The different components of neural networks are discussed in section 1.3. Multilayer neural networks are introduced in section 1.4. The importance of nonlinearity is discussed in section 1.5. Advanced topics in deep learning are discussed in section 1.6. Some notable benchmarks used by the deep learning community are discussed in section 1.7. A summary is provided in section 1.8.

## The Basic Idea of Neural Networks

All neural networks compute functions from input nodes to output nodes. Each node has a variable inside it which is the result of a function computation or is fixed externally (in case of *input nodes*). A neural network is structured as a directed acyclic graph. The edges of the neural network are parameterized with weights, so that the functions computed at individual nodes are affected by the weights of the incoming edges as well as the variables in the nodes at the tails of these edges. The overall function computed by a network is result of cascading function computations at individual nodes; by setting the weights on the edges appropriately, one can compute almost any function from the input to the output. In a *single-layer network*, a set of inputs is directly connected to one or more output nodes, and the output nodes compute a function of their inputs and the weights on the edges. In multi-layer neural networks, the neurons are arranged in layered fashion, in which the input and output layers are separated by a group of hidden layers of nodes.

The goal of a neural network is to *learn* a function that relates one or more inputs to one or more outputs with the use of *training examples*. In other words, we only have examples of inputs and outputs, and the neural network "constructs" a function that relates the

inputs to the outputs. This "construction" is achieved by setting the weights on the edges in such a way that the computations from inputs to outputs in the neural network match the observed data. Setting the edge weights in a data-driven manner is at the heart of all neural network learning. This process is also referred to as *training*.

For simplicity, we first consider the case where there are $d$ inputs and a single *binary* output drawn from $\{-1, +1\}$. Therefore, each training instance is of the form $(\overline{X}, y)$, where each $\overline{X} = [x_1, \ldots x_d]$ contains $d$ feature variables, and $y \in \{-1, +1\}$ contains the *observed value* of the binary class variable. By "observed value" we refer to the fact that it is given to us as a part of the training data, and our goal is to predict the class variable for cases in which it is not observed. The variable $y$ is referred to as the *target variable*, and it represents the all-important property that the machine learning algorithm is trying to predict. In other words, we want to learn the function $f(\cdot)$, where we have the following:

$$y = f_{\overline{W}}(\overline{X})$$

Here, $\overline{X}$ corresponds to the feature vector, and $\overline{W}$ is a weight vector that needs to be computed in a data-driven manner in order to "construct" the prediction function. The function $f(\overline{X})$ has been subscripted with a weight vector to show that it depends on $\overline{W}$ as well. Most machine learning problems reduce to the following optimization problem in one form or the other:

$$\text{Minimize}_{\overline{W}} \text{ Mismatching between } y \text{ and } f_{\overline{W}}(\overline{X})$$

The mismatching between predicted and observed values is penalized with the use of a *loss function*. For example, in a credit-card fraud detection application, the features in the vector $\overline{X}$ might represent various properties of a credit card transaction (e.g., amount and type of transaction), and the binary class variable $y \in \{1, -1\}$ might represent whether or not this transaction is fraudulent. Clearly, in this type of application, one would have historical cases in which the class variable is observed (i.e., transactions together with their fraudulent indicator), and other (current) cases of transactions in which the class variable has not yet been observed but needs to be predicted. This second type of data is referred to as the *testing data*. Therefore, one must first construct the function $f(\overline{X})$ with the use of the training data (in an indirect way by deriving the weights of the neural network), and then use this function in order to predict the value of $y$ using this function for cases in which the target variable $y$ is not known. In the next section, we will discuss a neural network with a single computational layer that can be trained with this type of data.

## 1.2 Single Computational Layer: The Perceptron

The simplest neural network is referred to as the perceptron. This neural network contains a single input layer and an output node. The basic architecture of the perceptron is shown in Figure 1.3(a). Since the training data has $d$ inputs and a single output, we assume that the neural network has $d$ inputs denoted by the row vector $\overline{X}$ (which is the argument of the function of to be learned) and a single output $y$. The input layer contains $d$ nodes that transmit the $d$ features $x_1 \ldots x_d$ contained in the row vector $\overline{X} = [x_1 \ldots x_d]$. We use row vectors for features (instead of the usual convention of using column vectors) since feature vectors are often extracted from rows of data matrices. The input layer is incident edges of weight $w_1 \ldots w_d$, contained in the column vector $\overline{W} = [w_1 \ldots w_d]^T$, and all these edges are incident on a single output node. The input layer does not perform any computation

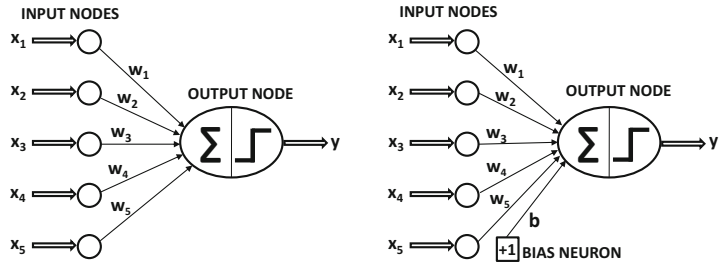
in its own right. The linear function $\overline{W} \cdot \overline{X}^T = \sum_{i=1}^{d} w_i x_i$ is computed at the output node. Subsequently, the sign of this real value is used in order to predict the dependent variable of $\overline{X}$. Therefore, the prediction $\hat{y}$ is computed as follows:

$$\hat{y} = f(\overline{X}) = \text{sign}\{\overline{W} \cdot \overline{X}^T\} = \text{sign}\{\sum_{j=1}^{d} w_j x_j\} \tag{1.1}$$

The sign function maps a real value to either $+1$ or $-1$, which is appropriate for binary classification. Note the circumflex on top of the variable $\hat{y}$ to indicate that it is a predicted value rather than the (original) observed value $y$. One can already see that the neural network creates a basic form of the function $f(\cdot)$ to be learned, although the weights $\overline{W}$ are unknown and need to be inferred in a data-driven manner from the training data. In most cases, a *loss function* is to used to quantify a nonzero (positive) cost when the observed value $y$ of the class variable is different from the predicted value $\hat{y}$. The weights of the neural network are learned in order to minimize the aggregate loss over all training instances.

Initially, the weights in column vector $\overline{W}$ are unknown and they may be set randomly. As a result, the prediction $\hat{y}$ will be random for a given input $\overline{X}$, and it may not match the observed value $y$. The goal of the learning process is to use these errors to modify the weights, so that the neural network predictions become more accurate. Therefore, a neural network starts with a basic form of the function $f(\overline{X})$ while allowing some ambiguities in the form of *parametrization*. By learning the parameters, also referred to as the weights, one also learns the function computed by the neural network. This is a general principle in machine learning, where the basic structure of a function relating two or more variables is chosen, and some parameters are left unspecified. These parameters are then learned in a data-driven manner, so that the functional relationships between the two variables are consistent with the observed data. This process requires the formulation of an optimization problem.

The architecture of the perceptron is shown in Figure 1.3(a), in which a single input layer transmits the features to the output node. The edges from the input to the output contain the weights $w_1 \ldots w_d$ with which the features are multiplied and added at the output node. Subsequently, the sign function is applied in order to convert the aggregated value into a class label. The sign function serves the role of an *activation function*, and we will see many other examples of this type of function later in this chapter. The value of the variable in a node in a neural network is also sometimes referred to as an *activation*. It is noteworthy that the perceptron contains two layers, although the input layer does not




(a) Perceptron without bias (b) Perceptron with bias

Figure 1.3: The basic architecture of the perceptron

perform any computation and only transmits the feature values. This is true across all neural architectures, where the input layer only performs a transmission role. The input layer is not included in the count of the number of layers in a neural network. Since the perceptron contains a single *computational* layer, it is considered a single-layer network.

At the time that the perceptron algorithm was proposed by Rosenblatt [421], the weights were learned using a heuristic update process with actual hardware circuits, and it was not presented in terms of a formal notion of optimization of loss functions in machine learning (as is common today). However, the goal was always to minimize the error in prediction, even if a formal optimization formulation was not presented. The perceptron algorithm was, therefore, heuristically designed to minimize the number of misclassifications. The training algorithm works by feeding each input data instance $\overline{X}$ into the network one by one (or in small batches) to create the prediction $\hat{y} = \text{sign}\{\overline{W} \cdot \overline{X}^T\}$. When this prediction is different from the observed class $y \in \{-1, +1\}$, the weights are updated using the error value $(y - \hat{y})$ so that this error is less likely to occur after the update. Specifically, when the data point $\overline{X}$ is fed into the network, the weight vector $\overline{W}$ (which is a column vector) is updated as follows:

$$\overline{W} \Leftarrow \overline{W} + \alpha(y - \hat{y})\overline{X}^T \tag{1.2}$$

The (heuristic) rationale for the above update is that it always increases the dot product of the weight vector and $\overline{X}^T$ by a quantity proportional to $(y - \hat{y})$, which tends to improve the prediction for that training instance. The parameter $\alpha$ regulates the *learning rate* of the neural network, although it can be set to 1 in the special case of the perceptron (as it only scales the weights). The perceptron algorithm repeatedly cycles through all the training examples in random order and iteratively adjusts the weights until convergence is reached. A single training data point may be cycled through many times. Each such cycle is referred to as an *epoch*.

Since the value of $(y - \hat{y}) \in \{-2, 0, +2\}$ is always equal to $2y$ in cases where $y \neq \hat{y}$, one can write the perceptron update as follows:

$$\overline{W} \Leftarrow \overline{W} + \alpha y \overline{X}^T \tag{1.3}$$

Note that a factor of 2 has been absorbed in the learning rate, and the update is *only* performed for examples where $y \neq \hat{y}$. For correctly classified training instances, no update is performed.

The type of model proposed in the perceptron is a *linear model*, in which the equation $\overline{W} \cdot \overline{X}^T = 0$ defines a linear hyperplane. Here, $\overline{W} = [w_1 \ldots w_d]^T$ is a $d$-dimensional vector that is perpendicular to this hyperplane. Furthermore, the value of $\overline{W} \cdot \overline{X}^T$ is positive for values of values of $\overline{X}$ on one side of the hyperplane, and it is negative for values of $\overline{X}$ on the other side. This type of model performs particularly well when the data is *linearly separable*. Linear separability refers to the case in which the two classes can be separated with a linear hyperplane. Examples of linearly separable and inseparable data are shown in Figure 1.4. In linearly separable cases, a weight vector $\overline{W}$ can always be found by the perceptron algorithm for which the sign of $\overline{W} \cdot \overline{X}_i^T$ matches $y_i$ for each training instance. It has been shown [421] that the perceptron algorithm always converges to provide zero error on the training data when the data are linearly separable. However, the perceptron algorithm is not guaranteed to converge in instances where the data are not linearly separable.

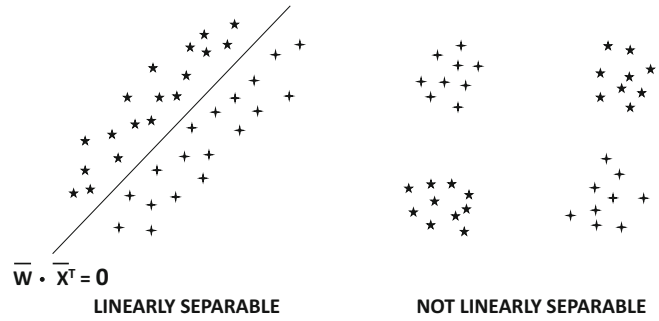



Figure 1.4: Examples of linearly separable and inseparable data in two classes

### 1.2.1 Use of Bias

In many settings, there is an invariant part of the prediction, which is referred to as the *bias*. For example, consider a setting in which the feature variables are mean centered, but the mean of the binary class prediction from $\{-1, +1\}$ is not 0. This will tend to occur in situations in which the binary class distribution is highly imbalanced. In such a case, the aforementioned approach is not sufficient for prediction since the different training points sum to zero, whereas the class variable do not. This is because adding up all the predictions over the training points (ignoring the sign operator), we obtain the following undesirable property:

$$\overline{W} \cdot \underbrace{\sum_i \overline{X}_i^T}_{=0} \neq \underbrace{\sum_i y_i}_{\neq 0}$$

When the differences are large, this model will perform poorly evenon the training data. We need to incorporate an additional bias variable $b$ that captures the invariant part of the prediction:

$$\hat{y} = \text{sign}\{\overline{W} \cdot \overline{X}^T + b\} = \text{sign}\{\sum_{j=1}^{d} w_j x_j + b\} \tag{1.4}$$

The bias can be incorporated as the weight of an edge by using a *bias neuron*. This is achieved by adding a neuron that always transmits a value of 1 to the output node. The weight of the edge connecting the bias neuron to the output node provides the bias variable. An example of a bias neuron is shown in Figure 1.3(b). Another approach that works well with single-layer architectures is to use a *feature engineering trick* in which an additional feature is created with a constant value of 1. The coefficient of this feature provides the bias, and one can then work with Equation 1.1. Throughout this book, biases will not be explicitly used (for simplicity in architectural representations) because they can be incorporated with bias neurons. However, they are often important to use in practice, as the effect on prediction accuracy can be quite large.

### 1.2.2 What Objective Function Is the Perceptron Optimizing?

Most machine learning algorithms (including neural networks) can be posed as loss optimization problems, wherein gradient descent updates are used to minimize the loss. The original perceptron paper by Rosenblatt [421] did not formally propose a loss function. In those years, these implementations were achieved using actual hardware circuits. The

Figure 1.5: The perceptron algorithm was originally implemented using hardware circuits. The image depicts the Mark I perceptron machine built in 1958. (Courtesy: Smithsonian Institute)

original *Mark I perceptron* was intended to be a machine rather than an algorithm, and custom-built hardware was used to create it (cf. Figure 1.5). The neural architecture was motivated by the (biological) *McCulloch-Pitts* model [332] of the neuron rather than a mathematical model. The general goal was to minimize the number of classification errors with a heuristic update process (in hardware) that changed weights in the "correct" direction whenever errors were made. This heuristic update strongly resembled gradient descent but it was not derived as a gradient-descent method. However, later work reverse engineered the loss function from the heuristic updates. This section will introduce this retrospective loss function.

Consider a training instance $(\overline{X}_i, y_i)$, where $\overline{X}_i$ is a row vector containing the features and $y_i$ is the observed class variable. The *perceptron criterion* [41] is a loss function that penalizes the training instance when the sign of $\overline{W} \cdot \overline{X}_i^T$ is different from $y_i$:

$$L_i = \max\{-y_i(\overline{W} \cdot \overline{X}_i^T), 0\} \tag{1.5}$$

Furthermore, large absolute values of $\overline{W} \cdot \overline{X}_i^T$ that also do not match the sign of $y_i$ are penalized to a greater degree. Gradient descent performs updates of $\overline{W}$ in the negative direction of the gradient of $L_i$ (with respect to $\overline{W}$). One can use calculus to verify that the gradient of the loss is as follows:

$$\frac{\partial L_i}{\partial \overline{W}} = \left[\frac{\partial L_i}{\partial w_1}, \ldots \frac{\partial L_i}{\partial w_d}\right]^T = \begin{cases} -y_i \overline{X}_i^T & \text{if sign}\{\overline{W} \cdot \overline{X}_i^T\} \neq y_i \\ 0 & \text{otherwise} \end{cases}$$

Note the use of the matrix calculus notation $\frac{\partial L_i}{\partial \overline{W}}$, which is the derivative of a scalar with respect to a vector, and is simply the $d$-dimensional vector $\left[\frac{\partial L_i}{\partial w_1}, \ldots \frac{\partial L_i}{\partial w_d}\right]^T$. The negative of the vector is the direction with the fastest rate of reduction of the loss, which provides the rationale for the perceptron update:

$$\begin{aligned} \overline{W} &\Leftarrow \overline{W} - \alpha \frac{\partial L_i}{\partial \overline{W}} \\ &= \overline{W} + \alpha y_i \overline{X}_i^T \quad \text{[For misclassified instances]} \end{aligned}$$

This loss function exposes some of the weaknesses of the updates in the original algorithm; one can set $\overline{W}$ to the zero vector *irrespective of the training data set* in order to obtain the optimal loss value of 0. In spite of this fact, the perceptron updates continue to converge to a meaningful solution in *linearly separable training data sets* and a nonzero initialization of $\overline{W}$. In linearly separable data sets, a nonzero weight vector $\overline{W}$ exists in which the sign of $\overline{W} \cdot \overline{X}_i^T$ correctly matches the sign of $y_i$ for *each and every* training instance $(\overline{X}_i, y_i)$. However, the behavior of the perceptron algorithm for data that are not linearly separable is rather arbitrary, and the resulting solution is sometimes not even a good approximate separator of the classes. The direct proportionality of the loss to the *magnitude* of the weight vector can dilute the goal of class separation by simply reducing the magnitude of the weight vector; it is possible for updates to worsen the number of misclassifications significantly while improving the loss. Because of this fact, the (vanilla) perceptron is not stable and can yield solutions of widely varying quality (for inseparable classes). Several variations of the perceptron were therefore proposed for inseparable data, and a natural approach is to always keep track of the best solution in terms of the number of misclassifications [133]. This approach of always keeping the best solution in one's "pocket" is referred to as the *pocket algorithm*.

## 1.3 The Base Components of Neural Architectures

The neural architecture of the perceptron algorithm uses a single output node, and the sign activation function. These aspects of neural architectures are a critical part of their design, and they may vary with the model at hand. This section discusses some of the common components of neural architectures.

### 1.3.1 Choice of Activation Function

The choice of activation function is a critical part of neural network design. Different types of nonlinear functions such as the *sign*, *sigmoid*, or *hyperbolic tangents* are commonly used. We have already seen the use of sign activation in the perceptron. We use the notation $\Phi$ to denote the activation function. A single-layer network with column vector $\overline{W}$ of weights and input (row) vector $\overline{X}$ would have a prediction of the following form:

$$\hat{y} = \Phi(\overline{W} \cdot \overline{X}^T) \tag{1.6}$$

The most basic activation function $\Phi(\cdot)$ is the linear activation, which is also referred to as the identity activation:

$$\Phi(v) = v$$

The linear activation function is often used in the output node, when the target is a real value.

The classical activation functions that were used early in the development of neural networks were the sign, sigmoid, and the hyperbolic tangent functions:

$$\Phi(v) = \text{sign}(v) \text{ (sign function)}$$

$$\Phi(v) = \frac{1}{1 + e^{-v}} \text{ (sigmoid function)}$$

$$\Phi(v) = \frac{e^{2v} - 1}{e^{2v} + 1} \text{ (tanh function)}$$

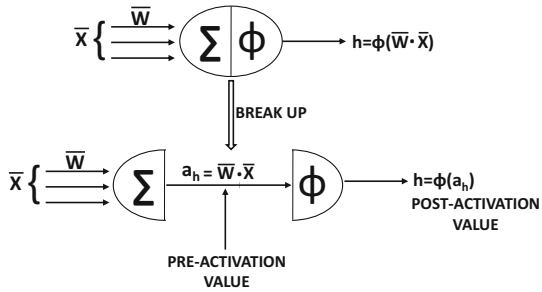



Figure 1.6: Pre- and post-activation values within a neuron

The break-up of the neuron computations into two separate values is shown in Figure 1.6. A neuron really computes two functions within the node, which is why we have incorporated the summation symbol $\Sigma$ as well as the activation symbol $\Phi$ within a neuron. The value computed before applying the activation function $\Phi(\cdot)$ will be referred to as the *pre-activation value*, whereas the value computed after applying the activation function is referred to as the *post-activation value*. An important point that emerges from Figure 1.6 is that one could treat a node with a nonlinear activation as two separate computational nodes, one of which performs the linear transformation $a_h = \overline{W} \cdot \overline{X}$ and the second performs the nonlinear transformation $\Phi(a_h)$. Indeed, this type of decoupled architecture is often used in order to simplify various types of analytical results in neural networks.

While the sign activation can be used to map to binary outputs at prediction time, its non-differentiability prevents its use for creating the loss function at training time. For example, while the perceptron uses the sign function for prediction, the perceptron criterion uses only linear activation. The sigmoid activation outputs a value in $(0, 1)$, which is helpful in interpreting outputs as probabilities. The tanh function has a shape similar to that of the sigmoid function, except that it is horizontally stretched and vertically translated/re-scaled to $[-1, 1]$. The tanh and sigmoid functions are related as follows (see Exercise 3):

$$\tanh(v) = 2 \cdot \text{sigmoid}(2v) - 1$$

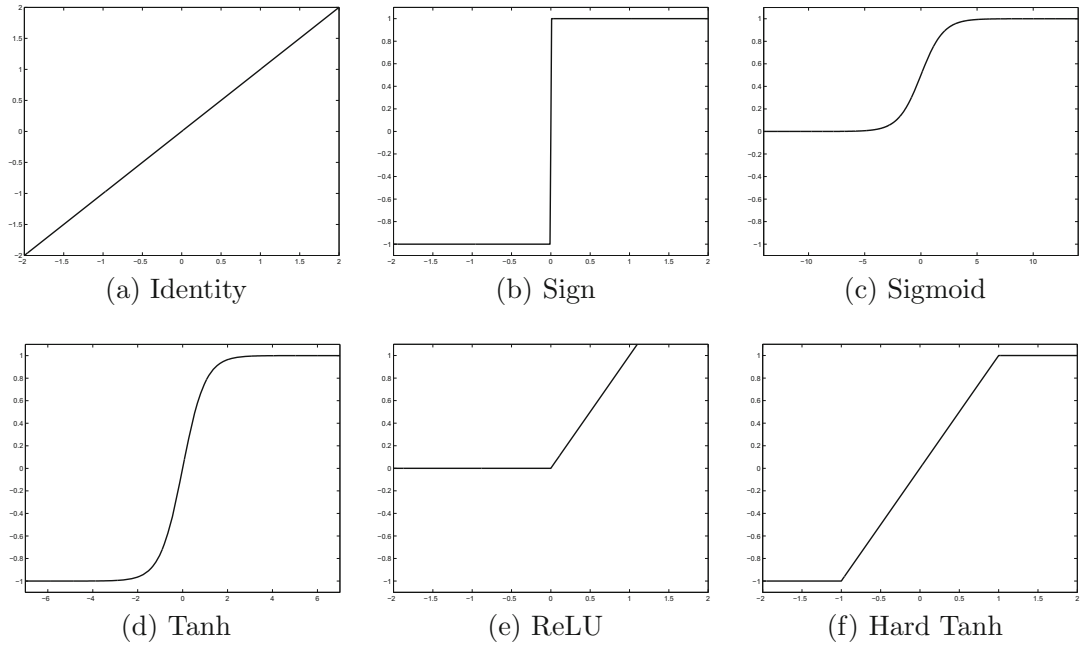
The tanh function is preferable to the sigmoid when the outputs of the computations are desired to be both positive and negative. The sigmoid and the tanh functions have been the historical tools of choice for incorporating nonlinearity in the neural network. In recent years, however, a number of piecewise linear activation functions have become more popular:

$$\Phi(v) = \max\{v, 0\} \text{ (Rectified Linear Unit [ReLU])}$$
$$\Phi(v) = \max\{\min[v, 1], -1\} \text{ (hard tanh)}$$

The ReLU and hard tanh activation functions have substantially replaced the sigmoid and soft tanh activation functions in modern neural networks (cf. Chapter 4).

Pictorial representations of all the aforementioned activation functions are illustrated in Figure 1.7. It is noteworthy that all activation functions shown here are monotonic. Furthermore, other than the identity activation function, most[1] of the other activation functions *saturate* at large absolute values of the argument at which increasing further does not change the activation much. As we will see later, such nonlinear activation functions are

[1]The ReLU shows asymmetric saturation.



(a) Identity (b) Sign (c) Sigmoid

(d) Tanh (e) ReLU (f) Hard Tanh

Figure 1.7: Various activation functions

also very useful in larger and more complex neural networks, because they help in creating more powerful compositions of different types of functions. Many of these functions are referred to as *squashing* functions, as they map the outputs from an arbitrary range to bounded outputs. The use of a nonlinear activation plays a fundamental role in increasing the modeling power of a network. If a network used only linear activations, it would not provide better modeling power than a single-layer linear network. This issue is discussed in section 1.5.

### 1.3.2 Softmax Activation Function

The softmax activation function is unique in that it is *almost always used in the output layer* to map $k$ real values into $k$ probabilities of discrete events. For example, consider the $k$-way classification problem in which each data record needs to be mapped to one of $k$ unordered class labels. In such cases, $k$ output values can be used, with a *softmax activation function* with respect to $k$ real-valued outputs $\overline{v} = [v_1, \ldots, v_k]$ at the nodes in a given layer. This activation function maps real values to probabilities that sum to 1. Specifically, the activation function for the $i$th output is defined as follows:

$$\Phi(\overline{v})_i = \frac{\exp(v_i)}{\sum_{j=1}^{k} \exp(v_j)} \quad \forall i \in \{1, \ldots, k\} \tag{1.7}$$

An example of the softmax function with three outputs is illustrated in Figure 1.8, and the values $v_1$, $v_2$, and $v_3$ are also shown in the same figure. Note that the three outputs correspond to the probabilities of the three classes, and they convert the three outputs of the final hidden layer into probabilities with the softmax function. The final hidden layer often uses linear (identity) activations, when it is input into the softmax layer. Furthermore,

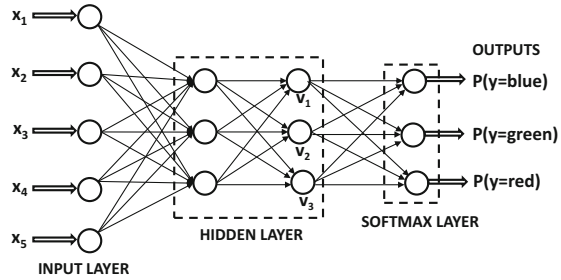



Figure 1.8: An example of multiple outputs for categorical classification with the use of a softmax layer

there are no weights associated with the softmax layer, since it is only converting real-valued outputs into probabilities. Each output is the probability of a particular class.

### 1.3.3 Common Loss Functions

The choice of the loss function is critical in defining the outputs in a way that is sensitive to the application at hand. For example, least-squares regression with numeric outputs requires a simple squared loss of the form $(y - \hat{y})^2$ for a single training instance with target $y$ and prediction $\hat{y}$. For probabilistic predictions of categorical data, two types of loss functions are used, depending on whether the prediction is binary or whether it is multiway:

1. **Binary targets (logistic regression):** In this case, it is assumed that the observed value $y$ is drawn from $\{-1, +1\}$, and the prediction $\hat{y}$ uses a sigmoid activation function to output $\hat{y} \in (0, 1)$, which indicates the probability that the observed value $y$ is 1. Then, the negative logarithm of $|y/2 - 0.5 + \hat{y}|$ provides the loss. This is because $|y/2 - 0.5 + \hat{y}|$ indicates the probability that the prediction is correct.

2. **Categorical targets:** In this case, if $\hat{y}_1 \ldots \hat{y}_k$ are the probabilities of the $k$ classes (using the softmax activation of Equation 1.8), and the $r$th class is the ground-truth class, then the loss function for a single instance is defined as follows:

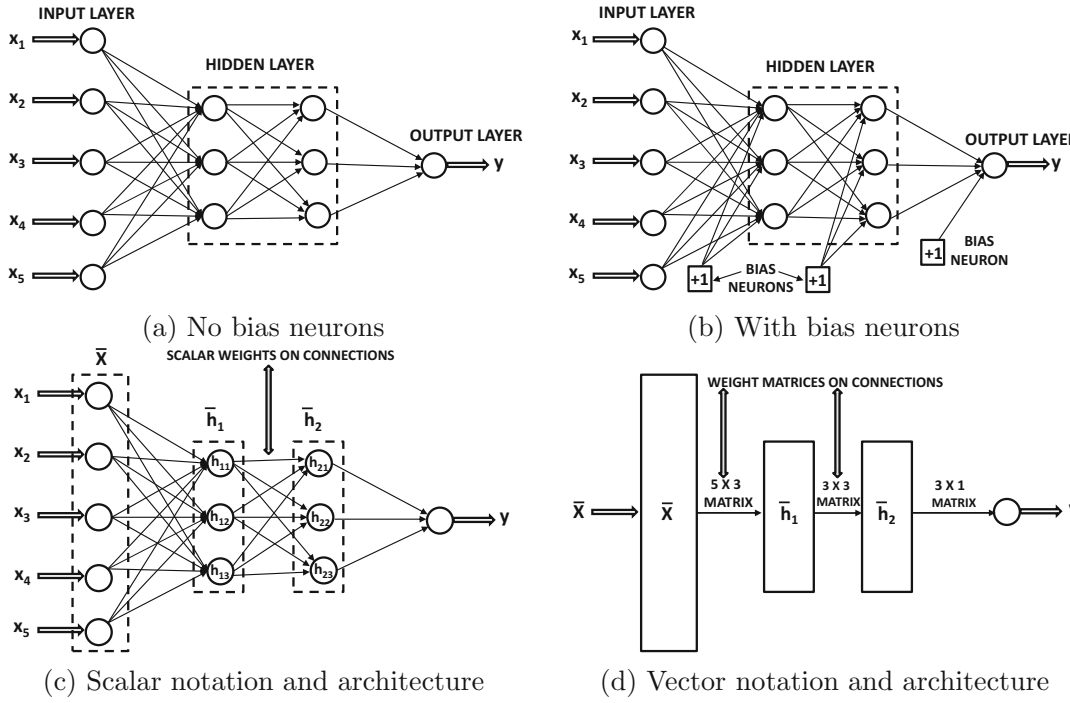
$$L = -\log(\hat{y}_r) \tag{1.8}$$

   This type of loss function implements multinomial logistic regression, and it is referred to as the *cross-entropy loss*. Note that binary logistic regression is identical to multinomial logistic regression, when the value of $k$ is set to 2 in the latter.

The key point to remember is that the nature of the output nodes, the activation function, and the loss function depend on the application at hand.

## 1.4 Multilayer Neural Networks

Multilayer neural networks contain more than one computational layer. The perceptron contains an input and output layer, of which the output layer is the only computation-performing layer. The input layer transmits the data to the output layer, and all computations are completely visible to the user. Multilayer neural networks contain multiple computational layers; the additional intermediate layers (between input and output) are referred to as *hidden layers* because the computations performed are not visible to the user.




(a) No bias neurons

(b) With bias neurons

(c) Scalar notation and architecture

(d) Vector notation and architecture

Figure 1.9: The basic architecture of a feed-forward network with two hidden layers and a single output layer. Even though each unit contains a single scalar variable, one often represents all units within a single layer as a single vector unit. Vector units are often represented as rectangles and have connection *matrices* between them.

The specific architecture of multilayer neural networks is referred to as *feed-forward* networks, because successive layers feed into one another in the forward direction from input to output. The default architecture of feed-forward networks assumes that all nodes in one layer are connected to those of the next layer. Therefore, the architecture of the neural network is almost fully defined, once the number of layers and the number/type of nodes in each layer have been defined. The only remaining detail is the loss function that is optimized in the output layer. Like the perceptron criterion, the loss function penalizes undesirable deviations of predicted outputs of the neural network from the observed outputs in the training data.

As in the case of single-layer networks, bias neurons can be used both in the hidden layers and in the output layers. Examples of multilayer networks with or without the bias neurons are shown in Figures 1.9(a) and (b), respectively. In each case, the neural network contains three layers. Note that the input layer is often not counted, because it simply transmits the data and no computation is performed in that layer. If a neural network contains $p_1 \ldots p_k$ units in each of its $k$ layers, then the (column) vector representations of these outputs, denoted by $\overline{h}_1 \ldots \overline{h}_k$ have dimensionalities $p_1 \ldots p_k$. Therefore, the number of units in each layer is referred to as the *dimensionality* of that layer.

The weights of the connections between the input layer and the first hidden layer are contained in a *matrix* $W_1$ with size $p_1 \times d$, whereas the weights between the $r$th hidden layer and the $(r+1)$th hidden layer are denoted by the $p_{r+1} \times p_r$ matrix denoted by $W_r$. Note

that the number of units in the $(r+1)$th layer defines the number of rows, whereas the number of units in the $r$th layer defines the number of columns. If the output layer contains $o$ nodes, then the final matrix $W_{k+1}$ of a $(k+1)$-layered network is of size $o \times p_k$. The $d$-dimensional input vector $\overline{x}$ is transformed into the outputs using the following recursive equations:

$$\overline{h}_1 = \Phi(W_1 \overline{x}) \qquad \text{[Input to Hidden Layer]}$$
$$\overline{h}_{p+1} = \Phi(W_{p+1} \overline{h}_p) \quad \forall p \in \{1 \ldots k-1\} \qquad \text{[Hidden to Hidden Layer]}$$
$$\overline{o} = \Phi(W_{k+1} \overline{h}_k) \qquad \text{[Hidden to Output Layer]}$$

Although activation functions like the sigmoid function have scalar[2] arguments, they can be applied in *element-wise* fashion to their vector arguments. In other words, the activation function is applied to each of the elements of the vector to create a vector of equal length containing the outputs in the same order. One can also represent neural network architectures with vector variables. Many architectural diagrams combine the units in a single layer to create a single vector unit, which is represented as a *rectangle* rather than a *circle*. For example, the architectural diagram in Figure 1.9(c) (with scalar units) has been transformed to a vector-based neural architecture in Figure 1.9(d). Note that the weights on the connections between the vector units are now matrices. It is noteworthy that even though the *connection matrix* between the 5-dimensional input layer and the first 3-dimensional hidden layer has been annotated as a $5 \times 3$ matrix in Figure 1.9(d), one would need to use a $3 \times 5$ matrix $W_1$ in order to multiply the 5-dimensional column vector of inputs. *Therefore, the weight matrix dimensions are the transposes of the connection matrix dimensions.*

An implicit assumption in the vector-based neural architecture is that all units in a layer use the same activation function, which is applied in element-wise fashion to that layer. This constraint is usually not a problem, because most neural architectures use the same activation function throughout the computational pipeline, with the only deviation caused by the nature of the output layer. Throughout this book, neural architectures in which units contain vector variables will be depicted with rectangular units, whereas scalar variables will correspond to circular units.

### 1.4.1 The Multilayer Network as a Computational Graph

It is helpful to view a neural network as a *computational graph*, which is constructed by piecing together many basic parametric models. Neural networks are fundamentally more powerful than their building blocks because the parameters of these models are learned *jointly* to create a highly optimized composition function of these models.

A multilayer network evaluates compositions of functions computed at individual nodes. A path of length 2 in the neural network in which the function $f(\cdot)$ follows $g(\cdot)$ can be considered a composition function $f(g(\cdot))$. Furthermore, if $g_1(\cdot)$, $g_2(\cdot) \ldots g_k(\cdot)$ are the functions computed in layer $m$, and a particular layer-$(m+1)$ node computes $f(\cdot)$, then the composition function computed by the layer-$(m+1)$ node in terms of the layer-$m$ inputs is $f(g_1(\cdot), \ldots g_k(\cdot))$. It is noteworthy that each of these functions is parameterized because of the presence of edge weights. Therefore, we have the following:

> A multilayer network computes a nested composition of parameterized multivariate functions. The overall function computed from the inputs to the outputs

[2]Some activation functions such as the softmax (which are typically used in the output layers) naturally have vector arguments.

can be controlled very closely by the choice of parameters. The notion of **learning** refers to the setting of the parameters to make the overall function consistent with observed input-output pairs.

The repeated nesting and the large number of nodes in each layer combine to make it difficult to even express the input-to-output function of a neural network in a compact and comprehensible way. This is a major difference from traditional machine learning algorithms in which the prediction function can often be crisply written in closed form.

The use of nonlinear activation functions is the key to increasing the power of multiple layers. If all layers use an identity activation function, then a multilayer network is no more powerful than a single-layer network (see next section). It has been shown [218] that a network with a single (nonlinear) hidden layer and a single (linear) output layer can compute almost any "reasonable" function. As a result, neural networks are often referred to as *universal function approximators*, although this theoretical claim is not always easy to translate into practical usefulness.

The weights are learned by penalizing differences between the observed and predicted output for the $i$th training instance with the use of a loss $L_i$ (analogous to the perceptron criterion). The weights $\overline{W}$ of the neural network are then updated with gradient descent to reduce each loss $L_i$:

$$\overline{W} \Leftarrow \overline{W} - \alpha \frac{\partial L_i}{\partial \overline{W}} \tag{1.9}$$

Similar to the perceptron, training instances are fed to the neural network one by one to make these updates.

How does one compute these gradients of the form $\frac{\partial L_i}{\partial \overline{W}}$? Multilayer neural networks (often) do not create straightforward loss functions like the perceptron that can be expressed (concisely) in closed form. The availability of closed-form loss functions in traditional machine learning is really a luxury that is helpful in using calculus to differentiate the loss function with respect to the parameters and perform updates like Equation 1.9. The fact that the loss functions of multilayer neural networks are often too awkward (and massive) to write in closed form creates challenges from the perspective of gradient descent. In this respect, the crown jewel of the neural network paradigm is the *backpropagation algorithm*; this algorithm can work out the complicated parameter update steps based on the structure of the underlying computational graph and the functions computed in each node. The analyst does not have to spend the time and effort to explicitly compute the loss function in closed form and work out the derivatives with the use of calculus. Working out these steps is often the most difficult part of most machine learning algorithms, and an important contribution of the neural network paradigm is to bring modular thinking into machine learning. In other words, the modularity in neural network design translates to modularity in learning its parameters; the specific name for the latter type of modularity is "backpropagation." This makes the design of neural networks more of an (experienced) engineer's task rather than a mathematical exercise. The backpropagation algorithm is discussed in detail in Chapter 2.

The "building block" description is particularly appropriate for multilayer neural networks. Very often, off-the-shelf softwares for building neural networks[3] provide analysts with access to these building blocks. The analyst is able to specify the number and type of units in each layer along with an off-the-shelf or customized loss function. A deep neural network containing tens of layers can often be described in a few hundred lines of code. This

[3]Examples include Torch [597], Theano [598], and TensorFlow [599].

makes the process of trying different types of architectures relatively painless for the analyst. Building a neural network with many of the off-the-shelf softwares is often compared to a child constructing a toy from building blocks that appropriately fit with one another. Each block is like a unit (or a layer of units) with a particular type of activation. All these off-the-shelf softwares have the goal of learning the weights of the neural network with the backpropagation algorithm.

## 1.5 The Importance of Nonlinearity

At its most basic level, a neural network is a computational graph that performs compositions of simpler functions to provide a more complex function. Much of the power of deep learning arises from the fact that the *repeated* composition of functions has significant expressive power. Not all base functions are equally good at achieving this goal. In fact, the nonlinear squashing functions used in neural networks are not arbitrarily chosen, but are carefully designed because of certain types of properties. For example, imagine a situation in which the identity activation function is used in each layer, so that only linear functions are computed. In such a case, the resulting neural network is no stronger than a single-layer, linear network:

**Theorem 1.5.1** *A multi-layer network that uses only the identity activation function in all its layers reduces to a single-layer network.*

**Proof:** Consider a network containing $k$ hidden layers, and therefore contains a total of $(k+1)$ computational layers (including the output layer). The corresponding $(k+1)$ weight matrices between successive layers are denoted by $W_1 \ldots W_{k+1}$. Let $\overline{x}$ be the $d$-dimensional column vector corresponding to the input, $\overline{h}_1 \ldots \overline{h}_k$ be the column vectors corresponding to the hidden layers, and $\overline{o}$ be the $m$-dimensional column vector corresponding to the output. Then, we have the following recurrence condition for multi-layer networks:

$$\overline{h}_1 = \Phi(W_1\overline{x}) = W_1\overline{x}$$
$$\overline{h}_{p+1} = \Phi(W_{p+1}\overline{h}_p) = W_{p+1}\overline{h}_p \quad \forall p \in \{1 \ldots k-1\}$$
$$\overline{o} = \Phi(W_{k+1}\overline{h}_k) = W_{k+1}\overline{h}_k$$

In all the cases above, the activation function $\Phi(\cdot)$ has been set to the identity function. Then, by eliminating the hidden layer variables, we obtain the following:

$$\overline{o} = \underbrace{W_{k+1}W_k \ldots W_1}_{W_{xo}}\overline{x}$$

Note that one can replace the matrix $W_{k+1}W_k \ldots W_1$ with the new $d \times m$ matrix $W_{xo}$, and learn the coefficients of $W_{xo}$ instead of those of all the matrices $W_1$, $W_2 \ldots W_{k+1}$, without loss of expressivity. In other words, we have the following:

$$\overline{o} = W_{xo}\overline{x}$$

However, this condition is exactly identical to that of linear regression with multiple outputs [7]. Therefore, a multilayer neural network with identity activations does not gain over a single-layer network in terms of expressivity. ■

The aforementioned result is for the case of regression modeling with numeric target variables. A similar result holds true for binary target variables. In the special case, where

all layers use identity activation and the final layer uses a single output with sign activation for prediction, the multilayer neural network reduces to the perceptron. This result is summarized below.

**Lemma 1.5.1** *Consider a multilayer network in which all hidden layers use identity activation and the single output node with linear activation that uses the perceptron criterion as the loss function. This neural network reduces to the single-layer perceptron.*

Since multilayer neural networks perform repeated compositions of functions of the form $f(g(\cdot))$, the above result can also be stated as follows:

**Observation 1.5.1** *The composition of linear functions is always a linear function. The repeated composition of simple nonlinear functions can be a very complex nonlinear function.*

These observations suggest that deep networks largely make sense only when the activation functions in intermediate layers are non-linear. Typically, the functions like sigmoid and tanh are *squashing* functions in which the output is bounded within an interval, and the gradients are largest near zero values. For large absolute values of their arguments, these functions are said to reach *saturation* where increasing the absolute value of the argument further does not change its value significantly.

The universal approximation result of neural networks [218] posits that of combination of many squashing functions (like sigmoid) in a single hidden layer followed by a linear layer can be used to approximate any function well. Therefore, a two-layer network is sufficient as long as the number of hidden units is large enough. However, some kind of basic non-linearity in the activation function is always required in order to model the turns and twists in an arbitrary function. To understand this point, note that all 1-dimensional functions can be approximated as a sum of scaled/translated step functions and most of the activation functions discussed in this chapter (e.g., sigmoid) look awfully like step functions (see Figure 1.7). This basic idea is the essence of the universal approximation theorem of neural networks. However, this theoretical result cannot always be translated into practical usefulness with a limited amount of data.

### 1.5.1 Nonlinear Activations in Action

The previous section provides a concrete proof of the fact that a neural network with only linear activations does not gain from increasing the number of layers in it. For example, consider the two-class data set illustrated in Figure 1.10, which is represented in two dimensions denoted by $x_1$ and $x_2$. There are two instances, A and B, of the class denoted by '*' with coordinates $(1, 1)$ and $(-1, 1)$, respectively. There is also a single instance B of the class denoted by '+' with coordinates $(0, 1)$, A neural network with only linear activations will never be able to classify the training data perfectly because the points are not linearly separable.

On the other hand, consider a situation in which the hidden units have ReLU activation, and they learn the two new features $h_1$ and $h_2$, which are as follows:

$$h_1 = \max\{x_1, 0\}$$
$$h_2 = \max\{-x_1, 0\}$$

Note that these goals can be achieved by using appropriate weights from the input to hidden layer, and also applying a ReLU activation unit. The latter achieves the goal of thresholding negative values to 0. We have indicated the corresponding weights in the

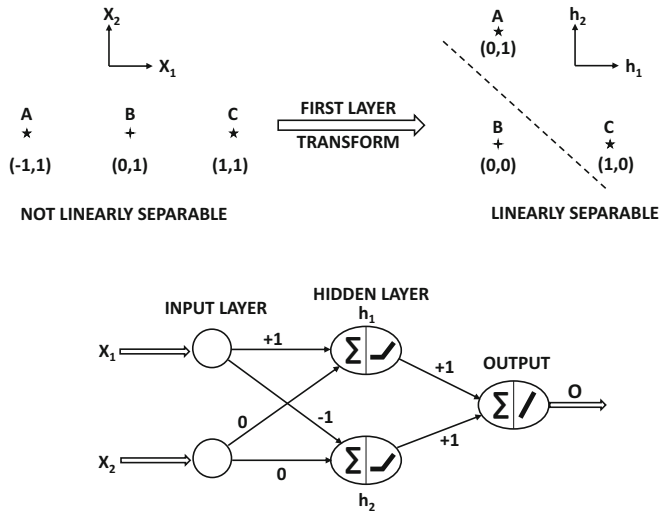



Figure 1.10: The power of nonlinear activation functions in transforming a data set to linear separability

neural network shown in Figure 1.10. We have shown a plot of the data in terms of $h_1$ and $h_2$ in the same figure. The coordinates of the three points in the 2-dimensional hidden layer are $\{(1,0),(0,1),(0,0)\}$. It is immediately evident that the two classes become linearly separable in terms of the new hidden representation. In a sense, the task of the first layer was *representation learning* to enable the solution of the problem with a linear classifier. Therefore, if we add a single linear output layer to the neural network, it will be able to classify these training instances perfectly. The key point is that the use of the nonlinear ReLU function is crucial in ensuring this linear separability. *Activation functions enable nonlinear mappings of the data, so that the embedded points can become linearly separable.* In fact, if both the weights from hidden to output layer are set to 1 with a linear activation function, the output $O$ will be defined as follows:

$$O = h_1 + h_2 \tag{1.10}$$

This simple linear function separates the two classes because it always takes on the value of 1 for the two points labeled '*' and takes on 0 for the point labeled '+'. Therefore, much of the power of neural networks is hidden in the use of activation functions. The weights shown in Figure 1.10 will need to be *learned* in a data-driven manner, although there are many alternative choices of weights that can make the hidden representation linearly separable. Therefore, the learned weights may be different than the ones shown in Figure 1.10 if actual training is performed. Nevertheless, in the case of the perceptron, there is no choice of weights at which one could hope to classify this training data set perfectly because the data set is not linearly separable in the original space. In other words, the activation functions enable nonlinear transformations of the data, that become increasingly powerful with multiple layers. A sequence of nonlinear activations imposes a specific type of structure on the learned model, whose power increases with the depth of the sequence (i.e., number of layers in the neural network).

Another classical example is the XOR function in which the two points $\{(0,0),(1,1)\}$ belong to one class, and the other two points $\{(1,0),(0,1)\}$ belong to the other class.

It is possible to use ReLU activation to separate these two classes as well, although bias neurons will be needed in this case (see Exercise 1). The original backpropagation paper [425] discusses the XOR function, because this function was one of the motivating factors for designing multilayer networks and the ability to train them. The XOR function is considered a litmus test to determine the basic feasibility of a particular family of neural networks to properly predict nonlinearly separable classes. Although we have used the ReLU activation function above for simplicity, it is possible to use most of the other nonlinear activation functions to achieve the same goals. There are several types of neural architectures that are used commonly in various machine learning applications. The subsequent section will discuss these architectures.

## 1.6 Advanced Architectures and Structured Data

Numerous advanced architectures are available to address different types of data and learning settings. Some of these architectures have fallen out of favor in terms of practical application, but are nevertheless important because of the principles they embody; two such architectures are *radial basis function networks* and *restricted Boltzmann machines*. Both these architectures are closely related to classical machine learning methods. The former is a generalization of a classical machine learning method, known as the *support vector machine*, whereas the latter is a special case of a classical machine learning family referred to as *probabilistic graphical models*. Therefore, understanding these models helps in relating neural networks to classical machine learning methods. A discussion of radial basis function networks is provided in Chapter 6, and a discussion of restricted Boltzmann machine is provided in Chapter 7. Numerous other advanced topics such as *attention mechanisms*, *generative adversarial networks*, and *Kohonen self-organizing maps* are discussed in Chapter 12.

Many types of data have structure embedded in them, where the individual elements of the data are not independent of one another. For example, in a text document, successive words are not independent of one another, and in a time series, the successive elements of the series are not independent of one another. Similarly, in an image data set, adjacent pixels have values that are highly correlated with one another. In all these cases, it makes sense to use our domain-specific understanding of the data (e.g., text or image) in order to design specialized neural architectures. This book will discuss three well-known neural architectures that are specifically designed for sequence, image, and graph data.

1. *Sequence data:* Recurrent neural networks are designed for sequential data like text sentences, time-series, and other discrete sequences like biological sequences. In these cases, the input is of the form $\overline{x}_1 \ldots \overline{x}_n$, where $\overline{x}_t$ is a $d$-dimensional point received at the time-stamp $t$. For example, the vector $\overline{x}_t$ might contain the $d$ values at the $t$th tick of a multivariate time-series (with $d$ different series). In a text-setting, the vector $\overline{x}_t$ will contain the *one-hot encoded* word at the $t$th time-stamp. In one-hot encoding, we have a vector of length equal to the lexicon size, and the component for the relevant word has a value of 1. All other components are 0. The sequential dependencies among such inputs can be addressed with recurrent neural networks, which are discussed in Chapter 8.

2. *Image data:* The spatial nature of image data is accommodated with layers that are spatially structured. In other words, the features in the layer are spatially related to one another just like pixels in an image. Such spatial layers require the use of different

types of operations in intermediate nodes, which take the spatial arrangement of features into account. One such important operation is the *convolution operation*, from which a convolutional neural network derives its name. Convolutional neural networks are discussed in Chapter 9.

3. *Graph data:* Many real-world networked entities such as the Web and various types of social networks are represented as graphs. Graph neural networks implicitly create a neural network structure that is based on the graph that it learns it. The construction of graph neural networks is based on principles of both recurrent networks and convolutional networks. Graph neural networks are discussed in Chapter 10.

Convolutional neural networks have historically been the most successful of all types of neural networks. They are used widely for image recognition, object detection/localization, and even text processing. The performance of these networks has recently exceeded that of humans in the problem of image classification [194].

## 1.7 Two Notable Benchmarks

The benchmarks used in the neural network literature are dominated by data from the domain of computer vision. Although traditional machine learning data sets like the UCI repository [625] can be used for testing neural networks, the general trend is towards using data sets from perceptually oriented data domains that can be visualized well. Although there are a variety of data sets drawn from the text and image domains, two of them stand out because of their ubiquity in deep learning papers. Although both are data sets drawn from computer vision, the first of them is simple enough that it can also be used for testing generic applications beyond the field of vision. In the following, we provide a brief overview of these two data sets.

### 1.7.1 The MNIST Database of Handwritten Digits

The MNIST database, which stands for *Modified National Institute of Standards and Technology* database, is a large database of handwritten digits [287]. As the name suggests, this data set was created by modifying an original database of handwritten digits provided by NIST. The data set contains 60,000 training images and 10,000 testing images. Each image is a scan of a handwritten digit from 0 to 9, and the differences between different images are a result of the differences in the handwriting of different individuals. These individuals were American Census Bureau employees and American high school students. The original black and white images from NIST were size normalized to fit in a $20 \times 20$ pixel box while preserving their aspect ratio and centered in a $28 \times 28$ image by computing the center of mass of the pixels. The images were translated to position this point at the center of the $28 \times 28$ field. Each of these $28 \times 28$ pixel values takes on a value from 0 to 255, depending on where it lies in the grayscale spectrum. The labels associated with the images correspond to the ten digit values. Examples of the digits in the MNIST database are illustrated in Figure 1.11. The size of the data set is rather small, and it contains only a simple object corresponding to a digit. Therefore, one might argue that the MNIST database is a toy data set. However, its small size and simplicity is also an advantage because it can be used as a laboratory for quick testing of machine learning algorithms. Furthermore, the simplification of the data set by virtue of the fact that the digits are (roughly) centered makes it easy to use it to test algorithms beyond computer vision. Computer vision algorithms require

Figure 1.11: Examples of handwritten digits in the MNIST database

specialized assumptions such as translation invariance. The simplicity of this data set makes these assumptions unnecessary. It has been remarked by Geoff Hinton [623] that the MNIST database is used by neural network researchers in much the same way as biologists use fruit flies for early and quick results (before serious testing on more complex organisms).

Although the matrix representation of each image is suited to a convolutional neural network, one can also convert it into a multidimensional representation of $28 \times 28 = 784$ dimensions. This conversion loses some of the spatial information in the image, but this loss is not debilitating (at least in the case of the MNIST data set) because of its relative simplicity. In fact, the use of a simple support vector machine on the 784-dimensional representation can provide an impressive error rate of about 0.56%. A straightforward 2-layer neural network on the multidimensional representation (without using the spatial structure in the image) generally does worse than the support vector machine across a broad range of parameter choices! A deep neural network without any special convolutional architecture can achieve an error rate of 0.35% [75]. Deeper neural networks and convolutional neural networks (that do use spatial structure) can reduce the error rate to as low as 0.21% by using an ensemble of five convolutional networks [419]. Therefore, even on this simple data set, one can see that the relative performance of neural networks with respect to traditional machine learning is sensitive to the specific architecture used in the former.

Finally, it should be noted that the 784-dimensional non-spatial representation of the MNIST data is used for testing all types of neural network algorithms beyond the domain of computer vision. Even though the use of the 784-dimensional (flattened) representation is not appropriate for a vision task, it is still useful for testing the general effectiveness of non-vision oriented (i.e., generic) neural network algorithms. For example, the MNIST data is frequently used to test data reconstruction algorithms for multidimensional data and not just image data. Even when an MNIST image is reconstructed using a tabular (i.e., not image domain-specific) algorithm, one can visualize the reconstructed pixels to obtain a feel of what the algorithm is doing with the data. This visual exploration often gives the researcher some insights that are not available with arbitrary data sets like those obtained from the UCI Machine Learning Repository [625]. In this sense, the MNIST data set tends to have broader usability than many other types of data sets.

### 1.7.2 The ImageNet Database

The *ImageNet* database [605] is a huge database of over 14 million images drawn from 1000 different categories. Its class coverage is exhaustive enough that it covers most types of images that one would encounter in everyday life. This database is organized according to

a *WordNet* hierarchy of nouns [341]. The *WordNet* database is a data set containing the relationships among English words using the notion of *synsets*. The *WordNet* hierarchy has been successfully used for machine learning in the natural language domain, and therefore it is natural to design an image data set around these relationships.

The *ImageNet* database is famous for the fact that an annual *ImageNet Large Scale Visual Recognition Challenge (ILSVRC)* [606] is held using this dataset. This competition has a very high profile in the vision community and receives entries from most major research groups in computer vision. The entries to this competition have resulted in many of the state-of-the-art image recognition architectures today, including the methods that have surpassed human performance on some narrow tasks like image classification [194]. Because of the wide availability of known results on these data sets, it is a popular alternative for benchmarking. The data set is large and diverse enough to be representative of the key visual concepts within the image domain. As a result, convolutional neural networks are often trained on this data set; the pretrained network can be used to extract features from an arbitrary image. This image representation is defined by the hidden activations in the penultimate layer of the neural network. Such an approach creates new multidimensional representations of image data sets that are amenable for use with traditional machine learning methods. One can view this approach as a kind of transfer learning in which the visual concepts in the *ImageNet* data set are transferred to unseen data objects for other applications.

## 1.8 Summary

Although a neural network can be viewed as a simulation of the learning process in living organisms, a more direct understanding of neural networks is as computational graphs. Such computational graphs perform recursive composition of simpler functions in order to learn more complex functions. Since these computational graphs are parameterized, the problem generally boils down to learning the parameters of the graph in order to optimize a loss function. The simplest types of neural networks are often basic machine learning models like least-squares regression. The real power of neural networks is unleashed by using more complex combinations of the underlying functions. The main advantage of the neural paradigm is that one can use the backpropagation algorithm to compute gradients. The design of deep learning methods in specific domains such as text and images requires carefully crafted architectures, such as recurrent neural networks and convolutional neural networks.

## 1.9 Bibliographic Notes and Software Resources

A proper understanding of neural network design requires a solid understanding of machine learning algorithms, and especially the linear models based on gradient descent. The reader is recommended to refer to [2, 3, 40, 187] for basic knowledge on machine learning methods. Numerous surveys and overviews of neural networks in different contexts may be found in [28, 29, 208, 283, 356, 450]. Classical books on neural networks for pattern recognition may be found in [41, 192], whereas more recent perspectives on deep learning may be found in [154]. A recent text mining book [7] also discusses recent advances in deep learning for text analytics. An overview of the relationships between deep learning and computational neuroscience may be found in [186, 249].

The perceptron algorithm was proposed by Rosenblatt [421]. To address the issue of stability, the *pocket algorithm* [133], the *Maxover* algorithm [542], and other margin-based

methods [128]. Other early algorithms of a similar nature included the Widrow-Hoff [550] and the Winnow algorithms [255]. The Winnow algorithm uses multiplicative updates instead of additive updates, and is particularly useful when many features are irrelevant. The original idea of backpropagation was based on the idea of differentiation of composition of functions as developed in control theory [54, 247]. The use of dynamic programming to perform gradient-based optimization of variables that are related via a directed acyclic graph has been a standard practice since the sixties. However, the ability to use these methods for neural network training had not yet been observed at the time. In 1969, Minsky and Papert published a book on perceptrons [342], which was largely negative about the potential of being able to properly train multilayer neural networks. The book showed that a single perceptron had limited expressiveness, and no one knew how to train multiple layers of perceptrons anyway. Minsky was an influential figure in artificial intelligence, and the negative tone of his book contributed to the first winter in the field of neural networks. The adaptation of dynamic programming methods to backpropagation in neural networks was first proposed by Paul Werbos in his PhD thesis in 1974 [543]. However, Werbos's work could not overcome the strong views against neural networks that had already become entrenched at the time. The backpropagation algorithm was proposed again by Rumelhart *et al.* in 1986 [424, 425]. Rumelhart *et al.*'s work is significant for the beauty of its presentation, and it was able to address at least some of the concerns raised earlier by Minsky and Papert. This is one of the reasons that the Rumelhart *et al.* paper is considered very influential from the perspective of backpropagation, even though it was certainly not the first to propose the method. A discussion of the history of the backpropagation algorithm may be found in the book by Paul Werbos [544].

At this point, the field of neural networks was only partially resurrected, as there were still problems with training neural networks. Furthermore, backpropagation turned out to be less effective at training deeper networks because of the vanishing and exploding gradient problems. However, by this time, it was already hypothesized by several prominent researchers that existing algorithms would yield large performance improvements with increases in data, computational power, and algorithmic experimentation. The coupling of big data frameworks with GPUs turned out to be a boon for neural network research in the late 2000s. With reduced cycle times for experimentation enabled by increased computational power, tricks like pretraining started showing up in the late 2000s [208]. The publicly obvious resurrection of neural networks occurred after the year 2011 with the resounding victories [263] of neural networks in deep learning competitions for image classification. The consistent victories of deep learning algorithms in these competitions laid the foundation for the explosion in popularity we see today. Notably, the differences of these winning architectures from the ones that were developed more than two decades earlier are modest (but essential).

The theoretical expressiveness of neural networks was recognized early in its development. For example, early work recognized that a neural network with a single hidden layer can be used to approximate any function [218]. A further result is that certain neural architectures like recurrent networks are Turing complete [464]. The latter means that neural networks can potentially simulate any algorithm. Of course, there are numerous practical issues associated with neural network training, as to why these exciting theoretical results do not always translate into real-world performance.

### Video Lectures

Deep learning has a significant number of free video lectures available on resources such as *YouTube* and *Coursera*. Two of the most authoritative resources include Geoff Hinton's course on YouTube [623] and Andrew Ng's course at *Coursera* [624]. *Coursera* has multiple offerings on deep learning, and offers a group of related courses in the area. During the writing of this book, an accessible course from Andrew Ng was also added to the offerings. A course on convolutional neural networks from Stanford University is freely available on *YouTube* [246]. The Stanford class by Karpathy, Johnson, and Fei-Fei [246] is on convolutional neural networks, although it does an excellent job in covering broader topics in neural networks. The initial parts of the course deal with vanilla neural networks and training methods.

Numerous topics in machine learning [92] and deep learning [93] are covered by Nando de Freitas in a lectures available on *YouTube*. Another interesting class on neural networks is available from Hugo Larochelle at the Universite de Sherbrooke [268]. A deep learning course by Ali Ghodsi at the University of Waterloo is available at [143]. David Silver's course on reinforcement learning is available at [641].

### Software Resources

Deep learning is supported by numerous software frameworks like *Caffe* [596], *Torch* [597], *Theano* [598], and *TensorFlow* [599]. Extensions of *Caffe* to Python and MATLAB are available. *Caffe* was developed at the University of California at Berkeley, and it is written in C++. It provides a high-level interface in which one can specify the architecture of the network, and it enables the construction of neural networks with very little code writing and relatively simple scripting. The main drawback of *Caffe* is the relatively limited documentation available. *Theano* [35] is Python-based, and it provides high-level packages like *Keras* [600] and *Lasagne* [601] as interfaces. *Theano* is based on the notion of computational graphs, and most of the capabilities provided around it use this abstraction explicitly. *TensorFlow* [599] is also strongly oriented towards computational graphs, and is the framework proposed by Google. *Torch* [597] is written in a high-level language called *Lua*, and it is relatively friendly to use. In recent years, *Torch* has gained some ground compared to other frameworks. Support for GPUs is tightly integrated in *Torch*, which makes it relatively easy to deploy *Torch*-based applications on GPUs. Many of these frameworks contain pretrained models from computer vision and text mining, which can be used to extract features. Many off-the-shelf tools for deep learning are available from the **DeepLearning4j** repository [614]. IBM has a *PowerAI* platform that offers many machine learning and deep learning frameworks on top of IBM Power Systems [622]. Notably, as of the writing of this book, this platform also has a free edition available for certain uses.

## 1.10 Exercises

1. Consider the case of the XOR function in which the two points $\{(0,0),(1,1)\}$ belong to one class, and the other two points $\{(1,0),(0,1)\}$ belong to the other class. Show how you can use the ReLU activation function to separate the two classes in a manner similar to the example in Figure 1.10.

2. Show the following properties of the sigmoid and tanh activation functions (denoted by $\Phi(\cdot)$ in each case):

(a) Sigmoid activation: $\Phi(-v) = 1 - \Phi(v)$

(b) Tanh activation: $\Phi(-v) = -\Phi(v)$

(c) Hard tanh activation: $\Phi(-v) = -\Phi(v)$

**3.** Show that the tanh function is a re-scaled sigmoid function with both horizontal and vertical stretching, as well as vertical translation:

$$\tanh(v) = 2\text{sigmoid}(2v) - 1$$

**4.** Consider a data set in which the two points $\{(-1, -1), (1, 1)\}$ belong to one class, and the other two points $\{(1, -1), (-1, 1)\}$ belong to the other class. Start with perceptron parameter values at $(0, 0)$, and work out a few point-wise gradient-descent updates with $\alpha = 1$. While performing the gradient-descent updates, cycle through the training points in any order.

(a) Does the algorithm converge in the sense that the change in objective function becomes extremely small over time?

(b) Explain why the situation in (a) occurs.

**5.** For the data set in Exercise 4, where the two features are denoted by $(x_1, x_2)$, define a new 1-dimensional representation $z$ denoted by the following:

$$z = x_1 \cdot x_2$$

Is the data set linearly separable in terms of the 1-dimensional representation corresponding to $z$? Explain the importance of nonlinear transformations in classification problems.

**6.** Implement the perceptron in a programming language of your choice.

**7.** Show that the derivative of the sigmoid activation function is at most 0.25, irrespective of the value of its argument. At what value of its argument does the sigmoid activation function take on its maximum value?

**8.** Show that the derivative of the tanh activation function is at most 1, irrespective of the value of its argument. At what value of its argument does the tanh activation take on its maximum value?

**9.** Consider a network with two inputs $x_1$ and $x_2$. It has two hidden layers, each of which contain two units. Assume that the weights in each layer are set so that top unit in each layer applies sigmoid activation to the sum of its inputs and the bottom unit in each layer applies tanh activation to the sum of its inputs. Finally, the single output node applies ReLU activation to the sum of its two inputs. Write the output of this neural network *in closed form* as a function of $x_1$ and $x_2$. This exercise should give you an idea of the complexity of functions computed by neural networks.

**10.** Compute the partial derivative of the closed form computed in the previous exercise with respect to $x_1$. Is it practical to compute derivatives for gradient descent in neural networks by using closed-form expressions (as in traditional machine learning)?

**11.** Consider a 2-dimensional data set in which all points with $x_1 > x_2$ belong to the positive class, and all points with $x_1 \leq x_2$ belong to the negative class. Therefore, the true separator of the two classes is linear hyperplane (line) defined by $x_1 - x_2 = 0$. Now create a training data set with 20 points randomly generated inside the unit square in the positive quadrant. Label each point depending on whether or not the first coordinate $x_1$ is greater than its second coordinate $x_2$. Implement the perceptron algorithm, train it on the 20 points above, and test its accuracy on 1000 randomly generated points inside the unit square. Generate the test points using the same procedure as the training points.

**12.** Suppose you know (as domain knowledge) that the real-valued output of a neural model *always* lies in the range $[a, b]$. Show how you can use an activation function in the output or hidden layer(s) to model this fact.


# Chapter 2

# The Backpropagation Algorithm

"Don't re-invent the wheel, just re-align it." – Anthony J. D'Angelo

## 2.1 Introduction

This chapter will introduce the backpropagation algorithm, which is the key to learning in multilayer neural networks. In the early years, methods for training multilayer networks were not known, primarily because of the unfamiliarity of the computer science community with ideas that were used quite frequently in control theory [54, 247]. In their influential book, Minsky and Papert [342] strongly argued against the prospects of neural networks because of the (perceived) inability to train multilayer networks. Therefore, neural networks stayed out of favor as a general area of research till the eighties. At this point, an algorithm for training neural networks was popularized[1] by Rumelhart *et al.* [424, 425] in the form of the "backpropagation" algorithm. This algorithm had its roots in control theory, and was re-invented several times [54, 247, 543, 544]. The introduction of this algorithm rekindled an interest in neural networks. However, several computational, stability, and overfitting challenges were found in the use of this algorithm. As a result, research in the field of neural networks again fell from favor.

At the turn of the century, several advances again brought popularity to neural networks. Not all of these advances were algorithm-centric. For example, increased data availability and computational power have played the primary role in this resurrection. However, some changes to the basic backpropagation algorithm and clever methods for initialization have also helped. It has also become easier in recent years to perform the intensive experimentation required for making algorithmic adjustments due to the reduced testing cycle times

[1] Although the backpropagation algorithm was popularized by the Rumelhart *et al.* papers [424, 425], it had (independently) been studied earlier in the context of control theory. Crucially, Paul Werbos's forgotten (and eventually rediscovered) thesis [543] in 1974 discussed how these backpropagation methods could be used in neural networks. Nevertheless, Rumelhart *et al.*'s papers in 1986 were significant in contributing to a better understanding and appreciation of these ideas.



C. C. Aggarwal, *Neural Networks and Deep Learning*,
https://doi.org/10.1007/978-3-031-29642-0_2

(caused by improved computational hardware). Therefore, increased data, computational power, and reduced experimentation time (for algorithmic tweaking) went hand-in-hand.

**Chapter Organization**

This chapter is organized as follows. The computational graph abstraction for neural networks is introduced in section 2.2. Section 2.3 discusses the backpropagation algorithm from the perspective of computational graphs. The generalization of the computational graph abstraction to neural networks is discussed in section 2.4. The vector-centric view of backpropagation is introduced in section 2.5. Important details and special cases are discussed in section 2.6. Feature preprocessing and initialization issues are discussed in section 2.7. The interpretability of backpropagation is presented in section 2.8. The summary is presented in section 2.9.

## 2.2 The Computational Graph Abstraction

A *computational graph* is a more general abstraction than a neural network, and it is defined as follows:

**Definition 2.2.1 (Directed Acyclic Computational Graph)** *A directed acyclic computational graph is a directed acyclic graph of nodes, where each node contains a variable. Edges* **might** *be associated with learnable parameters. A variable in a node is either fixed externally (for input nodes with no incoming edges), or it is a computed as a function of the variables in the tail ends of edges incoming into the node and the learnable parameters on the incoming edges.*

Note that the above definition of computational graphs does not make any assumptions on the type of function computed in a node, although it is common to use a combination of a linear function and an activation function in a neural network. Furthermore, the nodes of the computational graph need not be arranged in layers, as is common in the case of neural networks. The main restriction is that the graph needs to be acyclic in order for computations of variables to be performed in a clearly defined way.

The variables in some of the nodes are observable, and are referred to as *output nodes*. These output nodes are associated with loss functions that are computed using observed input-output pairs in the training data. For each observed input, the loss function quantifies how well the predicted values in output nodes match the observed values of the output (when input node variables are fixed to the observed inputs). The choice of loss function depends on the application at hand. For example, one can model least-squares regression by using as many input nodes as the number of input variables (regressors), and a single output node containing the predicted regressand. In this model, directed edges exist from each input node to the solitary output node, and the parameter on each such edge corresponds to the weight $w_i$ associated with that input variable $x_i$ (cf. Figure 2.1). The output node, which is the only computational node, computes the following function of the variables $x_1 \ldots x_d$ in the $d$ input nodes:

$$\hat{o} = f(x_1, x_2, \ldots, x_d) = \sum_{i=1}^{d} w_i x_i$$

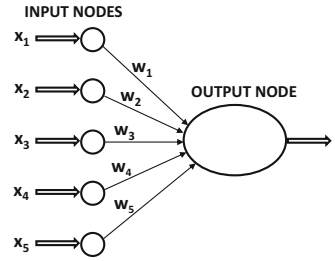



Figure 2.1: A single-layer computational graph that can perform linear regression

The computational graph in Figure 2.1 is a rather rudimentary one in which the computed function is not particularly complex. However, the closed-form representation of the function computed by a neural network becomes increasingly unwieldy and complex as one moves from a single-layer network to multiple layers.

### 2.2.1 Computational Graphs Create Complex Functions

Any computational graph (such as a neural network) evaluates compositions of functions computed at individual nodes. A path of length 2 in a computational graph in which the function $f(\cdot)$ follows $g(\cdot)$ can be considered a composition function $f(g(\cdot))$. It is this type of *recursive nesting* of functions that makes it hard to express the output (or loss function) of a computational graph in closed form as a *direct* function of its *inputs* and *edge-specific parameters*. The inability to easily express the optimization function in closed form in terms of the edge-specific parameters (as is common in all machine learning problems) causes difficulties in computing the derivatives needed for gradient descent.

Consider a variable $x$ at a node in a computational graph with only three nodes containing a path of length 2. The first node applies the function $g(x)$, whereas the second node applies the function $f(\cdot)$ to the result. Such a graph computes the function $f(g(x))$, and it is shown in Figure 2.2. The example shown in Figure 2.2 uses the case when $f(x) = \cos(x)$ and $g(x) = x^2$. Therefore, the overall function is $\cos(x^2)$. Now, consider another setting in which both $f(x)$ and $g(x)$ are set to the same function, which is the sigmoid function:

$$f(x) = g(x) = \frac{1}{1 + \exp(-x)}$$

Then, the global function evaluated by the computational graph is as follows:

$$f(g(x)) = \frac{1}{1 + \exp\left[-\frac{1}{1+\exp(-x)}\right]} \tag{2.1}$$

This simple graph already computes a rather awkward composition function. Trying to find the derivative of this composition function algebraically becomes increasingly tedious as we increase the complexity of the computational graph.

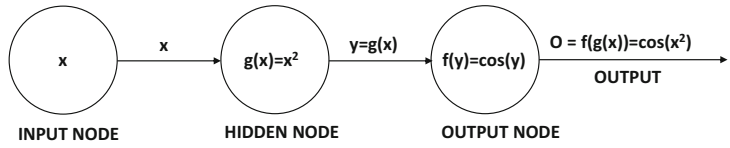



Figure 2.2: A simple computational graph with an input node and two computational nodes

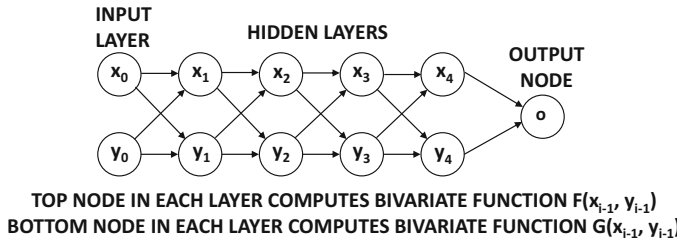



Figure 2.3: The awkwardness of recursive nesting caused by a computational graph

Consider a case in which the functions $g_1(\cdot)$, $g_2(\cdot) \ldots g_k(\cdot)$ are the functions computed in layer $m$, and they feed into a particular layer-$(m+1)$ node that computes the multivariate function $f(\cdot)$ that uses the values computed in the previous layer as arguments. Therefore, the layer-$(m+1)$ function computes $f(g_1(\cdot), \ldots g_k(\cdot))$. This type of multivariate composition function already appears rather awkward. As we increase the number of layers, a function that is computed several edges downstream will have as many layers of nesting as the length of the path from the source to the final output. For example, if we have a computational graph which has 10 layers, and 2 nodes per layer, the overall composition function would have $2^{10}$ nested "terms" if one were to try to write it on a piece of paper (in closed form). In fact, such a "closed-form" function will not even fit on any reasonably sized sheet of paper that we are accustomed to writing on!

In order to understand this point, consider the function in Figure 2.3. In this case, we have two nodes in each layer other than the output layer. The output layer simply sums its inputs. Each hidden layer contains two nodes. The variables in the $i$th layer are denoted by $x_i$ and $y_i$, respectively. The input nodes (variables) use subscript 0, and therefore they are denoted by $x_0$ and $y_0$ in Figure 2.3. The two computed functions in the $i$th layer are $F(x_{i-1}, y_{i-1})$ and $G(x_{i-1}, y_{i-1})$, respectively.

In the following, we will write the expression for the variable in each node in order to show the increasing complexity with increasing number of layers:

$$x_1 = F(x_0, y_0)$$
$$y_1 = G(x_0, y_0)$$
$$x_2 = F(x_1, y_1) = F(F(x_0, y_0), G(x_0, y_0))$$
$$y_2 = G(x_1, y_1) = G(F(x_0, y_0), G(x_0, y_0))$$

We can already see that the expressions have already started looking unwieldy. On computing the values in the next layer, this becomes even more obvious:

$$x_3 = F(x_2, y_2) = F(F(F(x_0, y_0), G(x_0, y_0)), G(F(x_0, y_0), G(x_0, y_0)))$$
$$y_3 = G(x_2, y_2) = G(F(F(x_0, y_0), G(x_0, y_0)), G(F(x_0, y_0), G(x_0, y_0)))$$

An immediate observation is that the complexity and length of the closed-form function increases *exponentially* with the path lengths in the computational graphs. This type of complexity further increases in the case when optimization parameters are associated with the edges, and one tries to express the outputs/losses in terms of the inputs and the parameters on the edges. This is obviously a problem, if we try to use the standard approach of first expressing each variable in closed form in terms of the optimization parameters on the edges in order to differentiate the closed-form expression. Here, a key point is that the derivatives of the output with respect to various variables in the computational graph are related to one another with the use of the chain rule of differential calculus. Therefore, the chain rule of differential calculus needs to be applied repeatedly to update derivatives of the output with respect to the variables in the computational graph. This approach is referred to as the backpropagation algorithm, because the derivatives of the output with respect to the variables close to the output are simpler to compute (and are therefore computed first while propagating them backwards towards the inputs). Furthermore, derivatives are computed *numerically* for the specific values of each input-output pair in a training point, rather than computing them *algebraically* than substituting the values of the features.

## 2.3 Backpropagation in Computational Graphs

To learn the weights of a computational graph, an input-output pair is selected from the training data and the variables in the input nodes are instantiated with the corresponding input variables in the training data. Then, the computations are performed in the *forward direction* on the directed acyclic graph to determine the *predicted values* of the outputs. If these *predicted values* are different from the *observed values* in the training data, a loss function is used to quantify the error. The gradients of the loss function are computed with respect to the weights in the computational graph. These gradients are used to update the weights. Therefore, the overall approach for training a computational graph is as follows:

1. Use the attribute values from the input portion of a training data point to fix the values in the input nodes. Repeatedly select a node for which the values in all incoming nodes have already been computed and apply the node-specific function to also compute its variable. Such a node can be found in a directed acyclic graph by processing the nodes in order of increasing distance from input nodes. Repeat the process until the values in all nodes (including the output nodes) have been computed. If the values on the output nodes do not match the observed values of the output in the training point, compute the loss value. This phase is referred to as the *forward phase*.

2. Compute the gradient of the loss with respect to the weights on the edges. This phase is referred to as the *backwards phase*. The rationale for calling it a "backwards phase" is that derivatives of the loss with respect to weights near the output (where the loss function is computed) are easier to compute and are computed first. The derivatives become increasingly complex as we move towards edge weights away from the output (in the backwards direction) and the chain rule is used repeatedly to compute them.

3. Update the weights in the negative direction of the gradient.

One cycles through the training points repeatedly until convergence is reached. A single cycle through all the training points is referred to as an *epoch*.

Although we want to compute the gradient of the loss function with respect to the *weights* in a computational graph. it turns out that *the derivatives of the node variables*

*with respect to one another can be easily used to compute the derivative of the loss function with respect to the weights on the edges.* Therefore, in this discussion, we will focus on the computation of the derivatives of the variables with respect to one another. Later, we will show how these derivatives can be converted into gradients of loss functions with respect to weights.

### 2.3.1 Computing Node-to-Node Derivatives with the Chain Rule

As discussed in an earlier section, one can express the function in a computational graph in terms of the nodes in early layers using an awkward closed-form expression that uses nested compositions of functions. If one were to indeed compute the derivative of this closed-form expression, it would require the use of the chain rule of differential calculus in order to deal with the repeated composition of functions. However, a blind application of the chain rule is rather wasteful in this case because many of the expressions in different portions of the inner nesting are identical, and one would be repeatedly computing the same derivative. The key idea in *automatic differentiation over computational graphs* is to recognize the fact that structure of the computational graph already provides all the information about which terms are repeated. We can avoid repeating the differentiation of these terms by using the structure of the computational graph itself to store intermediate results (by working backwards starting from output nodes to compute derivatives)! This is a well-known idea from dynamic programming, which has been used frequently in control theory [54, 247]. In neural networks, this same algorithm is referred to as *backpropagation*. It is noteworthy that the applications of this idea in control theory were well-known to the traditional optimization community in 1960 [54, 247], although they remained unknown to researchers in the field of artificial intelligence for a while (who coined the term "backpropagation" in the 1980s to independently propose and describe this idea in the context of neural networks).

The simplest version of the chain rule is defined for a univariate composition of functions:

$$\frac{\partial f(g(x))}{\partial x} = \frac{\partial f(g(x))}{\partial g(x)} \cdot \frac{\partial g(x)}{\partial x} \tag{2.2}$$

This variant is referred to as the *univariate chain rule*. Note that each term on the right-hand side is a *local gradient* because it computes the derivative of a *local* function with respect to its immediate argument rather than a recursively derived argument. The basic idea is that a composition of functions is applied on the input $x$ to yield the final output, and the gradient of the final output is given by the product of the local gradients along that path. Each local gradient only needs to worry about its specific input and output, which simplifies the computation. An example is shown in Figure 2.2 in which the function $f(y)$ is $\cos(y)$ and $g(x) = x^2$. Therefore, the composition function is $\cos(x^2)$. On using the univariate chain rule, we obtain the following:

$$\frac{\partial f(g(x))}{\partial x} = \underbrace{\frac{\partial f(g(x))}{\partial g(x)}}_{-\sin(g(x))} \cdot \underbrace{\frac{\partial g(x)}{\partial x}}_{2x} = -2x \cdot \sin(x^2)$$

The example of Figure 2.2 is a rather simple case in which the computational graph is a single path. In general, a computational graph with good expressive power will not be a single path. In neural networks, a single node feeds its output to multiple nodes. This case is also a challenging one from the perspective of computing derivatives, because the closed form of the global function with respect to variables in earlier nodes becomes rather

large and unwieldy. Consider the simple case in which we have a single input $x$, and we have $k$ independent computational nodes that compute the functions $g_1(x), g_2(x), \ldots g_k(x)$. If these nodes are connected to a single output node computing the function $f()$ with $k$ arguments, then the resulting function that is computed is $f(g_1(x), \ldots g_k(x))$. In such cases, the *multivariate chain rule* needs to be used. The multivariate chain rule is defined as follows:

$$\frac{\partial f(g_1(x), \ldots g_k(x))}{\partial x} = \sum_{i=1}^{k} \frac{\partial f(g_1(x), \ldots g_k(x))}{\partial g_i(x)} \cdot \frac{\partial g_i(x)}{\partial x} \tag{2.3}$$

It is easy to see that the multivariate chain rule of Equation 2.3 is a simple generalization of that in Equation 2.2.

One can also view the multivariate chain rule in a path-centric fashion rather than a node-centric fashion. *For any pair of source-sink nodes, the multivariate chain rule can be recursively applied to derive the fact that the derivative of the variable in the sink node with respect to the variable in the source node is simply the sum of the expressions arising from the univariate chain rule being applied to all paths existing between that pair of nodes.* While this view leads to a direct expression for the derivative between any pair of nodes, it leads to an excessive amount of repeated computation. This is because the number of paths between a pair of nodes is exponentially related to the path length. In order to show the repetitive nature of the operations, we work with a very simple closed-form function with a single input $x$:

$$o = \sin(x^2) + \cos(x^2) \tag{2.4}$$

The resulting computation graph is shown in Figure 2.4. In this case, the multivariate chain rule is applied to compute the derivative of the output $o$ with respect to $x$. This is achieved by applying the univariate chain rule to each of the two paths from $x$ to $o$ and summing the expressions:

$$\begin{aligned}
\frac{\partial o}{\partial x} &= \underbrace{\frac{\partial K(p,q)}{\partial p}}_{1} \cdot \underbrace{g'(y)}_{-\sin(y)} \cdot \underbrace{f'(x)}_{2x} + \underbrace{\frac{\partial K(p,q)}{\partial q}}_{1} \cdot \underbrace{h'(z)}_{\cos(z)} \cdot \underbrace{f'(x)}_{2x} \\
&= -2x \cdot \sin(y) + 2x \cdot \cos(z) \\
&= -2x \cdot \sin(x^2) + 2x \cdot \cos(x^2)
\end{aligned}$$

It is easy to see that the application of the chain rule on the computational graph correctly evaluates the derivative, which is $-2x \cdot \sin(x^2) + 2x \cdot \cos(x^2)$. In this simple example, there

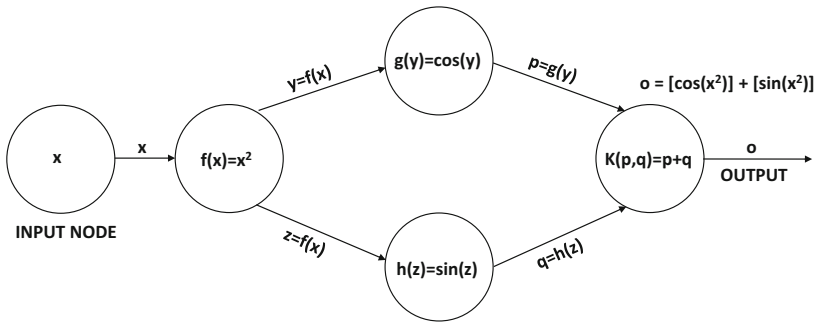



Figure 2.4: A simple computational function that illustrates the chain rule.

are two paths, both of which compute the function $f(x) = x^2$. As a result, the same function is differentiated *twice*, once for each path. Furthermore, the global gradient of $o$ with respect to $x$ is obtained by computing the product of the local gradients along each of the two paths and then adding them. This type of repetition can have severe effects for large multilayer networks containing many shared nodes, where the same function might be differentiated hundreds of thousands of times as a portion of the nested recursion. It is this *repeated and wasteful* approach to the computation of the derivative, that it is impractical to express the global function of a computational graph in closed form and explicitly differentiate it.

One can summarize the path-centric view of the multivariate chain rule as follows:

**Lemma 2.3.1 (Pathwise Aggregation Lemma)** *Consider a directed acyclic computational graph in which the ith node contains variable $y(i)$. The local derivative $z(i, j)$ of the directed edge $(i, j)$ in the graph is defined as $z(i, j) = \frac{\partial y(j)}{\partial y(i)}$. Let a non-null set of paths $\mathcal{P}$ exist from a node $s$ in the graph to node $t$. Then, the value of $\frac{\partial y(t)}{\partial y(s)}$ is given by computing the product of the local gradients along each path in $\mathcal{P}$, and summing these products over all paths in $\mathcal{P}$.*

$$\frac{\partial y(t)}{\partial y(s)} = \sum_{P \in \mathcal{P}} \prod_{(i,j) \in P} z(i, j) \tag{2.5}$$

This lemma can be easily shown by applying the multivariate chain rule (Equation 2.3) recursively over the computational graph. Although the use of the path-wise aggregation lemma is a wasteful approach for computing the derivative of $y(t)$ with respect to $y(s)$, it helps us develop an exponential-time algorithm for computing the derivative, which is relatively simple to understand.

### An Exponential-Time Algorithm

The pathwise aggregation lemma provides a natural exponential-time algorithm, which is roughly[2] similar to the steps one would go through by expressing the computational function in closed form with respect to a particular variable and then differentiating it. Specifically, the pathwise aggregation lemma leads to the following exponential-time algorithm to compute the derivative of the output $o$ with respect to a variable $x$ in the graph:

1. Use computational graph to compute the value $y(i)$ of each node $i$ in a forward phase.
2. Compute the local partial derivatives $z(i, j) = \frac{\partial y(j)}{\partial y(i)}$ on each edge in the computational graph.
3. Let $\mathcal{P}$ be the set of all paths from an input node with value $x$ to the output $o$. For each path $P \in \mathcal{P}$ compute the product of each local derivative $z(i, j)$ on that path.
4. Add up these values over all paths in $\mathcal{P}$.

In general, a computational graph will have an exponentially increasing number of paths with depth and one must add the product of the local derivatives over all paths. An example is shown in Figure 2.5, in which we have five layers, each of which has only two units.

[2]It is not exactly similar in cases where the composition of functions in the computational graph leads to a simplified closed form. However, if one cannot simplify or "collapse" the composition of functions in the computational graph to a simplified form, the number of steps would be similar in the two cases.

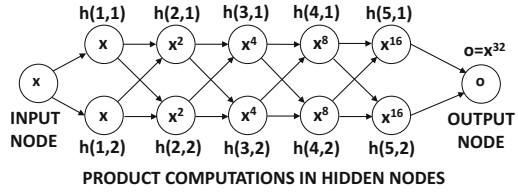



Figure 2.5: The number of paths in a computational graph increases exponentially with depth. In this case, the chain rule will aggregate the product of local derivatives along $2^5 = 32$ paths.

Therefore, the number of paths between the input and output is $2^5 = 32$. The $j$th hidden unit of the $i$th layer is denoted by $h(i, j)$. Each hidden unit is defined as the product of its inputs:

$$h(i,j) = h(i-1,1) \cdot h(i-1,2) \quad \forall j \in \{1,2\} \tag{2.6}$$

In this case, the output is $x^{32}$, which is expressible in closed form, and can be differentiated easily with respect to $x$. In other words, we do not really need computational graphs in order to perform the differentiation. However, we will use the exponential-time algorithm to elucidate its workings. The derivatives of each $h(i, j)$ with respect to its two inputs are the values of the complementary inputs, because the partial derivative of the multiplication of two variables is the complementary variable:

$$\frac{\partial h(i,j)}{\partial h(i-1,1)} = h(i-1,2), \quad \frac{\partial h(i,j)}{\partial h(i-1,2)} = h(i-1,1)$$

The pathwise aggregation lemma implies that the value of $\frac{\partial o}{\partial x}$ is the product of the local derivatives (which are the complementary input values in this particular case) along all 32 paths from the input to the output:

$$\begin{aligned}\frac{\partial o}{\partial x} &= \sum_{j_1,j_2,j_3,j_4,j_5 \in \{1,2\}^5} \underbrace{\prod h(1,j_1)}_{x} \underbrace{h(2,j_2)}_{x^2} \underbrace{h(3,j_3)}_{x^4} \underbrace{h(4,j_4)}_{x^8} \underbrace{h(5,j_5)}_{x^{16}} \\ &= \sum_{\text{All 32 paths}} x^{31} = 32x^{31}\end{aligned}$$

This result is, of course, consistent with what one would obtain on differentiating $x^{32}$ directly with respect to $x$. However, an important observation is that it requires $2^5$ aggregations to compute the derivative in this way for a relatively simple graph. More importantly, *we repeatedly differentiate the same function computed in a node* for aggregation. For example, the differentiation of the variable $h(3, 1)$ is performed 16 times because it appears in 16 paths from $x$ to $o$.

Obviously, this is an inefficient approach to compute gradients. For a network with 100 nodes in each layer and three layers, we will have a million paths. *Nevertheless, this is exactly what we do in traditional machine learning when our prediction function is a complex composition function.* Manually working out the details of a complex composition function is tedious and impractical beyond a certain level of complexity. It is here that one can apply dynamic programming (which is guided by the structure of the computational graph) in order to store important intermediate results. By using such an approach, one can minimize repeated computations, and achieve polynomial complexity.

### 2.3.2 Dynamic Programming for Computing Node-to-Node Derivatives

In graph theory, computing all types of path-aggregative values over directed acyclic graphs is done using dynamic programming. Consider a directed acyclic graph in which the value $z(i, j)$ (interpreted as local partial derivative of variable in node $j$ with respect to variable in node $i$) is associated with edge $(i, j)$. In other words, if $y(p)$ is the variable in the node $p$, we have the following:

$$z(i, j) = \frac{\partial y(j)}{\partial y(i)} \tag{2.7}$$

An example of such a computational graph is shown in Figure 2.6. In this case, we have associated the edge $(2, 4)$ with the corresponding partial derivative. We would like to compute the product of $z(i, j)$ over each path $P \in \mathcal{P}$ from source node $s$ to output node $t$ and then add them in order to obtain the partial derivative $S(s, t) = \frac{\partial y(t)}{\partial y(s)}$:

$$S(s, t) = \sum_{P \in \mathcal{P}} \prod_{(i,j) \in P} z(i, j) \tag{2.8}$$

Let $A(i)$ be the set of nodes at the end points of outgoing edges from node $i$. We can compute the aggregated value $S(i, t)$ for each intermediate node $i$ (between source node $s$ and output node $t$) using the following well-known dynamic programming update:

$$S(i, t) \Leftarrow \sum_{j \in A(i)} S(j, t) z(i, j) \tag{2.9}$$

This computation can be performed backwards starting from the nodes directly incident on $o$, since $S(t, t) = \frac{\partial y(t)}{\partial y(t)}$ is already known to be 1. This is because the partial derivative of a variable with respect to itself is always 1. Therefore one can describe the pseudocode of this algorithm as follows:

Initialize $S(t, t) = 1$;
**repeat**
  Select an unprocessed node $i$ such that the values of $S(j, t)$ all of its outgoing nodes $j \in A(i)$ are available;
  Update $S(i, t) \Leftarrow \sum_{j \in A(i)} S(j, t) z(i, j)$;
**until** all nodes have been selected;

Note that the above algorithm always selects a node $i$ for which the value of $S(j, t)$ is available for all nodes $j \in A(i)$. Such a node is always available in directed acyclic graphs, and the node selection order will always be in the backwards direction starting from node

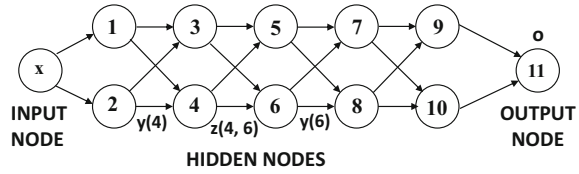



Figure 2.6: Example of computational graph with edges corresponding to local partial derivatives

$t$. Therefore, the above algorithm will work only when the computational graph does not have cycles.

The algorithm discussed above is used by the network optimization community for computing all types of path-centric functions between *source-sink* node pairs $(s, t)$ on directed acyclic graphs, which would otherwise require exponential time. For example, one can even use a variation of the above algorithm to find the longest path in a directed acyclic graph [8]. When applied on computational graphs corresponding to neural networks, the above algorithm is referred to as the backpropagation algorithm.

Interestingly, *the aforementioned dynamic programming update is exactly the multivariate chain rule of Equation 2.3, which is repeated in the backwards direction starting at the output node where the local gradient is known.* This is because we derived the path-aggregative form of the loss gradient (Lemma 2.3.1) using this chain rule in the first place. The main difference is that we apply the rule in a particular order in order to minimize computations. Since this point is important, we summarize it below:

> Using dynamic programming to efficiently aggregate the product of local gradients along the exponentially many paths in a computational graph results in a dynamic programming update that is identical to the multivariate chain rule of differential calculus. The main point of dynamic programming is to apply this rule in a particular order, so that the derivative computations at different nodes are not repeated.

This approach is the backbone of the backpropagation algorithm used in neural networks. The above gradients are used to update the weights of the neural network. In the case where we have multiple output nodes $t_1, \ldots t_p$, one can initialize each $S(t_r, t_r)$ to 1, and then apply the same approach for each $t_r$.

### Example of Computing Node-to-Node Derivatives

In order to show how the backpropagation approach works, we will provide an example of computation of node-to-node derivatives in a graph containing 10 nodes (see Figure 2.7). A variety of functions are computed in various nodes, such as the sum function (denoted by '+'), the product function (denoted by '*'), and the trigonometric sine/cosine functions. The variables in the 10 nodes are denoted by $y(1) \ldots y(10)$, where the variable $y(i)$ belongs to the $i$th node in the figure. Two of the edges incoming into node 6 also have the weights $w_2$ and $w_3$ associated with them. Other edges do not have weights associated with them.

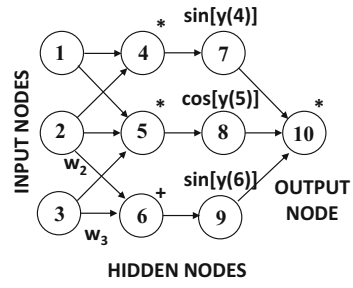



Figure 2.7: Example of node-to-node derivative computation

The functions computed in the various layers are as follows:

Layer 1: $y(4) = y(1) \cdot y(2)$, $y(5) = y(1) \cdot y(2) \cdot y(3)$, $y(6) = w_2 \cdot y(2) + w_3 \cdot y(3)$
Layer 2: $y(7) = \sin(y(4))$, $y(8) = \cos(y(5))$, $y(9) = \sin(y(6))$
Layer 3: $y(10) = y(7) \cdot y(8) \cdot y(9)$

We would like to compute the derivative of $y(10)$ with respect to each of the inputs $y(1)$, $y(2)$, and $y(3)$. One possibility is to simply express the $y(10)$ in closed form in terms of the inputs $y(1)$, $y(2)$ and $y(3)$, and then compute the derivative. By recursively using the above relationships, it is easy to show that $y(10)$ can be expressed in terms of $y(1)$, $y(2)$, and $y(3)$ as follows:

$$y(10) = \sin(y(1) \cdot y(2)) \cdot \cos(y(1) \cdot y(2) \cdot y(3)) \cdot \sin(w_2 \cdot y(2) + w_3 \cdot y(3))$$

As discussed earlier computing the closed-form derivative is not practical for larger networks. Furthermore, since one needs to compute the derivative of the output with respect to each and every node in the network, such an approach would also required closed-form expressions in terms of upstream nodes like $y(4)$, $y(5)$ and $y(6)$. All this tends to increase the amount of repeated computation. Luckily, backpropagation frees us from this repeated computation, since the derivative in $y(10)$ with respect to each and every node is computed by the backwards phase.

The algorithm starts, by initializing the derivative of the output $y(10)$ with respect to itself, which is 1:

$$S(10, 10) = \frac{\partial y(10)}{\partial y(10)} = 1$$

Subsequently, the derivatives of $y(10)$ with respect to all the variables on its incoming nodes are computed. Since $y(10)$ is expressed in terms of the variables $y(7)$, $y(8)$, and $y(9)$ incoming into it, this is easy to do, and the results are denoted by $z(7, 10)$, $z(8, 10)$, and $z(9, 10)$ (which is consistent with the notations used earlier in this chapter). Therefore, we have the following:

$$z(7, 10) = \frac{\partial y(10)}{\partial y(7)} = y(8) \cdot y(9)$$
$$z(8, 10) = \frac{\partial y(10)}{\partial y(8)} = y(7) \cdot y(9)$$
$$z(9, 10) = \frac{\partial y(10)}{\partial y(9)} = y(7) \cdot y(8)$$

Subsequently, we can use these values in order to compute $S(7, 10)$, $S(8, 10)$, and $S(9, 10)$ using the recursive backpropagation update:

$$S(7, 10) = \frac{\partial y(10)}{\partial y(7)} = S(10, 10) \cdot z(7, 10) = y(8) \cdot y(9)$$
$$S(8, 10) = \frac{\partial y(10)}{\partial y(8)} = S(10, 10) \cdot z(8, 10) = y(7) \cdot y(9)$$
$$S(9, 10) = \frac{\partial y(10)}{\partial y(9)} = S(10, 10) \cdot z(9, 10) = y(7) \cdot y(8)$$

Next, we compute the derivatives $z(4,7)$, $z(5,8)$, and $z(6,9)$ associated with all the edges incoming into nodes 7, 8, and 9:

$$z(4,7) = \frac{\partial y(7)}{\partial y(4)} = \cos[y(4)]$$
$$z(5,8) = \frac{\partial y(8)}{\partial y(5)} = -\sin[y(5)]$$
$$z(6,9) = \frac{\partial y(9)}{\partial y(6)} = \cos[y(6)]$$

These values can be used to compute $S(4,10)$, $S(5,10)$, and $S(6,10)$:

$$S(4,10) = \frac{\partial y(10)}{\partial y(4)} = S(7,10) \cdot z(4,7) = y(8) \cdot y(9) \cdot \cos[y(4)]$$
$$S(5,10) = \frac{\partial y(10)}{\partial y(5)} = S(8,10) \cdot z(5,8) = -y(7) \cdot y(9) \cdot \sin[y(5)]$$
$$S(6,10) = \frac{\partial y(10)}{\partial y(6)} = S(9,10) \cdot z(6,9) = y(7) \cdot y(8) \cdot \cos[y(6)]$$

In order to compute the derivatives with respect to the input values, one now needs to compute the values of $z(1,3)$, $z(1,4)$, $z(2,4)$, $z(2,5)$, $z(2,6)$, $z(3,5)$, and $z(3,6)$:

$$z(1,4) = \frac{\partial y(4)}{\partial y(1)} = y(2)$$
$$z(2,4) = \frac{\partial y(4)}{\partial y(2)} = y(1)$$
$$z(1,5) = \frac{\partial y(5)}{\partial y(1)} = y(2) \cdot y(3)$$
$$z(2,5) = \frac{\partial y(5)}{\partial y(2)} = y(1) \cdot y(3)$$
$$z(3,5) = \frac{\partial y(5)}{\partial y(3)} = y(1) \cdot y(2)$$
$$z(2,6) = \frac{\partial y(6)}{\partial y(2)} = w_2$$
$$z(3,6) = \frac{\partial y(6)}{\partial y(3)} = w_3$$

These partial derivatives can be backpropagated to compute $S(1,10)$, $S(2,10)$, and $S(3,10)$:

$$\begin{aligned} S(1,10) &= \frac{\partial y(10)}{\partial y(1)} = S(4,10) \cdot z(1,4) + S(5,10) \cdot z(1,5) \\ &= y(8) \cdot y(9) \cdot \cos[y(4)] \cdot y(2) - y(7) \cdot y(9) \cdot \sin[y(5)] \cdot y(2) \cdot y(3) \\ S(2,10) &= \frac{\partial y(10)}{\partial y(2)} = S(4,10) \cdot z(2,4) + S(5,10) \cdot z(2,5) + S(6,10) \cdot z(2,6) \\ &= y(8) \cdot y(9) \cdot \cos[y(4)] \cdot y(1) - y(7) \cdot y(9) \cdot \sin[y(5)] \cdot y(1) \cdot y(3) + \\ &\quad + y(7) \cdot y(8) \cdot \cos[y(6)] \cdot w_2 \end{aligned}$$

$$S(3,10) = \frac{\partial y(10)}{\partial y(3)} = S(5,10) \cdot z(3,5) + S(6,10) \cdot z(3,6)$$
$$= -y(7) \cdot y(9) \cdot \sin[y(5)] \cdot y(1) \cdot y(2) + y(7) \cdot y(8) \cdot \cos[y(6)] \cdot w_3$$

Note that the use of a backward phase has the advantage of computing the derivative of $y(10)$ (output node variable) with respect to all the hidden and input node variables. These different derivatives have many sub-expressions in common, although the derivative computation of these sub-expressions is not repeated. This is the advantage of using the backwards phase for derivative computation as opposed to the use of closed-form expressions.

Because of the tedious nature of the closed-form expressions for outputs, the algebraic expressions for derivatives are also very long and awkward (no matter how we compute them). One can see that this is true even for the simple computational graph of this section (containing just ten nodes). For example, if one examines the derivative of $y(10)$ with respect to each of nodes $y(1)$, $y(2)$ and $y(3)$, the algebraic expression wraps into multiple lines. Furthermore, one cannot avoid the presence of repeated subexpressions within the algebraic derivative. This is counter-productive because our original goal of the backwards algorithm was to avoid the repeated computation endemic to the use of traditional derivative evaluation with closed-form expressions. Therefore, one does not *algebraically* compute these types of expressions in real-world networks. One would first *numerically* compute all the node variables for a *specific* input. Subsequently, one would *numerically* carry the derivatives backward, so that one does not have to carry the large algebraic expressions (with many repeated sub-expressions) in the backwards direction. The advantage of carrying numerical expressions is that multiple terms get consolidated into a single numerical value, which is specific to a particular input. Although one must repeat the backwards computation algorithm for each training point, it is still a better choice than computing the (massive) symbolic derivative in one shot and substituting the values in different training points. This is the reason that such an approach is referred to as *numerical differentiation* rather than *symbolic differentiation*. In much of machine learning, one first computes the algebraic derivative before substituting numerical values of the variables in the expression (for the derivative) to perform gradient-descent updates. This is different from the case of computational graphs, where the backwards algorithm is *numerically* applied to each training point.

### 2.3.3 Converting Node-to-Node Derivatives into Loss-to-Weight Derivatives

Most computational graphs define loss functions with respect to output node variables. One needs to compute the derivatives with respect to weights on *edges* rather than the node variables (in order to update the weights). In general, the node-to-node derivatives can be converted into loss-to-weight derivatives with a few additional applications of the univariate and multivariate chain rule.

Consider the case in which we have computed the node-to-node derivative of output variables in nodes indexed by $t_1, t_2, \ldots t_p$ with respect to the variable in node $i$ using the dynamic programming approach in the previous section. Therefore, the computational graph has $p$ output nodes in which the corresponding variable values are $y(t_1) \ldots y(t_p)$ (since the indices of the output nodes are $t_1 \ldots t_p$). The loss function is denoted by $L(y(t_1), \ldots y(t_p))$. We would like to compute the derivative of this loss function with respect to *the weights in the incoming edges of* $i$. For the purpose of this discussion, let $w_{ji}$ be the weight of an edge from node index $j$ to node index $i$. Therefore, we want to compute the derivative of the loss

function with respect to $w_{ji}$. In the following, we will abbreviate $L(y_{t_1}, \ldots y_{t_p})$ with $L$ for compactness of notation:

$$\frac{\partial L}{\partial w_{ji}} = \left[\frac{\partial L}{\partial y(i)}\right] \frac{\partial y(i)}{\partial w_{ji}} \qquad \text{[Univariate chain rule]}$$

$$= \left[\sum_{k=1}^{p} \frac{\partial L}{\partial y(t_k)} \frac{\partial y(t_k)}{\partial y(i)}\right] \frac{\partial y(i)}{\partial w_{ji}} \qquad \text{[Multivariate chain rule]}$$

Here, it is noteworthy that the loss function is typically a closed-form function of the variables in the node indices $t_1 \ldots t_p$, which (for example) could be a least-squares function. Therefore, each derivative of the loss $L$ with respect to $y(t_i)$ is easy to compute. Furthermore, the value of each $\frac{\partial y(t_k)}{\partial y(i)}$ for $k \in \{1 \ldots p\}$ can be computed using the dynamic programming algorithm of the previous section. The value of $\frac{\partial y_i}{\partial w_{ji}}$ is a derivative of the **local** function at each node, which usually has a simple form. Therefore, the loss-to-weight derivatives can be computed relatively easily, once the node-to-node derivatives have been computed using dynamic programming.

Although one can apply the pseudocode of page 38 to compute $\frac{\partial y(t_k)}{\partial y(i)}$ for each $k \in \{1 \ldots p\}$, it is more efficient to collapse all these computations into a single backwards algorithm. In practice, one initializes the derivatives at the output nodes to the loss derivatives $\frac{\partial L}{\partial y(t_k)}$ for each $k \in \{1 \ldots p\}$ rather than the value of 1 (as shown in the pseudocode of page 38). Subsequently, the entire loss derivative $\Delta(i) = \frac{\partial L}{\partial y(i)}$ is propagated backwards. Therefore, the modified algorithm for computing the loss derivative with respect to the *node variables* as well as the *edge variables* is as follows:

Initialize $\Delta(t_r) = \frac{\partial L}{\partial y(t_k)}$ for each $k \in \{1 \ldots p\}$;
**repeat**
  Select an unprocessed node $i$ such that the values of $\Delta(j)$ all of its outgoing nodes $j \in A(i)$ are available;
  Update $\Delta(i) \Leftarrow \sum_{j \in A(i)} \Delta(j) z(i, j)$;
**until** all nodes have been selected;
**for each edge** $(j, i)$ with weight $w_{ji}$ **do** compute $\frac{\partial L}{\partial w_{ji}} = \Delta(i) \frac{\partial y(i)}{\partial w_{ji}}$

In the above algorithm, $y(i)$ denotes the variable at node $i$. The key difference from the algorithm on page 38 is in the nature of the initialization and the addition of a final step computing the edge-wise derivatives. However, the core algorithm for computing the node-to-node derivatives remains an integral part of this algorithm. In fact, one can convert all the weights on the edges into additional "input" nodes containing weight parameters, and also add computational nodes that multiply the weights with the corresponding variables at the tail nodes of the edges. Furthermore, a computational node can be added that computes the loss from the output node(s). For example, the architecture of Figure 2.7 can be converted to that in Figure 2.8. Performing only node-to-node derivative computation in Figure 2.8 from the loss node to the weight nodes is equivalent to loss-to-weight derivative computation. In other words, *loss-to-weight derivative computation in a weighted graph is equivalent to node-to-node derivative computation in a modified computational graph.* The derivative of the loss with respect to each weight can be denoted by the vector $\frac{\partial L}{\partial \overline{W}}$ (in matrix calculus[3]

[3]In matrix calculus, the gradient of a scalar with respect to a vector is a vector of the same length in which the $i$th component is the partial derivative of the scalar with respect to the $i$th component of the vector. With appropriate choice of convention, the gradient of scalar with respect to a column vector maps to a column vector.

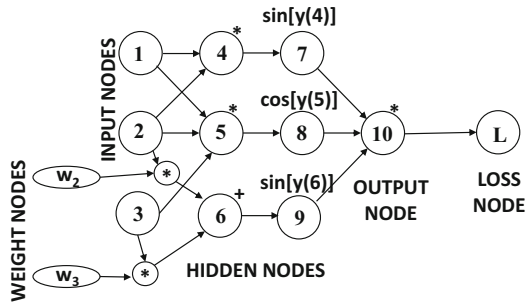



Figure 2.8: Converting loss-to-weight derivatives into node-to-node derivative computation based on Figure 2.7. Note the extra weight nodes and an extra loss node where loss is computed.

notation), where $\overline{W}$ denotes the weight vector. Subsequently, the standard gradient descent update can be performed:

$$\overline{W} \Leftarrow \overline{W} - \alpha \frac{\partial L}{\partial \overline{W}} \tag{2.10}$$

Here, $\alpha$ is the *learning rate* that controls the size of the steps. This type of update is performed to convergence by repeating the process with different inputs in order to learn the weights of the computational graph.

**Example of Computing Loss-to-Weight Derivatives**

Consider the case of Figure 2.7 in which the loss function is defined by $L = \log[y(10)^2]$, and we wish to compute the derivative of the loss with respect to the weights $w_2$ and $w_3$. In such a case, the derivative of the loss with respect to the weights is given by the following:

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial y(10)} \frac{\partial y(10)}{\partial y(6)} \frac{\partial y(6)}{\partial w_2} = \left[\frac{2}{y(10)}\right] [y(7) \cdot y(8) \cdot \cos[y(6)]]\, y(2)$$

$$\frac{\partial L}{\partial w_3} = \frac{\partial L}{\partial y(10)} \frac{\partial y(10)}{\partial y(6)} \frac{\partial y(6)}{\partial w_3} = \left[\frac{2}{y(10)}\right] [y(7) \cdot y(8) \cdot \cos[y(6)]]\, y(3)$$

Note that the quantity $\frac{\partial y(10)}{\partial y(6)}$ has been obtained using the example in the previous section on node-to-node derivatives. In practice, these quantities are not computed algebraically. This is because the aforementioned algebraic expressions can be extremely awkward for large networks. Rather, for each numerical input set $\{y(1), y(2), y(3)\}$, one computes the different values of $y(i)$ in a forward phase. Subsequently, the derivatives of the loss with respect to each node variable (and incoming weights) are computed in a backwards phase. Again, these values are computed numerically for a specific input set $\{y(1), y(2), y(3)\}$. The numerical gradients can be used in order to update the weights for learning purposes.

## 2.4 Backpropagation in Neural Networks

In this section, we will describe how the generic algorithm based on computational graphs can be used in order to perform the backpropagation algorithm in neural networks. The key idea is that specific variables in the neural network need to be defined as nodes of the

computational-graph abstraction. The same neural network can be represented by different types of computational graphs, depending on which variables in the neural network are used to create computational graph nodes. The precise methodology for performing the backpropagation updates depends heavily on this design choice.

Consider the case of a neural network that first applies a linear function with weights $w_{ij}$ on its inputs to create the pre-activation value $a(i)$, and then applies the activation function $\Phi(\cdot)$ in order to create the output $h(i)$:

$$h(i) = \Phi(a(i))$$

The variables $h(i)$ and $a(i)$ are shown in Figure 2.9. In this case, it is noteworthy that there are several ways in which the computational graph can be created. For example, one might create a computational graph in which each node contains the post-activation value $h(i)$, and therefore we are implicitly setting $y(i) = h(i)$. A second choice is to create a computational graph in which each node contains the pre-activation variable $a(i)$ and therefore we are setting $y(i) = a(i)$. It is even possible to create a decoupled computational graph containing both $a(i)$ and $h(i)$; in the last case, the computational graph will have twice as many nodes as the neural network. In all these cases, a relatively straightforward special-case/simplification of the pseudocodes in the previous section can be used for learning the gradient:

1. The post-activation value $y(i) = h(i)$ could represent the variable in the $i$th computational node in the graph. Therefore, each computational node in such a graph *first* applies the linear function, *and then* applies the activation function. The post-activation value is shown in Figure 2.9. In such a case, the value of $z(i, j) = \frac{\partial y(j)}{\partial y(i)} = \frac{\partial h(j)}{\partial h(i)}$ in the pseudocode of page 43 is $w_{ij}\Phi'_j$. Here, $w_{ij}$ is the weight of the edge from $i$ to $j$ and $\Phi'_j = \frac{\partial \Phi(a(j))}{\partial a(j)}$ is the local derivative of the activation function at node $j$ with respect to its argument. The value of each $\Delta(t_r)$ at output node $t_r$ is simply the derivative of the loss function with respect to $h(t_r)$. The final derivative with respect to the weight $w_{ji}$ (in the final line of the pseudocode on page 43) is equal to $\Delta(i)\frac{\partial h(i)}{\partial w_{ji}} = \Delta(i)h(j)\Phi'_i$.

2. The pre-activation value (after applying the linear function), which is denoted by $a(i)$, could represent the variable in each computational node $i$ in the graph. Note the subtle distinction between the work performed in computational nodes and neural network nodes. Each *computational* node *first* applies the activation function to each of its inputs before applying a linear function, whereas these operations are performed in the reverse order in a neural network. The structure of the computational graph is roughly similar to the neural network, except that the first layer of computational

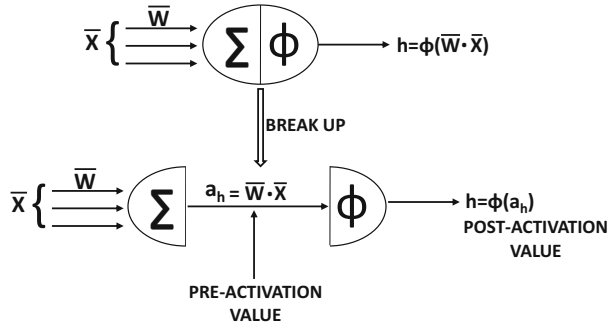



Figure 2.9: Pre- and post-activation values within a neuron

nodes do not contain an activation. In such a case, the value of $z(i, j) = \frac{\partial y(j)}{\partial y(i)} = \frac{\partial a(j)}{\partial a(i)}$ in the pseudocode of page 43 is $\Phi_i' w_{ij}$. Note that $\Phi(a(i))$ is being differentiated with respect to its argument in this case, rather than $\Phi(a(j))$ as in the case of the post-activation variables. The value of the loss derivative with respect to the pre-activation variable $a(t_r)$ in the $r$th output node $t_r$ needs to account for the fact that it is a pre-activation value, and therefore, we cannot directly use the loss derivative with respect to post-activation values. Rather the post-activation loss derivative needs to be *multiplied with the derivative* $\Phi_{t_r}'$ *of the activation function at that node*. The final derivative with respect to the weight $w_{ji}$ (final line of pseudocode on page 43) is equal to $\Delta(i)\frac{\partial a(i)}{\partial w_{ji}} = \Delta(i)h(j)$.

The use of pre-activation variables for backpropagation is more common than the use of post-activation variables. Therefore, we present the backpropagation algorithm in a crisp pseudocode with the use of pre-activation variables. Let $t_r$ be the index of the $r$th output node. Then, the backpropagation algorithm with pre-activation variables may be presented as follows:

Initialize $\Delta(t_r) = \frac{\partial L}{\partial y(t_r)} = \Phi'(a(t_r))\frac{\partial L}{\partial h(t_r)}$ for each output node $t_r$ with $r \in \{1 \ldots k\}$;
**repeat**
  Select an unprocessed node $i$ such that the values of $\Delta(j)$ all of its outgoing nodes $j \in A(i)$ are available;
  Update $\Delta(i) \Leftarrow \Phi_i' \sum_{j \in A(i)} w_{ij}\Delta(j)$;
**until** all nodes have been selected;
**for each edge** $(j, i)$ with weight $w_{ji}$ **do** compute $\frac{\partial L}{\partial w_{ji}} = \Delta(i)h(j)$;

It is also possible to use both pre-activation and post-activation variables as separate nodes of the computational graph. In section 2.5.3, we will combine this approach with a vector-centric representation.

The backpropagation algorithm is quite general in terms of the types of architectures where it can be used. It is most commonly used in feed-forward neural networks, where all input nodes feed into the first hidden layer, the hidden layers successively feed into one another, and the final hidden layer feeds into the output layer. The hidden layer generally does not take inputs, and the loss is generally not computed over the values in the hidden layers. Because of this focus, it is easy to forget that the backpropagation algorithm can be used on *any type of parameterized computational graph*, as long as it is acyclic. For example, a neural network is proposed in [535] that allows random bags of features as direct inputs to hidden layers (see Figure 2.10). Even in this case, the backpropagation pseudocode discussed above would work because it only requires a directed acyclic graph. Furthermore, one could easily experiment with any type of function being used in a computational node (beyond well-known linear summation and activation functions), as long as it can be differentiated.

### 2.4.1 Some Useful Derivatives of Activation Functions

All of the updates in the previous section require the use of the derivatives of various activation functions. For this reason, the derivatives of these activation functions are used repeatedly in this book. This section provides details of these derivatives.

1. *Linear and sign activations:* The derivative of the linear activation function is 1 at all places. The derivative of sign$(v)$ is 0 at all values of $v$ other than at $v = 0$, where it is discontinuous and non-differentiable. Because of the zero gradient and non-differentiability of this activation function, it is rarely used in the loss function

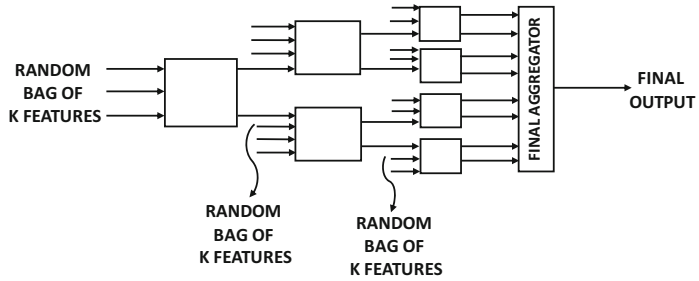
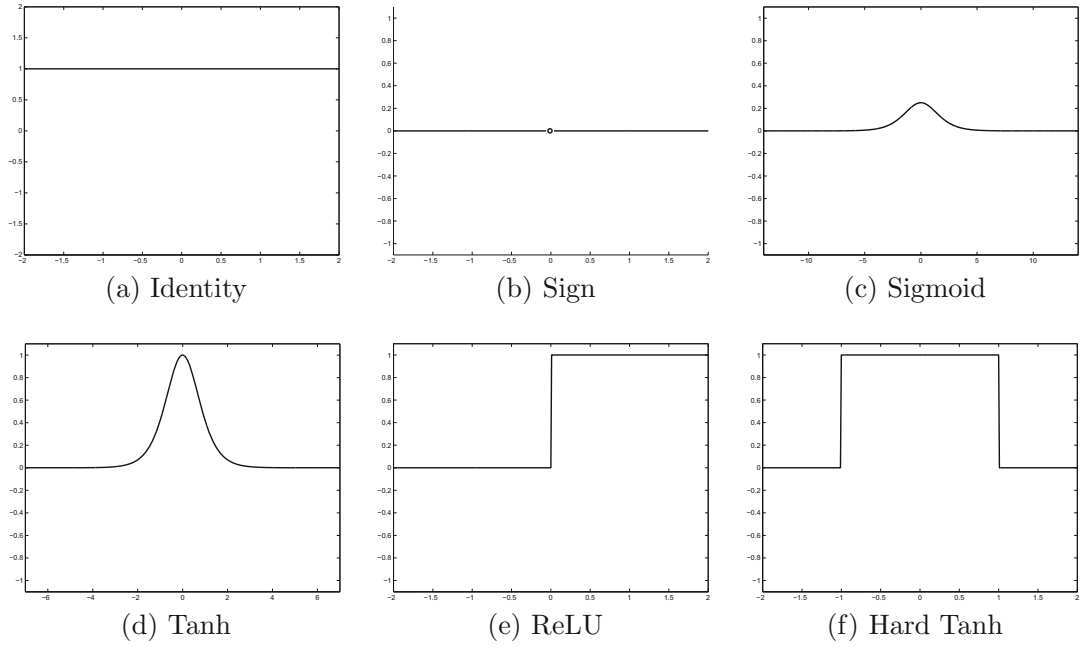



Figure 2.10: An example of an unconventional architecture in which inputs occur to layers other than the first hidden layer.



(a) Identity (b) Sign (c) Sigmoid

(d) Tanh (e) ReLU (f) Hard Tanh

Figure 2.11: The derivatives of various activation functions

even when it is used for prediction at testing time. The derivatives of the linear and sign activations are illustrated in Figures 2.11(a) and (b), respectively.

2. *Sigmoid activation:* The derivative of sigmoid activation is particularly simple, when it is expressed in terms of the *output* of the sigmoid, rather than the input. Let $o$ be the output of the sigmoid function with argument $v$:

$$o = \frac{1}{1 + \exp(-v)} \tag{2.11}$$

Then, one can write the derivative of the activation as follows:

$$\frac{\partial o}{\partial v} = \frac{\exp(-v)}{(1 + \exp(-v))^2} \tag{2.12}$$

The key point is that this sigmoid can be written more conveniently in terms of the outputs:

$$\frac{\partial o}{\partial v} = o(1 - o) \tag{2.13}$$

The derivative of the sigmoid is often used as a function of the output rather than the input. The derivative of the sigmoid activation function is illustrated in Figure 2.11(c).

3. *Tanh activation:* As in the case of the sigmoid activation, the tanh activation is often used as a function of the output $o$ rather than the input $v$:

$$o = \frac{\exp(2v) - 1}{\exp(2v) + 1} \tag{2.14}$$

One can then compute the derivative as follows:

$$\frac{\partial o}{\partial v} = \frac{4 \cdot \exp(2v)}{(\exp(2v) + 1)^2} \tag{2.15}$$

One can also write this derivative in terms of the output $o$:

$$\frac{\partial o}{\partial v} = 1 - o^2 \tag{2.16}$$

The derivative of the tanh activation is illustrated in Figure 2.11(d).

4. *ReLU and hard tanh activations:* The ReLU takes on a partial derivative value of 1 for non-negative values of its argument, and 0, otherwise. The hard tanh function takes on a partial derivative value of 1 for values of the argument in $[-1, +1]$ and 0, otherwise. The derivatives of the ReLU and hard tanh activations are illustrated in Figures 2.11(e) and (f), respectively.

### 2.4.2 Examples of Updates for Various Activations

Since updates with pre-activation variables are more common, we provide the concrete updates with the use of pre-activation variables. We repeat the update discussed in the pseudocode of the previous section with pre-activation variables:

$$\Delta(i) \Leftarrow \Phi_i' \sum_{j \in A(i)} w_{ij} \Delta(j)$$

Since $\Delta(i)$ corresponds to the pre-activation variables, the post-activation values are denoted by $h(i) = \Phi(a(i))$. In the following, we provide the instantiation of the above pre-activation update with the use of different types of activation functions. Note that these updates are always expressed in terms of post-activation values $h(i) = \Phi[a(i)]$, even though the computational graph implicitly uses pre-activation variables:

$$\Delta(i) \Leftarrow \sum_{j \in A(i)} w_{ij} \Delta(j) \quad \text{[Linear]}$$

$$\Delta(i) \Leftarrow h(i)[1 - h(i)] \sum_{j \in A(i)} w_{ij} \Delta(j) \quad \text{[Sigmoid]}$$

$$\Delta(i) \Leftarrow [1 - h(i)^2] \sum_{j \in A(i)} w_{ij} \Delta(j) \quad [\text{Tanh}]$$

Note that the derivative of the sigmoid can be written in terms of its *output* value $h(i)$ as $h(i)[1 - h(i)]$. Similarly, the tanh derivative can be expressed as $[1 - h(i)^2]$. The derivatives of different activation functions are discussed in section 2.4.1 of Chapter 1. For the ReLU function, the value of $\Delta(i)$ can be computed in case-wise fashion:

$$\Delta(i) \Leftarrow \begin{cases} \sum_{j \in A(i)} w_{ij} \Delta(j) & \text{if } 0 < a(i) \\ 0 & \text{otherwise} \end{cases}$$

A similar recurrence can be shown for the hard tanh function except that the update condition is slightly different:

$$\Delta(i) \Leftarrow \begin{cases} \sum_{j \in A(i)} w_{ij} \Delta(j) & \text{if } -1 < a(i) < 1 \\ 0 & \text{otherwise} \end{cases}$$

It is noteworthy that the ReLU and tanh are non-differentiable at exactly the condition boundaries. However, this is rarely a problem in practical settings, in which one works with finite precision.

### The Special Case of Softmax

Softmax activation is a special case because the function is not computed with respect to one input, but with respect to multiple inputs. Therefore, one cannot use exactly the same type of update, as with other activation functions. As discussed in Equation 1.7 of Chapter 1, the softmax activation function is a vector-to-vector function that converts $k$ real-valued predictions $v_1 \ldots v_k$ into output probabilities $o_1 \ldots o_k$ using the following relationship:

$$o_i = \frac{\exp(v_i)}{\sum_{j=1}^{k} \exp(v_j)} \quad \forall i \in \{1, \ldots, k\} \tag{2.17}$$

Note that if we try to use the chain rule to backpropagate the derivative of the loss $L$ with respect to $v_1 \ldots v_k$, then one has to compute each $\frac{\partial L}{\partial o_i}$ and also each $\frac{\partial o_i}{\partial v_j}$. This backpropagation of the softmax is greatly simplified, when we take two facts into account:

1. The softmax is almost always used in the output layer.
2. The softmax is almost always paired with the *cross-entropy loss*. If $y_1 \ldots y_k \in \{0, 1\}$, be the one-hot encoded (observed) outputs for the $k$ mutually exclusive classes, then the cross-entropy loss is defined as follows:

$$L = -\sum_{i=1}^{k} y_i \log(o_i) \tag{2.18}$$

The key point is that the value of $\frac{\partial L}{\partial v_i}$ has a particularly simple form in the case of the softmax:

$$\frac{\partial L}{\partial v_i} = \sum_{j=1}^{k} \frac{\partial L}{\partial o_j} \cdot \frac{\partial o_j}{\partial v_i} = o_i - y_i \tag{2.19}$$

The reader is encouraged to work out the detailed derivation of the result above; it is tedious, but relatively straightforward algebra. The derivation is enabled by the fact that the value of $\frac{\partial o_j}{\partial v_i}$ in Equation 2.19 can be shown to be equal to $o_i(1 - o_i)$ when $i = j$ (which is the same as sigmoid), and otherwise can be shown to be equal to $-o_i o_j$ (see Exercise 7).

In this case, we have decoupled the backpropagation update of the softmax activation from the updates of the weighted layers. In general, it is helpful to create a view of backpropagation in which the linear matrix multiplications and activation layers are decoupled because it greatly simplifies the updates. This view will be discussed in the next section.

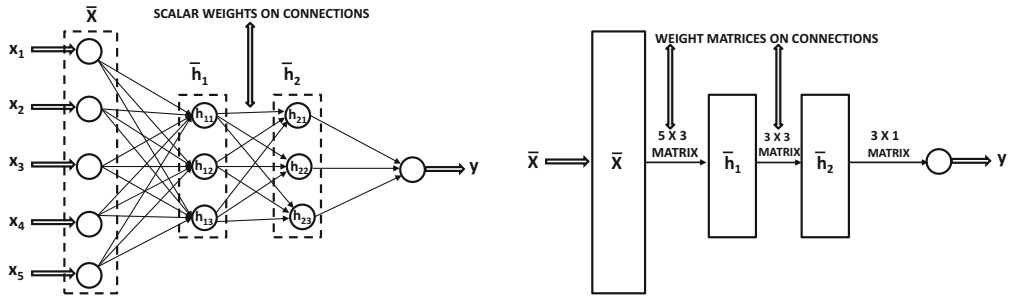
## 2.5 The Vector-Centric View of Backpropagation

In this section, we will discuss the vector-centric view of backpropagation. First, we recap the vector architecture of neural networks discussed in Chapter 1. The corresponding $(k + 1)$ weight matrices between successive layers are denoted by $W_1 \ldots W_{k+1}$. Let $\overline{x}$ be the $d$-dimensional column vector corresponding to the input, $\overline{h}_1 \ldots \overline{h}_k$ be the column vectors corresponding to the hidden layers, and $\overline{o}$ be the $m$-dimensional column vector corresponding to the output. Then, we have the following recurrence condition for multi-layer networks:

$$\overline{h}_1 = \Phi(W_1 \overline{x}) = W_1 \overline{x}$$
$$\overline{h}_{p+1} = \Phi(W_{p+1} \overline{h}_p) = W_{p+1} \overline{h}_p \quad \forall p \in \{1 \ldots k-1\}$$
$$\overline{o} = \Phi(W_{k+1} \overline{h}_k) = W_{k+1} \overline{h}_k$$

The function $\Phi(\cdot)$ is applied in element-wise fashion. A key problem here is that each of the above functions is a *vector composition* of a linear layer and a nonlinear activation layer. Furthermore, the output is an even more complex composition function of earlier layers.

A comparison of the scalar architecture is provided with the vector architecture in Figure 2.12. It is noteworthy that the *connection matrix* between the input layer and the first hidden layer is of size $5 \times 3$, since there are 5 inputs. However, in order to apply the linear transformation $W_1 \overline{x}$ to the 5-dimensional column vector $\overline{x}$, the weight matrix will be of size $3 \times 5$, so that $\Phi(W_1 \overline{x})$ is a 3-dimensional vector. A key point is that the entire neural network can be expressed as a single path with vector-centric operations, which greatly simplifies the topology of the computational graph. However, all functions in the vector-centric view of neural networks are vector-to-vector functions. Therefore, one needs to use the vector-to-vector derivatives and a corresponding chain rule. With this machinery,




(a) Scalar notation and architecture (b) Vector notation and architecture

Figure 2.12: A feed-forward network with two hidden layers and a single output layer.

backpropagation becomes much simpler, as one has an extremely simple topology of a single path (at least in the case of layer-wise neural networks).

### 2.5.1 Derivatives with Respect to Vectors

The backpropagation algorithm requires the computation of node-to-node derivatives and loss-to-node derivatives as intermediate steps. In the vector-centric view, one wants to compute derivatives with respect to entire layers of nodes. To do so, one must use *matrix calculus* notation, which allows derivatives of scalars and vectors with respect to other vectors.

For example, consider the case where one wishes to compute the derivative of a scalar loss $L$ with respect to a vector layer $\overline{x}$, where $\overline{x} = [x_1 \ldots x_d]^T$ is a $d$-dimensional column vector. The derivative of a scalar with respect to a column vector is another column vector, using the *denominator layout convention*[4] of matrix calculus. This derivative is denoted by $\frac{\partial L}{\partial \overline{x}}$, and is simply the gradient. This notation is a scalar-to-vector derivative, which always returns a vector. Therefore, we have the following:

$$\nabla L = \frac{\partial L}{\partial \overline{x}} = \left[\frac{\partial L}{\partial x_1} \cdots \frac{\partial L}{\partial x_d}\right]^T$$

The matrix calculus notation also allows derivatives of vectors with respect to vectors. For example, the derivative of an $m$-dimensional column vector $\overline{h} = [h_1, \ldots, h_m]^T$ with respect to a $d$-dimensional column vector $\overline{x} = [x_1, \ldots, x_d]^T$ is a $d \times m$ matrix in the denominator layout convention. The $(i, j)$th entry of this matrix is the derivative of $h_j$ with respect to $x_i$:

$$\left[\frac{\partial \overline{h}}{\partial \overline{x}}\right]_{ij} = \frac{\partial h_j}{\partial x_i} \tag{2.20}$$

This matrix is closely related to the *Jacobian matrix* in matrix calculus. The $(i, j)$th element of the Jacobian is always $\frac{\partial h_i}{\partial w_j}$, and therefore it is the transpose of the matrix $\frac{\partial \overline{h}}{\partial \overline{x}}$ shown in Equation 2.20:

$$\text{Jacobian}(\overline{h}, \overline{x}) = \left[\frac{\partial \overline{h}}{\partial \overline{x}}\right]^T$$

The transposition occurs because of the use of denominator layout. In fact the Jacobian matrix is exactly equal to $\frac{\partial \overline{h}}{\partial \overline{x}}$ in the numerator layout convention of matrix calculus. However, we will consistently work with the denominator convention in this book.

Two special cases of vector-to-vector derivatives are very useful in neural networks. The first is the linear propagation $\overline{h} = W\overline{x}$, which occurs in linear layers of neural networks. In such a case, $\frac{\partial \overline{h}}{\partial \overline{x}}$ can be shown to be $W^T$. The second is when an *element-wise* activation function $\overline{h} = \Phi(\overline{x})$ is applied to the $d$-dimensional input vector $\overline{x}$ to create another $d$-dimensional vector. In such a case, $\frac{\partial \overline{h}}{\partial \overline{x}}$ can be shown to be the $d \times d$ diagonal matrix in which the $i$th diagonal entry is the derivative $\Phi'(x_i) = \frac{\partial \Phi(x_i)}{\partial x_i}$. Here, $x_i$ is the $i$th component of the vector $\overline{x}$.

### 2.5.2 Vector-Centric Chain Rule

As discussed above, a layered neural network uses repeated compositions of vector functions. Therefore, the function computed by a layered neural network is of the nested form

[4] In the numerator layout convention, the derivative of a scalar with respect to a column vector is a row vector. The choice of convention provides equivalent results with some transpositional changes.

$F_k(F_{k-1}(\ldots F_1(\overline{x})))$. The vectored chain rule is useful for computing derivatives of such functions, and is defined as follows:

**Theorem 2.5.1 (Vectored Chain Rule)** *Consider a composition function of the following form:*

$$\overline{o} = F_k(F_{k-1}(\ldots F_1(\overline{x})))$$

*Assume that each $F_i(\cdot)$ takes as input an $n_i$-dimensional column vector and outputs an $n_{i+1}$-dimensional column vector. Therefore, the input $\overline{x}$ is an $n_1$-dimensional vector and the final output $\overline{o}$ is an $n_{k+1}$-dimensional vector. For brevity, denote the vector output of $F_i(\cdot)$ by $\overline{h}_i$. Then, the vectored chain rule asserts the following:*

$$\underbrace{\left[\frac{\partial \overline{o}}{\partial \overline{x}}\right]}_{n_1 \times n_{k+1}} = \underbrace{\left[\frac{\partial \overline{h}_1}{\partial \overline{x}}\right]}_{n_1 \times n_2} \underbrace{\left[\frac{\partial \overline{h}_2}{\partial \overline{h}_1}\right]}_{n_2 \times n_3} \cdots \underbrace{\left[\frac{\partial \overline{h}_{k-1}}{\partial \overline{h}_{k-2}}\right]}_{n_{k-1} \times n_k} \underbrace{\left[\frac{\partial \overline{o}}{\partial \overline{h}_{k-1}}\right]}_{n_k \times n_{k+1}}$$

It is easy to see that the size constraints of matrix multiplication are respected in this case.

### 2.5.3 A Decoupled View of Vector-Centric Backpropagation

In the previous discussion, two equivalent ways of computing the updates based on pre-activation and post-activation values are provided. In each case, *one is really backpropagating through a linear matrix multiplication and an activation computation simultaneously.* However, in many real implementations, the linear computations and the activation computations are decoupled as separate "layers," and one separately backpropagates through the two layers. Furthermore, we use a vector-centric representation of the neural network, so that operations on vector representations of layers are vector-to-vector operations such as a matrix multiplication in a linear layer (cf. Figure 2.12). This view greatly simplifies the computations, since the entire neural network is a single path (albeit a vector-centric one). Therefore, one can create a neural network in which activation layers are alternately arranged with linear layers, as shown in Figure 2.13. Note that the activation layers can use identity activation if needed. Activation layers (usually) perform one-to-one, element-wise computations on the vector components with the activation function $\Phi(\cdot)$, whereas linear layers perform all-to-all computations by multiplying with the coefficient matrix $W$. Then, for each pair of matrix multiplication and activation function layers, the following forward and backward steps need to be performed:

1. Let $\overline{z}_i$ and $\overline{z}_{i+1}$ be the column vectors of activations in the forward direction when the matrix of linear transformations from the $i$th to the $(i+1)$th layer is denoted[5] by $W$. Furthermore, let $\overline{g}_i$ and $\overline{g}_{i+1}$ be the backpropagated vectors of gradients in the two layers. Each element of $\overline{g}_i$ is the partial derivative of the loss function with respect to a hidden variable in the $i$th layer. Then, we have the following:

$$\overline{z}_{i+1} = W\overline{z}_i \qquad \text{[Forward Propagation]}$$
$$\overline{g}_i = W^T \overline{g}_{i+1} \qquad \text{[Backward Propagation]}$$

[5]Strictly speaking, we should use the notation $W_i$ instead of $W$, although we omit the subscripts here for simplicity, since the entire discussion in this section is devoted to the linear transforms between a single pair of layers.

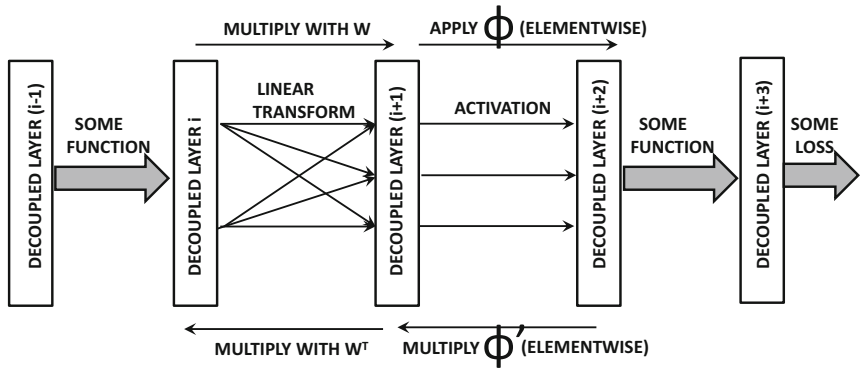



Figure 2.13: A decoupled view of backpropagation

Table 2.1: Examples of different functions and their backpropagation updates between layers $i$ and $(i+1)$. The hidden values and gradients in layer $i$ are denoted by $\overline{z}_i$ and $\overline{g}_i$. Some of these computations use $I(\cdot)$ as the binary indicator function.

| Function | Type | Forward | Backward |
|---|---|---|---|
| Linear | Many-Many | $\overline{z}_{i+1} = W\overline{z}_i$ | $\overline{g}_i = W^T\overline{g}_{i+1}$ |
| Sigmoid | One-One | $\overline{z}_{i+1}$ =sigmoid$(\overline{z}_i)$ | $\overline{g}_i = \overline{g}_{i+1} \odot \overline{z}_{i+1} \odot (\overline{1} - \overline{z}_{i+1})$ |
| Tanh | One-One | $\overline{z}_{i+1}$ =tanh$(\overline{z}_i)$ | $\overline{g}_i = \overline{g}_{i+1} \odot (\overline{1} - \overline{z}_{i+1} \odot \overline{z}_{i+1})$ |
| ReLU | One-One | $\overline{z}_{i+1} = \overline{z}_i \odot I(\overline{z}_i > 0)$ | $\overline{g}_i = \overline{g}_{i+1} \odot I(\overline{z}_i > 0)$ |
| Hard Tanh | One-One | Set to $\pm 1$ ($\notin [-1, +1]$)<br>Copy ($\in [-1, +1]$) | Set to 0 ($\notin [-1, +1]$)<br>Copy ( $\in [-1, +1]$) |
| Max | Many-One | Maximum of inputs | Set to 0 (non-maximal inputs)<br>Copy (maximal input) |
| Arbitrary function $f_k(\cdot)$ | Anything | $\overline{z}_{i+1}^{(k)} = f_k(\overline{z}_i)$ | $\overline{g}_i = J^T\overline{g}_{i+1}$<br>$J$ is Jacobian (Equation 2.21) |

2. Now consider a situation where the activation function $\Phi(\cdot)$ is applied to each node in layer $(i+1)$ to obtain the activations in layer $(i+2)$. Then, we have the following:

$$\overline{z}_{i+2} = \Phi(\overline{z}_{i+1}) \quad \text{[Forward Propagation]}$$
$$\overline{g}_{i+1} = \overline{g}_{i+2} \odot \Phi'(\overline{z}_{i+1}) \quad \text{[Backward Propagation]}$$

Here, $\Phi_i(\cdot)$ and its derivative $\Phi_i'(\cdot)$ are applied in element-wise fashion to vector arguments. The symbol $\odot$ indicates elementwise multiplication.

Note the extraordinary simplicity once the activation is decoupled from the matrix multiplication in a layer. The forward and backward computations are shown in Figure 2.13. Furthermore, the derivatives of $\Phi(\overline{z}_{i+1})$ can often be computed in terms of the outputs of the next layer. Based on section 2.4.2, one can show the following for sigmoid activations:

$$\Phi'(\overline{z}_{i+1}) = \Phi(\overline{z}_{i+1}) \odot (1 - \Phi(\overline{z}_{i+1}))$$
$$= \overline{z}_{i+2} \odot (1 - \overline{z}_{i+2})$$

Examples of different types of backpropagation updates for various forward functions are shown in Table 2.1. In this case, we have used layer indices of $i$ and $(i+1)$ for *both* linear

transformations and activation functions (rather than using $(i+2)$ for activation function). Note that the second to last entry in the table corresponds to the maximization function. This type of function is useful for *max-pooling* operations in convolutional neural networks.

Given the vector of gradients in a layer, one only has to apply the operations shown in the final column of Table 2.1 to obtain the gradients with respect to the previous layer. In the table, the vector indicator function $I(\overline{x} > 0)$ is an *element-wise* indicator function that returns a binary vector of the same size as $\overline{x}$; the $i$th output component is set to 1 when the $i$th component of $\overline{x}$ is larger than 0. The notation $\overline{1}$ indicates a column vector of 1s.

An arbitrary function on the $i$th layer of the network (beyond the simple linear matrix multiplications and element-wise activations) can be handled by assuming that the $k$th activation in layer-$(i+1)$ is obtained by applying an arbitrary function $f_k(\overline{z}_i)$ on the vector $\overline{z}_i$ of activations in layer-$i$. Then, backpropagation can be performed with the help of the Jacobian matrix $J = [J_{kr}]$, which is defined as follows:

$$J_{kr} = \frac{\partial f_k(\overline{z}_i)}{\partial \overline{z}_i^{(r)}} \tag{2.21}$$

Here, $\overline{z}_i^{(r)}$ is the $r$th element in $\overline{z}_i$. Let $J$ be the matrix whose elements are $J_{kr}$. Then, it is easy to see that the backpropagation update from layer-$(i+1)$ to layer-$i$ can be written as follows:

$$\overline{g}_i = J^T \overline{g}_{i+1} \tag{2.22}$$

Writing backpropagation equations as matrix multiplications is often beneficial from an implementation-centric point of view, such as acceleration with Graphics Processor Units (cf. section 4.9.1).

### Derivatives with Respect to Weight Matrices

Upon performing the backpropagation, one only obtains the loss-to-node derivatives but not the loss-to-weight derivatives. Note that the elements in $\overline{g}_i$ represent gradients of the loss with respect to the *activations* in the $i$th layer, and therefore an additional step is needed to compute gradients with respect to the *weights*. The gradient of the loss with respect to a weight between the $p$th unit of the $(i-1)$th layer and the $q$th unit of $i$th layer is obtained by multiplying the $p$th element of $\overline{z}_{i-1}$ with the $q$th element of $\overline{g}_i$. One can also achieve this goal using a vector-centric approach, by simply computing the *outer product* of $\overline{g}_i$ and $\overline{z}_{i-1}$. In other words, the entire matrix $M$ of derivatives of the loss with respect to the elements of the weight matrix between the $(i-1)$th layer and the $i$th layer is given by the following:

$$M = \overline{g}_i \overline{z}_{i-1}^T \tag{2.23}$$

Since $M$ is given by the product of a column vector and a row vector of sizes equal to two successive layers, it is a matrix of exactly the same size as the weight matrix between the two layers. The $(q, p)$th element of $M$ yields the derivative of the loss with respect to the weight between the $p$th element of $\overline{z}_{i-1}$ and $q$th element of $\overline{z}_i$.

### Example of Vector-Centric Backpropagation

In order to explain vector-specific backpropagation, we will use an example in which the linear layers and activation layers have been decoupled. Figure 2.14 shows an example of

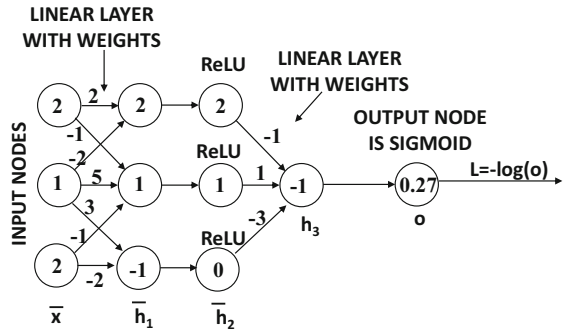



Figure 2.14: Example of decoupled neural network with vector layers $\overline{x}$, $\overline{h}_1$, $\overline{h}_2$, $h_3$, and $o$: variable values are shown within the nodes

a neural network with two computational layers, but they appear as four layers, since the activation layers have been decoupled as separated layers from the linear layers. The vector for the input layer is denoted by the 3-dimensional column vector $\overline{x}$, and the vectors for the computational layers are $\overline{h}_1$ (3-dimensional), $\overline{h}_2$ (3-dimensional), $h_3$ (1-dimensional), and output layer $o$ (1-dimensional). The loss function is $L = -\log(o)$. These notations are annotated in Figure 2.14. The input vector $\overline{x}$ is $[2, 1, 2]^T$, and the weights of the edges in the two linear layers are annotated in Figure 2.14. Missing edges between $\overline{x}$ and $\overline{h}_1$ are assumed to have zero weight. In the following, we will provide the details of both the forward and the backwards phase.

**Forward phase:** The first hidden layer $\overline{h}_1$ is related to the input vector $\overline{x}$ with the weight matrix $W$ as $\overline{h}_1 = W\overline{x}$. We can reconstruct the weights matrix $W$ and then compute $\overline{h}_1$ for forward propagation as follows:

$$W = \begin{bmatrix} 2 & -2 & 0 \\ -1 & 5 & -1 \\ 0 & 3 & -2 \end{bmatrix}; \quad \overline{h}_1 = W\overline{x} = \begin{bmatrix} 2 & -2 & 0 \\ -1 & 5 & -1 \\ 0 & 3 & -2 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \\ -1 \end{bmatrix}$$

The hidden layer $\overline{h}_2$ is obtained by applying the ReLU function in element-wise fashion to $\overline{h}_1$ during the forward phase. Therefore, we obtain the following:

$$\overline{h}_2 = \text{ReLU}(\overline{h}_1) = \text{ReLU} \begin{bmatrix} 2 \\ 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix}$$

Subsequently, the $1 \times 3$ weight matrix $W_2 = [-1, 1, -3]$ is used to transform the 3-dimensional vector $\overline{h}_2$ to the 1-dimensional "vector" $h_3$ as follows:

$$h_3 = W_2\overline{h}_2 = [-1, 1, -3] \begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix} = -1$$

The output $o$ is obtained by applying the sigmoid function to $h_3$. In other words, we have the following:

$$o = \frac{1}{1 + \exp(-h_3)} = \frac{1}{1+e} \approx 0.27$$

The point-specific loss is $L = -\log_e(0.27) \approx 1.3$.

**Backwards phase:** In the backward phase, we first start by initializing $\frac{\partial L}{\partial o}$ to $-1/o$, which is $-1/0.27$. Then, the 1-dimensional "gradient" $g_3$ of the hidden layer $h_3$ is obtained by using the backpropagation formula for the sigmoid function in Table 2.1:

$$g_3 = o(1-o) \underbrace{\frac{\partial L}{\partial o}}_{-1/o} = o - 1 = 0.27 - 1 = -0.73 \tag{2.24}$$

The gradient $\overline{g}_2$ of the hidden layer $\overline{h}_2$ is obtained by multiplying $g_3$ with the transpose of the weight matrix $W_2 = [-1, 1, -3]$:

$$\overline{g}_2 = W_2^T g_3 = \begin{bmatrix} -1 \\ 1 \\ -3 \end{bmatrix} (-0.73) = \begin{bmatrix} 0.73 \\ -0.73 \\ 2.19 \end{bmatrix}$$

Based on the entry in Table 2.1 for the ReLU function, the gradient $\overline{g}_2$ can be propagated backwards to $\overline{g}_1 = \frac{\partial L}{\partial \overline{h}_1}$ by copying the components of $\overline{g}_2$ to $\overline{g}_1$, when the corresponding components in $\overline{h}_1$ are positive; otherwise, the components of $\overline{g}_1$ are set to zero. Therefore, the gradient $\overline{g}_1 = \frac{\partial L}{\partial \overline{h}_1}$ can be obtained by simply copying the first and second components of $\overline{g}_2$ to the first and second components of $\overline{g}_1$, and setting the third component of $\overline{g}_1$ to 0. In other words, we have the following:

$$\overline{g}_1 = \begin{bmatrix} 0.73 \\ -0.73 \\ 0 \end{bmatrix}$$

Note that we can also compute the gradient $\overline{g}_0 = \frac{\partial L}{\partial \overline{x}}$ of the loss with respect to the input layer $\overline{x}$ by simply computing $\overline{g}_0 = W^T \overline{g}_1$. However, this is not really needed for computing loss-to-weight derivatives.

**Computing loss-to-weight derivatives:** So far, we have only shown how to compute loss-to-node derivatives in this particular example. These need to be converted to loss-to-weight derivatives with the additional step of multiplying with a hidden layer. Let $M$ be the loss-to-weight derivatives for the weight matrix $W$ between the two layers. Note that there is a one-to-one correspondence between the positions of the elements of $M$ and $W$. Then, the matrix $M$ is defined as follows:

$$M = \overline{g}_1 \overline{x}^T = \begin{bmatrix} 0.73 \\ -0.73 \\ 0 \end{bmatrix} [2, 1, 2] = \begin{bmatrix} 1.46 & 0.73 & 1.46 \\ -1.46 & -0.73 & -1.46 \\ 0 & 0 & 0 \end{bmatrix}$$

Similarly, one can compute the loss-to-weight derivative matrix $M_2$ for the $1 \times 3$ matrix $W_2$ between $\overline{h}_2$ and $h_3$:

$$M_2 = g_3 \overline{h}_2^T = (-0.73)[2, 1, 0] = [-1.46, -0.73, 0]$$

Note that the size of the matrix $M_2$ is identical to that of $W_2$, although the weights of the missing edges should not be updated.

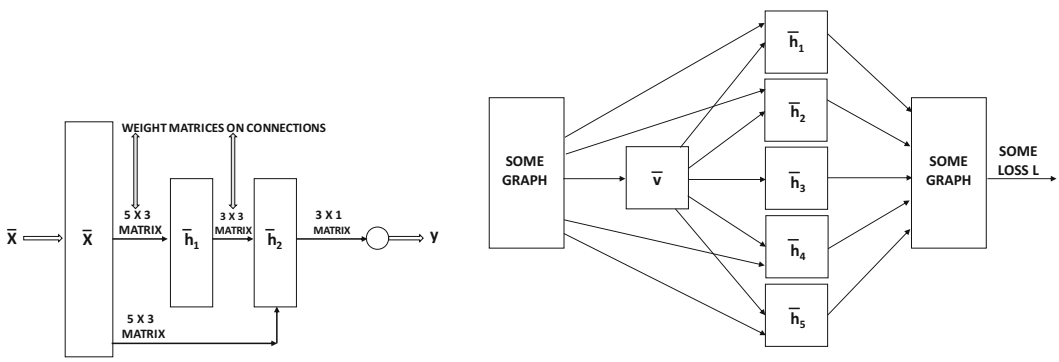
### 2.5.4 Vector-Centric Backpropagation with Non-Layered Architectures

Most neural networks are layered architectures in which all nodes in layer $i$ are connected to those from layer-$(i+1)$. As a result, the computational graph appears as a single path. What happens when the computational graph has an arbitrary structure? For example, the computational graph in Figure 2.15(a) has a *skip connection* between alternate layers. This type of situation does occur in some recent convolutional architectures like *ResNet* [193, 194]. In such a case, we might have a situation where we have multiple nodes $\overline{h}_1 \ldots \overline{h}_s$ between node $\overline{v}$ and a network in later layers, as shown in Figure 2.15(b). Assume that the vector $\overline{h}_i$ has dimensionality $m_i$. In such a case, the partial derivative turns out to be a simple generalization of the case used for a single path:

$$\frac{\partial L}{\partial \overline{v}} = \sum_{i=1}^{s} \underbrace{\frac{\partial \overline{h}_i}{\partial \overline{v}}}_{d \times m_i} \underbrace{\frac{\partial L}{\partial \overline{h}_i}}_{m_i \times 1} = \sum_{i=1}^{s} \text{Jacobian}(\overline{h}_i, \overline{v})^T \frac{\partial L}{\partial \overline{h}_i}$$

In most layered neural networks, we only have a single path and we rarely have to deal with the case of branches. Such branches might, however, arise in the case of neural networks with skip connections (see Figures 2.15(a) and (b)). However, even in these types of complicated network architectures, *each node only has to worry about its local outgoing edges during backpropagation*. This is the key point of the backpropagation paradigm, where one does not have to worry about the global structure of the graph during gradient computation.

Consider the case where we have $p$ output nodes containing vector-valued variables, which have indices denoted by $t_1 \ldots t_p$, and the variables in it are $\overline{y}(t_1) \ldots \overline{y}(t_p)$. In such a case, the loss function $L$ might be function of all the components in these vectors. Assume that the $i$th node contains a column vector of variables denoted by $\overline{y}(i)$. Furthermore, in the denominator layout of matrix calculus, each $\overline{\Delta}(i) = \frac{\partial L}{\partial \overline{y}(i)}$ is a column vector with dimensionality equal to that of $\overline{y}(i)$. It is this *vector* of loss derivatives that will be propagated backwards. The vector-centric algorithm for computing derivatives is as follows:




(a) An unconventional architecture (b) Vector-centric graph with multiple paths

Figure 2.15: Examples of vector-centric computational graphs

Initialize $\overline{\Delta}(t_k) = \frac{\partial L}{\partial \overline{y}(t_k)}$ for each output node $t_k$ for $k \in \{1 \ldots p\}$;
**repeat**
  Select an unprocessed node $i$ such that the values of $\overline{\Delta}(j)$ all of its outgoing nodes $j \in A(i)$ are available;
  Update $\overline{\Delta}(i) \Leftarrow \sum_{j \in A(i)} \text{Jacobian}(\overline{y}(j), \overline{y}(i))^T \overline{\Delta}(j)$;
**until** all nodes have been selected;
**for** the vector $\overline{w}_i$ of edges incoming to each node $i$ **do** compute $\frac{\partial L}{\partial \overline{w}_i} = \frac{\partial \overline{y}(i)}{\partial \overline{w}_i} \overline{\Delta}(i)$

In the final step of the above pseudocode, the derivative of vector $\overline{y}(i)$ with respect to the vector $\overline{w}_i$ is computed, which is itself the transpose of a Jacobian matrix. This final step converts a vector of partial derivatives with respect to node variables into a vector of partial derivatives with respect to weights incoming at a node.

## 2.6 The Not-So-Unimportant Details

In this section, we will discuss some important special cases and details of the backpropagation algorithm.

### 2.6.1 Mini-Batch Stochastic Gradient Descent

So far, all updates to the weights of a neural network are performed in point-specific fashion, which is referred to as *stochastic* gradient descent. In this section, we provide a justification for this choice along with related variants like *mini-batch stochastic gradient descent*. We also provide an understanding of the advantages and disadvantages of various choices.

Most machine learning problems can be recast as optimization problems over a *linearly additive sum* of the loss functions on the individual training data points. However, one rarely uses all the points at a single time, and one might *stochastically* select a point in order to update the parameters to reduce that point-specific loss. In a neural network, this process is natural because the simple methods we have introduced so far process the points one at a time in forwards and backwards propagation in order to iteratively minimize point-specific losses. However, the loss over the *entire data set* is really defined as the sum of the losses over individual training points. One can write the loss function of a neural network as the sum of point-specific losses:

$$L = \sum_{i=1}^{n} L_i \tag{2.25}$$

Here, $L_i$ is the loss contributed by the $i$th training point. In gradient descent, one tries to minimize the loss function of the neural network by moving the parameters along the negative direction of the gradient of the loss *over all points*. For example, in the case of the perceptron, the parameters correspond to $\overline{W} = (w_1 \ldots w_d)$. Therefore, one would try to compute the loss of the underlying objective function over all points simultaneously and perform gradient descent. Therefore, in traditional gradient descent, one would try to perform gradient-descent steps such as the following:

$$\overline{W} \Leftarrow \overline{W} - \alpha \left( \frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2} \ldots \frac{\partial L}{\partial w_d} \right) \tag{2.26}$$

This type of derivative can also be written succinctly in vector notation (i.e., matrix calculus notation):

$$\overline{W} \Leftarrow \overline{W} - \alpha \frac{\partial L}{\partial \overline{W}} \tag{2.27}$$

Correspondingly, it is easy to show that the true update of the perception should be the following (based on Equation 2.25):

$$\overline{W} \Leftarrow \overline{W} - \frac{\partial L}{\partial \overline{W}} = \overline{W} - \sum_{i=1}^{n} \frac{\partial L_i}{\partial \overline{W}} \tag{2.28}$$

This is possible to achieve in the perceptron, but if one were to generalize the above updates to multilayer architectures, one would need to *simultaneously* run all examples forwards and backwards through the network in order to compute the gradient. Note that the intermediate activations and derivatives *for each training instance* would need to be maintained *simultaneously* over hundreds of thousands of neural network nodes. This can be exceedingly large in most practical settings. It is, therefore, impractical to simultaneously run all examples through the network to compute the gradient with respect to the *entire data set* in one shot. The point-at-a-time update introduced so far is really a *practical approximation* of the true update by updating with the use of $L_i$ rather than $L$:

$$\overline{W} \Leftarrow \overline{W} - \alpha \frac{\partial L_i}{\partial \overline{W}} \tag{2.29}$$

Assuming a random ordering of points, each update can be viewed as a probabilistic approximation of the true update, although the long-term effect of repeated updates is approximately the same. This type of probabilistic approximation is, therefore, referred to as *stochastic* gradient descent. The main advantages of stochastic gradient descent are that it is fast and memory efficient, albeit[6] at the expense of accuracy.

The main issue with stochastic gradient descent is that the point-at-a-time approach can sometimes behave in an unstable way, because individual points in the training data might be mislabeled or have other errors. However, a middle ground is possible with the use of *mini-batch stochastic gradient descent*. The idea is to use a *batch* of points in each update rather than one point. In mini-batch stochastic descent, one uses a batch $B = \{j_1 \ldots j_m\}$ of training points for the update:

$$\overline{W} \Leftarrow \overline{W} - \alpha \sum_{i \in B} \frac{\partial L_i}{\partial \overline{W}} \tag{2.30}$$

The data used in machine learning problems often have a high level of redundancy in terms of the knowledge captured by different training points, and therefore the gradient obtained from a sample of points is usually quite accurate. Furthermore, the weights are often incorrect to such a degree at the beginning of the learning process that even a small sample of points can be used to create an excellent estimate of the gradient. This observation provides a practical foundation for the success of mini-batch stochastic gradient-descent, which often exhibits the best trade-off between stability, speed, and memory requirements.

When using mini-batch stochastic gradient descent, the outputs of each layer are matrices instead of vectors, and forward propagation requires the multiplication of the weight matrix with the activation matrix. The same is true for backward propagation in which matrices of gradients are maintained. Therefore, the implementation of mini-batch stochastic gradient descent increases the memory requirements, which is a limiting factor on the size of

[6] On the other hand, stochastic gradient descent also has the indirect benefit of *regularization* (cf. Chapter 5), and therefore the degradation is often not too significant. Sometimes, it even improves over vanilla gradient descent in terms of performance on out-of-sample data. However, it can occasionally provide poor results with certain types of data sets.

the mini-batch. However, memory is rarely a limiting factor in practical terms, because one does not gain much accuracy in gradient computation by increasing the batch size beyond a few hundred points. It is common to use powers of 2 as the size of the mini-batch, because this choice often provides the best efficiency on most hardware architectures; commonly used values are 32, 64, 128, or 256. Although the use of mini-batch stochastic gradient descent is ubiquitous in neural network learning, most of this book will use a single point for the update (i.e., pure stochastic gradient descent) for simplicity in presentation.

### 2.6.2 Learning Rate Decay

A constant learning rate is not desirable because it poses a dilemma to the analyst. In general, it is desirable to select a high learning rate at the beginning of the algorithm, with a lower learning rate for later phases. The dilemma is as follows. A lower learning rate used early on will cause the algorithm to take too long to come even close to an optimal solution. On the other hand, a large initial learning rate will allow the algorithm to come reasonably close to a good solution at first; however, the algorithm will then oscillate around the point for a very long time, or diverge in an unstable way, if the high rate of learning is maintained. In either case, maintaining a constant learning rate is not ideal. Allowing the learning rate to decay over time can naturally achieve the desired learning-rate adjustment to avoid these challenges.

The two most common decay mechanisms are *exponential decay* and *inverse decay*, which define the learning rate $\alpha_t$ in terms of the initial decay rate $\alpha_0$ and epoch $t$ as follows:

$$\alpha_t = \alpha_0 \exp(-k \cdot t) \qquad \text{[Exponential Decay]}$$

$$\alpha_t = \frac{\alpha_0}{1 + k \cdot t} \qquad \text{[Inverse Decay]}$$

The parameter $k$ controls the rate of the decay. Another approach is to use step decay in which the learning rate is reduced by a particular factor every few epochs. For example, the learning rate might be multiplied by 0.5 every 5 epochs. A common approach is to track the loss on a held-out portion of the training data set, and reduce the learning rate whenever this loss stops improving. In some cases, the analyst might even babysit the learning process, and use an implementation in which the learning rate can be changed manually depending on the progress. This type of approach can be used with simple implementations of gradient descent, although it does not address many of the other problematic issues. Therefore, more complex algorithms for gradient descent are discussed in Chapter 4.

### 2.6.3 Checking the Correctness of Gradient Computation

The backpropagation algorithm is quite complex, and one might occasionally check the correctness of gradient computation. This can be performed easily with the use of numerical methods. Consider a particular weight $w$ of a randomly selected edge in the network. Let $L(w)$ be the current value of the loss. The weight of this edge is perturbed by adding a small amount $\epsilon > 0$ to it. Then, the forward algorithm is executed with this perturbed weight and the loss $L(w + \epsilon)$ is computed. Then, the partial derivative of the loss with respect to $w$ can be shown to be the following:

$$\frac{\partial L(w)}{\partial w} \approx \frac{L(w + \epsilon) - L(w)}{\epsilon} \tag{2.31}$$

When the partial derivatives do not match closely enough, it is easy to detect that an error must have occurred in computation. One needs to perform the above estimation for

only two or three checkpoints in the training process, which is quite efficient. However, it might be advisable to perform the checking over a modest subset of the parameters at these checkpoints. In order to determine whether the gradients are "close enough," one has to account for the absolute magnitudes of these gradients with the use of *relative ratios*.

Let the backpropagation-determined derivative be denoted by $G_e$, and the aforementioned estimation be denoted by $G_a$. Then, the relative ratio $\rho$ is defined as follows:

$$\rho = \frac{|G_e - G_a|}{|G_e + G_a|} \tag{2.32}$$

Typically, the ratio should be less than $10^{-6}$, although for some activation functions like the ReLU in which sharp changes in derivatives occur at particular points, it is possible for the numerical gradient to be different from the computed gradient. In such cases, the ratio should still be less than $10^{-3}$. One can use this numerical approximation to check the correctness of the gradients with respect to a subset of weights. If there are millions of parameters, then one can test a sample of the derivatives for a quick check of correctness. It is also advisable to perform this check at two or three points in the training because the checks at initialization might correspond to special cases that do not hold at later stages.

### 2.6.4 Regularization

As you will learn in Chapter 5, it is important to regularize the stochastic gradient-descent updates by *shrinking* the weights every time an update is made. For example, the stochastic gradient-descent update for the $i$th data point, as shown in Equation 2.29, is repeated here:

$$\overline{W} \Leftarrow \overline{W} - \alpha \frac{\partial L_i}{\partial \overline{W}} \tag{2.33}$$

In regularization, an additional shrinking factor $(1 - \alpha\lambda)$ is applied to the weight vector, where $\lambda > 0$ is the regularization parameter:

$$\overline{W} \Leftarrow \overline{W}(1 - \alpha\lambda) - \alpha \frac{\partial L_i}{\partial \overline{W}} \tag{2.34}$$

As discussed in Chapter 5, regularization often improves the accuracy on out-of-sample data at prediction time (although it might worsen the accuracy on training data).

### 2.6.5 Loss Functions on Hidden Nodes

In some forms of sparse feature learning, even the outputs of the hidden nodes have loss functions associated with them. This occurs frequently in order to encourage specific properties of the hidden layer. In such cases, the backpropagation algorithm requires only minor modifications in which the gradient flow in the backwards direction includes the losses at the hidden nodes (by treating them in a similar manner to output nodes). This can be achieved by simple aggregation of the gradient flows resulting from different losses. At a fundamental level, the backpropagation methodology remains the same.

Consider the case in which the loss function $L_i^h$ is associated with the hidden node $h(i)$. Furthermore, let $\Delta(i)$ denote the gradient flow from all nodes reachable from $i$. Then, the standard pre-activation based update is modified to add the effect of hidden node losses:

$$\Delta(i) \Leftarrow [\Phi_i' \sum_{j \in A(i)} w_{ij} \Delta(j)] + \Phi_i' \frac{\partial L_i^h}{\partial h(i)} \tag{2.35}$$

Note that the second term in the above update accounts for the effect of the loss on the hidden node $i$. This term simply adds to the gradient flow during backpropagation. This addition has a similar algebraic form to the value of the initialization of the output nodes. Therefore, the overall framework of backpropagation remains almost identical, with the main difference being that the backpropagation algorithm picks up additional contributions from the losses at the hidden nodes.

### 2.6.6 Backpropagation Tricks for Handling Shared Weights

A very common approach for regularizing neural networks is to use *shared weights*. The basic idea is that if one has some semantic insight that a similar function will be computed in different nodes of the network, then the weights associated with those nodes will be constrained to be the same value. Some examples are as follows:

1. In an *autoencoder* simulating *singular value decomposition* (cf. section 3.4.1.2 of Chapter 3), the weights in the input layer and the output layer are shared.
2. In a recurrent neural network for text (cf. Chapter 8), the weights in different temporal layers are shared, because each word in a sentence shares the same *language model*.
3. In a convolutional neural network, the same weight *filters* are used over the entire image input (cf. Chapter 9), as pixels are interpreted in a spatially consistent way.

Sharing weights in a semantically insightful way is one of the key tricks to successful neural network design. When one can identify particular sets of nodes sharing similar semantic relationships, it makes sense to use shared weights.

At first glance, the computation of the loss gradient with respect to the shared weights might seem to be an onerous task because of the complexity of loss functions in computational graphs. However, a key trick can be borrowed from differential calculus to enable simple computation. Let $w$ be a shared weight across $T$ nodes in the network, and the corresponding weight copies be denoted by $w_1 \ldots w_T$. Let the loss function be $L$. Then, it is easy to use the chain rule to show the following:

$$\frac{\partial L}{\partial w} = \sum_{i=1}^{T} \frac{\partial L}{\partial w_i} \cdot \underbrace{\frac{\partial w_i}{\partial w}}_{=1} = \sum_{i=1}^{T} \frac{\partial L}{\partial w_i}$$

In other words, all we have to do is to pretend that these weights are independent, compute their derivatives, and add them! Therefore, we simply have to execute the backpropagation algorithm without any change and then sum up the gradients of different copies of the shared weight. This simple observation forms the basis of learning algorithms in many key neural network architectures such as recurrent, convolutional, and graph neural networks.

## 2.7 Tuning and Preprocessing

There are several important issues associated with the setup of the neural network, preprocessing, and initialization. First, the *hyperparameters* of the neural network (such as the learning rates and regularization parameters) need to be selected. Feature preprocessing and initialization can also be rather important. Neural networks tend to have larger parameter spaces compared to other machine learning algorithms, which magnifies the effect of preprocessing and initialization in many ways. In the following, we will discuss the basic methods

used for feature preprocessing and initialization. Strictly speaking, advanced methods like *pretraining* can also be considered initialization techniques. However, the discussion of this technique is deferred to the next chapter because it requires a deeper understanding of the model generalization issues associated with neural network training.

### 2.7.1 Tuning Hyperparameters

Neural networks have a large number of *hyperparameters* such as the learning rate, the weight of regularization, and so on. The term "hyperparameter" is used to specifically refer to the parameters regulating the design of the model (like learning rate and regularization), and they are different from the more fundamental parameters representing the weights of connections in the neural network. In Bayesian statistics, the notion of hyperparameter is used to control the prior distribution, although we use this definition in a somewhat loose sense here. In a sense, there is a two-tiered organization of parameters in the neural network, in which primary model parameters like weights are optimized with backpropagation only after fixing the hyperparameters either manually or with the use of a *tuning* phase. As we will discuss in section 5.3 of Chapter 5, the hyperparameters should not be tuned using the same data used for gradient descent. Rather, a portion of the data is held out as validation data, and the performance of the algorithm is tested on the validation set with various choices of hyperparameters. This type of approach ensures that the tuning process does not overfit to the training data set (while providing poor test data performance).

How should the candidate hyperparameters be selected for testing? The most well-known technique is *grid search*, in which a set of values is selected for each hyperparameter. In the most straightforward implementation of grid search, all combinations of selected values of the hyperparameters are tested in order to determine the optimal choice. One issue with this procedure is that the number of hyperparameters might be large, and the number of points in the grid increases *exponentially* with the number of hyperparameters. For example, if we have 5 hyperparameters, and we test 10 values for each hyperparameter, the training procedure needs to be executed $10^5 = 100000$ times to test its accuracy. Although one does not run such testing procedures to completion, the number of runs is still too large to be reasonably executed for most settings of even modest size. Therefore, a commonly used trick is to first work with coarse grids. Later, when one narrows down to a particular range of interest, finer grids are used. One must be careful when the optimal hyperparameter selected is at the edge of a grid range, because one would need to test beyond the range to see if better values exist.

The testing approach may at times be too expensive even with the coarse-to-fine-grained process. It has been pointed out [37] that grid-based hyperparameter exploration is not necessarily the best choice. In some cases, it makes sense to randomly sample the hyperparameters uniformly within the grid range. As in the case of grid ranges, one can perform multi-resolution sampling, where one first samples in the full grid range. One then creates a new set of grid ranges that are geometrically smaller than the previous grid ranges and centered around the optimal parameters from the previously explored samples. Sampling is repeated on this smaller box and the entire process is iteratively repeated multiple times to refine the parameters.

Another key point about sampling many types of hyperparameters is that the *logarithms* of the hyperparameters are sampled uniformly rather than the hyperparameters themselves. Two examples of such parameters include the regularization rate and the learning rate. For example, instead of sampling the learning rate $\alpha$ between 0.1 and 0.001, we first sample $\log(\alpha)$ uniformly between $-1$ and $-3$, and then exponentiate it to the power of 10. It is

more common to search for hyperparameters in the logarithmic space, although there are some hyperparameters that should be searched for on a uniform scale.

Finally, a key point about large-scale settings is that it is sometimes impossible to run these algorithms to completion because of the large training times involved. For example, a single run of a convolutional neural network in image processing might take a couple of weeks. Trying to run the algorithm over many different choices of parameter combinations is impractical. However, one can often obtain a reasonable estimate of the broader behavior of the algorithm in a short time. Therefore, the algorithms are often run for a certain number of epochs to test the progress. Runs that are obviously poor or diverge from convergence can be quickly killed. In many cases, multiple threads of the process with different hyperparameters can be run, and one can successively terminate or add new sampled runs. In the end, only one winner is allowed to train to completion. Sometimes a few winners may be allowed to train to completion, and their predictions will be averaged as an ensemble.

The general problem of searching for optimal neural network configurations is referred to as *Neural Architecture Search*, which has a much wider scope than what is discussed in this chapter [115]. Examples include *reinforcement learning* (cf. section 11.8.5) for architectural search or *Bayesian optimization* [41, 317] for parameter search. For smaller networks, it is possible to use libraries such as *Hyperopt* [637], *Spearmint* [639], and *SMAC* [638].

### 2.7.2 Feature Preprocessing

The feature processing methods used for neural network training are not very different from those in other machine learning algorithms. There are two forms of feature preprocessing used in machine learning algorithms:

1. *Additive preprocessing and mean-centering:* In many algorithms, it can be useful to mean-center the data in order to remove certain types of bias effects. Many algorithms in traditional machine learning (such as principal component analysis) also work with the assumption of mean-centered data. In such cases, a vector of column-wise means is subtracted from each data point. Mean-centering is often paired with *standardization*, which is discussed in the section of feature normalization.

   A second type of pre-processing is used when it is desired for all feature values to be non-negative. In such a case, the absolute value of the most negative entry of a feature is added to the corresponding feature value of each data point. The latter is typically combined with min-max normalization, which is discussed below.

2. *Feature normalization:* A common type of normalization is to divide each feature value by its standard deviation. When this type of feature scaling is combined with mean-centering, the data is said to have been *standardized*. The basic idea is that each feature is presumed to have been drawn from a *standard* normal distribution with zero mean and unit variance.

   The other type of feature normalization is useful when the data needs to be scaled in the range $(0, 1)$. Let $min_j$ and $max_j$ be the minimum and maximum values of the $j$th attribute. Then, each feature value $x_{ij}$ for the $j$th dimension of the $i$th point is scaled by min-max normalization as follows:

$$x_{ij} \Leftarrow \frac{x_{ij} - min_j}{max_j - min_j} \tag{2.36}$$

Feature normalization often does ensure better performance, because it is common for the relative values of features to vary by more than an order of magnitude. In such cases, parameter learning faces the problem of ill-conditioning, in which the loss function has an inherent tendency to be more sensitive to some parameters than others. As we will see in Chapter 4, this type of ill-conditioning affects the performance of gradient descent. Therefore, it is advisable to perform the feature scaling up front.

**Whitening**

Another form of feature pre-processing is referred to as *whitening*, in which the axis-system is rotated to create a new set of *de-correlated* features, each of which is scaled to unit variance. Typically, principal component analysis is used to achieve this goal.

Principal component analysis can be viewed as the application of singular value decomposition *after* mean-centering a data matrix (i.e., subtracting the mean from each column). Let $D$ be an $n \times d$ data matrix that has already been mean-centered. Let $C$ be the $d \times d$ co-variance matrix of $D$ in which the $(i, j)$th entry is the co-variance between the dimensions $i$ and $j$. Because the matrix $D$ is mean-centered, we have the following:

$$C = \frac{D^T D}{n} \propto D^T D \tag{2.37}$$

The eigenvectors of the co-variance matrix provide the de-correlated directions in the data. Furthermore, the eigenvalues provide the variance along each of the directions. Therefore, if one uses the top-$k$ eigenvectors (i.e., largest $k$ eigenvalues) of the covariance matrix, most of the variance in the data will be retained and the noise will be removed. One can also choose $k = d$, but this will often result in the variances along the near-zero eigenvectors being dominated by numerical errors in the computation. It is a bad idea to include dimensions in which the variance is caused by computational errors, because such dimensions will contain little useful information for learning application-specific knowledge. Furthermore, the whitening process will scale each transformed feature to unit variance, which will blow up the errors along these directions. At the very least, it is advisable to use some threshold like $10^{-5}$ on the magnitude of the eigenvalues. Therefore, as a practical matter, $k$ will rarely be exactly equal to $d$. Alternatively, one can add $10^{-5}$ to each eigenvalue for regularization before scaling each dimension.

Let $P$ be a $d \times k$ matrix in which each column contains one of the top-$k$ eigenvectors. Then, the data matrix $D$ can be transformed into the $k$-dimensional axis system by post-multiplying with the matrix $P$. The resulting $n \times k$ matrix $U$, whose rows contain the transformed $k$-dimensional data points, is given by the following:

$$U = DP \tag{2.38}$$

Note that the variances of the columns of $U$ are the corresponding eigenvalues, because this is the property of the de-correlating transformation of principal component analysis. In whitening, each column of $U$ is scaled to unit variance by dividing it with its standard deviation (i.e., the square root of the corresponding eigenvalue). The transformed features are fed into the neural network. Since whitening might reduce the number of features, this type of preprocessing might also affect the architecture of the network, because it reduces the number of inputs.

One important aspect of whitening is that one might not want to make a pass through a large data set to estimate its covariance matrix. In such cases, the covariance matrix

and columnwise means of the original data matrix can be estimated on a sample of the data. The $d \times k$ eigenvector matrix $P$ is computed in which the columns contain the top-$k$ eigenvectors. Subsequently, the following steps are used for each data point: (i) The mean of each column is subtracted from the corresponding feature; (ii) Each $d$-dimensional row vector representing a training data point (or test data point) is post-multiplied with $P$ to create a $k$-dimensional row vector; (iii) Each feature of this $k$-dimensional representation is divided by the square-root of the corresponding eigenvalue.

The basic idea behind whitening is that data is assumed to be generated from an independent Gaussian distribution along each principal component. By whitening, one assumes that each such distribution is a *standard* normal distribution, and provides equal importance to the different features. Note that after whitening, the scatter plot of the data will roughly have a spherical shape, even if the original data is elliptically elongated with an arbitrary orientation. The idea is that the uncorrelated concepts in the data have now been scaled to equal importance (on an a priori basis), and the neural network can decide which of them to emphasize in the learning process. Another issue is that when different features are scaled very differently, the activations and gradients will be dominated by the "large" features in the initial phase of learning. This might hurt the relative learning rate of some important weights in the network.

### 2.7.3 Initialization

Initialization is particularly important in neural networks because of the stability issues associated with neural network training. As you will learn in section 4.3.2, neural networks often exhibit stability problems in the sense that the activations of each layer either become successively weaker or successively stronger. The effect is exponentially related to the depth of the network, and is therefore particularly severe in deep networks. One way of ameliorating this effect to some extent is to choose good initialization points in such a way that the gradients are stable across the different layers.

One possible approach to initialize the weights is to generate random values from a Gaussian distribution with zero mean and a small standard deviation, such as $10^{-2}$. Typically, this will result in small random values that are both positive and negative. One problem with this initialization is that it is not sensitive to the number of inputs to a specific neuron. For example, if one neuron has only 2 inputs and another has 100 inputs, the output of the former is far more sensitive to the average weight because of the additive effect of more inputs (which will show up as a much larger gradient). In general, it can be shown that the variance of the outputs linearly scales with the number of inputs, and therefore the standard deviation scales with the square root of the number of inputs. To balance this fact, each weight is initialized to a value drawn from a normal distribution with zero mean and standard deviation $\sqrt{1/r_{in}}$, where $r_{in}$ is the number of inputs to that neuron. Bias neurons are always initialized to zero weight. Alternatively, one can initialize the weight to a value that is uniformly distributed in $[-1/\sqrt{r_{in}}, 1/\sqrt{r_{in}}]$.

More sophisticated rules for initialization consider the fact that the nodes in different layers interact with one another to contribute to output sensitivity. Therefore a larger outdegree also contributes to an increasing magnitude of the activations. Let $r_{in}$ and $r_{out}$ respectively be the fan-in and fan-out for a particular neuron. One suggested initialization rule, referred to as *Xavier initialization* or *Glorot initialization* is to use a normal distribution with zero mean and standard deviation of $\sqrt{2/(r_{in} + r_{out})}$.

An important consideration in using randomized methods is that *symmetry breaking* is important. if all weights are initialized to the same value (such as 0), all updates will move

in lock-step in a layer. As a result, identical features will be created by the neurons in a layer. It is important to have a source of asymmetry among the neurons to begin with.

## 2.8 Backpropagation Is Interpretable

One of the common refrains about neural networks has been their lack of interpretability [99]. However, it turns out that one can use backpropagation in order to determine the features that contribute the most to the classification of a particular test instance. This provides the analyst with an understanding of the relevance of each feature to classification. This approach also has the useful property that it can be used for feature selection [422]. Consider a test instance $\overline{X} = [x_1, \ldots x_d]$, for which the multilabel output scores of the neural network are $o_1 \ldots o_k$. Furthermore, let the output of the winning class among the $k$ outputs be $o_m$, where $m \in \{1 \ldots k\}$. The goal is to identify the features that are most sensitive to the classification of this test instance. Therefore, for each attribute $x_i$, we would like to determine the sensitivity of the output $o_m$ to $x_i$, which amounts to computing the absolute magnitude of $\frac{\partial o_m}{\partial x_i}$. The sign of this derivative also tells us whether increasing $x_i$ slightly from its current value increases or decreases the score of the winning class. For classes other than the winning class, the derivative also provides some understanding of the sensitivity, but this additional sensitivity is secondary in importance to that of the winning class, particularly when the number of classes is large. The value of $\frac{\partial o_m}{\partial x_i}$ can be computed with a straightforward application of node-to-node derivative computation (cf. section 2.3.1) all the way to the input layer rather than focusing to loss-to-weight derivatives.

One can also use this approach for feature selection by aggregating the absolute value of the gradient over all classes and all correctly classified training instances. The features with the largest aggregate sensitivity over the whole training data are the most relevant. Strictly speaking, one does not need to aggregate this value over all classes, but one can simply use only the winning class for correctly classified training instances. However, the original work in [422] aggregates this value over all classes and all instances.

These types of interpretation become particularly intuitive in computer vision [482] (see section 9.5.1 of Chapter 9), because the visual effects are sometimes spectacular. For example, for an image of a dog, the analysis will tell us which features (i.e., pixels) result in the image being considered a dog. As a result, we can create a black-and-white *saliency image* in which the portion corresponding to a dog is emphasized in light color against a dark background (cf. Figure 9.13 of Chapter 9). Backpropagation can also be used to determine the sensitivity of hidden features with respect to the original data features. For any given hidden feature $h_m$, the value of $\partial h_m / \partial x_i$ tells us how sensitive the hidden feature might be to a particular input. For some types of data like image data, this sensitivity can be plotted on the pixels, which provides a high level of understanding of how features are engineered. For example, each hidden feature often corresponds to a frequently occurring shape, which becomes successively complex in later layers (cf. Figure 9.16 in Chapter 9).

## 2.9 Summary

This chapter introduces the backpropagation algorithm in detail. A general view of backpropagation is discussed in the context of computational graphs. Subsequently, this general view is followed up by a discussion of how it is used in neural networks. Backpropagation only provides the gradient steps that are needed for effective learning. This algorithm

needs to be paired with computational algorithms for descent. Many of these algorithms are discussed in Chapter 4.

## 2.10 Bibliographic Notes and Software Resources

The original idea of backpropagation was based on idea of differentiation of composition of functions as developed in control theory [54, 247] under the ambit of *automatic differentiation*. The adaptation of these methods to neural networks was proposed by Paul Werbos in his PhD thesis in 1974 [543], although a more modern form of the algorithm was proposed by Rumelhart *et al.* in 1986 [424]. A discussion of the history of the backpropagation algorithm may be found in the book by Paul Werbos [544].

A discussion of algorithms for hyperparameter optimization in neural networks and other machine learning algorithms may be found in [36, 38, 507]. The random search method for hyperparameter optimization is discussed in [37]. The use of *Bayesian optimization* for hyperparameter tuning is discussed in [41, 317, 478]. Numerous libraries are available for Bayesian tuning such as *Hyperopt* [637], *Spearmint* [639], and *SMAC* [638].

The rule that the initial weights should depend on both the fan-in and fan-out of a node in proportion to $\sqrt{2/(r_{in} + r_{out})}$ is based on [147]. The analysis of initialization methods for rectifier neural networks is provided in [193]. Evaluations and analysis of the effect of feature preprocessing on neural network learning may be found in [284, 551]. The use of rectifier linear units for addressing some of the training challenges is discussed in [148].

### Software Resources

The backpropagation algorithm is supported by almost all deep learning frameworks like *Caffe* [596], *Torch* [597], *Theano* [598], and *TensorFlow* [599]. *Caffe* is also available in Python and MATLAB. All these frameworks provide numerous optimization algorithms that are discussed in Chapter 4.

## 2.11 Exercises

**1.** Consider the following recurrence:

$$(x_{t+1}, y_{t+1}) = (f(x_t, y_t), g(x_t, y_t)) \tag{2.39}$$

Here, $f()$ and $g()$ are multivariate functions.

(a) Derive an expression for $\frac{\partial x_{t+2}}{\partial x_t}$ in terms of only $x_t$ and $y_t$.

(b) Can you draw an architecture of a neural network corresponding to the above recursion for $t$ varying from 1 to 5? Assume that the neurons can compute any function you want.

**2.** Consider a two-input neuron that multiplies its two inputs $x_1$ and $x_2$ to obtain the output $o$. Let $L$ be the loss function that is computed at $o$. Suppose that you know that $\frac{\partial L}{\partial o} = 5$, $x_1 = 2$, and $x_2 = 3$. Compute the values of $\frac{\partial L}{\partial x_1}$ and $\frac{\partial L}{\partial x_2}$.

**3.** Consider a neural network with three layers including an input layer. The first (input) layer has four inputs $x_1$, $x_2$, $x_3$, and $x_4$. The second layer has six hidden units corresponding to all pairwise multiplications. The output node $o$ simply adds the values

in the six hidden units. Let $L$ be the loss at the output node. Suppose that you know that $\frac{\partial L}{\partial o} = 2$, and $x_1 = 1$, $x_2 = 2$, $x_3 = 3$, and $x_4 = 4$. Compute $\frac{\partial L}{\partial x_i}$ for each $i$.

**4.** How does your answer to the previous question change when the output $o$ is computed as a maximum of its six inputs rather than its sum?

**5.** The chapter discusses (cf. Table 2.1) how one can perform a backpropagation of an arbitrary function by using the multiplication with the Jacobian matrix. Discuss why one must be careful in using this matrix-centric approach.[Hint: Compute the Jacobian with respect to sigmoid function]

**6.** Consider the loss function $L = x^2 + y^{10}$. Implement a simple steepest-descent algorithm to plot the coordinates as they vary from the initialization point to the optimal value of 0. Consider two different initialization points of $(0.5, 0.5)$ and $(2, 2)$ and plot the trajectories in the two cases at a constant learning rate. What do you observe about the behavior of the algorithm in the two cases?

**7.** Consider the softmax activation function in the output layer, in which real-valued outputs $v_1 \ldots v_k$ are converted into probabilities as follows (according to Equation 2.17):

$$o_i = \frac{\exp(v_i)}{\sum_{j=1}^{k} \exp(v_j)} \quad \forall i \in \{1, \ldots, k\}$$

(a) Show that the value of $\frac{\partial o_i}{\partial v_j}$ is $o_i(1 - o_i)$ when $i = j$. In the case that $i \neq j$, show that this value is $-o_i o_j$.

(b) Use the above result to show the correctness of Equation 2.19:

$$\frac{\partial L}{\partial v_i} = o_i - y_i$$

Assume that we are using the cross-entropy loss $L = -\sum_{i=1}^{k} y_i \log(o_i)$, where $y_i \in \{0, 1\}$ is the one-hot encoded class label over different values of $i \in \{1 \ldots k\}$.

**8.** Consider a variation of neural architecture in Figure 2.12(b) in which a skip-connection exists between the input $\overline{x}$ and hidden layer $\overline{h}_2$ with recurrence equations as follows:

$$\overline{h}_1 = \text{ReLU}(W_1 \overline{x}), \quad \overline{h}_2 = \text{ReLU}(W_2 \overline{x} + W_3 \overline{h}_1), \quad y = W_4 \overline{h}_2$$

Here, $W_1$, $W_2$, $W_3$, and $W_4$ are matrices of appropriate size. Use the vector-centric backpropagation algorithm to derive the expressions for $\frac{\partial y}{\partial \overline{h}_2}$, $\frac{\partial y}{\partial \overline{h}_1}$, and $\frac{\partial y}{\partial \overline{x}}$ in terms of the matrices and activation values in intermediate layers.

**9. Interpretability:** Consider a neural network for a binary classification problem in which the output $o$ (created by a sigmoid activation) indicates the probability that the class label is $+1$. Discuss how you can use backpropagation to find which hidden units contribute heavily to the classification of a particular test instance.

**10.** Let $f(x)$ be defined as follows:

$$f(x) = \sin(x) + \cos(x)$$

Consider the function $f(f(f(f(x))))$. Write this function in closed form to obtain an appreciation of the awkwardly long function. Evaluate the derivative of this function at $x = \pi/3$ radians by using a computational graph abstraction.

**11.** Consider a multilayer neural network with positive inputs in which each activation function produces a positive output (e.g., sigmoid). Show that all weights feeding into the same neuron will increase or decrease together in a gradient descent step. [This is one of the reasons that tanh has better convergence properties than the sigmoid.]

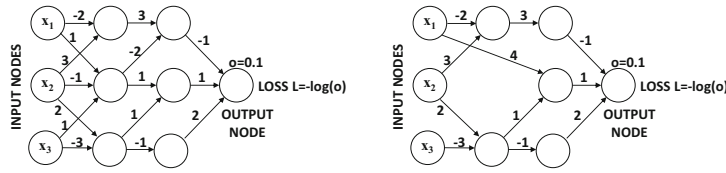
**12. Forward Mode Differentiation:** The backpropagation algorithm needs to compute node-to-node derivatives of *output* nodes with respect to all other nodes, and therefore computing gradients in the backwards direction makes sense (see pseudocode on page 38). However, one can also compute the node-to-node derivatives $\frac{\partial x}{\partial s_i}$ of each node $x$ with respect to *source* (input) nodes $s_1 \ldots s_k$. Propose a variation of the pseudocode of page 38 that computes node-to-node gradients in the forward direction.

**13. All-pairs node-to-node derivatives:** Let $y(i)$ be the variable in node $i$ in a directed acyclic computational graph containing $n$ nodes and $m$ edges. Consider the case where one wants to compute $S(i, j) = \frac{\partial y(j)}{\partial y(i)}$ for all pairs of nodes in a computational graph, so that at least one directed path exists from node $i$ to node $j$. Propose an $O(n^2 m)$ algorithm for all-pairs derivative computation. [Hint: The pathwise aggregation lemma is helpful. First compute $S(i, j, t)$, which is the portion of $S(i, j)$ in the lemma belonging to paths of length exactly $t$, and set up an iterative relationship for increasing $t$.]

**14.** Use the pathwise aggregation lemma to compute the derivative of $y(10)$ with respect to each of $y(1)$, $y(2)$, and $y(3)$ as an algebraic expression (cf. Figure 2.7). You should get the same derivative as obtained in the text of the chapter.

**15.** Consider the computational graph of Figure 2.6. For a particular numerical input $x = a$, you find the unusual situation that the value $\frac{\partial y(j)}{\partial y(i)}$ is 0.3 for each and every edge $(i, j)$ in the network. Compute the numerical value of the partial derivative of the output with respect to the input $x$ (at $x = a$). Show the computations using both the pathwise aggregation lemma and the backpropagation algorithm.

**16.** Consider the computational graph of Figure 2.6. The upper node in each layer computes $\sin(x + y)$ and the lower node in each layer computes $\cos(x + y)$ with respect to its two inputs. For the first hidden layer, there is only a single input $x$, and therefore the values $\sin(x)$ and $\cos(x)$ are computed. The final output node computes the product of its two inputs. The single input $x$ is 1 radian. Compute the numerical value of the partial derivative of the output with respect to the input $x$ (at $x = 1$ radian). Show computations using both pathwise aggregation and the backpropagation.




(a) Exercise 17 (b) Exercise 18

Figure 2.16: Computational graphs for Exercises 17 and 18

**17.** Consider the computational graph shown in Figure 2.16(a), in which the local derivative $\frac{\partial y(j)}{\partial y(i)}$ is shown for each edge $(i, j)$, where $y(k)$ denotes the activation of node $k$. The output $o$ is 0.1, and the loss $L$ is given by $-\log(o)$. Compute the value of $\frac{\partial L}{\partial x_i}$ for each input $x_i$ using both path-wise aggregation and backpropagation.

**18.** Consider the computational graph shown in Figure 2.16(b), in which the local derivative $\frac{\partial y(j)}{\partial y(i)}$ is shown for each edge $(i, j)$, where $y(k)$ denotes the activation of node $k$. The output $o$ is 0.1, and the loss $L$ is given by $-\log(o)$. Compute the value of $\frac{\partial L}{\partial x_i}$ for each input $x_i$ using both path-wise aggregation and backpropagation.


# Chapter 3

# Machine Learning with Shallow Neural Networks

"Simplicity is the ultimate sophistication."– Leonardo da Vinci

## 3.1 Introduction

Conventional machine learning often uses optimization and gradient-descent methods for learning parameterized models. Neural networks are also parameterized models that are learned with continuous optimization methods. In all these cases, the machine learning model constructs a loss function in closed form, and gradient descent is used in order to learn the optimal parameters. This chapter will show that a wide variety of optimization-centric methods in machine learning can be captured with *very simple* neural network architectures containing one or two layers. In fact, neural networks can be viewed as more powerful generalizations of these simple models, in which the depth creates a loss function that is difficult to express in closed form, but is also more powerful in being to learn rich structure from the data. It is useful to show these parallels early on, as this allows the understanding of the design of a deep network as a composition of the basic units that one often uses in machine learning. Furthermore, showing this relationship provides an appreciation of the specific way in which traditional machine learning is different from neural networks, and of the cases in which one can hope to do better with neural networks.

Complex neural architectures are often an overkill in instances where data availability is limited. Additionally, it is easier to optimize traditional machine learning models in data-lean settings. On the other hand, neural networks have an increasing advantage with more data because they retain the flexibility to model more complex functions where warranted. Figure 3.1 illustrates this point in which it is shown that the advantage of neural networks increases with data availability.



C. C. Aggarwal, *Neural Networks and Deep Learning*,
https://doi.org/10.1007/978-3-031-29642-0_3

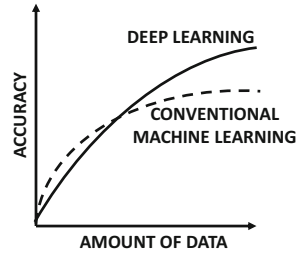



Figure 3.1: Re-visiting Figure 1.2: The effect of increased data availability on accuracy.

One way of viewing deep learning models is as a network connecting simpler computational units like the perceptron. Two examples of such simpler models are *linear regression* and *logistic regression*. A computational graph of these types of units obviously looks like a composition of the prediction functions used in simpler machine learning models. Although many neural networks can be viewed in this way, this point of view does not fully capture the complexity and the style of thinking involved in deep learning models. For example, several models (such as recurrent neural networks or convolutional neural networks) perform this stacking in a particular way with a *domain-specific understanding* of the input data. The ability to put together the basic units in a clever way is a key architectural skill required by practitioners in deep learning. Nevertheless, it is also important to learn the properties of the basic models in machine learning, since they are used repeatedly in deep learning as elementary units of computation. This chapter will, therefore, explore these basic models.

This chapter will primarily discuss two classes of models for machine learning:

1. *Supervised models:* The supervised models discussed in this chapter primarily correspond to linear models and their variants. These include methods like least-squares regression, support vector machines, and logistic regression. Multiclass variants of these models will also be studied.

2. *Unsupervised models:* The unsupervised models discussed in this chapter primarily correspond to dimensionality reduction and matrix factorization.

This chapter assumes that the reader has a basic familiarity with the classical machine learning models. Nevertheless, a brief overview of each model will also be provided to the uninitiated reader.

### Chapter Organization

The next section will discuss some basic models for classification and regression, such as least-squares regression, binary Fisher discriminant, support vector machine, and logistic regression. The multiway variants of these models will be discussed in section 3.3. The use of *autoencoders* for unsupervised learning is discussed in section 3.4. Several domain-specific applications are also discussed, such as recommender systems (cf. section 3.5), text embeddings (cf. section 3.6), and graph embeddings (cf. section 3.7). A summary is given in section 3.8.

## 3.2 Neural Architectures for Binary Classification Models

In this section, we will discuss some basic architectures for machine learning models such as least-squares regression and classification. As we will see, the corresponding neural architectures are minor variations of the perceptron model in machine learning. The main difference is in the choice of the activation function used in the final layer, and the loss function used on these outputs. This will be a recurring theme throughout this chapter, where we will see that small changes in neural architectures can result in distinct models from traditional machine learning.

Throughout this section, we will work with a single-layer network with $d$ input nodes and a single output node. The coefficients of the connections from the $d$ input nodes to the output node are denoted by $\overline{W} = [w_1 \ldots w_d]^T$. Therefore, the weight vector is assumed to be a column vector. On the other hand, the training instances are rows of data matrices, and therefore each training instance $\overline{X}_i$ of feature variables is assumed to be a row vector. Furthermore, the bias will not be explicitly shown because it can be seamlessly modeled as the coefficient of an additional dummy input with a constant value of 1.

### 3.2.1 Revisiting the Perceptron

Let $[\overline{X}_i, y_i]$ be a training instance, in which the observed value $y_i$ is predicted from the row vector $\overline{X}_i$ of feature variables using the following relationship:

$$\hat{y}_i = \text{sign}(\overline{W} \cdot \overline{X}_i^T) \tag{3.1}$$

Here, $\overline{W}$ is the $d$-dimensional column vector $[w_1, \ldots w_d]^T$ learned by the perceptron. Note the circumflex on top of $\hat{y}_i$ to indicate that it is a predicted value rather than an observed value. In general, the goal of training is to ensure that the prediction $\hat{y}_i$ is as close as possible to the observed value $y_i$. The gradient-descent steps of the perceptron are focused on reducing the number of misclassifications, and therefore the updates are proportional to the difference $(y_i - \hat{y}_i)$ between the observed and predicted values based on the discussion in Figure 1.3 of Chapter 1:

$$\overline{W} \Leftarrow \overline{W} + \alpha(y_i - \hat{y}_i)\overline{X}_i^T \tag{3.2}$$

A gradient-descent update that is proportional to the difference between the observed and predicted values is often caused by a squared-loss function such as $(y_i - \hat{y}_i)^2$ (see next section). However, this seemingly obvious connection to squared loss is misleading because both $y_i$ and $\hat{y}_i$ are discrete values from $\{-1, +1\}$, and the true loss function is different from the non-differentiable value $(y_i - \hat{y}_i)^2$.

The perceptron is one of the few learning models in which the loss function was reverse engineered well after the (heuristically defined) "gradient descent" updates were proposed. To obtain the *differentiable* loss function, it is helpful to rewrite the perceptron update with the use of an indicator function $I(\cdot) \in \{0, 1\}$ that takes on 1 when the misclassification condition (i.e., $y_i\hat{y}_i < 0$) in its argument is satisfied:

$$\overline{W} \Leftarrow \overline{W} + \alpha y_i \overline{X}_i^T \, [I(y_i\hat{y}_i < 0)] \tag{3.3}$$

This rewrite from Equation 3.2 to Equation 3.3 uses the fact that $y_i = (y_i - \hat{y}_i)/2$ for misclassified points when $y_i, \hat{y}_i \in \{-1, +1\}$. Furthermore, one can absorb a constant factor

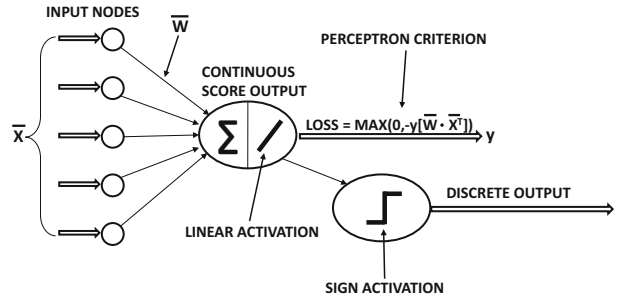



Figure 3.2: An extended architecture of the perceptron with both continuous and discrete outputs for training and prediction

of 2 within the learning rate. The update can be shown to be equivalent to gradient descent with the loss function $L_i$ (specific to the $i$th training example) as follows:

$$L_i = \max\{0, -y_i \hat{y}_i\} = \max\{0, -y_i(\overline{W} \cdot \overline{X}_i^T)\} \tag{3.4}$$

This loss function is referred to as the *perceptron criterion*, which is correspondingly reflected in Figure 3.3(a). It is easy to show that the update of Equation 3.3 is a stochastic gradient-descent update:

$$\overline{W} \Leftarrow \overline{W} - \alpha \frac{\partial L_i}{\partial \overline{W}} \tag{3.5}$$

Therefore, the perceptron updates are also stochastic gradient-descent updates, although the loss function was reverse engineered historically after the (heuristic) updates were proposed.

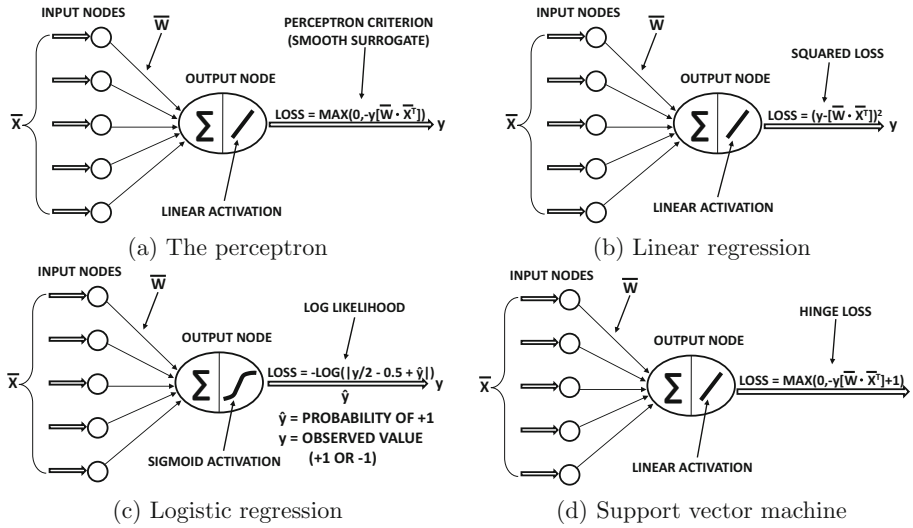
The perceptron architecture in Figure 3.3(a) uses *linear* activations to compute the continuous loss function, which is different from the sign activation used in the perceptron architecture of Chapter 1. The former is referred to as the *training architecture*, whereas the latter is referred to as the *predictive architecture*. Modern conventions on neural networks almost always depict training architectures by default. The perceptron is somewhat of an exception in often showing the predictive architecture because of the way in which its updates were initially proposed without the use of a formal loss function and heuristically defined updates with discrete outputs. One can, in fact, create an extended architecture for the perceptron (cf. Figure 3.2), in which both continuous values (for training) and discrete values (for prediction) are output. Throughout the remainder of this book, we focus on training architectures whenever an illustration of a neural network is provided.

### 3.2.2 Least-Squares Regression

In least-squares regression, the training data contains $n$ different training pairs $[\overline{X}_1, y_1] \ldots [\overline{X}_n, y_n]$, where each $\overline{X}_i$ is a $d$-dimensional row-vector of the feature variables, and each $y_i$ is a *real-valued* target. The fact that the target is real-valued is important, because the underlying problem is then referred to as *regression* rather than *classification*. Least-squares regression is the oldest of all learning problems, and the original formulation is owed to several nineteenth century mathematicians such as Legendre and Gauss (well before the advent of computers).

In least-squares regression, the target variable is related to the feature variables using the following relationship:

$$\hat{y}_i = \overline{W} \cdot \overline{X}_i^T \tag{3.6}$$




(a) The perceptron (b) Linear regression

(c) Logistic regression (d) Support vector machine

Figure 3.3: Basic machine learning models are small variants of one another

Note the transposition of the training instance since it is a row vector, whereas $\overline{W}$ is a column vector. Note the presence of the circumflex on top of $\hat{y}_i$ to indicate that it is a predicted value. The error of the prediction, $e_i$, is given by $e_i = (y_i - \hat{y}_i)$. Here, $\overline{W} = [w_1 \ldots w_d]^T$ is a $d$-dimensional column vector of weights that needs to be learned so as to minimize the total squared error on the training data, which is $\sum_{i=1}^{n} e_i^2$. The portion of the loss that is specific to the $i$th training instance is given by the following:

$$L_i = e_i^2 = (y_i - \hat{y}_i)^2 \tag{3.7}$$

This loss can be simulated with the use of an architecture similar to the perceptron except that we use squared loss instead of the perceptron criterion. This architecture is shown in Figure 3.3(b), whereas the perceptron architecture is shown in Figure 3.3(a).

As in the perceptron algorithm, the stochastic gradient-descent steps are determined by computing the gradient of $e_i^2$ with respect to $\overline{W}$, when the training pair $(\overline{X}_i, y_i)$ is presented to the neural network. This gradient can be computed as follows:

$$\frac{\partial e_i^2}{\partial \overline{W}} = -e_i \overline{X}_i^T \tag{3.8}$$

Therefore, the gradient-descent updates for $\overline{W}$ are computed using the above gradient and step-size $\alpha$:

$$\overline{W} \Leftarrow \overline{W} + \alpha e_i \overline{X}_i^T$$

One can rewrite the above update as follows:

$$\overline{W} \Leftarrow \overline{W} + \alpha (y_i - \hat{y}_i) \overline{X}_i^T \tag{3.9}$$

the update above looks identical to the perceptron update of Equation 3.2. The updates are, however, not exactly identical because of how the predicted value $\hat{y}_i$ is computed in the two cases. In the case of the perceptron, the sign function is applied to $\overline{W} \cdot \overline{X}_i^T$ in order to compute

the binary value $\hat{y}_i$ and therefore the error $(y_i - \hat{y}_i)$ can only be drawn from $\{-2, +2\}$. In least-squares regression, the prediction $\hat{y}_i$ is a real value without the application of the sign function. This idea can also be carried on binary targets with Widrow-Hoff learning.

#### 3.2.2.1 Widrow-Hoff Learning

One can apply least-squares regression directly to minimize the squared distance of the *real-valued* prediction $\hat{y}_i$ from the observed binary targets $y_i \in \{-1, +1\}$, and this special case of regression is referred to as *least-squares classification*. The gradient-descent update is the same as the one shown in Equation 3.9, which *looks* identical to Equation 3.2 for the perceptron. However, the least-squares classification method is different from the perceptron, because the perceptron uses $\hat{y}_i = \text{sign}\{\overline{W} \cdot \overline{X}_i^T\}$ in the update, whereas least-squares classification uses $\hat{y}_i = \overline{W} \cdot \overline{X}_i^T$. The direct application of least-squares regression to binary targets is referred to as *Widrow-Hoff learning*. The Widrow-Hoff learning rule was proposed in 1960. However, the method was not a fundamentally new one, as it is a direct application of the ancient least-squares regression method to binary targets.

The neural architecture for classification with the Widrow-Hoff method is illustrated in Figure 3.3(b). The gradient-descent steps in both the perceptron and the Widrow-Hoff method would be given by Equation 3.9, except for differences in how $(y_i - \hat{y}_i)$ is computed. In the case of the perceptron, the value $\hat{y}_i$ is an integer, and therefore the $(y_i - \hat{y}_i)$ would always be drawn from $\{-2, +2\}$. In the case of the Widrow-Hoff method, these errors can be arbitrary real values, since $\hat{y}_i$ is set to $\overline{W} \cdot \overline{X}_i^T$ without using the sign function. The learning rule of Equation 3.9, when applied to binary targets in $\{-1, +1\}$, can be alternatively referred to as least-squares classification, least mean-squares algorithm (LMS), the Widrow-Hoff learning rule, delta rule, or Adaline. The family of least-squares classification methods has been rediscovered several times in the literature under different names and with different motivations.

The loss function of the Widrow-Hoff method can also be rewritten in a different form because of its binary responses:

$$L_i = (y_i - \hat{y}_i)^2 = \underbrace{y_i^2}_{1}(y_i - \hat{y}_i)^2$$
$$= (\underbrace{y_i^2}_{1} - \hat{y}_i y_i)^2 = (1 - \hat{y}_i y_i)^2$$

This type of encoding is possible only when the target variable $y_i$ is drawn from $\{-1, +1\}$ because we can use $y_i^2 = 1$. It is helpful to convert the Widrow-Hoff objective function to this (equivalent) form because it can be more easily related to other objective functions for binary targets (like the perceptron or the support vector machine). In fact, the term $\hat{y}_i y_i$ appears repeatedly in various loss functions for binary targets. For example, the perceptron criterion is simply $\max\{-\hat{y}_i y_i, 0\}$, whereas the loss function of the support vector machine (see next section) is $\max\{-\hat{y}_i y_i + 1, 0\}$. Almost all the binary classification models discussed in this chapter are modified versions of least-squares classification, which address some key weaknesses in the least-squares classification loss function. These weaknesses are a consequence of blindly treating binary labels as real targets, and will be discussed over the course of this chapter in the context of the better loss functions used by other algorithms.

The gradient-descent updates (cf. Equation 3.9) of least-squares regression can be rewritten slightly for Widrow-Hoff learning because of binary response variables:

$$\begin{aligned}\overline{W} &\Leftarrow \overline{W} + \alpha(y_i - \hat{y}_i)\overline{X}_i^T \quad \text{[For numeric as well as binary responses]} \\ &= \overline{W} + \alpha y_i(1 - y_i\hat{y}_i)\overline{X}_i^T \quad \text{[Only for binary responses, since } y_i^2 = 1\text{]}\end{aligned}$$

The second form of the update is helpful in relating it to perceptron, which uses an indicator function value of 1 when $1 - y_i\hat{y}_i > 0$ in lieu of $(1 - y_i\hat{y}_i)$.

#### 3.2.2.2 Closed Form Solutions

The special case of least-squares regression and classification is solvable in closed form (without gradient-descent) by using the *pseudo-inverse* of the $n \times d$ training data matrix $D$, whose rows are $\overline{X}_1 \ldots \overline{X}_n$. Let the $n$-dimensional column vector of dependent variables be denoted by $\overline{y} = [y_1 \ldots y_n]^T$. The pseudo-inverse of matrix $D$ is defined as follows:

$$D^+ = (D^T D)^{-1} D^T \tag{3.10}$$

Then, the optimal column-vector $\overline{W}$ can be shown [6] to be the following:

$$\overline{W} = D^+ \overline{y} \tag{3.11}$$

If regularization is incorporated, the coefficient vector $\overline{W}$ is given by the following:

$$\overline{W} = (D^T D + \lambda I)^{-1} D^T \overline{y} \tag{3.12}$$

Here, $\lambda > 0$ is the regularization parameter. However, it is more common to use gradient-descent methods like the Widrow-Hoff rule rather than using the matrix form of the solution.

### 3.2.3 Support Vector Machines

Consider the training data set of $n$ instances denoted by $[\overline{X}_1, y_1]$, $[\overline{X}_2, y_2]$, $\ldots$ $[\overline{X}_n, y_n]$. Each $\overline{X}_i$ is a row vector of feature variables and $y_i$ is the class variable drawn from $\{-1, +1\}$. The neural architecture of the support-vector machine is identical to that of least-squares classification (i.e., Widrow-Hoff method), and the only difference is in the choice of loss function. As in the case of least-squares classification, the prediction $\hat{y}_i$ for the training point $\overline{X}_i$ is obtained by applying the identity activation function on $\overline{W} \cdot \overline{X}_i^T$.

$$\hat{y}_i = \overline{W} \cdot \overline{X}_i^T$$

Here, $\overline{W} = [w_1, \ldots w_d]^T$ contains the column vector of $d$ weights for the $d$ different inputs into the single-layer network. Therefore, the output of the neural network is $\hat{y}_i = \overline{W} \cdot \overline{X}_i^T$ for computing the loss function, although a test instance is predicted by applying the sign function to the output.

The loss function $L_i$ for the $i$th training instance in the support-vector machine is defined as follows:

$$L_i = \max\{0, 1 - y_i\hat{y}_i\} = \max\{0, 1 - y_i(\overline{W} \cdot \overline{X}_i^T)\} \tag{3.13}$$

This loss is referred to as the *hinge-loss*, and the corresponding neural architecture is illustrated in Figure 3.3(c). Note that instances in which $y_i\hat{y}_i$ is strongly positive (i.e., larger

than 1) are considered correct predictions, and are therefore not penalized by the loss function. This is different from the Widrow-Hoff loss of $(1 - y_i\hat{y}_i)^2$, which penalizes instances even when the prediction score is "confidently" correct about the true label. For example, the Widrow-Hoff loss function would heavily penalize an instance with true label $y_i = 1$ and prediction score $\hat{y}_i = +10^6$. The support vector machine does not penalize these instances. In fact, Hinton proposed the loss value of $[\max\{(1 - \hat{y}_i y_i), 0\}]^2$ to repair this problem in Widrow-Hoff loss [200], and it turned out to be identical to the loss function of the $L_2$-SVM (independently proposed much later).

The overall idea behind the SVM loss function is that a positive-class training instance is only penalized for the predicted score being less than 1 (i.e., the model not being correct enough in a "confident" way), and a negative-class training instance is only penalized for the prediction score being greater than -1 (i.e., not being correct enough in a "confident" way). The need to be "confident" in order to avoid penalties is referred to as the notion of *margin* in the SVM.

It is helpful to compare this loss function with the Widrow-Hoff loss value of $(1 - y_i\hat{y}_i)^2$, in which predictions are penalized for being *different* from the target values. Comparing the hinge loss of Equation 3.13 with the perceptron criterion of Equation 3.4 is even more revealing, because it suggests that the hinge loss is horizontally shifted from the perceptron criterion by one unit. Note that the perceptron does not keep the constant term of 1 on the right-hand side, whereas the hinge loss keeps this constant within the maximization function and therefore triggers updates when the model is not confident enough about correctly predicted instances. The relationship between the perceptron criterion and the hinge loss is shown in Figure 3.4. This shift is a result of the use of the notion of margin in the SVM; it is important because it gives more stability to the SVM in noisy data sets.

The stochastic gradient-descent method computes the partial derivative of the point-wise loss function $L_i$ with respect to the elements in $\overline{W}$. The gradient is computed as follows:

$$\frac{\partial L_i}{\partial \overline{W}} = \begin{cases} -y_i \overline{X}_i^T & \text{if } y_i\hat{y}_i < 1 \\ 0 & \text{otherwise} \end{cases} \tag{3.14}$$

Therefore, the stochastic gradient method samples a point and checks whether $y_i\hat{y}_i < 1$. If this is the case, an update is performed that is proportional to $y_i \overline{X}_i^T$:

$$\overline{W} \Leftarrow \overline{W} + \alpha y_i \overline{X}_i^T \, [I(y_i\hat{y}_i < 1)] \tag{3.15}$$

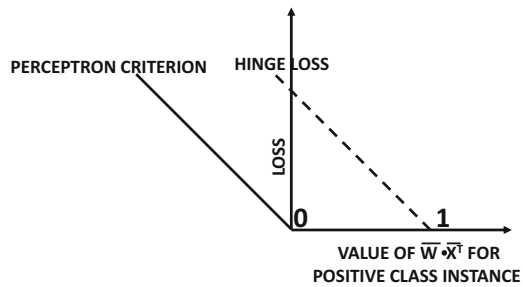



Figure 3.4: Perceptron criterion versus hinge loss

Here, $I(\cdot) \in \{0, 1\}$ is the indicator function that takes on the value of 1 when the condition in its argument is satisfied. This approach is the simplest version[1] of the primal update for SVMs [468].

The above update is *identical* to that of a perceptron (cf. Equation 3.3), except that the condition for making this update in the perceptron is $y_i \hat{y}_i < 0$, whereas the corresponding condition in the support vector machine is $y_i \hat{y}_i < 1$. Therefore, a perceptron makes the update only when a point is misclassified, whereas the support vector machine also makes updates for points that are classified correctly, albeit not very confidently. This neat relationship among their updates is a direct result of the relationships among their loss functions.

As discussed in section 1.2.2 of Chapter 1, the loss function of a perceptron has several weaknesses, which causes it to perform poorly when the data is not linearly separable. In such cases, the perceptron updates might not even converge. On the other hand, the support vector machine will usually converge, irrespective of whether the classes are linearly separable or not. For this reason, the linear support vector machine is also referred to as the *perceptron of optimal stability*.

### 3.2.4 Logistic Regression

Logistic regression is a probabilistic model that classifies the instances in terms of probabilities. Let $[\overline{X}_1, y_1], [\overline{X}_2, y_2], \ldots [\overline{X}_n, y_n]$ be a set of $n$ training pairs in which $\overline{X}_i$ is a row vector containing the $d$-dimensional features of the $i$th training point and $y_i \in \{-1, +1\}$ is a binary class variable. As in the case of a perceptron, a single-layer architecture with weight vector $\overline{W} = [w_1 \ldots w_d]^T$ is used. Logistic regression applies the soft sigmoid function to $\overline{W} \cdot \overline{X}_i^T$ in order to estimate the *probability* that $y_i$ is 1:

$$\hat{y}_i = P(y_i = 1) = \frac{1}{1 + \exp(-\overline{W} \cdot \overline{X}_i^T)} \tag{3.16}$$

Note that $P(y_i = 1)$ is 0.5 when $\overline{W} \cdot \overline{X}_i^T = 0$. Therefore, the hyperplane $\overline{W} \cdot \overline{X}_i^T = 0$ can still be considered a linear separator, although a nonzero probability of a class can be obtained on either side of the separator (with the break-even point being the separator itself). This makes logistic regression a probabilistic predictor.

We will now describe how the probabilistic training procedure. For positive samples in the training data, we want to maximize $P(y_i = 1)$ and for negative samples, we want to maximize $P(y_i = -1)$. For positive samples satisfying $y_i = 1$, one wants to maximize $\hat{y}_i$ and for negative samples satisfying $y_i = -1$, one wants to maximize $1 - \hat{y}_i$. One can write this case-wise maximization in the form of a consolidated expression of always maximizing $|y_i/2 - 0.5 + \hat{y}_i|$. The products of these probabilities must be maximized over all training instances to maximize the likelihood $\mathcal{L}$:

$$\mathcal{L} = \prod_{i=1}^{n} |y_i/2 - 0.5 + \hat{y}_i| \tag{3.17}$$

[1] In the case of the SVM, regularization is considered particularly important, although we omit it here for simplicity of comparison. The regularized update is $\overline{W} \Leftarrow \overline{W}(1 - \alpha\lambda) + \alpha y_i \overline{X}_i^T\ [I(y_i \hat{y}_i < 1)]$.

The goal is to maximize $\mathcal{L}$, which corresponds to a *maximum-likelihood model*. Using the negative logarithm of $\mathcal{L}$ converts the product-wise maximization is converted to additive minimization over training instances.

$$\mathcal{LL} = -\log(\mathcal{L}) = \sum_{i=1}^{n} \underbrace{-\log(|y_i/2 - 0.5 + \hat{y}_i|)}_{L_i} \tag{3.18}$$

Therefore, the loss function is set to $L_i = -\log(|y_i/2 - 0.5 + \hat{y}_i|)$ for each training instance. Additive forms of the objective function are particularly convenient for the types of stochastic gradient updates that are common in neural networks. The overall architecture and loss function is illustrated in Figure 3.3(d).

Let the loss for the $i$th training instance be denoted by $L_i$, which is also annotated in Equation 3.18. Then, the gradient of $L_i$ with respect to the weights in $\overline{W}$ can be computed as follows:

$$\frac{\partial L_i}{\partial \overline{W}} = -\frac{\text{sign}(y_i/2 - 0.5 + \hat{y}_i)}{|y_i/2 - 0.5 + \hat{y}_i|} \cdot \frac{\partial \hat{y}_i}{\partial \overline{W}} = -\frac{\text{sign}(y_i/2 - 0.5 + \hat{y}_i)}{|y_i/2 - 0.5 + \hat{y}_i|} \cdot \frac{\exp(-\overline{W} \cdot \overline{X}_i^T)\overline{X}_i^T}{\left[1 + \exp(-\overline{W} \cdot \overline{X}_i^T)\right]^2}$$

One can substitute the two cases from $\{-1, +1\}$ for $y_i$ and the value of $\hat{y}_i$ from Equation 3.16 to obtain the following:

$$\frac{\partial L_i}{\partial \overline{W}} = \begin{cases} -\frac{\overline{X}_i^T}{1+\exp(\overline{W} \cdot \overline{X}_i^T)} & \text{if } y_i = 1 \\ \frac{\overline{X}_i^T}{1+\exp(-\overline{W} \cdot \overline{X}_i^T)} & \text{if } y_i = -1 \end{cases}$$

Note that one can concisely write the above gradient as follows:

$$\frac{\partial L_i}{\partial \overline{W}} = -\frac{y_i \overline{X}_i^T}{1 + \exp(y_i \overline{W} \cdot \overline{X}_i^T)} = -\left[\text{Probability of mistake on } (\overline{X}_i, y_i)\right] (y_i \overline{X}_i^T) \tag{3.19}$$

Therefore, the gradient-descent updates of logistic regression are as follows:

$$\overline{W} \Leftarrow \overline{W} + \alpha \frac{y_i \overline{X}_i^T}{1 + \exp[y_i(\overline{W} \cdot \overline{X}_i^T)]} \tag{3.20}$$

Just as the perceptron and the Widrow-Hoff algorithms use (various functions of) the *magnitudes* of the mistakes to make updates, the logistic regression method uses the *probabilities* of the mistakes to make updates. This is a natural consequence of the probabilistic nature of the loss function.

It is noteworthy that the loss function of logistic regression can also be written directly in terms of the weights as follows:

$$L_i = \log(1 + \exp(-y_i[\overline{W} \cdot \overline{X}_i^T])) \tag{3.21}$$

The loss function of Equation 3.21 makes it possible to directly compare the loss functions of various models, which will be explored in the next section.

### 3.2.5 Comparison of Different Models

In order to explain the difference in loss functions between the perceptron, Widrow-Hoff learning, the support vector machine, and logistic regression, we list their loss functions in a table below:

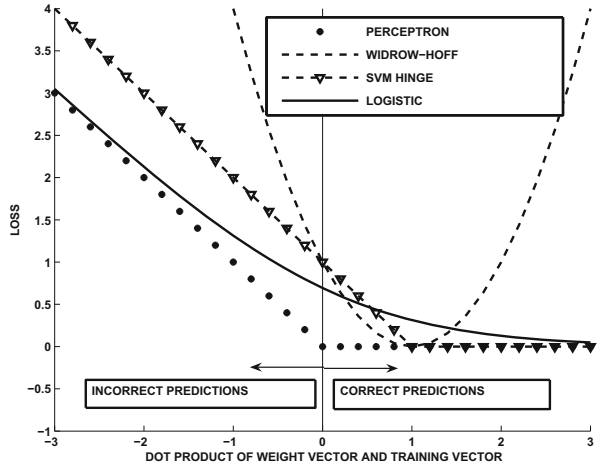



Figure 3.5: The loss functions of different variants of the perceptron for a **positive class** instance. Key observations: (i) The SVM loss is shifted from the perceptron (surrogate) loss by exactly one unit to the right; (ii) the logistic loss is a smooth variant of the SVM loss; (iii) the Widrow-Hoff loss is the only case in which points are increasingly penalized for classifying points "too correctly."

| **Model** | **Loss function** $L_i$ for $(\overline{X}_i, y_i)$ |
|---|---|
| Perceptron | $\max\{0, -y_i \cdot (\overline{W} \cdot \overline{X}_i^T)\}$ |
| Widrow-Hoff learning | $(y_i - \overline{W} \cdot \overline{X}_i^T)^2 = \{1 - y_i \cdot (\overline{W} \cdot \overline{X}_i^T)\}^2$ |
| Support vector machine (Hinge) | $\max\{0, 1 - y_i \cdot (\overline{W} \cdot \overline{X}_i^T)\}$ |
| Support vector machine (Hinton's $L_2$-Loss) [200] | $[\max\{0, 1 - y_i \cdot (\overline{W} \cdot \overline{X}_i^T)\}]^2$ |
| Logistic Regression | $\log(1 + \exp[-y_i(\overline{W} \cdot \overline{X}_i^T)])$ |

For the case of the support vector machine, we have added Hinton's $L_2$-loss to the table, which has not been discussed in this chapter. This loss function is simply the square of the hinge loss, and it is historically important because it was proposed before SVMs were discovered by the machine learning community. Therefore, the $L_2$-SVM was proposed as an unnamed loss function in a neural architecture before it was formally proposed in traditional machine learning as the "support vector machine"! It is evident that all losses are functions of $\overline{W} \cdot \overline{X}_i^T$ and they penalize the training instance in different ways when $\overline{W} \cdot \overline{X}_i^T$ is different from the observed value $y_i$.

We have also shown the value of different loss functions for a *positive* training instance $[\overline{X}_i, y_i]$ at different values of $\overline{W} \cdot \overline{X}_i^T$ in Figure 3.5. The $X$-axis shows the value of $\overline{W} \cdot \overline{X}_i^T$ and the $Y$-axis shows the loss for an instance in which $y_i = +1$. Because of the fact that $y_i = +1$, the loss function should not penalize the instance less by increasing $\overline{W} \cdot \overline{X}_i^T$, and it should ideally penalize the instance very little when $\overline{W} \cdot \overline{X}_i^T \geq 1$. This is indeed the case for the support-vector machine, in which the hinge-loss function flattens out suddenly to zero loss when $\overline{W} \cdot \overline{X}_i^T$ exceeds the target value of $+1$. In other words, only misclassified points or points that are too close to the decision boundary $\overline{W} \cdot \overline{X}^T = 0$ are penalized. The perceptron criterion is identical in shape to the hinge loss, except that it is shifted by one unit to the left. Logistic regression has a loss that has a similar shape to the SVM loss

function, although it does penalize training instances slightly for values of $\overline{W} \cdot \overline{X}_i^T \geq 1$. Notably, this loss is monotonically decreasing with increasing value of $\overline{W} \cdot \overline{X}_i^T$, which is desirable for a positive-class instance. The Widrow-Hoff method is the only case in which a positive training point is penalized for having too large a positive value of $\overline{W} \cdot \overline{X}_i^T$. In other words, the Widrow-Hoff method penalizes points for being classified "too correctly." This is a potential problem with the Widrow-Hoff objective function, and many of the loss functions in machine learning (e.g., Hinton's $L_2$-SVM loss) were motivated by the need to repair the Widrow-Hoff loss in a one-sided way.

It is noteworthy that all the derived updates in this section typically correspond to stochastic gradient-descent updates that are encountered in traditional machine learning. The updates are the same whether or not we use a neural architecture to represent the models for these algorithms. Our main point in going through this exercise is to show that rudimentary special cases of neural networks are instantiations of well-known algorithms in the machine learning literature. The key point is that with greater availability of data one can incorporate additional nodes and depth to increase the model's power, explaining the superior behavior of neural networks with larger data sets (cf. Figure 3.1).

## 3.3 Neural Architectures for Multiclass Models

All the models discussed so far in this chapter are designed for binary classification. In this section, we will discuss how one can design multiway classification models by changing the architecture of the perceptron slightly, and allowing multiple output nodes.

### 3.3.1 Multiclass Perceptron

Consider a setting with $k$ different classes. Therefore, each training instance $[\overline{X}_i, c(i)]$ is of the form where the value of $\overline{X}_i$ is a $d$-dimensional row vector of features, and $c(i) \in \{1 \ldots k\}$ is the index of the correct class for the $i$th training instance. In such a case, we would like to find $k$ columns vectors of coefficients $\overline{W}_1 \ldots \overline{W}_k$ simultaneously so that the value of $\overline{W}_{c(i)} \cdot \overline{X}_i^T$ is larger than $\overline{W}_r \cdot \overline{X}_i^T$ for any $r \neq c(i)$. This is because one always predicts the training instance to the class $r$ with the largest value of $\overline{W}_r \cdot \overline{X}_i^T$. In other words, the predicted value $hatc(i)$ of the class of the $i$th training instance is given by the following:

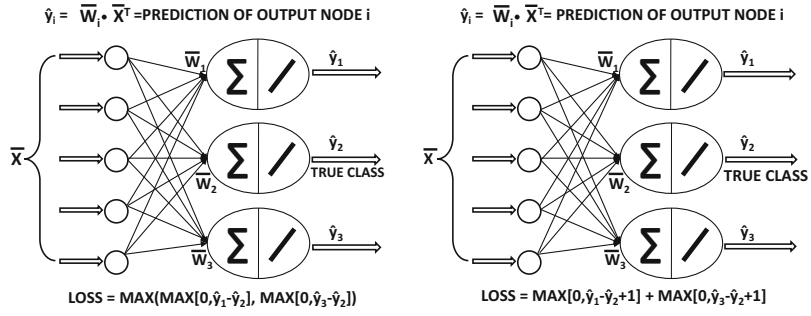
$$\hat{c}(i) = \text{argmax}_r \overline{W}_r \cdot \overline{X}_i^T$$

Note the circumflex on top of $c(i)$ indicating that it is a predicted value rather than observed value. Therefore, the loss function for the $i$th training instance penalizes deviation from the aforementioned desiderata on the separators as follows:

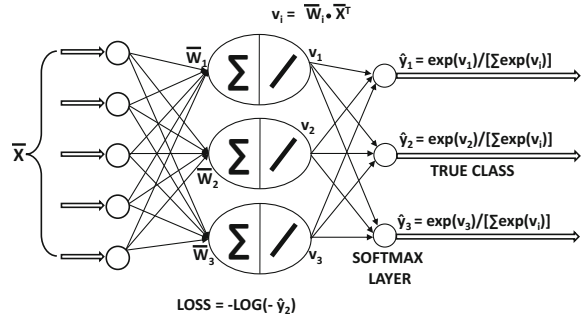
$$L_i = \max_{r:r \neq c(i)} \max(\overline{W}_r \cdot \overline{X}_i^T - \overline{W}_{c(i)} \cdot \overline{X}_i^T, 0) \tag{3.22}$$

The loss is positive whenever the wrong class $r \neq c(i)$ has the largest value of $\overline{W}_r \cdot \overline{X}_i^T$. The multiclass perceptron is illustrated in Figure 3.6(a). As in all neural network models, one can use gradient-descent in order to determine the updates. For a correctly classified instance, the gradient is always 0, and there are no updates. For a misclassified instance, the gradients are as follows:

$$\frac{\partial L_i}{\partial \overline{W}_r} = \begin{cases} -\overline{X}_i^T & \text{if } r = c(i) \\ \overline{X}_i^T & \text{if } r \neq c(i) \text{ is most misclassified prediction} \\ 0 & \text{otherwise} \end{cases} \tag{3.23}$$




(a) Multiclass perceptron (b) Multiclass SVM




(c) Multinomial logistic regression

Figure 3.6: Multiclass models: In each case, class 2 is assumed to be the ground-truth class.

Therefore, the stochastic gradient-descent method is applied as follows. Each training instance is fed into the network. If the correct class $r = c(i)$ receives the largest of output $\overline{W}_r \cdot \overline{X}_i^T$, then no update needs to be executed. Otherwise, the following update is made to each separator $\overline{W}_r$ for learning rate $\alpha > 0$:

$$\overline{W}_r \Leftarrow \overline{W}_r + \begin{cases} \alpha \overline{X}_i^T & \text{if } r = c(i) \\ -\alpha \overline{X}_i^T & \text{if } r \neq c(i) \text{ is most misclassified prediction} \\ 0 & \text{otherwise} \end{cases} \tag{3.24}$$

Only two of the separators are always updated at a given time. In the special case that $k = 2$, these gradient updates reduce to the perceptron because both the separators $\overline{W}_1$ and $\overline{W}_2$ will be related as $\overline{W}_1 = -\overline{W}_2$ if the descent is started at $\overline{W}_1 = \overline{W}_2 = 0$. Another quirk that is specific to the unregularized perceptron is that it is possible to use a learning rate of $\alpha = 1$ without affecting the learning because the value of $\alpha$ only has the effect of scaling the weight when starting with $\overline{W}_j = 0$ (see Exercise 2). This property is, however, not true for other linear models in which the value of $\alpha$ does affect the learning.

### 3.3.2 Weston-Watkins SVM

As in the case of the multiclass perceptron, it is assumed that the $i$th training instance is denoted by $[\overline{X}_i, c(i)]$, where $\overline{X}_i$ contains the $d$-dimensional row vector of feature variables, and $c(i)$ contains the class index drawn from $\{1, \ldots, k\}$. One wants to learn $d$-dimensional column vectors $\overline{W}_1 \ldots \overline{W}_k$ representing the coefficients (ideally) satisfying the condition

the class index $r$ with the largest value of $\overline{W}_r \cdot \overline{X}_i^T$ is predicted to be the correct class $c(i)$. The Weston-Watkins loss function [548] $L_i$ for the $i$th training instance $[\overline{X}_i, c(i)]$ penalizes deviation from this ideal condition, and is defined as follows:

$$L_i = \sum_{r:r \neq c(i)} \max(\overline{W}_r \cdot \overline{X}_i^T - \overline{W}_{c(i)} \cdot \overline{X}_i^T + 1, 0) \tag{3.25}$$

The neural architecture of the Weston-Watkins SVM is illustrated in Figure 3.6(b). It is instructive to compare the objective function of the Weston-Watkins SVM (Equation 3.25) with that of the multiclass perceptron (Equation 3.22). First, for each class $r \neq c(i)$, if the prediction $\overline{W}_r \cdot \overline{X}_i^T$ lags behind that of the true class by less than a margin amount of 1, then a loss is incurred for that class. Furthermore, the losses over all such classes $r \neq c(i)$ are *added*, rather than taking the maximum of the losses. These two differences accomplish the following intuitive goals:

1. The multiclass perceptron only updates the linear separator of a class that is predicted *most* incorrectly along with the linear separator of the true class. On the other hand, the Weston-Watkins SVM updates the separator of *any* class that is predicted more favorably than the true class.

2. Not only does the Weston-Watkins SVM update the separator in the case of misclassification, but it also updates the separators in cases where an incorrect class gets a prediction that is "uncomfortably close" to the true class. This is based on the notion of margin.

The gradient of the loss function with respect to each $\overline{W}_r$ is computed as follows. In the event that the loss function $L_i$ is 0, the gradient of the loss function is 0 as well. Therefore, no update is required. However, if the loss function is non-zero, we have either a misclassified or a "barely correct" prediction in which the second-best and best class prediction are not sufficiently distinguished by the model. In such a case, an update needs to be performed. The loss function of Equation 3.25 is created by adding up the contributions of the $(k-1)$ separators belonging to the incorrect classes. Let $\delta(r, \overline{X}_i)$ be a 0/1 indicator function, which is 1 when the $r$th class separator contributes positively to the loss function in Equation 3.25. In such a case, the gradient of the loss function is as follows:

$$\frac{\partial L_i}{\partial \overline{W}_r} = \begin{cases} -\overline{X}_i^T [\sum_{j \neq r} \delta(j, \overline{X}_i)] & \text{if } r = c(i) \\ \overline{X}_i^T [\delta(r, \overline{X}_i)] & \text{if } r \neq c(i) \end{cases} \tag{3.26}$$

This results in the following stochastic gradient-descent step:

$$\overline{W}_r \Leftarrow \overline{W}_r + \alpha \begin{cases} \overline{X}_i^T [\sum_{j \neq r} \delta(j, \overline{X}_i)] & \text{if } r = c(i) \\ -\overline{X}_i^T [\delta(r, \overline{X}_i)] & \text{if } r \neq c(i) \end{cases} \tag{3.27}$$

Here, the parameter $\alpha$ denotes the learning rate. In most cases, the weight vectors are also shrunk with a regularization parameter $\lambda > 0$. One can implement regularization by adding the shrinkage update $\overline{W} \Leftarrow \overline{W}(1 - \alpha\lambda)$ after each loss-wise update.

### 3.3.3 Multinomial Logistic Regression (Softmax Classifier)

Multinomial logistic regression can be considered the multi-way generalization of logistic regression, just as the Weston-Watkins SVM is the multiway generalization of the binary

SVM. As in the case of the multiclass perceptron, it is assumed that the input to the model is a training data set containing pairs of the form $[\overline{X}_i, c(i)]$, where $\overline{X}_i$ is a row vector of features, and $c(i) \in \{1 \ldots k\}$ is the index of the class of $d$-dimensional row vector $\overline{X}_i$. As in the case of the previous two models, the class $r$ with the largest value of $\overline{W}_r \cdot \overline{X}_i^T$ is predicted to be the label of the data point $\overline{X}_i$. However, in this case, there is an additional probabilistic interpretation of $\overline{W}_r \cdot \overline{X}_i^T$ in terms of the posterior probability $P(r|\overline{X}_i)$ that the class $r$ is predicted given the data point $\overline{X}_i$. This estimation can be naturally accomplished with the softmax activation function:

$$P(r|\overline{X}_i) = \frac{\exp(\overline{W}_r \cdot \overline{X}_i^T)}{\sum_{j=1}^{k} \exp(\overline{W}_j \cdot \overline{X}_i^T)} \tag{3.28}$$

Note that large values of $\overline{W}_r \cdot \overline{X}_i^T$ map to large probabilities, and the probabilities over all classes always sum to 1. The loss function $L_i$ for the $i$th training instance is defined by the cross-entropy loss, which is the negative logarithm of the probability of the true class. The neural architecture of the softmax classifier is illustrated in Figure 3.6(c).

The cross-entropy loss may be expressed in terms of either the input features or in terms of the softmax pre-activation values $v_r = \overline{W}_r \cdot \overline{X}_i^T$ as follows:

$$\begin{aligned} L_i &= -\log[P(c(i)|\overline{X}_i)] = -\overline{W}_{c(i)} \cdot \overline{X}_i^T + \log[\sum_{j=1}^{k} \exp(\overline{W}_j \cdot \overline{X}_i^T)] \\ &= -v_{c(i)} + \log[\sum_{j=1}^{k} \exp(v_j)] \end{aligned}$$

Therefore, the partial derivative of $L_i$ with respect to $v_r$ can be computed as follows:

$$\frac{\partial L_i}{\partial v_r} = \begin{cases} -\left(1 - \frac{\exp(v_r)}{\sum_{j=1}^{k} \exp(v_j)}\right) & \text{if } r = c(i) \\ \left(\frac{\exp(v_r)}{\sum_{j=1}^{k} \exp(v_j)}\right) & \text{if } r \neq c(i) \end{cases} \tag{3.29}$$

$$= \begin{cases} -(1 - P(r|\overline{X}_i)) & \text{if } r = c(i) \\ P(r|\overline{X}_i) & \text{if } r \neq c(i) \end{cases} \tag{3.30}$$

The gradient of the loss of the $i$th training instance with respect to the separator of the $r$th class is computed by using the chain rule of differential calculus in terms of its pre-activation value $v_j = \overline{W}_j \cdot \overline{X}_i^T$:

$$\frac{\partial L_i}{\partial \overline{W}_r} = \sum_j \left(\frac{\partial L_i}{\partial v_j}\right)\left(\frac{\partial v_j}{\partial \overline{W}_r}\right) = \frac{\partial L_i}{\partial v_r} \underbrace{\frac{\partial v_r}{\partial \overline{W}_r}}_{\overline{X}_i^T} \tag{3.31}$$

In the above simplification, we used the fact that $v_j$ has a zero gradient with respect to $\overline{W}_r$ for $j \neq r$. The value of $\frac{\partial L_i}{\partial v_r}$ in Equation 3.31 can be substituted from Equation 3.30 to obtain the following result:

$$\frac{\partial L_i}{\partial \overline{W}_r} = \begin{cases} -\overline{X}_i^T (1 - P(r|\overline{X}_i)) & \text{if } r = c(i) \\ \overline{X}_i^T P(r|\overline{X}_i) & \text{if } r \neq c(i) \end{cases} \tag{3.32}$$

Note that we have expressed the gradient indirectly using probabilities (based on Equation 3.28) both for brevity and for intuitive understanding of how the gradient is related to the probability of making different types of mistakes. Each of the terms $[1 - P(r|\overline{X}_i)]$ and $P(r|\overline{X}_i)$ is the probability of making a mistake for an instance with label $c(i)$ with respect to the predictions for the $r$th class. The separator for the $r$th class is updated as follows:

$$\overline{W}_r \Leftarrow \overline{W}_r + \alpha \begin{cases} \overline{X}_i^T (1 - P(r|\overline{X}_i)) & \text{if } r = c(i) \\ -\overline{X}_i^T P(r|\overline{X}_i) & \text{if } r \neq c(i) \end{cases} \tag{3.33}$$

Here, $\alpha$ is the learning rate. The softmax classifier updates all the $k$ separators for each training instance, unlike the multiclass perceptron and the Weston-Watkins SVM, each of which updates only a small subset of separators (or no separator) for each training instance. This is a consequence of probabilistic modeling, in which the notion of correctness is not absolute.

## 3.4 Unsupervised Learning with Autoencoders

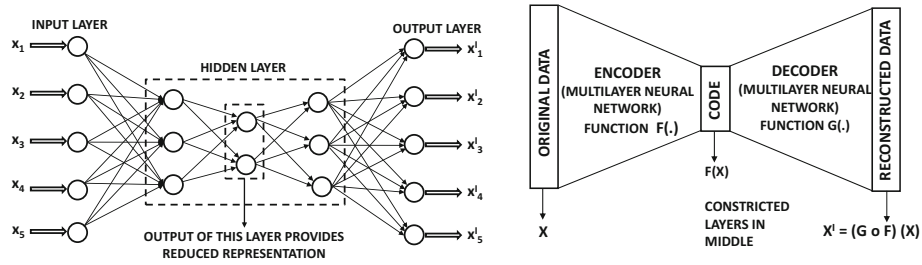
Autoencoders represent a fundamental architecture that is used for various types of unsupervised learning applications. The simplest autoencoders with linear layers map to well-known dimensionality reduction techniques like *singular value decomposition*. However, deep autoencoders with nonlinearity map to complex models that might not exist in traditional machine learning. Therefore, the goal of this section is to show two things:

1. Classical dimensionality reduction methods like singular value decomposition are special cases of shallow neural architectures.

2. By adding depth and nonlinearity to the basic architecture, one can generate sophisticated nonlinear embeddings of the data. While nonlinear embeddings are also available in traditional machine learning, the latter is limited to loss functions that can be expressed compactly in closed form. The loss functions of deep neural architectures are no longer compact; however, they provide unprecedented flexibility in controlling the properties of the embedding by making various types of architectural changes (and allowing backpropagation to take care of the complexities of differentiation).

The basic idea of an autoencoder is to have an output layer with the same dimensionality as the inputs. The idea is to try to reconstruct the input data instance. An autoencoder *replicates* the data from the input to the output, and is therefore sometimes referred to as a *replicator neural network*. Although reconstructing the data might seem like a trivial matter by simply copying the data forward from one layer to another, this is not possible when the number of units in the middle are *constricted*. In other words, the number of units in each middle layer is typically fewer than that in the input (or output). As a result, one cannot simply copy the data from one layer to another. Therefore, the activations in the constricted layers hold a reduced representation of the data; the net effect is that this type of reconstruction is inherently *lossy*. This general representation of the autoencoder is given in Figure 3.7(a), where an architecture is shown with three constricted layers. Note that the output layer has the same number of units as the input layer. The loss function of this neural network uses the sum-of-squared differences between the input and the output feature values in order to force the output to be as similar as possible to the input.

It is common (but not necessary) for an $M$-layer autoencoder to have a symmetric architecture between the input and output, where the number of units in the $k$th layer is




(a) Three hidden layers (b) General schematic

Figure 3.7: The basic schematic of the autoencoder

the same as that in the $(M-k+1)$th layer. Furthermore, the value of $M$ is often odd, as a result of which the $(M+1)/2$th layer is the most constricted layer. The symmetry in the architecture often extends to the fact that the weights outgoing from the $k$th layer are tied to those incoming to the $(M-k)$th layer in many architectures. Here, we are counting the (non-computational) input layer as the first layer, and therefore the minimum number of layers in an autoencoder would be three, corresponding to the input layer, constricted layer, and the output layer.

The reduced representation of the data in the most constricted layer is also sometimes referred to as the *code*, and the number of units in this layer is the dimensionality of the reduction. The initial part of the neural architecture before the bottleneck is referred to as the *encoder* (because it creates a reduced code), and the final part of the architecture is referred to as the *decoder* (because it reconstructs from the code). The general schematic of the autoencoder is shown in Figure 3.7(b).

### 3.4.1 Linear Autoencoder with a Single Hidden Layer

Matrix factorization is one of the most widely studied problems in unsupervised learning, and it is used for dimensionality reduction, clustering, and predictive modeling in recommender systems. In matrix factorization, we want to factorize the $n \times d$ matrix $D$ into an $n \times k$ matrix $U$ and a $d \times k$ matrix $V$:

$$D \approx UV^T \tag{3.34}$$

Here, $k \ll n$ is the rank of the factorization. The matrix $U$ contains the reduced representation of the data, and the matrix $V$ contains the basis vectors. In traditional machine learning, this problem is solved by minimizing the *Frobenius norm* of the *residual matrix* denoted by $(D-UV^T)$. The squared Frobenius norm of a matrix is the sum of the squares of the entries in the matrix. Therefore, one can write the objective function of the optimization problem as follows:

$$\text{Minimize } J = \|D - UV^T\|_F^2$$

Here, the notation $\|\cdot\|_F$ indicates the Frobenius norm. The parameter matrices $U$ and $V$ need to be learned in order to optimize the aforementioned error. Although it is relatively easy to derive the gradient-descent steps [7] for this optimization problem (without worrying about neural networks at all), our goal here is to capture this optimization problem within a neural architecture. Going through this exercise helps us show that simple matrix factorization is a special case of an autoencoder architecture, which sets the stage for understanding the gains obtained with deeper autoencoders.

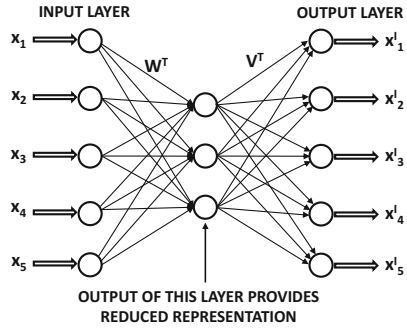



Figure 3.8: A basic autoencoder with a single layer

From the perspective of neural networks, one would need to design the architecture in such a way that the rows of $D$ are fed to the neural network, and the reduced rows of $U$ are obtained from the constricted layer when the original data is fed to the neural network. The single-layer autoencoder is illustrated in Figure 3.8, where the hidden layer contains $k$ units. The rows of $D$ are input into the autoencoder, whereas the $k$-dimensional rows of $U$ are the activations of the hidden layer. Note that the $k \times d$ matrix of weights in the decoder must be $V^T$, since it is necessary to be able to multiply the rows of $U$ to reconstruct the rows of $D \approx UV^T$ according to the optimization model discussed above. As shown in subsequent sections, the $d \times k$ matrix $W^T$ of weights in the encoder can also be expressed in terms of $D$, $U$, and $V$.

The vector of values in a particular layer of the network can be obtained by multiplying the vector of values in the previous layer with the matrix of weights connecting the two layers (with linear activation). Here, $\overline{u}_i$ is the row vector containing the $i$th row of $U$ and $\overline{X}'_i$ is the reconstruction of the $i$th row $\overline{X}_i$ of $D$. One first encodes $\overline{X}_i$ with the matrix $W^T$, and then decodes (reconstructs) it back to $\overline{X}'_i$ using $V^T$:

$$\overline{u}_i = \overline{X}_i W^T \tag{3.35}$$

$$\overline{X}'_i = \overline{u}_i V^T \tag{3.36}$$

Although neural network activations have been chosen to be column vectors throughout the book, we have used (equivalent) row-vector variants here (and forward propagation matrix operations) in order to show the similarity with traditional matrix factorization models. For example, it is not difficult to see that Equation 3.36 is a row-wise variant of the matrix-centric reconstruction in Equation 3.34. The autoencoder minimizes the sum-of-squared differences between the input and the output, which is equivalent to minimizing $\sum_i \|\overline{X}_i - \overline{u}_i V^T\|^2$. The latter is precisely the row-wise variant of the objective $\|D - UV^T\|_F^2$ of matrix factorization. Therefore, the neural architecture has exactly the same loss function as matrix factorization. However, the rows $\overline{u}_i$ in the neural network are generated as linear transformations of $D$ with matrix $W$, which will cause specific relationships (i.e., constraints) to exist between $D$, $\overline{u}_i$ and $V$. It needs to be shown that this constraint does not affect the nature of the optimal solution. In the next section, we examine the nature of the linear transformation $W$.

**Deriving Encoder Weights**

As shown in Figure 3.8, the encoder weights are contained in the $k \times d$ matrix denoted by $W$. How is this matrix related to $V$? Note that the reduced representation $\overline{u}_i$ of the $i$th training instance $\overline{X}_i$ can be obtained by multiplying the input layer with $W^T$. Therefore, we have $\overline{u}_i = \overline{X}_i W^T$. We need to compute a closed-form solution to $W$ assuming *fixed* $V$. Since the neural network minimizes $\sum_i \|\overline{X}_i - \overline{X}_i'\|^2$, the problem boils down to minimizing $\sum_i \|\overline{X}_i - \overline{X}_i W^T V^T\|^2$ over fixed $\overline{X}_i$ and $V$. The optimal solution to this problem can be shown to be $W = V^+$, where $V^+ = (V^T V)^{-1} V^T$ is the pseudoinverse of $V$ (see Exercise 14). In other words, the optimal activation $\overline{u}_i$ is obtained by multiplying each row-vector $\overline{X}_i$ with the transposed pseudo-inverse $V^+$ of $V$:

$$\overline{u}_i = \overline{X}_i [V^+]^T$$

Therefore, the matrix $U$ of neural network activations is related to the data set and weights as $U = D[V^+]^T$. It is also well known in traditional matrix factorization (see [6]) that if a matrix $D$ is factorized to $UV^T$ with least-squares loss, then the optimal matrices $U$ and $V$ are related by $U = D[V^+]^T$. Therefore, all objective functions and relationships between parameters and neural network activations are exactly the same as those in traditional matrix factorization.

#### 3.4.1.1 Connections with Singular Value Decomposition

The single-layer autoencoder architecture is closely connected to *singular value decomposition (SVD)*. The loss function $\|D - UV^T\|_F^2$ of unconstrained matrix factorization is identical to that of SVD (and therefore this neural network) is identical to that of singular value decomposition (see, for example, [6]). The main difference is that the matrix factorization objective function has an infinite number of global optima, although SVD only corresponds to the specific global optimum in which the columns of $V$ are orthonormal and those of $U$ are mutually orthogonal (and such an alternate optimal solution always exists for the unconstrained problem [6]). The $L_2$-normalized columns of $U$ and $V$ are referred to as *left and right singular vectors*, respectively. One cannot guarantee these types of properties of $U$ and $V$ when using the gradient-descent updates with the neural network. Nevertheless, the subspace spanned by the $k$ columns of the matrix $V$ found by the neural network will be the same as that spanned by the top-$k$ basis vectors of SVD as long as the backpropagation algorithm works perfectly. The optimal solution $V$ can be related to the optimal solution $V_{svd}$ of SVD as $V = V_{svd} A$ for *some arbitrary* $k \times k$ *invertible matrix* $A$. The corresponding matrix $U$ is related to $U_{svd}$ as $U = U_{svd}(A^T)^{-1}$. One can easily confirm that $UV^T = U_{svd} V_{svd}^T$, and therefore the quality of reconstruction remains unaffected. The choice of $A$ depends on the specifics of the learning algorithm, gradient descent, or parameter initialization. One cannot easily control $A$ using the vanilla architecture of Figure 3.8.

#### 3.4.1.2 Sharing Weights in the Encoder and Decoder

A common practice that is used in the autoencoder construction is to share some of the weights between the encoder and the decoder. This is also referred to as *tying the weights*. In particular, the autoencoder has an inherently symmetric structure, in which the weights of the encoder and decoder are forced to be the same in symmetrically matching layers. In the single-layer case discussed above, the encoder and decoder weights are shared as follows:

$$W = V^T \tag{3.37}$$

In other words, the $d \times k$ matrix $V$ of weights is first used to transform the $d$-dimensional data point $\overline{X}$ into a $k$-dimensional representation. Then, the matrix $V^T$ of weights is used to reconstruct the data to its original representation.

The tying of the weights effectively means that one is minimizing $\|D - DVV^T\|_F^2$ over the entire data set $D$. Even though the tying of weights makes the optimization problem more constrained, *the optimal objective function value is not worsened at least in the case of this simple architecture*. However, this desirable property might not hold in the case in more complex architectures with nonlinear activation functions. One can also show that the optimal $V$ found by gradient descent has orthonormal columns when $D$ has at least $k$ linearly independent columns (see Exercise 14). This brings it closer to SVD in terms of which of the alternate optima are found. The main difference from SVD is that the optimal $V$ found by this architecture can be a $k$-dimensional rotreflection $V_{svd}P$ of the optimal basis matrix $V_{svd}$ of SVD (using *any* $k \times k$ orthogonal matrix $P$ depending on the vagaries of gradient descent). Furthermore, the $n \times k$ matrix $U$ obtained by stacking the activations $\overline{u}_1 \ldots \overline{u}_n$ will be related to $U_{svd}$ as $U = U_{svd}P$, and we will always have $UV^T = U_{svd}V_{svd}^T$. The columns of $U$ may not be mutually orthogonal.

How does one tie weights when there are multiple layers? In general, one would have an odd number of hidden layers and an even number of weight matrices. It is a common practice to match up the weight matrices in a symmetric way about the middle. In such a case, the symmetrically arranged hidden layers would need to have the same numbers of units. Even though it is not necessary to share weights between the encoder and decoder portions of the architecture, it reduces the number of parameters by a factor of 2. This is beneficial from the point of view of creating a more stable model that better reconstructs out-of-sample data (even though the in-sample reconstructions might worsen slightly).

The sharing of weights does require some changes to the backpropagation algorithm during training. However, these modifications are not very difficult. All that one has to do is to perform normal backpropagation by pretending that the weights are not tied in order to compute the gradients. Then, the gradients across different copies of the same weight are added in order to compute the gradient-descent steps. The logic for handing shared weights in this way is discussed in section 2.6.6 of Chapter 2.

### 3.4.2 Nonlinear Activation Functions and Depth

The single-layer autoencoder does not seem to achieve much because many off-the-shelf tools exist for singular value decomposition. However, the autoencoder can be made to achieve more complex goals by using nonlinear activations and multiple layers. For example, consider a situation in which the matrix $D$ is binary. In such a case, one can use the same neural architecture as shown in Figure 3.8, but one can also use a sigmoid function in the final layer to predict the output. This sigmoid layer is combined with negative log loss. Therefore, for a binary matrix $B = [b_{ij}]$, the model assumes the following:

$$B \sim \text{sigmoid}(UV^T) \tag{3.38}$$

Here, the sigmoid function is applied in element-wise fashion. Note the use of $\sim$ instead of $\approx$ in the above expression, which indicates that the binary matrix $B$ is an instantiation of random draws from Bernoulli distributions with corresponding parameters contained in sigmoid$(UV^T)$. The resulting factorization can be shown to be equivalent to *logistic matrix factorization*. The basic idea is that the $(i, j)$th element of $UV^T$ is the parameter of a Bernoulli distribution, and the binary entry $b_{ij}$ is generated from a Bernoulli distribution with these parameters. Therefore, $U$ and $V$ are learned using the log-likelihood loss of this

*generative* model. The log-likelihood loss implicitly tries to find parameter matrices $U$ and $V$ so that the probability of the matrix $B$ being generated by these parameters is maximized.

Logistic matrix factorization has only recently been proposed [235] as a sophisticated matrix factorization method for binary data, which is useful for recommender systems with *implicit feedback* ratings. Implicit feedback refers to the binary actions of users such as buying or not buying specific items. The solution methodology of this recent work on logistic matrix factorization [235] seems to be vastly different from SVD, and it is not based on a neural network approach. However, for a neural network practitioner, the change from the SVD model to that of logistic matrix factorization is a relatively small one, where only the final layer of the neural network needs to be changed. It is this modular nature of neural networks that makes them so attractive to engineers and encourages all types of experimentation. This benefit multiplies as one moves to deeper architectures.

The real power of autoencoders in the neural network domain is realized when deeper variants are used. For example, an autoencoder with three hidden layers is shown in Figure 3.7(a). One can increase the number of intermediate layers in order to further increase the representation power of the neural network. It is noteworthy that it is essential for some of the layers of the deep autoencoder to use a nonlinear activation function to increase its representation power. As shown in Lemma 1.5.1 of Chapter 1, no additional power is gained by a multilayer network when only linear activations are used. Although this result was shown in Chapter 1 for the classification problem, it is broadly true for any type of multilayer neural network (including an autoencoder).

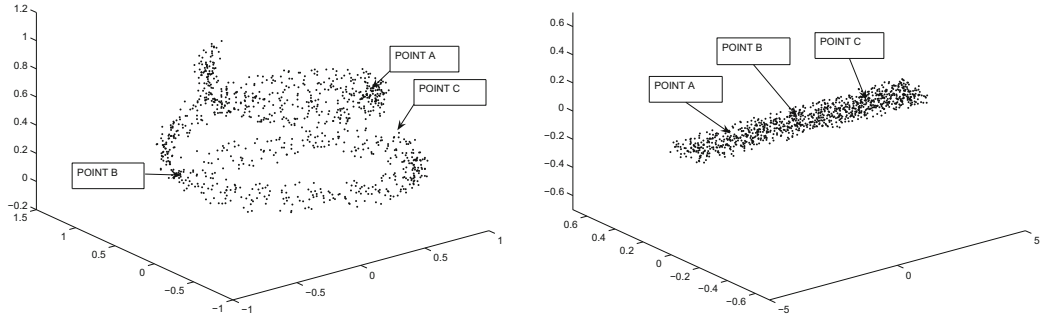
Deep networks with multiple layers provide an extraordinary amount of representation power. The multiple layers of this network provide *hierarchically* reduced representations of the data. For some data domains like images, hierarchically reduced representations are particularly natural. Indeed, for multilayer variants of the autoencoder, an exact counterpart often does not even exist in traditional machine learning. An important point is that even though the loss function might be too complex to express in closed form, the backpropagation approach rescues us from the challenges associated in computing the complicated gradient-descent steps.

Nonlinear dimensionality reduction methods can map the data into much lower dimensional spaces (with good reconstruction characteristics) than would be possible with methods like singular value decomposition. An example of a data set, which is distributed on a nonlinear spiral, is shown in Figure 3.9(a). This data set cannot be reduced to lower dimensionality using singular value decomposition (without causing significant reconstruction error). However, the use of nonlinear dimensionality reduction methods can flatten out the nonlinear spiral into a 2-dimensional representation. This representation is shown in Figure 3.9(b).

Autoencoders form the workhorse of unsupervised learning in the neural network domain. They are used for a host of applications, which will be discussed throughout this book. After training an autoencoder, it is not necessary to use both the encoder and decoder portions. For example, when using the approach for dimensionality reduction, one can use the encoder portion in order to create the reduced representations of the data. The reconstructions of the decoder might not be required at all.

### 3.4.3 Application to Visualization

It is possible to achieve extraordinarily compact reductions by using this approach. Greater reduction is always achieved by using nonlinear units, which implicitly map warped manifolds into linear hyperplanes. The superior reduction in these cases is because it is easier to




(a) A nonlinear pattern in three dimensions (b) A reduced data set in two dimensions

Figure 3.9: The effect of nonlinear dimensionality reduction. This figure is drawn for illustrative purposes only.

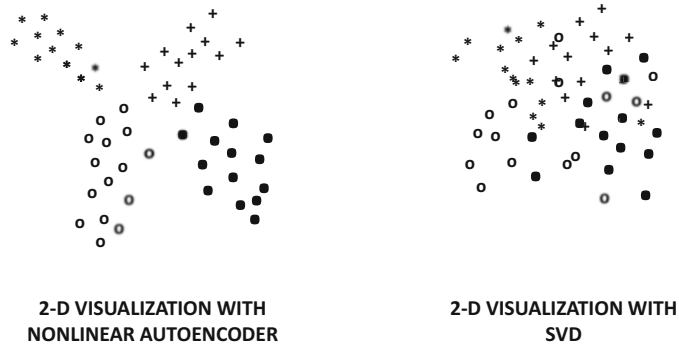



Figure 3.10: A depiction of the typical difference between the embeddings created by nonlinear autoencoders and singular value decomposition. Nonlinear and deep autoencoders are often able to separate out the entangled class structures in the underlying data, which is not possible within the constraints of linear transformations like singular value decomposition.

thread a warped surface (as opposed to a linear surface) through a larger number of points. This property of nonlinear autoencoders is often used for 2-dimensional visualizations of the data by creating a deep autoencoder in which the most compact hidden layer has only two dimensions. These two dimensions can then be mapped on a plane to visualize the points. In many cases, the class structure of the data is exposed in terms of well-separated clusters.

An illustrative example of the typical behavior of the dimensionality reduction obtained by autoencoders is shown in Figure 3.10, in which the 2-dimensional mapping created by a deep autoencoder seems to clearly separate out the different classes. On the other hand, the mapping created by singular value decomposition does not seem to separate the classes well. Figure 3.9, which provides a nonlinear spiral mapped to a linear hyperplane, clarifies the reason for this behavior. In many cases, the data may contain heavily entangled spirals (or other shapes). Linear dimensionality reduction methods cannot attain clear separation because nonlinearly entangled shapes are not linearly separable. On the other hand, deep autoencoders with nonlinearity are far more powerful and able to disentangle such shapes.

Deep autoencoders can sometimes be used as alternatives to other robust visualization methods like $t$-distributed stochastic neighbor embedding ($t$-SNE) [316]. The advantage of an autoencoder over $t$-SNE is that it is easier to generalize to out-of-sample data. When new data points are received, they can simply be passed through the encoder portion of the autoencoder in order to add them to the current set of visualized points.

### 3.4.4 Application to Outlier Detection

Dimensionality reduction is closely related to outlier detection, because outlier points are hard to encode and decode without losing substantial information. It is a well-known fact that if a matrix $D$ is factorized as $D \approx D' = UV^T$, then the low-rank matrix $D'$ is a de-noised representative of the data. After all, the compressed representation $U$ captures only the regularities in the data, and is unable to capture the unusual variations in specific points. As a result, reconstruction to $D'$ misses all these unusual variations.

The absolute values of the entries of $(D - D')$ represent the outlier scores of the matrix entries. Therefore, one can use this approach to find outlier entries, or add the squared scores of the entries in each row of $D$ to find the outlier score of that row. Therefore, one can identify outlier data points. Furthermore, by adding the squared scores in each column of $D$, one can find outlier features. This is useful for applications like feature selection in clustering, where a feature with a large outlier score can be removed because it adds noise to the clustering. Refer to the bibliographic notes for more details of outlier detection methods.

### 3.4.5 Application to Multimodal Embeddings

One can also use an autoencoder for embedding multimodal data in a joint latent space. Multimodal data is essentially data in which the input features are heterogeneous. For example, an image with descriptive tags can be considered multimodal data. Multimodal data pose challenges to mining applications because different features require different types of processing and treatment. By embedding the heterogeneous attributes in a unified space, one is removing this source of difficulty in the mining process. An autoencoder can be used to embed the heterogeneous data into a joint space. An example of such a setting is shown in Figure 3.11. This figure shows an autoencoder with only a single layer, although one might have multiple layers in general [368, 484]. Such joint spaces can be very useful in a

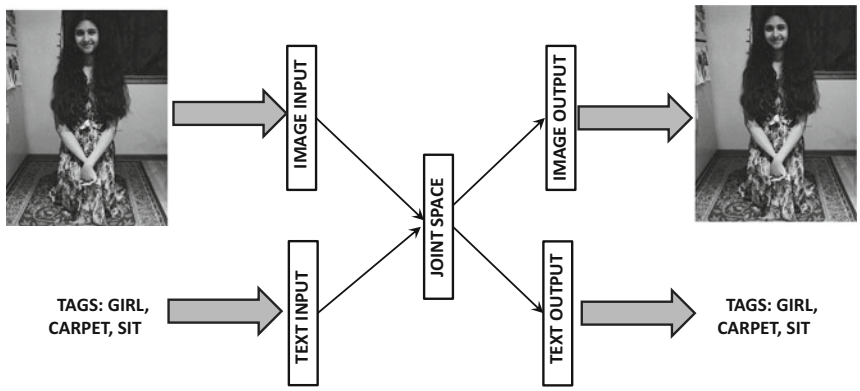



Figure 3.11: Multimodal embedding with autoencoders

variety of applications, because it often turns out to be hard to integrate data from multiple sources in known algorithms without changing the algorithms in an ad hoc manner. On the other hand, the integrated representation of the data in the hidden layer can be directly fed to any machine learning algorithm that works with multidimensional data.

### 3.4.6 Benefits of Autoencoders

Although several methods for nonlinear dimensionality reduction are known in machine learning, neural networks have some advantages over these methods:

1. Many nonlinear dimensionality reduction methods have a hard time mapping out-of-sample data points to reduced representations, unless these points are included in the training data up front. On the other hand, it is a simple matter to compute the reduced representation of an out-of-sample point by passing it through the network.

2. Neural networks allow more power and flexibility in the nonlinear data reduction by varying on the number and type of layers used in intermediate stages. Furthermore, by choosing specific types of activation functions in particular layers, one can engineer the nature of the reduction to the properties of the data. For example, it makes sense to use a logistic output layer with logarithmic loss for a binary data set.

From an architectural point of view, the amount of effort required by the analyst to change from one architecture to the other is often a few lines of code. This is because modern softwares for building neural networks often provide templates for describing the architecture of the neural network, where each layer is specified independently. In a sense, the neural network is "built" with well-known machine-learning units much like a child puts together building blocks of a toy. Backpropagation takes care of the details of optimization, while shielding the user from the complexities of the steps. Consider the significant mathematical differences between the specific details of SVD and logistic matrix factorization. Changing the output layer from linear to sigmoid (along with a change of loss function) can literally be a matter of changing a trivially small number of lines of code without touching most of the remaining code. This type of modularity is tremendously useful in application-centric settings.

## 3.5 Recommender Systems

One of the most interesting applications of matrix factorization is the design of neural architectures for recommender systems. Consider an $n \times d$ ratings matrix $D$ with $n$ users and $d$ items. The $(i, j)$th entry of the matrix is the rating of user $i$ for item $j$. However, most entries in the matrix are not observed (i.e., missing), which creates difficulties in using a traditional autoencoder architecture. This is because traditional autoencoders are designed for fully specified matrices, in which a single *row* of the matrix is input at one time. On the other hand, recommender systems are inherently suited to *elementwise* learning, in which a very small subset of ratings from a row may be available. As a practical matter, one might consider the input to a recommender system as a set of triplets of the following form:

$$\langle \text{RowId} \rangle, \langle \text{ ColumnId } \rangle, \langle \text{ Rating } \rangle$$

Our goal is to learn a $k$-dimensional parameter vector $\overline{u}_i$ for each user $i \in \{1 \ldots n\}$, and a $k$-dimensional parameter vector $\overline{v}_j$ for item $j \in \{1 \ldots d\}$. One would like to learn

these parameters so that $\overline{u}_i \cdot \overline{v}_j$ yields an approximation of the $(i, j)$th entry of the ratings matrix. Note that this is the same condition as traditional matrix factorization $D \approx UV^T$ by treating $\overline{u}_i$ and $\overline{v}_j$ as rows of $U$ and $V$, respectively. However, the difference between recommender systems and traditional matrix factorization is that the matrix $D$ is not fully specified; therefore, one must learn each $\overline{u}_i$ and $\overline{v}_j$ using triplet-centric input. This makes it difficult to use the traditional autoencoder architecture that is used in (fully specified) matrix factorization.

Therefore, a different type of neural architecture is used in which a one-hot encoded row index is mapped to the ratings in that row. The input layer contains $n$ input units, which is the same as the number of rows (users). The input is a *one-hot encoded* index of the row identifier; a one-hot encoding refers to the fact that one of the $n$ inputs is 1 and the remaining take on the value of 0. Therefore, the input uniquely specifies one of the $n$ users. The hidden layer contains $k$ units, where $k$ is the rank of the factorization. Finally, the output layer contains $d$ real-valued units with identity activation, where $d$ is the number of columns (items). The output is a vector containing the $d$ ratings (even though only a small subset of them are observed). All layers of the network are densely connected. The goal is to train the neural network by using the rows of the data matrix $D$ as training points. For each row of the data matrix, the one-hot encoded row index is input, and the network outputs all the ratings of that user. Furthermore, even though the network outputs all the ratings of that user, the ground truth information of only a subset of the ratings is available. Therefore, the loss function needs to account for this missing information.

It turns out that *the weights of the neural network encode the parameters in the matrices $U$ and $V$ required to reconstruct $D \approx UV^T$.* Consider a setting in which the $n \times k$ input-to-hidden matrix is $U$, and the $k \times d$ hidden-to-output matrix is $V^T$. The entries of the matrix $U$ are denoted by $u_{iq}$, and those of the matrix $V$ are denoted by $v_{jq}$. The $k$ weights from the $i$th user (input) to the $k$ hidden units create the $i$th row of $U$, which is denoted by $\overline{u}_i = [u_{i1}, \ldots, u_{ik}]$. The $k$ weights from the $k$ hidden units to the $j$th item (output) create the $j$th row of $V$, which is denoted by $\overline{v}_j = [v_{j1} \ldots v_{jd}]$. All layers use identity activation, and therefore there is no nonlinearity in this architecture. The loss function is the sum of the squares of the errors in the output layer. However, because of the missing entries, not all output nodes have an observed output value, and the updates are performed only with respect to entries that are known. The overall architecture of this neural network is illustrated in Figure 3.12. For any particular row-wise input we are training on a neural network that is a subset of this base network, depending on which entries are specified.

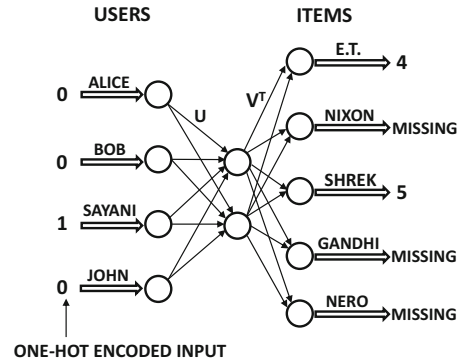



Figure 3.12: Row-index-to-value encoder for matrix factorization with missing values.

However, it is possible to give *predictions* for all outputs in the network (even though a loss function cannot be computed for missing entries).

How can we show that the aforementioned neural architecture performs matrix factorization? Consider a situation in which we are trying to reconstruct the ratings for the $r$th user using the neural network. Then, the input to the network is the one-hot encoded input vector $\overline{e}_r$ of length $n$, in which only the $r$th entry takes on the value of 1 and the remaining take on the value of 0. Assume that $\overline{e}_r$ is a row vector. Because of this input, the neural network simply copies the weights in $\overline{u}_r$ to the hidden units, which can also be written as $\overline{e}_r U$. Furthermore, the next layer of weights contains the matrix $V^T$, and therefore vector of $d$ outputs is obtained by multiplying the hidden activations $\overline{e}_r U$ with $V^T$, which is $\overline{e}_r U V^T$. One can also understand this process in the context of the fact that linear layers in a network perform matrix multiplication, and this neural network contains $U$ and $V^T$ as the weight matrices. In essence, when the $r$th user is input to the neural network as a one-hot encoded vector, the computations in the neural network pull out the $r$th row from the product $UV^T$ of the weight matrices $U$ and $V^T$ in the neural network. The corresponding $d$ values in the $r$th row of $UV^T$ appear at the output layer and represent the item-wise ratings predictions for the $r$th user. Therefore, all feature values are reconstructed in one shot.

How is training performed? The main attraction of this architecture is that one can perform the training either in row-wise fashion or in element-wise fashion. When performing the training in row-wise fashion, the one-hot encoded index for that row is input, and all *specified* entries of that row are used to compute the loss. The backpropagation algorithm is executed by aggregated the squared loss *only at output nodes where the ratings are specified*. From a theoretical point of view, each row is being trained on a slightly different neural network with a subset of the base output nodes (depending on which entries are observed), although the weights for the different neural networks are shared. This situation is shown in Figure 3.13, where the neural networks for the movie ratings of two different users, Bob and Sayani, are shown. For example, Bob is missing a rating for *Shrek*, as a result of which the corresponding output node is missing. However, since both users have specified a rating for *E.T.*, the $k$-dimensional hidden factors for this movie in matrix $V$ will be updated during backpropagation when either Bob or Sayani is processed.

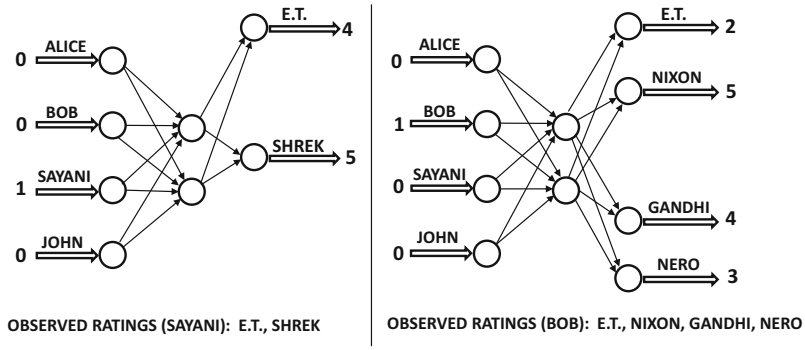



Figure 3.13: Output nodes are dropped for missing values at training time. All output nodes are materialized during prediction.

It is also possible to perform the training in element-wise fashion, where a single triplet is input. In such a case, the loss is computed only with respect to a single column index specified in the triplet. Consider the case where the row index is $i$, and the column index is $j$. In this specific case, and the single error computed at the output layer is $y - \hat{y} = e_{ij}$, which is the error in predicting rating $j$ for user $i$. the backpropagation algorithm essentially updates the weights on all the $k$ paths from node $j$ in the output layer to the node $i$ in the input layer. These $k$ paths pass through the $k$ nodes in the hidden layer. It is easy to show that the update along the $q$th such path is as follows:

$$u_{iq} \Leftarrow u_{iq}(1 - \alpha\lambda) + \alpha e_{ij} v_{jq}$$
$$v_{jq} \Leftarrow v_{jq}(1 - \alpha\lambda) + \alpha e_{ij} u_{iq}$$

Here, $\alpha$ is the step-size, and $\lambda$ is the regularization parameter. *These updates are identical to those used in stochastic gradient descent for matrix factorization in recommender systems.* However, an important advantage of the use of the neural architecture (over traditional matrix factorization) is that we can vary on it in so many different ways in order to enforce different properties. For example, we can incorporate multiple hidden layers to create more powerful models when data availability is greater. The resulting loss function might not even be (compactly) expressible in closed form (and therefore outside the ambit of traditional machine learning). For matrices with binary data, we can use a logistic layer in the output. This will result in *logistic matrix factorization.* For matrices with categorical entries (and count-centric weights attached to entries), one can use a softmax layer at the very end. This will result in *multinomial matrix factorization.* To date, we are not aware of a formal description of multinomial matrix factorization in traditional machine learning; yet, it is a simple modification of the neural architecture (implicitly) used by recommender systems. In general, it is often easy to stumble upon sophisticated models when working with neural architectures because of their modular structure. The neural network abstraction brings practitioners (without too much mathematical training) much closer to sophisticated methods in machine learning, while being shielded from the details of optimization by the backpropagation framework.

## 3.6 Text Embedding with Word2vec

Neural network methods have been used to learn word embeddings of text data. Such embeddings are learned from word-word context matrices. For a lexicon of $d$ words, the $(i, j)$th entry of such a $d \times d$ *word-word context* matrix contains the frequency of the number of times that word $j$ occurs in the context of word $i$. The notion of "context" refers to a window of length $t$ placed around the word in a sentence, and it therefore contains $2t$ words (not including the central "target" word creating the context). There are two ways in which the embedding can be generated. The first is to directly factorize the word-word context matrix. The second is to generate a neural model in which context words are target words are generated from one another by extracting windows of training points from the training sentences. As we will show in this section, *these two methods are equivalent.*

Consider a sentence containing the words $w_1 w_2 \ldots w_n$ in that sequence. One generates multiple training points from each sentence using context windows. There are two variants of *word2vec*, depending on whether target words are predicted from contexts or whether contexts are predicted from target words:

1. *Predicting target words from contexts:* This model tries to predict the $i$th word, $w_i$, in a sentence using a window of width $t$ around the word. Therefore, the words

$w_{i-t}w_{i-t+1}\ldots w_{i-1}w_{i+1}\ldots w_{i+t-1}w_{i+t}$ are used to predict the target word $w_i$. This model is also referred to as the *continuous bag-of-words (CBOW) model*.

2. *Predicting contexts from target words:* This model tries to predict the context $w_{i-t}w_{i-t+1}\ldots w_{i-1}w_{i+1}\ldots w_{i+t-1}w_{i+t}$ around word $w_i$, given the $i$th word in the sentence, denoted by $w_i$. This model is referred to as the *skip-gram model*. There are, however, two ways in which one can perform this prediction. The first technique is a *multinomial* model which predicts one word out of $d$ outcomes. The second model is a Bernoulli model, which predicts whether or not each context is present for a particular word. The second model uses *negative sampling* of contexts for better efficiency and accuracy.

Each of these methods will be discussed in this section.

### 3.6.1 Neural Embedding with Continuous Bag of Words

In the continuous bag-of-words (CBOW) model, the training pairs are all context-word pairs in which a window of context words is input, and a single target word is predicted. The context contains $2 \cdot t$ words, corresponding to $t$ words both before and after the target word. For notational ease, we will use the length $m = 2 \cdot t$ to define the length of the context. Therefore, the input to the system is a set of $m$ words. Without loss of generality, let the subscripts of these words be numbered so that they are denoted by $w_1 \ldots w_m$, and let the target (output) word in the middle of the context window be denoted by $w$. Note that $w$ can be viewed as a categorical variable with $d$ possible values, where $d$ is the size of the lexicon. The neural model predicts the probability $P(w|w_1w_2\ldots w_m)$ in the output node and creates a loss function that implicitly maximizes the product of these probabilities over all training samples (by minimizing the sum of the corresponding negative log-probabilities over all training instances).

The overall architecture of this model is illustrated in Figure 3.14. In the architecture, we have a single input layer with $m \times d$ nodes, a hidden layer with $p$ nodes, and an output layer with $d$ nodes. The nodes in the input layer are clustered into $m$ different groups, each of which has $d$ units. Each group with $d$ input units is the one-hot encoded input vector of one of the $m$ context words being modeled by CBOW. Only one of these $d$ inputs will be 1 and the remaining inputs will be 0. Therefore, one can represent an input $x_{ij}$ with two indices corresponding to contextual position and word identifier. Specifically, the input $x_{ij} \in \{0, 1\}$ contains two indices $i$ and $j$ in the subscript, where $i \in \{1 \ldots m\}$ is the position of the context, and $j \in \{1 \ldots d\}$ is the identifier of the word.

The hidden layer contains $p$ units, where $p$ is the dimensionality of the hidden layer in *word2vec*. Let $h_1, h_2, \ldots h_p$ be the outputs of the hidden layer nodes. Note that each of the $d$ words in the lexicon has $m$ different representatives in the input layer corresponding to the $m$ different context words, but the weight of each of these $m$ connections is the same. Such weights are referred to as shared. Sharing weights is a common trick used for regularization in neural networks, when one has specific insight about the domain at hand. Let the shared weight of each connection from the $j$th word in the lexicon to the $q$th hidden layer node be denoted by $u_{jq}$. Note that each of the $m$ groups in the input layer has connections to the hidden layer that are defined by the same $d \times p$ weight matrix $U$. This situation is shown in Figure 3.14.

It is noteworthy that $\overline{u}_j = [u_{j1}, u_{j2}, \ldots u_{jp}]$ (i.e., $j$th row of $U$) can be viewed as the $p$-dimensional embedding of the $j$th input word over the entire corpus, and $\overline{h} = [h_1 \ldots h_p]$ provides the embedding of a specific instantiation of an input context. Then, the output

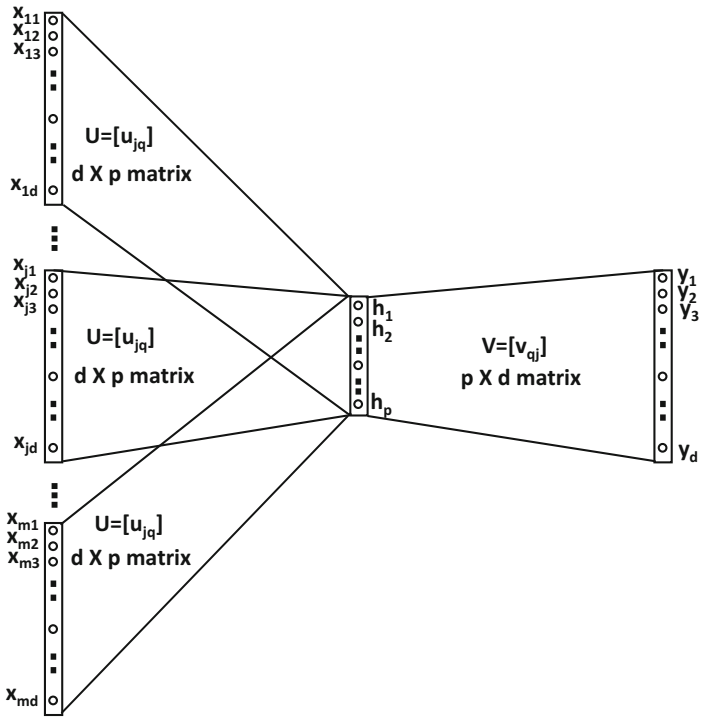



Figure 3.14: Word2vec: The CBOW model. Note the similarities and differences with Figure 3.12, which uses a single set of inputs with a linear output layer. One could equivalently choose to collapse the $m$ sets of $d$ one-hot encoded input nodes into a single set of $d$ *real-valued* input nodes, which are fed the averages of the $m$ one-hot encoded inputs to achieve the same effect.

of the hidden layer is obtained by averaging[2] the embeddings of the words present in the context. One can write this relationship in vector form:

$$\overline{h} = \frac{1}{m} \sum_{i=1}^{m} \sum_{j=1}^{d} \overline{u}_j x_{ij} \tag{3.39}$$

In essence, the one-hot encodings of the input words are aggregated, which implies that the ordering of the words within the window of size $m$ does not affect the output of the model. This is the reason that the model is referred to as the continuous bag-of-words model.

The embedding $[h_1 \ldots h_p]$ is used to predict the probability that the target word is one of each of the $d$ outputs with the use of the softmax function. The weights in the output layer are parameterized with a $d \times p$ matrix $V = [v_{jq}]$. The $j$th row of $V$ is denoted by $\overline{v}_j$. Since the hidden-to-output matrix is a $p \times d$ matrix, it is annotated by $V^T$ instead of $V$ in Figure 3.14. Note that the multiplication of the hidden vector $\overline{h}$ with $V^T$ to create $\overline{h}V^T$ will create a real-valued vector $[\overline{h} \cdot \overline{v}_1, \ldots, \overline{h} \cdot \overline{v}_d]$. The softmax function is applied to this vector

[2]When $m$ is fixed over all sampled windows, it is possible to omit the factor of $m$ in the denominator on the right-hand side of Equation 3.39. This would make the transformation identical to that of a traditional linear layer.

to convert it to $d$ probabilities $\hat{y}_1 \ldots \hat{y}_d$ summing to 1. These outputs are the $d$ probabilities for the target word, given the context:

$$\hat{y}_j = P(y_j = 1 | w_1 \ldots w_m) = \frac{\exp(\overline{h} \cdot \overline{v}_j)}{\sum_{k=1}^{d} \exp(\overline{h} \cdot \overline{v}_k)} \tag{3.40}$$

The *ground-truth* value of only one of the outputs $y_1 \ldots y_d$ is 1 and the remaining values are 0 for a given training instance: follows:

$$y_j = \begin{cases} 1 & \text{if the target word } w \text{ is the } j \text{ th word} \\ 0 & \text{otherwise} \end{cases} \tag{3.41}$$

For a particular target word $w = r \in \{1 \ldots d\}$, the loss function is given by $L = -\log[P(y_r = 1 | w_1 \ldots w_m)] = -\log(\hat{y}_r)$. The use of the negative logarithm turns the multiplicative likelihoods over different training instances into an additive loss function using the negative log-likelihoods.

The updates on the rows of $U$ and $V$ are defined by computing the gradients using the backpropagation algorithm, as training instances are passed through the neural network one by one. The update equations with learning rate $\alpha$ are as follows:

$$\overline{u}_i \Leftarrow \overline{u}_i - \alpha \frac{\partial L}{\partial \overline{u}_i} \quad \forall i$$

$$\overline{v}_j \Leftarrow \overline{v}_j - \alpha \frac{\partial L}{\partial \overline{v}_j} \quad \forall j$$

Next, we provide expressions for the aforementioned partial derivatives of this loss function [337, 339, 420]. The probability of making a mistake in prediction on the $j$th word in the lexicon is defined by $|y_j - \hat{y}_j|$. However, we use *signed* mistakes $\epsilon_j$, in which only the correct word with $y_j = 1$ is given a positive mistake value, while all the other words in the lexicon receive negative mistake values. This is achieved by dropping the modulus:

$$\epsilon_j = y_j - \hat{y}_j \tag{3.42}$$

Note that $\epsilon_j$ is also the negative derivative of the cross-entropy loss with respect to $j$th input into the softmax layer (cf. Equation 2.19). Further backpropagation results in the following updates:

$$\overline{u}_i \Leftarrow \overline{u}_i + \frac{\alpha}{m} \sum_{j=1}^{d} \epsilon_j \overline{v}_j \quad [\forall \text{ words } i \text{ present in context window}]$$

$$\overline{v}_j \Leftarrow \overline{v}_j + \alpha \epsilon_j \overline{h} \quad [\forall \ j \text{ in lexicon}]$$

Here, $\alpha > 0$ is the learning rate. Repetitions of the same word $i$ in the context window trigger multiple updates of $\overline{u}_i$. The use of an aggregate context window in the input has a smoothing effect on the CBOW model, which is particularly helpful with smaller data sets.

The training examples of context-target pairs are presented one by one, and the weights are trained to convergence. After the final training, one obtains not one but two different embeddings corresponding to the $p$-dimensional rows of the matrix $U$ and the $p$-dimensional rows of the matrix $V$. The former type of embedding of words is referred to as the *input* embedding, whereas the latter is referred to as the *output* embedding. In the CBOW model, the input corresponds to the context, and therefore it makes sense to use the output embedding. However, the input embedding (or the sum/concatenation of input and output embeddings) can also be helpful for many tasks.

### 3.6.2 Neural Embedding with Skip-Gram Model

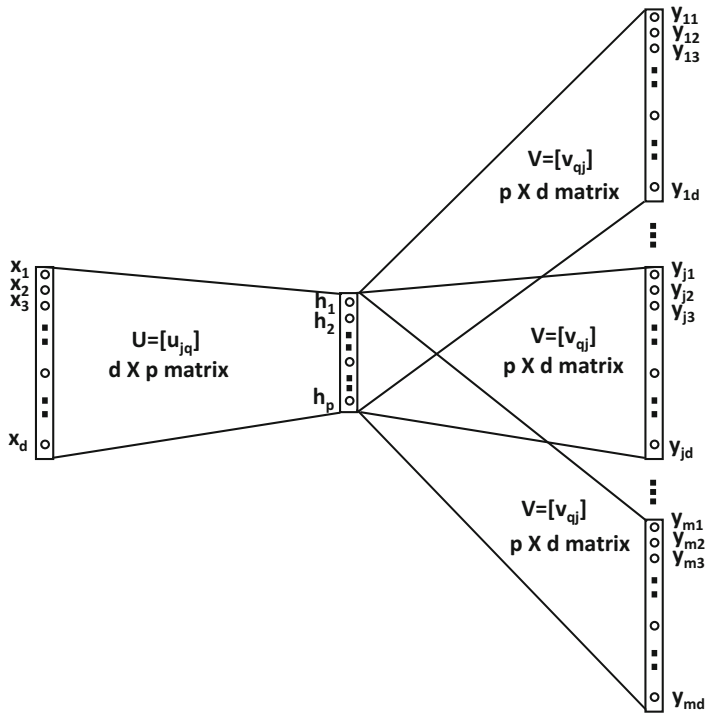
The skip-gram model uses a single target word $w$ as the input and outputs the $m$ context words denoted by $w_1 \ldots w_m$. Therefore, the goal is to predict $P(w_1, w_2 \ldots . w_m|w)$, which is different from the quantity $P(w|w_1 \ldots w_m)$ predicted in the CBOW model. As in the case of the continuous bag-of-words model, we can use one-hot encoding of the (categorical) input and outputs in the skip-gram model. After such an encoding, the skip-gram model will have $d$ binary inputs denoted by $x_1 \ldots x_d$ corresponding to the $d$ possible values of the single input word. Similarly, the output of each training instance is encoded as $m \times d$ values $y_{ij} \in \{0, 1\}$, where $i$ ranges from 1 to $m$ (size of context window), and $j$ ranges from 1 to $d$ (lexicon size). Each $y_{ij} \in \{0, 1\}$ indicates whether the $i$th contextual word takes on the $j$th possible value for that training instance. However, the $(i, j)$th output node only computes a soft probability value $\hat{y}_{ij} = P(y_{ij} = 1|w)$. Therefore, the probabilities $\hat{y}_{ij}$ in the output layer for fixed $i$ and varying $j$ sum to 1, since the $i$th contextual position takes on exactly one of the $d$ words. The hidden layer contains $p$ units denoted by the vector $\overline{h} = [h_1 \ldots h_p]$. Each input $x_j$ is connected to all the hidden nodes with a $d \times p$ matrix $U$ in which the $i$th row is denoted by $\overline{u}_i$.

Furthermore, the $p$ hidden nodes are connected to each of the $m$ groups of $d$ output nodes with the same set of shared weights. This set of shared weights between the $p$ hidden nodes and the $d$ output nodes of each of the context words is defined by the $p \times d$ matrix $V^T$. Therefore, $V$ is a $d \times p$ matrix in which the $j$th row is $\overline{v}_j$. Note that the input-output structure of the skip-gram model is an inverted version of the input-output structure of the CBOW model. The neural architecture of the skip-gram model is illustrated in Figure 3.15(a). However, in the case of the skip-gram model, one can collapse the $m$ identical outputs into a single output, and achieve the same results simply by using a *particular type of mini-batching* during stochastic gradient descent. In particular, all elements of a single context window are always forced to belong to the same mini-batch. This architecture is shown in Figure 3.15(b). Since the value of $m$ is small, this specific type of mini-batching has a very limited effect, and the simplified architecture of Figure 3.15(b) is sufficient to describe the model whether or not any specific type of mini-batching is used. For the purpose of further discussion, we will use the architecture of Figure 3.15(a).

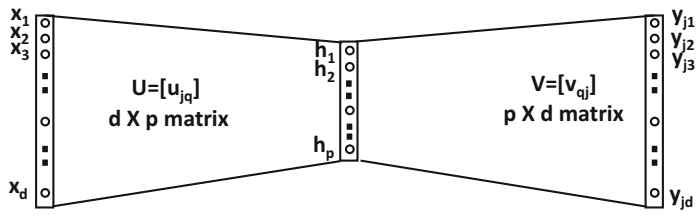
The output of the hidden layer can be computed from the input layer using the $d \times p$ matrix of weights $U = [u_{jq}]$ between the input and hidden layer as follows:

$$\overline{h} = \sum_{j=1}^{d} \overline{u}_j x_j \tag{3.43}$$

The above equation has a simple interpretation because of the one-hot encoding of the input word $w$ in terms of $x_1 \ldots x_d$. If the input word $w$ is the $r$th word, then one simply copies $\overline{u}_r$ to the hidden layer. In other words, the $r$th row $\overline{u}_r$ of $U$ is copied to the hidden layer. As discussed above, the hidden layer is connected to $m$ groups of $d$ output nodes, each of which is connected to the hidden layer with a $d \times p$ matrix $V = [v_{jq}]$. Since the hidden-to-output matrix is of size $p \times d$, the matrix $V^T$ is used instead of $V$. Each of these $m$ groups of $d$ output nodes computes the probabilities of the various words for a particular context word. The $j$th column of $V$ is denoted by $\overline{v}_j$ and represents the output embedding of the $j$th word. The output $\hat{y}_{ij}$ is the probability that the word in the $i$th context position takes on the $j$th word of the lexicon. However, since the same matrix $V$ is shared by all groups, the




(a) All elements in context window explicitly shown




**MINIBATCH THE m d-DIMENSIONAL OUTPUT VECTORS IN EACH CONTEXT WINDOW DURING STOCHASTIC GRADIENT DESCENT. THE SHOWN OUTPUTS $y_{jk}$ CORRESPOND TO THE jth OF m OUTPUTS.**

(b) All elements in context window not explicitly shown

Figure 3.15: Word2vec: The skip-gram model. Note the similarity with Figure 3.12, which uses a single set of linear outputs. One could also choose to collapse the $m$ sets of $d$ output nodes in (a) into a single set of $d$ outputs, and mini-batch the $m$ instances in a single context window during stochastic gradient descent to achieve the same effect. All elements in the mini-batch are explicitly shown in (a), whereas the elements of the mini-batch are not explicitly shown in (b). However, both are equivalent as long as the nature of mini-batching is respected.

neural network predicts the same multinomial distribution for each of the context words. Therefore, we have the following:

$$\hat{y}_{ij} = P(y_{ij} = 1|w) = \underbrace{\frac{\exp(\overline{h} \cdot \overline{v}_j)}{\sum_{k=1}^{d} \exp(\overline{h} \cdot \overline{v}_k)}}_{\text{Independent of context position } i} \qquad \forall i \in \{1 \ldots m\} \tag{3.44}$$

Note that the probability $\hat{y}_{ij}$ is the same for varying $i$ and fixed $j$, since the right-hand side of the above equation does not depend on the exact location $i$ in the context window.

The loss function for the backpropagation algorithm is the negative of the log-likelihood values of the ground truth $y_{ij} \in \{0, 1\}$ of a training instance. This loss function $L$ is given by the following:

$$L = -\sum_{i=1}^{m} \sum_{j=1}^{d} y_{ij} \log(\hat{y}_{ij}) \tag{3.45}$$

Note that the value outside the logarithm is a ground-truth binary value, whereas the value inside the logarithm is a predicted (probability) value. Since $y_{ij}$ is one-hot encoded for fixed $i$ and varying $j$, the objective function has only $m$ non-zero terms. For each training instance, this loss function is used in combination with backpropagation to update the weights of the connections between the nodes. The update equations with learning rate $\alpha$ are as follows:

$$\overline{u}_i \Leftarrow \overline{u}_i - \alpha \frac{\partial L}{\partial \overline{u}_i} \quad \forall i$$

$$\overline{v}_j \Leftarrow \overline{v}_j - \alpha \frac{\partial L}{\partial \overline{v}_j} \quad \forall j$$

Next, we derive the gradients above. The probability of making a mistake in predicting the $j$th word in the lexicon for the $i$th context is defined by $|y_{ij} - \hat{y}_{ij}|$. However, we use *signed* mistakes $\epsilon_{ij}$ in which only the predicted words (positive examples) have a positive probability. This is achieved by dropping the modulus:

$$\epsilon_{ij} = y_{ij} - \hat{y}_{ij} \tag{3.46}$$

Then, the updates for a particular input word $r$ and its output context are as follows:

$$\overline{u}_r \Leftarrow \overline{u}_r + \alpha \sum_{j=1}^{d} \left[ \sum_{i=1}^{m} \epsilon_{ij} \right] \overline{v}_j \qquad \text{[Only for input word } r\text{]}$$

$$\overline{v}_j \Leftarrow \overline{v}_j + \alpha \left[ \sum_{i=1}^{m} \epsilon_{ij} \right] \overline{h} \qquad \text{[For all words } j \text{ in lexicon]}$$

Here, $\alpha > 0$ is the learning rate. The $p$-dimensional rows of the matrix $U$ are used as the embeddings of the words. In other words, the convention is to use the input embeddings in the rows of $U$ rather than the output embeddings in the rows of $V$. It is stated in [294] that adding the input and output embeddings can help in some tasks (but hurt in others). The concatenation of the two can also be useful.

### Practical Issues

Several practical issues are associated with the accuracy and efficiency of the *word2vec* framework. Increasing the embedding dimensionality improves discrimination, but it requires a greater amount of data. In general, the typical embedding dimensionality is of the

order of several hundred, although it is possible to choose dimensionalities in the thousands for very large collections. The size of the context window typically varies between 5 and 10, with larger window sizes being used for the skip-gram model as compared to the CBOW model. Using a random window size is a variant that has the implicit effect of giving greater weight to words that are placed close together. The skip-gram model is slower but it works better for infrequent words and for larger data sets.

Another issue is that the effect of frequent and less discriminative words (e.g., "*the*") can dominate the results. Therefore, a common approach is to downsample the frequent words, which improves both accuracy and efficiency. Note that downsampling frequent words has the implicit effect of increasing the context window size because dropping a word in the middle of two words brings the latter pair closer. The words that are very rare are misspellings, and it is hard to create a meaningful embedding for them without overfitting. Therefore, such words are ignored.

From a computational point of view, the updates of output embeddings are expensive. This is caused by applying the softmax over a lexicon of $d$ words, which requires an update of each $\overline{v}_j$. One possibility is to use a *hierarchical softmax model* [337, 339, 420]. A more popular option changes the loss function (and neural architecture) slightly, which is referred to as *skipgram with negative sampling*. This approach is discussed in the next section.

### Skip-Gram with Negative Sampling

In the *skip-gram with negative sampling (SGNS)* model [339], both presence or absence of word-context pairs are used for training. The basic idea is that instead of directly predicting each of the $m$ words in the context window, we independently try to predict whether or not each of the $d$ words in the lexicon is present in the window. In other words, the final layer of Figure 3.15 is not a softmax prediction, but a Bernoulli layer of sigmoids. The output unit for each word at each context position in Figure 3.15 is a sigmoid providing a probability value that the position takes on that word. As the ground-truth values are also available, it is possible to use the logistic loss function over all the words. Therefore, in this point of view, even the prediction problem is defined differently. Of course, it is computationally inefficient to try to make binary predictions for all $d$ words, but the fact that the predictions of the different words are independent allows one to use only a subset of the negative outputs (since most words in the lexicon are not present). Therefore, the SGNS approach uses all the positive words in a context window and a *sample* of negative words. The number of negative samples is $k$ times the number of positive samples. Here, $k$ is a parameter controlling the sampling rate. Negative sampling becomes essential in this modified prediction problem to avoid learning trivial weights that predict all examples to 1. In other words, we cannot choose to avoid negative samples entirely (i.e., we cannot set $k = 0$). *This model is extremely similar to the recommender systems model with incomplete outputs*, as discussed in section 3.5. The main difference is that the "unavailability" of some of the outputs is caused by *voluntary* negative sampling rather than paucity of observed ratings (as in recommender systems).

How does one generate the negative samples? The vanilla unigram distribution samples words in proportion to their relative frequencies $f_1 \ldots f_d$ in the corpus. Better results are obtained [339] by sampling words in proportion to $f_j^{3/4}$ rather than $f_j$. As in all *word2vec* models, let $U$ be a $d \times p$ matrix representing the input embedding, and $V$ be a $d \times p$ matrix representing the output embedding. Let $\overline{u}_i$ be the $p$-dimensional row of $U$ (input embedding of $i$th word) and $\overline{v}_j$ be the $p$-dimensional row of $V$ (output embedding of $j$th word). Let $\mathcal{P}$ be the set of positive target-context word pairs in a context window, and $\mathcal{N}$ be the set

of negative target-context word pairs which are created by sampling. Therefore, the size of $\mathcal{P}$ is equal to the context window $m$, and that of $\mathcal{N}$ is $m \cdot k$. Then, the (minimization) objective function for each context window is obtained by summing up the logistic loss over the $m$ positive samples and $m \cdot k$ negative samples:

$$O = -\sum_{(i,j)\in\mathcal{P}} \log(P[\text{Predict } (i,j) \text{ to } 1]) - \sum_{(i,j)\in\mathcal{N}} \log(P[\text{Predict } (i,j) \text{ to } 0]) \tag{3.47}$$

$$= -\sum_{(i,j)\in\mathcal{P}} \log\left(\frac{1}{1+\exp(-\overline{u}_i \cdot \overline{v}_j)}\right) - \sum_{(i,j)\in\mathcal{N}} \log\left(\frac{1}{1+\exp(\overline{u}_i \cdot \overline{v}_j)}\right) \tag{3.48}$$

This modified objective function is used in the skip-gram with negative sampling (SGNS) model in order to update the weights of $U$ and $V$. SGNS is mathematically different from the basic skip-gram model discussed earlier. SGNS is not only efficient, but it also provides the best results among the different variants of skip-gram models.

**What Is the Actual Neural Architecture of SGNS?**

Even though the original *word2vec* paper seems to treat SGNS as an efficiency optimization of the skip-gram model, it is using a fundamentally different architecture in terms of the activation function used in the final layer. Unfortunately, the original *word2vec* paper does not explicitly point this out (and only provides the changed objective function), which causes confusion.

The modified neural architecture of SGNS is as follows. The softmax layer is no longer used in the SGNS implementation. Rather, each observed value $y_{ij}$ in Figure 3.15 is *independently* treated as a *binary* outcome, rather than as a multinomial outcome in which the probabilistic predictions of different outcomes at a contextual position depend on one another. Instead of using softmax to create the prediction $\hat{y}_{ij}$, it uses the sigmoid activation to create probabilistic predictions $\hat{y}_{ij}$, whether each $y_{ij}$ is 0 or 1. Then, one can add up the log loss of $\hat{y}_{ij}$ with respect to observed $y_{ij}$ over all $m \cdot d$ possible values of $(i, j)$ to create the full loss function of a context window. However, this is impractical because the number of zero values of $y_{ij}$ is too large and zero values are noisy anyway. Therefore, SGNS uses negative sampling to approximate this *modified* objective function. This means that for each context window, we are backpropagating from only a subset of the $m \cdot d$ outputs in Figure 3.15. The size of this subset is $m + m \cdot k$. This is where efficiency is achieved. However, since the final layer uses binary predictions (with sigmoids), it makes the SGNS architecture fundamentally different from the vanilla skip-gram model even in terms of the basic neural network it uses (i.e., logistic instead of softmax activation). The difference between the SGNS model and the vanilla skip-gram model is analogous to the difference between the Bernoulli and multinomial models in naïve Bayes classification (with negative sampling applied only to the Bernoulli model). Obviously, one cannot be considered a direct efficiency optimization of the other.

### 3.6.3 Word2vec (SGNS) is Logistic Matrix Factorization

Even though the work in [293] shows an *implicit* relationship between *word2vec* and matrix factorization, we provide a more direct relationship here. The architectures of the skip-gram models look suspiciously similar to those used in row index to value prediction in recommender systems (cf. section 3.5). The use of a backpropagation from a subset of observed outputs is similar to the negative sampling idea, except that the dropping of

outputs in negative sampling is performed for the purpose of efficiency. However, unlike the linear outputs of Figure 3.12 in section 3.5, the SGNS model uses logistic outputs to model binary predictions. The SGNS model of *word2vec* can be simulated with logistic matrix factorization. To understand the similarity with the problem setting of section 3.5, one can understand the predictions of a particular word-context window using the following triplets:

$$\langle \text{WordId} \rangle, \langle \text{Context WordId} \rangle, \langle 0/1 \rangle$$

Each context window produces $m \cdot d$ such triplets, although negative sampling only uses $m \cdot k + m$ of them, and *mini-batches* them during training. This mini-batching is another source of the difference between the architectures between Figures 3.12 and 3.15, wherein the latter has $m$ different groups of outputs to accommodate $m$ positive samples. However, these differences are relatively superficial, and one can still use logistic matrix factorization to represent the underlying model.

Let $B = [b_{ij}]$ be a binary matrix in which the $(i, j)$th value is 1 if word $j$ occurs at least once in the context of word $i$ in the data set, and 0 otherwise. The weight $c_{ij}$ for any word $(i, j)$ that occurs in the corpus is defined by the number of times word $j$ occurs in the context of word $i$. The weights of the zero entries in $B$ are defined as follows. For each row $i$ in $B$ we sample $k \sum_j b_{ij}$ different entries from row $i$, among the entries for which $b_{ij} = 0$, and the frequency with which the $j$th word is sampled is proportional to $f_j^{3/4}$. These are the negative samples, and one sets the weights $c_{ij}$ for the negative samples (i.e., those for which $b_{ij} = 0$) to the number of times that each entry is sampled. As in *word2vec*, the $p$-dimensional embeddings of the $i$th word and $j$th context are denoted by $\overline{u}_i$ and $\overline{v}_j$, respectively. The simplest way of factorizing is to use *weighted* matrix factorization of $B$ with the Frobenius norm:

$$\text{Minimize}_{U,V} \sum_{i,j} c_{ij}(b_{ij} - \overline{u}_i \cdot \overline{v}_j)^2 \tag{3.49}$$

Even though the matrix $B$ is of size $O(d^2)$, this matrix factorization only has a limited number of nonzero terms in the objective function, which have $c_{ij} > 0$. These weights are dependent on co-occurrence counts, but some zero entries also have positive weight. Therefore, the stochastic gradient-descent steps only have to focus on entries with $c_{ij} > 0$. Each cycle of stochastic gradient-descent is *linear* in the number of non-zero entries, as in the SGNS implementation of *word2vec*.

However, this objective function also looks somewhat different from *word2vec*, which has a logistic form. Just as it is advisable to replace linear regression with logistic regression in supervised learning of binary targets, one can use the same trick in matrix factorization of binary matrices [235]. We can change the squared error term to the familiar likelihood term $L_{ij}$, which is used in logistic regression:

$$L_{ij} = \left| b_{ij} - \frac{1}{1 + \exp(\overline{u}_i \cdot \overline{v}_j)} \right| \tag{3.50}$$

The value of $L_{ij}$ always lies in the range $(0, 1)$, and higher values indicate greater likelihood (which results in a maximization objective). The modulus in the above expression flips the sign only for the negative samples in which $b_{ij} = 0$. Now, one can optimize the following objective function in minimization form:

$$\text{Minimize}_{U,V} J = -\sum_{i,j} c_{ij} \log(L_{ij}) \tag{3.51}$$

The main difference from the objective function (cf. Equation 3.48) of *word2vec* is that this is a global objective function over all matrix entries, rather than a local objective function over a particular context window. Using mini-batch stochastic gradient-descent in matrix factorization (with an appropriately chosen mini-batch) makes the approach almost identical to *word2vec*'s backpropagation updates.

How can one interpret this type of factorization? Instead of $B \approx UV$, we have $B \approx f(UV)$, where $f(\cdot)$ is the sigmoid function. More precisely, this is a *probabilistic* factorization in which one computes the product of matrices $U$ and $V$, and then applies the sigmoid function to obtain the parameters of the Bernoulli distribution from which $B$ is generated:

$$P(b_{ij} = 1) = \frac{1}{1 + \exp(-\overline{u}_i \cdot \overline{v}_j)} \qquad \text{[Matrix factorization analog of logistic regression]}$$

It is also easy to verify from Equation 3.50 that $L_{ij}$ is $P(b_{ij} = 1)$ for positive samples and $P(b_{ij} = 0)$ for negative samples. Therefore, the objective function of the factorization is in the form of log-likelihood maximization. This type of logistic matrix factorization is commonly used [235] in recommender systems with binary data (e.g., user click-streams).

### Gradient Descent

It is also helpful to examine the gradient-descent steps of the factorization. One can compute the derivative of $J$ with respect to the input and output embeddings:

$$\begin{aligned}\frac{\partial J}{\partial \overline{u}_i} &= -\sum_{j:b_{ij}=1} \frac{c_{ij}\overline{v}_j}{1 + \exp(\overline{u}_i \cdot \overline{v}_j)} + \sum_{j:b_{ij}=0} \frac{c_{ij}\overline{v}_j}{1 + \exp(-\overline{u}_i \cdot \overline{v}_j)} \\ &= \underbrace{-\sum_{j:b_{ij}=1} c_{ij} P(b_{ij} = 0)\overline{v}_j}_{\text{Positive Mistakes}} + \underbrace{\sum_{j:b_{ij}=0} c_{ij} P(b_{ij} = 1)\overline{v}_j}_{\text{Negative Mistakes}}\end{aligned}$$

$$\begin{aligned}\frac{\partial J}{\partial \overline{v}_j} &= -\sum_{i:b_{ij}=1} \frac{c_{ij}\overline{u}_i}{1 + \exp(\overline{u}_i \cdot \overline{v}_j)} + \sum_{i:b_{ij}=0} \frac{c_{ij}\overline{u}_i}{1 + \exp(-\overline{u}_i \cdot \overline{v}_j)} \\ &= \underbrace{-\sum_{i:b_{ij}=1} c_{ij} P(b_{ij} = 0)\overline{u}_i}_{\text{Positive Mistakes}} + \underbrace{\sum_{i:b_{ij}=0} c_{ij} P(b_{ij} = 1)\overline{u}_i}_{\text{Negative Mistakes}}\end{aligned}$$

The optimization procedure uses gradient descent to convergence:

$$\overline{u}_i \Leftarrow \overline{u}_i - \alpha \frac{\partial J}{\partial \overline{u}_i} \quad \forall i$$
$$\overline{v}_j \Leftarrow \overline{v}_j - \alpha \frac{\partial J}{\partial \overline{v}_j} \quad \forall j$$

It is noteworthy that the derivatives can be expressed in terms of the probabilities of making mistakes in predicting $b_{ij}$. This is common in gradient descent with log-likelihood optimization. It is also noteworthy that the derivative of the SGNS objective in Equation 3.48 yields a similar form of the gradient. The only difference is that the derivative of the SGNS objective is expressed over a smaller batch of instances, defined by a context window. We can also solve the probabilistic matrix factorization with mini-batch stochastic gradient descent.

With an appropriate choice of the mini-batch, the stochastic gradient descent of matrix factorization becomes identical to the backpropagation update of SGNS. The only difference is that SGNS samples negative entries *for each set of updates* on the fly, whereas matrix factorization fixes the negative samples up front. Of course, on-the-fly sampling can also be used with matrix factorization updates. The similarity of SGNS to matrix factorization can also be inferred by observing that the architecture of Figure 3.15(b) is almost identical to the matrix factorization architecture for recommender systems in Figure 3.12. As in the case of recommender systems, SGNS has missing (negative) entries. This is caused by the fact the negative sampling uses only a subset of the zero values. The only difference between the two cases is that the architecture of SGNS caps the output layer with sigmoid units, whereas a linear layer is used for recommender systems. However, recommender systems with implicit feedback use *logistic matrix factorization* [235], which is similar to the *word2vec* setting.

## 3.7 Simple Neural Architectures for Graph Embeddings

Large networks have become very common because of their ubiquity in many social- and Web-centric applications. Graphs are structural entries containing *nodes* and *edges* connecting them. For example, in a social network, each person is a node, and a friendship link between two people is an edge. A friendship link is considered *undirected* because it is symmetric between the two nodes, whereas a follower-followee link is considered directed.

Graphs are typically represented with the use of *adjacency matrices*. For a graph with $n$ nodes, its adjacency matrix $A$ is an $m \times n$ matrix in which the $(i, j)$th entry is given by the weight of the directed edge from node $i$ to node $j$. In the event that the edges are not directed, the adjacency matrix symmetric.

In this particular exposition, we consider the case of very large networks like the Web, a social network, or a communication network. The goal is to embed the nodes into feature vectors, so that the graph captures the relationships between nodes. For simplicity we consider undirected graphs, although directed graphs with weights on the edges can be easily handled with very few changes to the exposition below.

Consider an $n \times n$ adjacency matrix $B = [b_{ij}]$ for a graph with $n$ nodes. The entry $b_{ij}$ is 1 if an undirected edge exists between nodes $i$ and $j$. Furthermore, the matrix $B$ is symmetric, because we have $b_{ij} = b_{ji}$ for an undirected graph. In order to determine the embedding, we would like to determine two $n \times p$ factor matrices $U$ and $V$, so that $B$ can be derived as a function of $UV^T$. In the simplest case, one can set $B$ to exactly $UV^T$, which is no different than a traditional matrix factorization method for factoring graphs [4]. However, for binary matrices, one can do better and use logistic matrix factorization instead. In other words, each entry of $B$ is generated using the matrix of Bernoulli parameters in $f(UV^T)$, where $f(\cdot)$ is the element-wise application of the sigmoid function to each entry of the matrix in its argument:

$$f(x) = \frac{1}{1 + \exp(-x)} \tag{3.52}$$

Therefore, if $\overline{u}_i$ is the $i$th row of $U$ and $\overline{v}_j$ is the $j$th row of $V$, we have the following:

$$b_{ij} \sim \text{Bernoulli distribution with parameter } f(\overline{u}_i \cdot \overline{v}_j) \tag{3.53}$$

This type of generative model is typically solved using a log-likelihood model. Furthermore, *the problem formulation is identical to the logistic matrix factorization equivalent of the SGNS model in word2vec.*

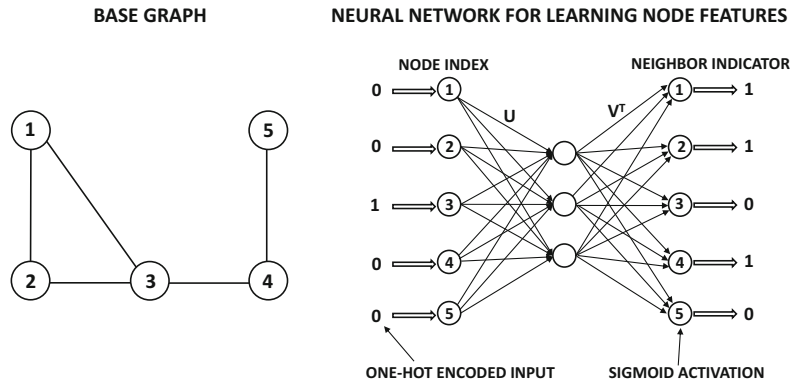



Figure 3.16: A graph of five nodes is shown together with a neural architecture for row index to neighbor indicator mapping. The shown input and output represent node 3 and its neighbors. Note the similarity to Figure 3.12. The main difference is that there are no missing values above, and the number of inputs is the same as the number of outputs for a square matrix. Both input and outputs are binary vectors. However, if negative sampling is used with sigmoid activation, most output nodes with zero values may be dropped.

Note that all *word2vec* models are logistic/multinomial variants of the model in Figure 3.12 that maps row indexes to values with linear activation. In order to explain this point, we show the neural architecture in Figure 3.16 for a toy graph containing 5 nodes. The input is the one-hot encoded index of a row in $B$ (i.e., node), and the output is the list of all 0/1 values for all nodes in the network. In this particular case, we have shown the input for node 3 and its corresponding output. Since the node 3 has three neighbors, the output vector contains three 1s. Note that this architecture is not very different from Figure 3.12 except that it uses sigmoid activations at the output (rather than linear activations). Furthermore, since the number of 0s is usually much greater[3] than the number of 1s in the output, it is possible to drop many of the 0s with the use of negative sampling. This type of negative sampling will create a situation similar to that of Figure 3.13. With this neural architecture, the gradient-descent steps will be identical to the SGNS model of *word2vec*. The main difference is that a node appears at most once as a neighbor of another node, whereas a word might appear more than once in the context of another word. Allowing arbitrary counts on the edges takes away this distinction.

### 3.7.1 Handling Arbitrary Edge Counts

The aforementioned discussion assumes that the weight of each edge is binary. Consider a setting in which an arbitrary count $c_{ij}$ is associated with the edge $(i, j)$. In such cases, both positive and negative sampling are required. The first step is to sample an edge $(i, j)$ from the network with probability proportional to $c_{ij}$. The input is, therefore, a one-hot encoded vector of the node at one end point (say, $i$) of this edge. The output is the one-hot encoding of node $j$. By default, both the input and output are $n$-dimensional vectors. However, if negative sampling is used, then one can reduce the output vector to a $(k + 1)$-

[3]This fact is not evident in the toy example of Figure 3.16. In practice, the degree of a node is a tiny fraction of the total number of nodes. For example, a person might have 100 friends in a social network of millions of nodes.

dimensional vector. Here, $k \ll n$ is a parameter that defines the sampling rate. A total of $k$ negative nodes are sampled with probabilities proportional to their (weighted) degrees[4] and the outputs of these nodes are 0s. One can compute the log-likelihood loss by treating each output as the outcome of a Bernoulli trial, where the parameter of the Bernoulli trial is the output of the sigmoid activation function. The gradient descent is performed with respect to this loss. This variant is an almost exact simulation of the SGNS variant of the *word2vec* model.

### 3.7.2 Beyond One-Hop Structural Models

The approach discussed in the previous section can be considered a one-hop structural model, wherein the immediate adjacency of nodes can be leveraged in order create the embeddings. In practice, one would like to use more structural information in order to improve the quality of the representation. There are two ways of achieving this goal, which are discussed below.

First, one can use various types of preprocessing in order to replace the edge counts $c_{ij}$ with multi-hop walk counts. For example, one can use random walks on the graph to (indirectly) generate $c_{ij}$. In this case, $c_{ij}$ can be interpreted as the number of times that node $j$ appears in the neighborhood of node $i$ because it was included in a breadth-first or depth-first walk starting at node $i$. One can view the value of $c_{ij}$ in the walk-based models as providing a more robust measure of the affinity between nodes $i$ and $j$, as compared to the raw weights in the original graph. Of course, there is nothing sacrosanct about using a random walk to improve the robustness of $c_{ij}$. This broad idea forms the basis of algorithms like *node2vec* [171] and *DeepWalk* [385].

The number of choices is almost unlimited in terms of how to generate this type of affinity value. All *link prediction methods* [304] generate such affinity values between pairs of nodes, which can be used to modify edge weights. Examples include the Adamic-Adar measure, the Jaccard coefficient, and the Katz measure, all of which are discussed in [304]. The Katz measure, is also closely related to the number of random walks between a pair of nodes.

The approach of using preprocessing can be viewed as a form of hand-crafted feature engineering in order to improve the input representation. This, of course, goes against the raison d'etre of neural networks, which are intended to be end-to-end systems. Such forms of multi-hop feature engineering can be achieved with the use of *graph neural networks*, which are discussed in detail in Chapter 10. Graph neural networks construct a neural network whose hidden-layer architecture heavily depends on the structure of the base graph for which it finds an embedding. We defer a further discussion to Chapter 10.

### 3.7.3 Multinomial Model

The vanilla skip-gram model of *word2vec* is a multinomial model. It is also possible to use a multinomial model to create the embedding. The only difference is that the final layer of the neural network in Figure 3.16 needs to use softmax activation (instead of the sigmoid activation function). Furthermore, negative sampling is not used in the multinomial model, and both input and output layers contain exactly $n$ nodes. As in the SGNS model, a single edge $(i, j)$ is sampled with probability proportional to $c_{ij}$ to create each input-output pair. The input is the one-hot encoding of $i$ and the output is the one-hot encoding of $j$. One

[4]The weighted degree of node $j$ is $\sum_r c_{rj}$.

can also use mini-batch sampling of edges to improve performance. The stochastic gradient-descent steps of this model are virtually similar to those used in the vanilla skip-gram model of *word2vec*.

## 3.8 Summary

This chapter discusses a number of neural models for supervised and unsupervised learning. The goal was to show that many of the traditional models used in machine learning are instantiations of relatively simple neural models. When a traditional machine learning technique like singular value decomposition is generalized to a neural representation, the results are roughly equivalent. However, the advantage of neural models is that they can usually be generalized to more powerful nonlinear models. Furthermore, it is relatively easy to experiment with nonlinear variants of traditional machine learning models with the use of neural networks. This chapter also discusses several practical applications of shallow models like recommender systems, text, and graph embeddings.

## 3.9 Bibliographic Notes and Software Resources

The perceptron algorithm was proposed by Rosenblatt [421], and a detailed discussion may be found in [421]. The Widrow-Hoff algorithm was proposed in [550] and is closely related to Tikhonov-Arsenin's work [516]. The Fisher discriminant was proposed by Ronald Fisher [125] in 1936, and is a specific case of the family of linear discriminant analysis methods [333]. Even though the Fisher discriminant uses an objective function that appears to be different from least-squares regression, it turns out to be a special case of least-squares regression in which the binary response variable is used as the regressand [40]. A detailed discussion of generalized linear models is provided in [331]. A variety of procedures such as *generalized iterative scaling*, *iteratively reweighted least-squares*, and *gradient descent* for multinomial logistic regression are discussed in [188]. The support-vector machine is generally credited to Cortes and Vapnik [85], although the primal method for $L_2$-loss SVMs was (indirectly) proposed several years earlier by Hinton [200] in the context of a neural network! This approach repairs the loss function in least-squares classification by keeping only one-half of the quadratic loss curve and setting the remaining to zero, so that it looks like a smooth version of hinge loss (try this on Figure 3.5). The specific significance of this contribution was lost within the broader literature on neural networks. The relationship of SVMs to least-squares classification is evident from related works [417], where it becomes evident that quadratic and hinge-loss SVMs are natural variations of regularized $L_2$-loss (i.e., Fisher discriminant) and $L_1$-loss classification that use the binary class variables as the regression responses [146]. The Weston-Watkins multiclass SVM was introduced in [548]. It was shown in [418] that the one-against-all approach to generalizing multiple classes seems to be as effective as the tightly integrated multiclass variants.

The earliest introduction of the autoencoder (in a more general form) is given in the backpropagation paper [424]. More detailed discussions of the autoencoder during its early years were provided in [47, 281]. A discussion of single-layer unsupervised learning may be found in [80]. Many types of regularized autoencoders (which generalize well to out-of-sample data with specific properties) are discussed in Chapter 5. The use of autoencoders for outlier detection is explored in [64, 191, 589], and a survey on the use in clustering is provided in [9]. The use of Restricted Boltzmann Machines for dimensionality reduction is discussed in [208].

An item-based autoencoder is discussed in [456], and this approach is a neural generalization of item-based neighborhood regression [261]. The main difference is that the regression weights are regularized with a constricted hidden layer. Similar works with different types of item-to-item models with the use of de-noising autoencoders are discussed in [488, 555]. A more direct generalization of matrix factorization methods may be found in [196], although the approach in [196] is slightly different from the simpler approach presented in this chapter. The incorporation of content in building recommender systems for deep learning is discussed in [533]. A multiview deep learning approach, which has also been extended to temporal recommender systems in a later work [481], is proposed in [112]. A survey of deep learning methods for recommenders may be found in [584]. The application of dimensionality reduction to recommender systems with the use of restricted Boltzmann machines may be found in [431].

The *word2vec* model was proposed in [337, 339], and a detailed exposition may be found in [420]. An alternative to the SGNS model for achieving greater efficiency is to use a hierarchical softmax layer, where each class corresponds to a leaf of a binary tree. For example, if the prediction is a target word, one can use the *WordNet* hierarchy [341] to guide the grouping. Further reorganization may be needed [355] because the *WordNet* hierarchy is not exactly a binary tree. Another option is to use Huffman encoding in order to create the binary tree [337, 339]. The basic ideas in *word2vec* have been extended to sentence- and paragraph-level embeddings, with a model, which is referred to as *doc2vec* [278]. An alternative of *word2vec* that uses a different type of matrix factorization is GloVe [384]. Multi-lingual word embeddings are presented in [10]. The extension of *word2vec* to graphs with node-level embeddings is provided in the *DeepWalk* [385] and *node2vec* [171] models. A detailed discussion of network embedding is provided in Chapter 10.

### Software Resources

Machine learning models like linear regression, SVMs, and logistic regression are available from *scikit-learn* [611]. The DISSECT (Distributional Semantics Composition Toolkit) [612] is a toolkit that uses word co-occurrence counts in order to create embeddings. The GloVe method is available from Stanford NLP [613] and the **gensim** library [410]. The *word2vec* tool is available under the Apache license [615], and as a **TensorFlow** version [616]. The **gensim** library has Python implementations of *word2vec* and *doc2vec* [410]. Java versions of *doc2vec*, *word2vec*, and GloVe may be found in the **DeepLearning4j** repository [614]. In several cases, one can simply download pre-trained versions of the representations (on a large corpus that is considered generally representative of text) and use them directly, as a convenient alternative to training for the specific corpus at hand. The *node2vec* software is available from the original author at [617].

## 3.10 Exercises

**1.** Consider the following loss function for training pair $(\overline{X}, y)$:

$$L = \max\{0, a - y(\overline{W} \cdot \overline{X})\}$$

The test instances are predicted as $\hat{y} = \text{sign}\{\overline{W} \cdot \overline{X}^T\}$. A value of $a = 0$ corresponds to the perceptron criterion and a value of $a = 1$ corresponds to the SVM. Show that any value of $a > 0$ leads to the SVM with an unchanged optimal solution when no regularization is used. What happens when regularization is used?

2. Based on Exercise 1, formulate a generalized objective for the Weston-Watkins SVM.

3. Consider the unregularized perceptron update for binary classes with learning rate $\alpha$. Show that using any value of $\alpha$ is inconsequential in the sense that it only scales up the weight vector by a factor of $\alpha$. Show that these results also hold true for the multiclass case. Do the results hold true when regularization is used?

4. Show that if the Weston-Watkins SVM is applied to a data set with $k = 2$ classes, the resulting updates are equivalent to the binary SVM updates discussed in this chapter.

5. Show that if multinomial logistic regression is applied to a data set with $k = 2$ classes, the resulting updates are equivalent to logistic regression updates.

6. Implement the softmax classifier using a deep-learning library of your choice.

7. In linear-regression-based neighborhood models, the rating of an item is predicted as a weighted combination of the ratings of other items of the same user, where the item-specific weights are learned with linear regression. Show how you can construct an autoencoder architecture to create this type of model. Discuss the relationship of this architecture with the matrix factorization architecture.

8. **Logistic matrix factorization:** Consider an autoencoder which has an input layer, a single hidden layer containing the reduced representation, and an output layer with sigmoid units. The hidden layer has linear activation:

   (a) Set up a negative log-likelihood loss function for the case when the input data matrix is known to contain binary values from $\{0, 1\}$.

   (b) Set up a negative log-likelihood loss function for the case when the input data matrix contains real values from $[0, 1]$.

9. **Non-negative matrix factorization with autoencoders:** Let $D$ be an $n \times d$ data matrix with non-negative entries. Show how you can approximately factorize $D \approx UV^T$ into two non-negative matrices $U$ and $V$, respectively, by using an autoencoder architecture with $d$ inputs and outputs. [Hint: Choose an appropriate activation function in the hidden layer, and modify the gradient-descent updates.]

10. **Probabilistic latent semantic analysis:** Refer to [216] for a definition of probabilistic latent semantic analysis. Propose a modification of the approach in Exercise 9 for probabilistic latent semantic analysis. [Hint: What is the relationship between non-negative matrix factorization and probabilistic latent semantic analysis?]

11. **Simulating a model combination ensemble:** In machine learning, a model combination ensemble averages the scores of multiple models in order to create a more robust classification score. Discuss how you can approximate the averaging of an Adaline and logistic regression with a two-layer neural network. Discuss the similarities and differences of this architecture with an actual model combination ensemble when backpropagation is used to train it. Show how to modify the training process so that the final result is a fine-tuning of the model combination ensemble.

12. **Simulating a stacking ensemble:** In machine learning, a stacking ensemble creates a higher-level classification model on top of features learned from first-level classifiers. Discuss how you can modify the architecture of Exercise 11, so that the first level of

classifiers correspond to an Adaline and a logistic regression classifier and the higher-level classifier corresponds to a support vector machine. Discuss the similarities and differences of this architecture with an actual stacking ensemble when backpropagation is used to train it. Show how you can modify the training process of the neural network so that the final result is a fine-tuning of the stacking ensemble.

**13.** Show that the stochastic gradient-descent updates of the perceptron, Widrow-Hoff learning, SVM, and logistic regression are all of the form $\overline{W} \Leftarrow \overline{W}(1 - \alpha\lambda) + \alpha y[\delta(\overline{X}, y)]\overline{X}^T$. Here, the mistake function $\delta(\overline{X}, y)$ is $1 - y(\overline{W} \cdot \overline{X}^T)$ for least-squares classification, an indicator variable for perceptron/SVMs, and a probability value for logistic regression. Assume that $\alpha$ is the learning rate, and $y \in \{-1, +1\}$. Write the specific forms of $\delta(\overline{X}, y)$ in each case.

**14.** The linear autoencoder discussed in the chapter is applied to each $d$-dimensional row of the $n \times d$ data set $D$ to create a $k$-dimensional representation. The encoder weights contain the $k \times d$ weight matrix $W$ and the decoder weights contain the $d \times k$ weight matrix $V$. Therefore, the reconstructed representation is $DW^TV^T$, and the aggregate loss value $\|DW^TV^T - D\|_F^2$ is minimized over the entire training data set.

(a) For a fixed value of $V$, show that the optimal matrix $W$ must satisfy $D^TD(W^TV^TV - V) = 0$.

(b) Use (a) to show that if the $n \times d$ matrix $D$ has rank $d$, we have $W^TV^TV = V$.

(c) Use (b) to show that $W = (V^TV)^{-1}V^T$. Assume that $V^TV$ is invertible.

(d) Repeat exercise parts (a), (b), and (c), when the encoder-decoder weights are tied as $W = V^T$. Show that the columns of $V$ must be orthonormal.

**15.** The autoencoder with shared weights in section 3.4.1.2 simulates SVD in terms of finding the same subspace in its weight matrix $V$ as the dominant right singular vectors, but the columns of this weight matrix represent a rotation of the dominant singular vectors of SVD. Discuss how you can use iterative training of an autoencoder architecture with successive addition of hidden units to exactly determine the dominant singular vectors of SVD.

**16. Maximum margin matrix factorization:** The hinge loss of the SVM is designed for binary classification. Use this inspiration to design a neural architecture for matrix factorization of binary matrices with entries drawn from $\{-1, +1\}$. Design a loss function like the hinge loss for matrix factorization.

**17. Multinomial matrix factorization:** Suppose that you have a matrix containing categorical entries like Zip Codes, ethnic identity, and so on. Discuss how you can leverage an autoencoder architecture in conjunction with appropriate activation and loss functions in order to perform the factorization. [Hint: Look at the output layer in softmax regression.]

**18.** Most matrices in the real world are mixed, and may contain a combination of real-valued, binary, and categorical columns. How do your answers to Exercises 16 and 17 provide a path to dealing with such matrices?

**19. Incomplete matrix with categorical entries:** Suppose that you have an incomplete matrix in which each entry is a categorical value value, and the different columns of the matrix correspond to different categorical attributes (e.g., Zip Code or ethnicity). You

want to predict missing entries. Draw inspirations from the recommender architecture discussed in the chapter to propose a neural model for this case.

**20.** Discuss how you would modify the recommender architecture discussed in the chapter when you have a $d$-dimensional attribute associated with each item (in addition to an incomplete matrix of ratings). Your architecture should build on top of the one already presented in the chapter.

**21.** For the graph embedding algorithm discussed in the chapter, what is the difference in the training procedure for undirected and directed graphs?

**22. Open-ended:** Using your insights from the *word2vec* CBOW model to propose a neural network for completing a set from its subset. Each set is an unordered set of discrete (possibly repeating) elements from a finite universe.


# Chapter 4

# Deep Learning: Principles and Training Algorithms

> "I hated every minute of training, but I said, 'Don't quit. Suffer now and live the rest of your life as a champion.'" –Muhammad Ali

## 4.1 Introduction

The great power of deep learning models comes with computational challenges. One key point is that the backpropagation algorithm is rather *unstable* to minor changes in the algorithmic setting, such as the initialization point used by the approach. This instability is particularly significant when one is working with very deep networks.

Neural network parameter optimization is a *multivariable optimization problem*. These variables correspond to the weights of the connections in various layers. Multivariable optimization problems often face stability challenges because one must perform the steps along each direction in the "right" proportion. This turns out to be particularly hard in deep networks, where the this proportion is hard to control. One issue is that *a gradient only provides a rate of change over an infinitesimal horizon in each direction*, whereas an actual step has a finite length. One needs to choose steps of reasonable size in order to make any real progress in optimization. The problem is that the gradients do change over a step of finite length, and in some cases they change drastically. The complex optimization surfaces presented by deep networks are particularly treacherous in this respect, and the problem is exacerbated with poorly chosen settings (such as the initialization point or the normalization of the input features). As a result, the (easily computable) steepest-descent direction is often not the best direction to use for retaining the ability to use large steps. Small step sizes lead to slow progress, whereas the optimization surface might change in unpredictable ways with the use of large step sizes. All these issues make neural network optimization more difficult than would seem at first glance. However, many of these problems can be



C. C. Aggarwal, *Neural Networks and Deep Learning*,
https://doi.org/10.1007/978-3-031-29642-0_4

avoided by carefully tailoring the gradient-descent steps to be more robust to the nature of the optimization surface. This chapter will discuss such algorithms.

**Chapter Organization**

This chapter is organized as follows. The next section discusses the benefits of increased depth. The common challenges of training deep networks are presented in section 4.3. Simple solutions based on neural architectures are presented in section 4.4. Gradient-descent strategies for deep learning are discussed in section 4.5. The Newton method is introduced in section 4.6, and its fast approximations are discussed in section 4.7. Batch normalization methods are introduced in section 4.8. A discussion of accelerated implementations of neural networks is found in section 4.9. The summary is presented in section 4.10.

## 4.2 Why Is Depth Beneficial?

As discussed in Chapter 1, the key to the benefits of depth lie in the use of nonlinear activation functions. In fact, the repeated composition of a linear function yields another linear function (cf. section 1.5); therefore, it does not increase the power of the learning algorithm. However, composing nonlinear functions repeatedly leads to inherently more powerful models. In general, each layer of the network constructs successively more complex features that are relevant to the predictive problem at hand. This principle is referred to as that of *hierarchical feature engineering*.

### 4.2.1 Hierarchical Feature Engineering: How Depth Reveals Rich Structure

Many deeper architectures with feed-forward architectures have multiple layers in which successive transformations of the inputs from the previous layer lead to increasingly sophisticated representations of the data. The values of each hidden layer for a particular input contain a transformed representation of the input point, which becomes increasingly informative about the target value we are trying to learn, as the layer gets closer to the output node. The nonlinear activations in intermediate layers play a critical role in the feature engineering process.

As shown in section 1.5.1 of Chapter 1, appropriately transformed feature representations are more amenable to predictive settings like classification. For example, if the final layer uses a linear separator to perform classification, the intermediate layers might transform the data to a representation where a linear separator works effectively. One way of viewing this division of labor between the hidden layers and final prediction layer is that the early layers create a feature representation that is more informative for a predictive task. The final layer then leverages this learned feature representation for prediction. This division of labor is shown in Figure 4.1. One can view this process as a kind of hierarchical feature engineering in which the features in earlier layers represent primitive characteristics of the data, whereas those in later layers represent complex characteristics with semantic significance to the class labels.

The idea behind deeper architectures is that they can better leverage *repeated regularities* in the data patterns in order to reduce the number of computational units and therefore *generalize* the learning even to areas of the data space where one does not have examples. Often these repeated regularities are learned by the neural network within the weights as the basis vectors of hierarchical features. Although a detailed proof [351] of this fact is beyond

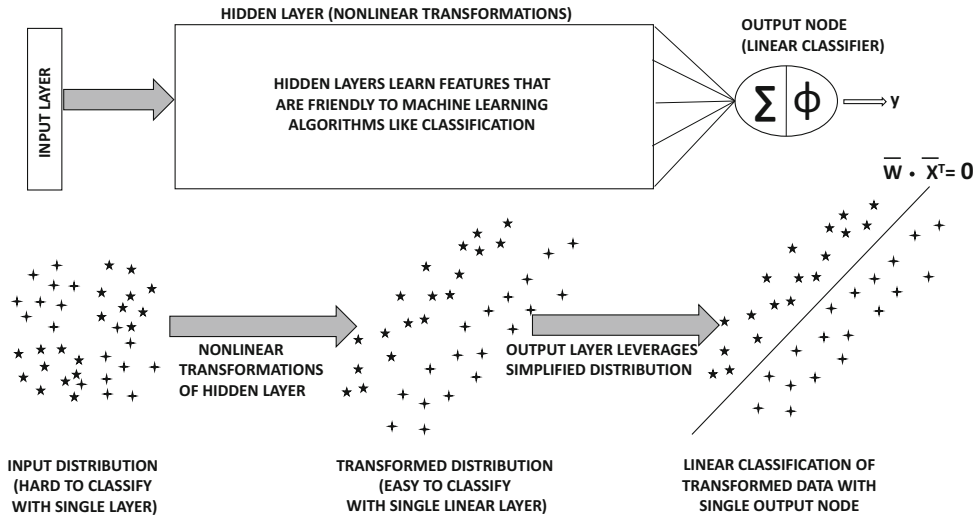



Figure 4.1: The feature engineering role of the hidden layers

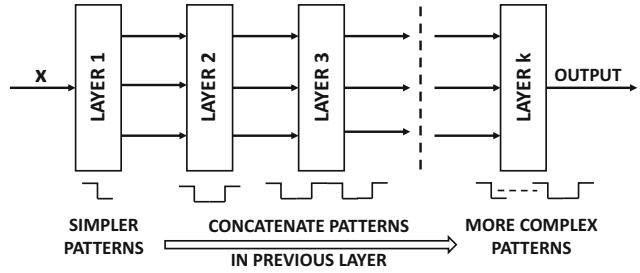



Figure 4.2: Deeper networks can learn more complex functions by composing the functions learned in earlier layers.

the scope of this book, we provide a simple example to elucidate this point. Consider a situation in which a 1-dimensional function is defined by 1024 repeated steps of the same size and height. A shallow network with one hidden layer and step activation functions would require at least 1024 units in order to model the function. However, a multilayer network would model a pattern of 1 step in the first layer, 2 steps in the next, 4 steps in the third, and $2^r$ steps in the $r$th layer. This situation is illustrated in Figure 4.2. Note that the pattern of 1 step is the simplest feature because it is repeated 1024 times, whereas a pattern of 2 steps is more complex. Therefore, the features (and the functions learned) in successive layers are hierarchically related. In this case, a total of 10 layers are required and a small number of constant nodes are required in each layer to model the joining of the two patterns from the previous layer. Therefore, the overall number of nodes required is an *order of magnitude less* than that required in the single-layer network. This means that the amount of data required for learning is also an order of magnitude less. The reason for this is that the multilayer network implicitly *looks for the repeated regularities and learns them* with a modest amount of data, rather than trying to explicitly learn every turn and twist of the target function.

One can also understand the importance of depth in terms of the fact that a multilayer neural network is a composition of functions, in which the depth of the composition corresponds to the depth of the neural network. For example if the layer $i$ computes the vector-to-vector function $f_i(\cdot)$, then the overall function computed by a neural network with $t$ computational layers is $f_t(f_{t-1}(\ldots(f_1(\overline{x}))))$ for input vector $\overline{x}$. We summarize this principle below:

> A repeated composition of relatively simple nonlinear functions (like neural network activation functions) can be used to create a very complex nonlinear function. The richness of the overall function computed by the neural network increases with the depth of the composition. Each layer of the neural network can learn successively more refined patterns from the patterns learned in previous layers.

This type of behavior is particularly evident in a visually interpretable way in some domains like convolutional neural networks for image data (cf. Chapter 9). In convolutional neural networks, the features in earlier layers capture detailed but primitive shapes like lines or edges from the data set of images. On the other hand, the features in later layers capture shapes of greater complexity like hexagons, honeycombs, and so forth, depending on the type of images provided as training data. Note that such semantically interpretable shapes often have closer correlations with class labels in the image domain. For example, almost any image will contain lines or edges, but images belonging to particular classes will be more likely to have hexagons or honeycombs. A later layer might model a complex shape like a face. On the other hand, a single layer would have difficulty in modeling every twist and turn of a face, because it would have a harder time in recognizing the repeated regularities of a human face, which are only possible with the perspective obtained by greater depth. This provides the deeper model with the ability to learn rich structure in the data. This property tends to make the representations of later layers easier to classify with simple models like linear classifiers. This process is illustrated in Figure 4.1. The features in earlier layers are used repeatedly as building blocks to create more complex features. This general principle of "putting together" simple features to create more complex features lies at the core of the successes achieved with neural networks.

## 4.3 Why Is Training Deep Networks Hard?

Increasing the depth of the network is not without its disadvantages. Deeper networks are often harder to train, and are also notoriously unstable to parameter choice. This is because depth increases the loss function complexity by recursive composition, as a result of which gradient descent behaves in unexpected ways.

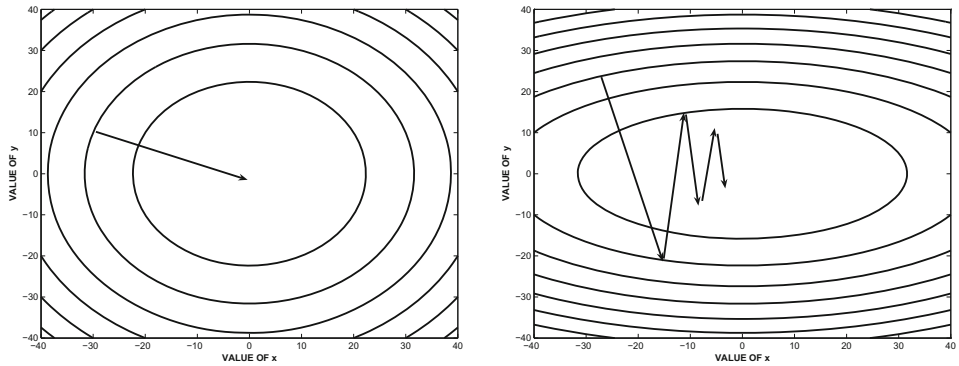
### 4.3.1 Geometric Understanding of the Effect of Gradient Ratios

An important problem that arises in the use of neural networks is the *vanishing and exploding gradient problem*. The basic idea underlying this problem (discussed in detail later in this chapter), is that the gradients of the loss function with respect to the weights in different layers is very different in magnitude. For example, the (magnitude of the) partial derivative with respect to a weight in the final layer might be $10^6$ times the magnitude of the partial derivative with respect to a weight in the first layer. Therefore, a gradient descent update that changes a parameter in the final layer by 0.2 units will (negligibly)

change the weight in the first layer by $2 \times 10^{-6}$. In other words, the gradients have *vanished* during backpropagation. Similarly, it is possible for the gradients to explode uncontrollably during backpropagation, where the magnitudes of the gradients in early layers are much larger than the ones in later layers. This can cause problems in the optimization process in being able to make updates of meaningful size.

The vanishing and exploding gradient problems are inherent to multivariable optimization, even in cases where there are no local optima. In fact, minor manifestations of this problem are encountered in almost any convex optimization problem. Therefore, in this section, we will consider the simplest possible case of a convex, quadratic objective function with a bowl-like shape and a single global minimum. In a single-variable problem, the path of steepest descent (which is the only path of descent), will always pass through the minimum point of the bowl (i.e., optimum objective function value). However, the moment we increase the number of variables in the optimization problem from 1 to 2, this is no longer the case. The key point to understand is that *with very few exceptions, the path of steepest descent in most loss functions is only an instantaneous direction of best movement, and is not the correct direction of descent in the longer term.* In other words, small steps with "course corrections" are always needed. When an optimization problem exhibits the vanishing gradient problem, it means that the only way to reach the optimum with steepest-descent updates is by using an *extremely large number of tiny updates and course corrections*, which is obviously very inefficient.'

In order to understand this point, we look at two bivariate loss functions in Figure 4.3. In this figure, the contour plots of the loss function are shown, in which each line corresponds to points in the XY-plane where the loss function has the same value. The direction of steepest descent is always perpendicular to this line. The first loss function is of the form $L = x^2 + y^2$, which takes the shape of a perfectly circular bowl, if one were to view the height as the objective function value. This loss function treats $x$ and $y$ in a symmetric way. The second loss function is of the form $L = x^2 + 4y^2$, which is an elliptical bowl. Note that this loss function is more sensitive to changes in the value of $y$ as compared to changes in the value of $x$, although the specific sensitivity depends on the position of the data point. In these cases, the *second-order* derivatives $\frac{\partial^2 L}{\partial x^2}$ and $\frac{\partial^2 L}{\partial y^2}$ are different in the case of the loss $L = x^2 + 4y^2$. The second-order derivatives are also referred to as the curvature, because they control how the slope changes in the two cases.




(a) Loss function is circular bowl $L = x^2 + y^2$

(b) Loss function is elliptical bowl $L = x^2 + 4y^2$

Figure 4.3: The effect of the shape of the loss function on steepest-gradient descent.

In the case of the circular bowl of Figure 4.3(a), the gradient points directly at the optimum solution, and one can reach the optimum in a single step, as long as the correct step-size is used. This is not quite the case in the loss function of Figure 4.3(b), in which the gradients are often more significant in the $y$-direction compared to the $x$-direction. Furthermore, the gradient never points to the optimal solution, as a result of which many course corrections are needed over the descent. A salient observation is that the steps along the $y$-direction are large, but subsequent steps undo the effect of previous steps. On the other hand, the progress along the $x$-direction is consistent but tiny. Although the situation of Figure 4.3(b) occurs in almost any optimization problem using steepest descent, the case of the vanishing gradient is an extreme manifestation of this behavior. The fact that a simple quadratic bowl (which is trivial compared to the typical loss function of a deep network) shows so much oscillation with the steepest-descent method is concerning. After all, the repeated composition of functions (as implied by the underlying computational graph) is highly *unstable* in terms of the sensitivity of the output to the parameters in different parts of the network. The problem of differing relative derivatives is extraordinarily large in real neural networks, in which we have millions of parameters and gradient ratios that vary by orders of magnitude. Furthermore, many activation functions have derivatives that are always less than 1, which tends to encourage the vanishing gradient problem by repeated multiplication of these derivatives (according to the chain rule) during backpropagation. As a result, the parameters in later layers with large descent components are often oscillating with large updates, whereas those in earlier layers make tiny but consistent updates. Therefore, neither the earlier nor the later layers make much progress in getting closer to the optimal solution. As a result, it is possible to get into situations where very little progress is made even after training for a long time. In the next section, we provide an understanding of this mechanism in the specific case of neural networks.

### 4.3.2 The Vanishing and Exploding Gradient Problems

Neural networks have millions of parameters in which some components of the gradients are extremely large as compared to others. This type of skewed gradient ratio is manifested in a particular way in neural networks, in which the partial derivatives with respect to weights in different *layers* are very different. This is primarily because of how a neural network computes a composition of functions across different layers, and how each layer of the composition incorporates a multiplicative factor to the gradient because of the mechanism of the chain rule of differential calculus. In order to understand this point, let us consider a very deep network that has a single node in each layer. We assume that there are $(m+1)$ layers, including the non-computational input layer. The weights of the edges between the various layers are denoted by $w_1, w_2, \ldots w_m$. Furthermore, assume that the sigmoid activation function $\Phi(\cdot)$ is applied in each layer. Let $x$ be the input, $h_1 \ldots h_{m-1}$ be the hidden values in the various layers, and $o$ be the final output. Let $\Phi'(h_t)$ be the derivative of the activation function in hidden layer $t$. Let $\frac{\partial L}{\partial h_t}$ be the derivative of the loss function with respect to the hidden activation $h_t$. The neural architecture is illustrated in Figure 4.4. Note that the output of each hidden layer is defined from the previous layer using the following relationship:

$$h_{t+1} = \Phi(w_{t+1} h_t)$$

Therefore, we have the following:

$$\frac{\partial h_{t+1}}{\partial h_t} = \Phi'(w_{t+1} h_t) w_{t+1}$$

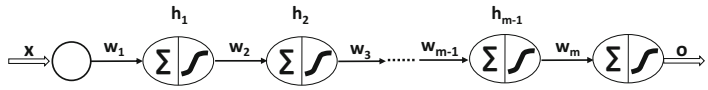



Figure 4.4: The vanishing and exploding gradient problems

One can use the chain rule in the backpropagation update to show the following relationship:

$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_{t+1}} \frac{\partial h_{t+1}}{\partial h_t} = \Phi'(w_{t+1} h_t) \cdot w_{t+1} \cdot \frac{\partial L}{\partial h_{t+1}} \tag{4.1}$$

Since the fan-in is 1 of each node, assume that the weights are initialized from a standard normal distribution. Therefore, each $w_t$ has an expected average magnitude of 1.

Let us examine the specific behavior of this recurrence in the case where the sigmoid activation is used. The derivative with a sigmoid with output $f \in (0, 1)$ is given by $f(1-f)$. This value takes on its maximum at $f = 0.5$, and therefore the value of $\Phi'(w_{t+1} h_t)$ is no more than 0.25 even at its maximum. Since the absolute value of $w_{t+1}$ is expected to be 1, it follows that each weight update will (typically) cause the value of $\frac{\partial L}{\partial h_t}$ to be less than 0.25 that of $\frac{\partial L}{\partial h_{t+1}}$. Therefore, after moving by about $r$ layers, this value will typically be less than $0.25^r$. Just to get an idea of the magnitude of this drop, if we set $r = 10$, then the gradient update magnitudes drop to $10^{-6}$ of their original values! It is also noteworthy that even though we have shown the derivatives with respect to the hidden layers, the derivatives with respect to weights are direct multiples of those with respect to hidden layers; the trends will be similar. Therefore, when backpropagation is used, the earlier layers will receive very small updates compared to the later layers. This problem is referred to as the *vanishing gradient problem*. Note that we could try to solve this problem by using an activation function with larger gradients and also initializing the weights to be larger. However, if we go too far in doing this, it is easy to end up in the opposite situation where the gradient *explodes* in the backward direction instead of vanishing. In general, unless we initialize the weight of every edge so that the product of the weight and the derivative of each activation is exactly 1, there will be considerable instability in the magnitudes of the partial derivatives. In practice, this is impossible with most activation functions because the derivative of an activation function will vary from iteration to iteration.

Although we have used an oversimplified example here with only one node in each layer, it is easy to generalize the argument to cases in which multiple nodes are available in each layer. In general, it is possible to show that the layer-to-layer backpropagation update includes a matrix multiplication (rather than a scalar multiplication). Just as repeated scalar multiplication is inherently unstable, so is repeated matrix multiplication. In particular, the loss derivatives in layer-$(i+1)$ are multiplied by a matrix referred to as the Jacobian (cf. Equation 2.21). The Jacobian contains the derivatives of the activations in layer-$(i+1)$ with respect to those in layer $i$. In certain cases like recurrent neural networks, the Jacobian is a square matrix and one can actually impose stability conditions with respect to the largest eigenvalue of the Jacobian. These stability conditions are rarely satisfied exactly, and therefore the model has an inherent tendency to exhibit the vanishing and exploding gradient problems. Furthermore, the effect of activation functions like the sigmoid tends to encourage the vanishing gradient problem. One can summarize this problem as follows:

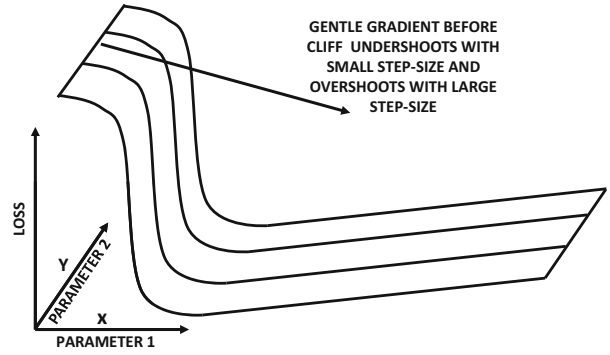



Figure 4.5: An example of a cliff in the loss surface

**Observation 4.3.1** *The relative magnitudes of the partial derivatives with respect to the parameters in different parts of the network tend to be very different, which creates problems for gradient-descent methods.*

### 4.3.3 Cliffs and Valleys

Gradient descent works with first-order derivatives. However, the steps taken in gradient descent are of a finite size, and they change along the course of the step. The rate of change of first-order derivatives can be quantified by the second-order derivatives, which is also informally referred to as the *curvature*. A particular example of a distressing topology from the perspective of gradient descent is that of a *cliff*.

An example of a loss surface is shown in Figure 4.5. In this case, there is a gently sloping surface that rapidly changes into a steeply descending surface; this type of surface is referred to as a cliff. However, if one computed only the first-order partial derivative with respect to the variable $x$ shown in the figure, one would only see a gentle slope. As a result, a small learning rate will lead to very slow learning, whereas increasing the learning rate can suddenly cause overshooting to a point far from the optimal solution. This problem is caused by the nature of the curvature (i.e., changing gradient), where the first-order gradient does not contain the information needed to control the size of the update. In many cases, the rate of change of gradient can be computed using the second-order derivative, which provides useful (additional) information.

Cliffs are not desirable because they manifest a certain level of instability in the loss function. This implies that a small change in some of the neural-network weights can either change the loss in a tiny way or suddenly change the loss by such a large amount that the resulting solution is even further away from the true optimum. As a result, it is easy to miss the optimum during a gradient-descent step. Such settings are often addressed with techniques that either modify the gradient in some way to favor directions of low curvature, or explicitly use the curvature (i.e., second-order derivative) of the loss function. This chapter will discuss several such algorithms.

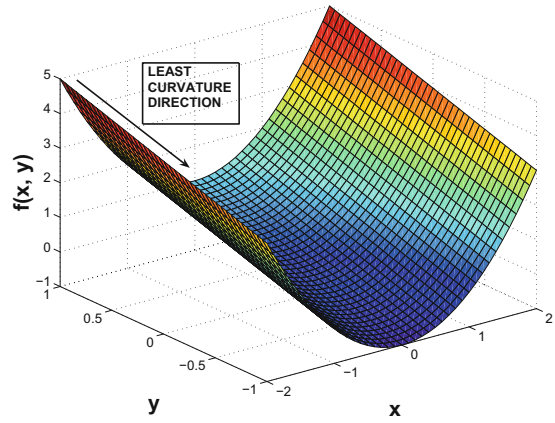



Figure 4.6: The curvature effect in valleys

The specific effect of curvature is particularly evident when one encounters loss functions in the shape of sloping or winding valleys. An example of a sloping valley is shown in Figure 4.6. A valley is a dangerous topography for a gradient-descent method, particularly if the bottom of the valley has a steep and rapidly changing surface (which creates a narrow valley). This is, of course, not the case in Figure 4.6, which is a relatively easier case. However, even in this case, the steepest-descent direction will often bounce along the sides of the valley, and move down the slope relatively slowly if the step-sizes are chosen inaccurately. In narrow valleys, the gradient-descent method will bounce along the steep sides of the valley even more violently without making much progress in the gently sloping direction, where the greatest *long-term* gains are present. It is again evident that choosing only the steepest-decent direction for steps is not helpful; rather, one must also favor low-curvature directions.

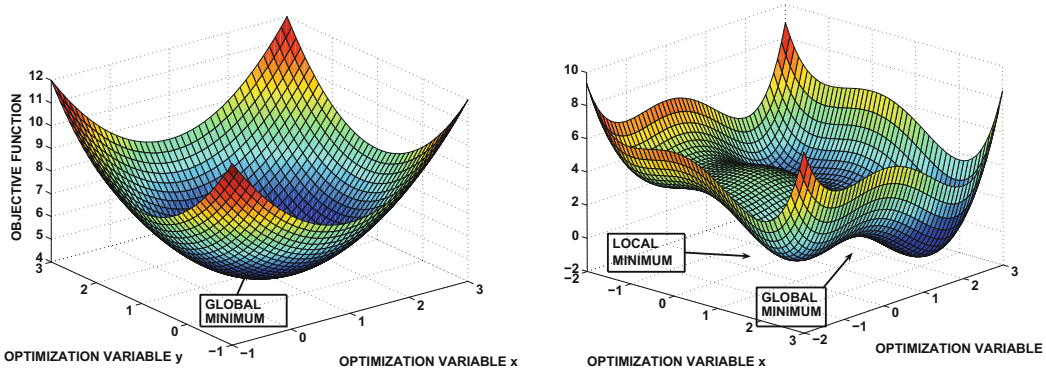
### 4.3.4 Convergence Problems with Depth

Very deep architectures have a difficult time in converging to an optimal solution during backpropagation. This problem has been explored in [193, 194], where it is conjectured that the rate of convergence of a deep network slows down exponentially with depth. Although the vanishing and exploding gradient problems also cause problems with convergence, the work in [193, 194] shows that using various tricks to reduce the vanishing and exploding gradient problems continue to cause problems with convergence. Therefore, the problem of convergence seems to be multi-faceted, and it cannot be easily explained by a single factor (like the vanishing and exploding gradient problems). One issue is that the partial derivatives in different layers are interdependent, and it takes a while for changes in weights in different layers to propagate to other layers (especially as depth increases). As a result, it is possible for the weights in different layers to move around without converging to an optimal solution.

### 4.3.5 Local Minima

Certain types of loss functions have a single global minimum. Such problems are referred to as *convex optimization problems*, and they represent the simplest case of optimization. A loss function $L(\overline{w})$ mapping to real values is said to be convex, if it satisfies the following for all weight vectors $\overline{w}_1$ and $\overline{w}_2$ and $\lambda \in (0, 1)$:

$$L(\lambda \overline{w}_1 + [1 - \lambda]\overline{w}_2) \leq \lambda L(\overline{w}_1) + (1 - \lambda)L(\overline{w}_2)$$




(a) Single global minimum
$g(x, y) = x^2 + y^2 - 2x - 2y + 6$

(b) Global and local minimum
$G(x, y) = ([x^4 + y^4]/4)$
$-([x^3 + y^3]/3) - x^2 - y^2 + 4$

Figure 4.7: Illustrations of local and global optima

A convex loss function is shaped like a bowl with a single bottom, whereas a non-convex function might have multiple "potholes" in the bowl. An example of a 2-dimensional convex loss function $g(x, y) = x^2 + y^2 - 2x - 2y + 6$ is illustrated in Figure 4.7(a), where the two horizontal axes are the variables and the vertical axis is the loss value. It is evident that this function has a single global minimum at $(x, y) = (1, 1)$. On the other hand, the loss function $G(x, y)$ in Figure 4.7(b) is not convex and it has multiple local minima.

The convexity of a function is important from the perspective of its behavior during gradient descent. When gradient descent reaches such a pothole, it might be unable to escape from that point because the gradient at the bottom of a pothole is 0 and all instantaneous directions of movement worsen the loss function. Most of the shallow neural networks discussed in Chapter 3 have convex loss functions, and therefore such a situation does not arise. A proof of convexity of many of these loss functions may be found in [6]. The general reason is that most of these shallow architectures can be explained by a single composition $g(f(\cdot))$ of a convex nonlinear loss function $g(\cdot)$ and a linear function $f(\cdot)$. Such a composition of functions maintains convexity of the loss function [6] as long as the convex nonlinear function is applied as the outer nest of this composition. For example, the linear function $f(x_1, x_2) = x_1 - x_2$ and quadratic function $g(x) = x^2$ are both convex, and the function $g(f(x_1, x_2)) = (x_1 - x_2)^2$ is convex. However, with increasing depth, the loss function is often not convex. This is primarily because most activation functions are convex, although composing a linear function and nonlinear convex function *multiple* times (or in the "wrong" order) is not always convex. For example, the linear function $f(x_1, x_2) = x_1 - x_2$ and quadratic function $g(x) = x^2$ are both convex, but the function $f(g(x_1), g(x_2)) = x_1^2 - x_2^2$ is not convex.

A deep neural network is the result of the composition of many linear dot products and nonlinear activation functions. In such cases, the overall function computed by the network is no longer convex in either the inputs or the weight variables. Therefore, gradient-descent is no longer guaranteed to converge to a global optimum. The procedure could easily get stuck in one of the "potholes" of the loss function; these spurious "minima" are also referred to as *local optima*.

Local minima are problematic only when their objective function values are significantly larger than that of the global minimum. In practice, however, this does not seem to be the case in neural networks. Many research results [91, 443] have shown that the local minima of real-life networks have very similar objective function values to the global minimum. As a result, their presence does not seem to cause a serious problem. Furthermore, some of the training methods discussed in this chapter, such as *momentum methods*, are very well suited to avoiding local optima during gradient descent.

## 4.4 Depth-Friendly Neural Architectures

Certain types of neural architectures are easier to training than others. The specific choice of the activation function and the way in which the units are connected both have an effect on gradient descent. This section will discuss various depth-friendly architectural tricks.

### 4.4.1 Activation Function Choice

The specific choice of activation function often has a considerable effect on the severity of the vanishing gradient problem. The derivatives of the sigmoid and the tanh activation functions are illustrated in Figure 4.8(a) and (b), respectively. The sigmoid activation function never has a gradient of more than 0.25, and therefore it is very prone to the vanishing gradient problem. Furthermore, it *saturates* at large absolute values of the argument, which refers to the fact that the gradient is almost 0. In such cases, the weights of the neuron change very slowly. Therefore, a few such activations within the network can significantly affect the gradient computations. The tanh function fares better than the sigmoid function because it has a gradient of 1 near the origin, but the gradient saturates rapidly at increasingly large

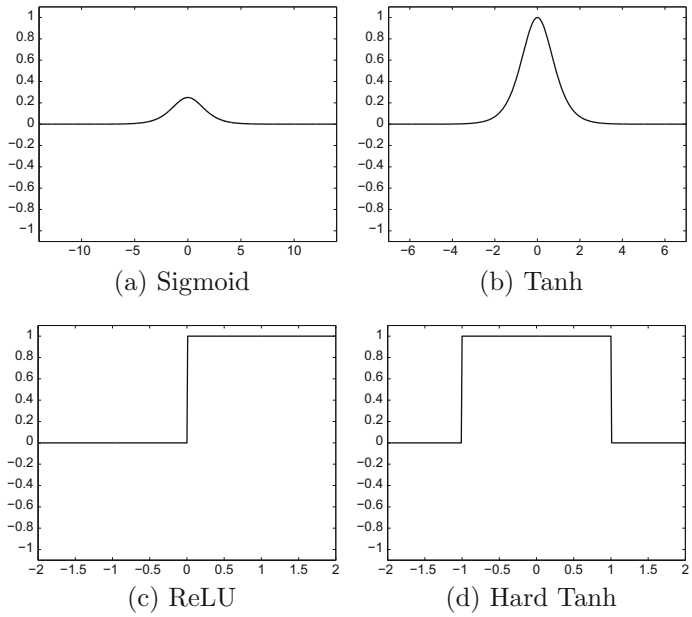



Figure 4.8: The derivatives of different activation functions

absolute values of the argument. Therefore, the tanh function will also be susceptible to the vanishing gradient problem.

In recent years, the use of the sigmoid and the tanh activation functions has been increasingly replaced with the ReLU and the hard tanh functions. The ReLU is faster to train because its gradient computation amounts to checking nonnegativity. The derivatives of the ReLU and the hard tanh functions are shown in Figures 4.8(c) and (d), respectively. It is evident that these functions take on the derivative of 1 in large intervals, although the derivative is 0 outside them. As a result, the vanishing gradient problem tends to occur less often, and these piecewise linear variants have become increasingly popular compared to their smooth counterparts. However, the use of the ReLU is only a partial fix because it does not account for the gradient flows associated with matrix multiplication across layers. Furthermore, the piecewise linear activations introduce a new problem of *dead neurons*.

### 4.4.2 Dying Neurons and "Brain Damage"

It is evident from Figure 4.8(c) that the derivative of the ReLU is zero for negative arguments. This can occur for a variety of reasons. For example, consider the case where the input into a neuron is always nonnegative, whereas all the weights have somehow been initialized to negative values. Therefore, the output will be 0. Another example is the case where a high learning rate is used. In such a case, the pre-activation values of the ReLU can jump to a range where the gradient is 0 irrespective of the input. In other words, high learning rates can "knock out" ReLU units. In such cases, the ReLU might not fire for any data instance. Once a neuron reaches this point, the gradient of the loss with respect to the weights just before the ReLU will always be zero. In other words, the weights of this neuron will never be updated further during training. Furthermore, its output will not vary across different choices of inputs and therefore will not play a role in discriminating between different instances. Such a neuron can be considered *dead*, which is considered a kind of permanent "brain damage" in biological parlance. The problem of dying neurons can be partially ameliorated by using learning rates that are somewhat modest. Another fix is to use the *leaky ReLU*.

#### 4.4.2.1 Leaky ReLU

The leaky ReLU is defined using an additional parameter $\alpha \in (0, 1)$:

$$\Phi(v) = \begin{cases} \alpha \cdot v & v \leq 0 \\ v & \text{otherwise} \end{cases} \tag{4.2}$$

Therefore, at negative values of $v$, the leaky ReLU can still propagate (i.e., *leak*) some gradient backwards, albeit at a reduced rate defined by $\alpha < 1$. Although $\alpha$ is a hyperparameter chosen by the user, it is also possible to learn it.

The gains with the leaky ReLU are not guaranteed. A key point is that dead neurons are not always a problem, because they represent a kind of pruning that defines the neural network structure. Therefore, a certain level of dropping of neurons can be viewed as a part of the learning process. After all, there are limitations to our ability to tune the number of neurons in each layer. Dying neurons do a part of this tuning for us. Indeed, the intentional pruning of *connections* is sometimes used as a strategy for regularization [288]. Of course, if a very large fraction of the neurons in the network are dead, that can be a problem as

well because much of the neural network will be inactive. Furthermore, it is undesirable for too many neurons to be knocked out during the early training phases, when the model is very poor.

#### 4.4.2.2 Maxout Networks

A recently proposed solution is the use of *maxout units* [155], which leverage two coefficient vectors $\overline{W_1}$ and $\overline{W_2}$ instead of a single one. The maxout unit computes the function $\max\{\overline{W_1} \cdot \overline{X}, \overline{W_2} \cdot \overline{X}\}$. In the event that bias neurons are used, the maxout activation is $\max\{\overline{W_1} \cdot \overline{X} + b_1, \overline{W_2} \cdot \overline{X} + b_2\}$. One can view the maxout as a generalization of the ReLU, because the ReLU is obtained by setting one of the coefficient vectors to 0. Even the leaky ReLU can be shown to be a special case of maxout, in which we set $\overline{W_2} = \alpha \overline{W_1}$ for $\alpha \in (0, 1)$. Like the ReLU, the maxout function is piecewise linear. However, it does not saturate at all, and is linear almost everywhere. In spite of its linearity, it has been shown [155] that maxout networks are universal function approximators. Maxout has advantages over the ReLU, and it enhances the performance of ensemble methods like *Dropout* (cf. section 5.5.4 of Chapter 5). However, one drawback with maxout is that it doubles the number of parameters.

### 4.4.3 Using Skip Connections

Most neural networks are arranged in layer-wise fashion, where connections from layer $i$ always flow to layer $(i + 1)$. A recent idea for improving the convergence behavior of a neural network is the use of *skip connections*, wherein nodes in layer $i$ allowed to connect to nodes in layer $(i + k)$ for $k > 1$. This idea is used extensively in image recognition, and a well-known network, referred to as *ResNet* [193, 194], has been designed with more than 100 layers. An example of a skip connection between layer $i$ and layer $(i+2)$ is illustrated in Figure 4.9. The basic idea behind skip connections is that the network uses as much depth as it needs to in order to learn features with varying levels of richness. Simple features are learned by using skip connections frequently, whereas intricate features are learned using a large number of layers. At the same time, the network is easier to train because most of basic features can be trained using a small number of layers. This basic idea is referred to as *residual learning*, because greater depth simply learns the "residual" patterns in the data that cannot be learned using a smaller number of layers. The general principle is to *use additional layers to learn more refined features but not forget the useful features learned in earlier layers.*

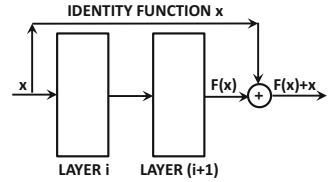



Figure 4.9: Example of a skip connection between alternate layers

## 4.5 Depth-Friendly Gradient-Descent Strategies

The steepest-descent direction is the optimal direction only from the perspective of infinitesimal steps, and it can worsen the objective function over a larger step. As a result, many course corrections are needed. A specific example of this phenomenon is discussed in section 4.3.1 in which minor differences in sensitivity to different features can cause a steepest-descent algorithm to have oscillations. The problem of oscillation and zigzagging is severe whenever the step is executed along a direction of *high curvature* in the loss function. This section discuss several clever learning strategies that work well in these settings.

### 4.5.1 Importance of Preprocessing and Initialization

The preprocessing and initialization strategies discussed in section 2.7 of Chapter 2 are extremely important for avoiding ill conditioning during gradient descent. In general, the gradients are stable across layers when the magnitudes of the activations across different layers are stable as well. Stability can be achieved by preprocessing and normalization as follows:

- *Importance of feature normalization:* When the variances of some features are much larger than others, it causes some weights to have much larger influence on the loss function than others. The varying sensitivity of the loss function to different weights can cause the type of zigzagging behavior discussed in section 4.3. Therefore, a common approach is to use standardization in order to make the variance of all inputs equal. Whitening also decorrelates the features and consolidates correlated moves of the weights into fewer non-redundant weights. Feature centering also helps because it causes different input feature values to have different signs — this ensures that all weights pointing into the same neuron from the input layer are not constrained to move in the same direction (which avoids zigzagging).

- *Importance of initialization:* A poor initialization in a deep network can exacerbate the vanishing and exploding gradient problems. With certain types of initialized weights, the vanishing and exploding gradient problems can be exacerbated in the very beginning, from which it becomes hard for gradient descent to recover. The initialized weights of the connections to a node are set to be proportional to the inverse of the square-root of the fan-in of that node by sampling them from a normal distribution with zero mean and standard deviation parameter set to $1/\sqrt{r_{in}}$. The idea is that each weight contributes a variance proportional to $1/r_{in}$, and the additive effect over nodes with different values of the fan-in becomes similar. However, this type of initialization does not account for the fact that the fan-out of the nodes also has an effect on the magnitudes of the activations in later layers. Let $r_{out}$ be the fan-out for a particular neuron. The *Xavier initialization* or *Glorot initialization* uses a normal distribution with zero mean and standard deviation of $\sqrt{2/(r_{in} + r_{out})}$.

Normalization of features is helpful not only in the initial layer, but also in later layers. As discussed in section 4.8, more benefits may be obtained by normalization of the activations of hidden layers by using an idea called *batch normalization*.

### 4.5.2 Momentum-Based Learning

Momentum-based methods address the issues of local optima, flat regions, and curvature-centric zigzagging by recognizing that *emphasizing medium-term to long-term directions of consistent movement* is beneficial, because they de-emphasize local distortions in the loss topology. Consequently, an aggregated measure of the feedback from previous steps is used in order to speed up the gradient-descent procedure. As an analogy, a marble that rolls down a sloped surface with many potholes and other distortions is often able to use its momentum to overcome such minor obstacles.

Consider a setting in which one is performing gradient-descent with respect to the parameter vector $\overline{W}$. The normal updates for gradient-descent with respect to the loss function $L$ (defined over a mini-batch of instances) are as follows:

$$\overline{V} \Leftarrow -\alpha \frac{\partial L}{\partial \overline{W}}; \quad \overline{W} \Leftarrow \overline{W} + \overline{V}$$

Here, $\alpha$ is the learning rate. We are using the matrix-calculus convention[1] that the derivative of a scalar $L$ with respect to a column vector $\overline{W}$ is the column vector $\nabla L$:

$$\nabla L = \frac{\partial L}{\partial \overline{W}} = \left[ \frac{\partial L}{\partial w_1} \cdots \frac{\partial L}{\partial w_d} \right]^T$$

In momentum-based descent, the vector $\overline{V}$ inherits a fraction $\beta$ of its velocity from its previous step in addition to the current gradient, where $\beta \in (0, 1)$ is the momentum parameter:

$$\overline{V} \Leftarrow \beta \overline{V} - \alpha \frac{\partial L}{\partial \overline{W}}; \quad \overline{W} \Leftarrow \overline{W} + \overline{V}$$

Setting $\beta = 0$ specializes to straightforward mini-batch gradient-descent. Larger values of $\beta \in (0, 1)$ help the approach pick up a consistent velocity $\overline{V}$ in the correct direction. The momentum parameter $\beta$ is also referred to as the *friction parameter*. The word "friction" is derived from the fact that small values of $\beta$ act as "brakes," much like friction. Setting $\beta > 1$ can cause instability, where the gradient descent might move in an uncontrolled way towards increasingly large loss values.

Momentum helps the gradient descent process in navigating flat regions and local optima. A good analogy for momentum-based methods is to visualize them in a similar way as a marble rolls down a bowl. As the marble picks up speed, it will be able to navigate flat regions of the surface quickly and escape form local potholes in the bowl. This is because the gathered momentum helps it escape potholes. Figure 4.10, which shows a marble rolling down a complex loss surface (picking up speed as it rolls down), illustrates this concept. The use of momentum will often cause the solution to slightly overshoot in the direction where velocity is picked up, just as a marble will overshoot when it is allowed to roll down a bowl. However, with the appropriate choice of $\beta$, it will still perform better than a situation in which momentum is not used. The momentum-based method will generally perform better because the marble gains speed as it rolls down the bowl; the quicker arrival at the optimal solution more than compensates for the overshooting of the target. Overshooting is desirable to the extent that it helps avoid local optima. The parameter $\beta$ controls the amount of friction that the marble encounters while rolling down the loss surface. While increased values of $\beta$ help in avoiding local optima, it might also increase oscillation at the end. In this sense, the momentum-based method has a neat interpretation in terms of the physics of a marble rolling down a complex loss surface.

[1] This convention is referred to as the *denominator layout* of matrix calculus [6]. In the numerator layout, the derivative of a scalar with respect to a column vector is a row vector.

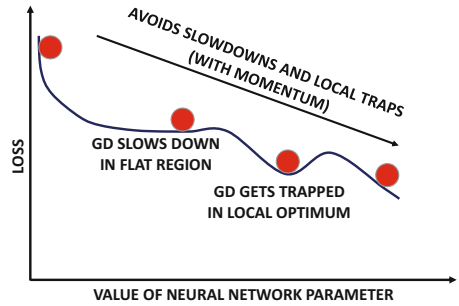



Figure 4.10: Effect of momentum in navigating complex loss surfaces.

In addition, momentum-based methods help in reducing the undesirable effects of curvature in the loss surface of the objective function. Momentum-based techniques recognize that zigzagging is a result of highly contradictory steps that cancel out one another and reduce the *effective* size of the steps in the correct (long-term) direction. An example of this scenario is illustrated in Figure 4.3(b). Simply attempting to increase the size of the step in order to obtain greater movement in the correct direction might actually move the current solution even further away from the optimum solution. In this point of view, it makes a lot more sense to move in an "averaged" direction of the last few steps, so that the zigzagging is smoothed out. This type of averaging is achieved by using the momentum from the previous steps. Oscillating directions do not contribute consistent velocity to the update.

With momentum-based descent, one is generally moving in a direction that often points closer to the optimal solution and the useless "sideways" oscillations are muted. The basic idea is to give greater preference to *consistent* directions over multiple steps, which have greater importance in the descent. This allows the use of larger steps in the correct direction without causing deteriorations in the sideways direction. As a result, learning is accelerated. An example of the use of momentum is illustrated in Figure 4.11. It is evident from Figure 4.11(a) that momentum increases the relative component of the gradient in the correct direction. The corresponding effects on the updates are illustrated in Figures 4.11(b) and (c). It is evident that momentum-based updates can reach the optimal solution in fewer updates. One can also understand this concept by visualizing the movement of a marble down the valley of Figure 4.6. As the marble gains speed down the gently sloping valley, the effects of bouncing along the sides of the valley will be muted over time.

### 4.5.3 Nesterov Momentum

The Nesterov momentum [364] is a modification of the traditional momentum method in which the gradients are computed at a point that would be reached after executing a $\beta$-discounted version of the previous step again (i.e., the momentum portion of the current step). This point is obtained by multiplying the previous update vector $\overline{V}$ with the friction parameter $\beta$ and then computing the gradient at $\overline{W} + \beta\overline{V}$. The idea is that this corrected gradient uses a better understanding of how the gradients will change because of the momentum portion of the update, and incorporates this information into the gradient portion of the update. Therefore, one is using a certain amount of lookahead in computing the updates. Let us denote the loss function by $L(\overline{W})$ at the current solution $\overline{W}$. In this case, it is important to explicitly denote the argument of the loss function because of the way in which the gradient is computed. Therefore, the update may be computed as follows:

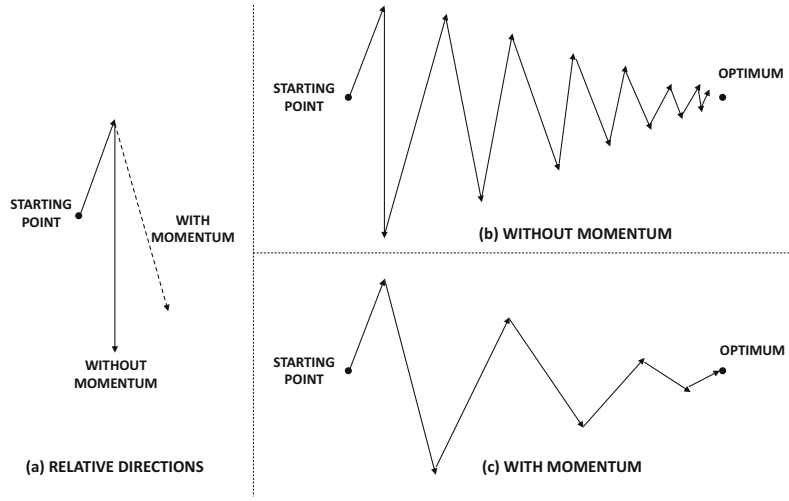



Figure 4.11: Effect of momentum in smoothing zigzagging updates

$$\overline{V} \Leftarrow \beta \overline{V} - \alpha \frac{\partial L(\overline{W} + \beta \overline{V})}{\partial \overline{W}}; \quad \overline{W} \Leftarrow \overline{W} + \overline{V}$$

Note that the *only* difference from the standard momentum method is in terms of *where* the gradient is computed. Using the value of the gradient a little further along the previous update can lead to faster convergence. In the previous analogy of the rolling marble, such an approach will start applying the "brakes" on the gradient-descent procedure when the marble starts reaching near the bottom of the bowl, because the lookahead will "warn" it about the reversal in gradient direction.

The Nesterov method works only in mini-batch gradient descent with modest batch sizes; using very small batches is a bad idea. In such cases, it can be shown that the Nesterov method reduces the error to $O(1/t^2)$ after $t$ steps, as compared to an error of $O(1/t)$ in the momentum method.

### 4.5.4 Parameter-Specific Learning Rates

The basic idea in the momentum methods of the previous section is to leverage the *consistency* in the gradient direction of certain parameters in order to speed up the updates. This goal can also be achieved more explicitly by having different learning rates for different parameters. The idea is that parameters with large partial derivatives are often oscillating and zigzagging, whereas parameters with small partial derivatives tend to be more consistent but move in the same direction. An early method, which was proposed in this direction, was the *delta-bar-delta* method [228]. This approach tracks whether the sign of each partial derivative changes or stays the same. If the sign of a partial derivative stays consistent, then it is indicative of the fact that the direction is correct. In such a case, the partial derivative in that direction increases. On the other hand, if the sign of the partial derivative flips all the time, then the partial derivative decreases. However, this kind of approach is designed for gradient descent rather than stochastic gradient descent, because the errors in stochastic gradient descent can get magnified. Therefore, a number of methods have been proposed that can work well even when the mini-batch method is used.

#### 4.5.4.1 AdaGrad

In the AdaGrad algorithm [110], one keeps track of the aggregated squared magnitude of the partial derivative with respect to each parameter over the course of the algorithm. The square-root of this value is *proportional* to the root-mean-square slope for that parameter (although the absolute value will increase with the number of epochs because of successive aggregation).

Let $A_i$ be the aggregate value for the $i$th parameter. Therefore, in each iteration, the following update is performed:

$$A_i \Leftarrow A_i + \left(\frac{\partial L}{\partial w_i}\right)^2 ; \quad \forall i \tag{4.3}$$

The update for the $i$th parameter $w_i$ is as follows:

$$w_i \Leftarrow w_i - \frac{\alpha}{\sqrt{A_i}} \left(\frac{\partial L}{\partial w_i}\right) ; \quad \forall i$$

If desired, one can use $\sqrt{A_i + \epsilon}$ in the denominator instead of $\sqrt{A_i}$ to avoid ill-conditioning. Here, $\epsilon$ is a small positive value such as $10^{-8}$.

Scaling the derivative inversely with $\sqrt{A_i}$ is a kind of "signal-to-noise" normalization because $A_i$ only measures the historical magnitude of the gradient rather than its sign; it encourages faster *relative* movements along gently sloping directions with consistent sign of the gradient. If the gradient component along the $i$th direction keeps wildly fluctuating between $+100$ and $-100$, this type of magnitude-centric normalization will penalize that component far more than another gradient component that consistently takes on the value in the vicinity of 0.1 (but with a consistent sign). For example, in Figure 4.11, the movements along the oscillating direction will be de-emphasized, and the movement along the consistent direction will be emphasized. However, absolute movements along all components will tend to slow down over time, which is the main problem with the approach. The slowing down is caused by the fact that $A_i$ is the *aggregate* value of the entire history of partial derivatives. This will lead to diminishing values of the scaled derivative. As a result, the progress of AdaGrad might prematurely become too slow, and it will eventually (almost) stop making progress. Another problem is that the aggregate scaling factors depend on ancient history, which can eventually become stale. The use of stale scaling factors can increase inaccuracy. As we will see later, most of the other methods use exponential averaging, which solves both problems.

#### 4.5.4.2 RMSProp

The RMSProp algorithm [204] uses a similar motivation as AdaGrad for performing the "signal-to-noise" normalization with the absolute magnitude $\sqrt{A_i}$ of the gradients. However, instead of simply adding the squared gradients to estimate $A_i$, it uses *exponential averaging*. Since one uses *averaging* to normalize rather than *aggregated* values, the progress is not slowed prematurely by a constantly increasing scaling factor $A_i$. The basic idea is to use a decay factor $\rho \in (0, 1)$, and weight the squared partial derivatives occurring $t$ updates ago by $\rho^t$. Note that this can be easily achieved by multiplying the current squared aggregate (i.e., *running* estimate) by $\rho$ and then adding $(1 - \rho)$ times the current (squared) partial derivative. The running estimate is initialized to 0. This causes some (undesirable) bias in

early iterations, which disappears over the longer term. Therefore, if $A_i$ is the exponentially averaged value of the $i$th parameter $w_i$, we have the following way of updating $A_i$:

$$A_i \Leftarrow \rho A_i + (1-\rho)\left(\frac{\partial L}{\partial w_i}\right)^2; \quad \forall i \tag{4.4}$$

The square-root of this value for each parameter is used to normalize its gradient. Then, the following update is used for (global) learning rate $\alpha$:

$$w_i \Leftarrow w_i - \frac{\alpha}{\sqrt{A_i}}\left(\frac{\partial L}{\partial w_i}\right); \quad \forall i$$

If desired, one can use $\sqrt{A_i + \epsilon}$ in the denominator instead of $\sqrt{A_i}$ to avoid ill-conditioning. Here, $\epsilon$ is a small positive value such as $10^{-8}$. Another advantage of RMSProp over AdaGrad is that the importance of ancient (i.e., stale) gradients decays exponentially with time. Furthermore, it can benefit by incorporating concepts of momentum within the computational algorithm (cf. sections 4.5.5.1 and 4.5.5.2). The drawback of RMSProp is that the running estimate $A_i$ of the second-order moment is biased in early iterations because it is initialized to 0.

#### 4.5.4.3 AdaDelta

The AdaDelta algorithm [578] uses a similar update as RMSProp, except that it eliminates the need for a global learning parameter by computing it as a function of incremental updates in previous iterations. Consider the update of RMSProp, which is repeated below:

$$w_i \Leftarrow w_i - \underbrace{\frac{\alpha}{\sqrt{A_i}}\left(\frac{\partial L}{\partial w_i}\right)}_{\Delta w_i}; \quad \forall i$$

We will show how $\alpha$ is replaced with a value that depends on the previous incremental updates. In each update, the value of $\Delta w_i$ is the increment in the value of $w_i$. As with the exponentially smoothed gradients $A_i$, we keep an exponentially smoothed value $\delta_i$ of the values of $\Delta w_i$ in previous iterations with the same decay parameter $\rho$:

$$\delta_i \Leftarrow \rho \delta_i + (1-\rho)(\Delta w_i)^2; \quad \forall i \tag{4.5}$$

For a given iteration, the value of $\delta_i$ can be computed using only the iterations before it because the value of $\Delta w_i$ is not yet available. On the other hand, $A_i$ can be computed using the partial derivative in the current iteration as well. This is a subtle difference between how $A_i$ and $\delta_i$ are computed. This results in the following AdaDelta update:

$$w_i \Leftarrow w_i - \underbrace{\sqrt{\frac{\delta_i}{A_i}}\left(\frac{\partial L}{\partial w_i}\right)}_{\Delta w_i}; \quad \forall i$$

It is noteworthy that a parameter $\alpha$ for the learning rate is completely missing from this update. The AdaDelta method shares some similarities with second-order methods because the ratio $\sqrt{\frac{\delta_i}{A_i}}$ in the update is a heuristic surrogate for the inverse of the second derivative of the loss with respect to $w_i$ [578]. As discussed in subsequent sections, many second-order methods like the Newton method also do not use learning rates.

### 4.5.5 Combining Parameter-Specific Learning and Momentum

Although parameter-specific learning can help in differentiating between the consistency of movement along different directions, they do not gain speed along a particular direction because of consistent movement (like momentum methods). Several methods combine parameter-specific learning with momentum methods in order to gain advantages from both.

#### 4.5.5.1 RMSProp with Nesterov Momentum

RMSProp can be combined with Nesterov momentum. Let $A_i$ be the squared aggregate of the $i$th weight. In such cases, we introduce the additional parameter $\beta \in (0, 1)$ and use the following updates:

$$v_i \Leftarrow \beta v_i - \frac{\alpha}{\sqrt{A_i}} \left( \frac{\partial L(\overline{W} + \beta \overline{V})}{\partial w_i} \right); \quad w_i \Leftarrow w_i + v_i; \quad \forall i$$

Note that the partial derivative of the loss function is computed at a shifted point, as is common in the Nesterov method. The weight $\overline{W}$ is shifted with $\beta\overline{V}$ while computing the partial derivative with respect to the loss function. The maintenance of $A_i$ is done using the shifted gradients as well:

$$A_i \Leftarrow \rho A_i + (1 - \rho) \left( \frac{\partial L(\overline{W} + \beta \overline{V})}{\partial w_i} \right)^2; \quad \forall i \tag{4.6}$$

Although this approach benefits from adding momentum to RMSProp, it does not correct for the initialization bias.

#### 4.5.5.2 Adam

The Adam algorithm uses a similar "signal-to-noise" normalization as AdaGrad and RMSProp; however, it also incorporates momentum into the update. In addition, it directly addresses the initialization bias inherent in the exponential smoothing of pure RMSProp.

As in the case of RMSProp, let $A_i$ be the exponentially averaged value of the $i$th parameter $w_i$. This value is updated in the same way as RMSProp with the decay parameter $\rho \in (0, 1)$:

$$A_i \Leftarrow \rho A_i + (1 - \rho) \left( \frac{\partial L}{\partial w_i} \right)^2; \quad \forall i \tag{4.7}$$

At the same time, an exponentially smoothed value of the gradient is maintained for which the $i$th component is denoted by $F_i$. This smoothing is performed with a different decay parameter $\rho_f$:

$$F_i \Leftarrow \rho_f F_i + (1 - \rho_f) \left( \frac{\partial L}{\partial w_i} \right); \quad \forall i \tag{4.8}$$

This type of exponentially smoothing of the gradient with $\rho_f$ is a variation of the momentum method discussed in section 4.5.2 (which is parameterized by a friction parameter $\beta$ instead of $\rho_f$). Then, the following update is used at learning rate $\alpha_t$ in the $t$th iteration:

$$w_i \Leftarrow w_i - \frac{\alpha_t}{\sqrt{A_i}} F_i; \quad \forall i$$

There are two key differences from the RMSProp algorithm. First, the gradient is replaced with its exponentially smoothed value in order to incorporate momentum. Second, the learning rate $\alpha_t$ now depends on the iteration index $t$, and is defined as follows:

$$\alpha_t = \alpha \underbrace{\left( \frac{\sqrt{1-\rho^t}}{1-\rho_f^t} \right)}_{\text{Adjust Bias}} \tag{4.9}$$

Technically, the adjustment to the learning rate is actually a bias correction factor that is applied to account for the unrealistic initialization of the two exponential smoothing mechanisms, and it is particularly important in early iterations. Both $F_i$ and $A_i$ are initialized to 0, which causes bias in early iterations. The two quantities are affected differently by the bias, which accounts for the ratio in Equation 4.9. It is noteworthy that each of $\rho^t$ and $\rho_f^t$ converge to 0 for large $t$ because $\rho, \rho_f \in (0, 1)$. As a result, the initialization bias correction factor of Equation 4.9 converges to 1, and $\alpha_t$ converges to $\alpha$. The default suggested values of $\rho_f$ and $\rho$ are 0.9 and 0.999, respectively, according to the original Adam paper [251]. Refer to [251] for details of other criteria (such as parameter sparsity) used for selecting $\rho$ and $\rho_f$. Like other methods, Adam uses $\sqrt{A_i + \epsilon}$ (instead of $\sqrt{A_i}$) in the denominator of the update for better conditioning. The Adam algorithm is extremely popular because it incorporates most of the advantages of other algorithms, and often performs competitively with respect to the best of the other methods [251].

### 4.5.6 Gradient Clipping

Gradient clipping is a technique that is used to deal with settings in which the partial derivatives along different directions have exceedingly different magnitudes. Some forms of gradient clipping make the different components of the partial derivatives more similar to one another. Two forms of gradient clipping are most common:

1. *Value-based clipping:* In value-based clipping, a minimum and maximum threshold are set on the gradient values. All partial derivatives that are less than the minimum are set to the minimum threshold. All partial derivatives that are greater than the maximum are set to the maximum threshold.

2. *Norm-based clipping:* In this case, the entire gradient vector is normalized by the $L_2$-norm of the entire vector. Note that this type of clipping does not change the relative magnitudes of the updates along different directions. However, for neural networks that share parameters across different layers (like *recurrent neural networks*), the effect of the two types of clipping is very similar. By clipping, one can achieve a better conditioning of the values, so that the updates from mini-batch to mini-batch are roughly similar. Therefore, it would prevent an anomalous gradient explosion in a particular mini-batch from affecting the solution too much.

By and large, the effects of gradient clipping are quite limited compared to many other methods. However, it is particularly effective in avoiding the exploding gradient problem in recurrent neural networks (cf. Chapter 8).

### 4.5.7 Polyak Averaging

One of the motivations for second-order methods is to avoid the kind of bouncing behavior caused by high-curvature regions. The example of the bouncing behavior caused in valleys

(cf. Figure 4.6) is another example of this setting. One way of achieving some stability with any learning algorithm is to create an exponentially decaying average of the parameters over time, so that the bouncing behavior is avoided. Let $\overline{W}_1 \ldots \overline{W}_T$, be the sequence of parameters found by any learning method over the full sequence of $T$ steps. In the simplest version of Polyak averaging, one simply computes the average of all the parameters as the final set $\overline{W}_T^f$:

$$\overline{W}_T^f = \frac{\sum_{i=1}^T \overline{W}_i}{T} \tag{4.10}$$

For simple averaging, we only need to compute $\overline{W}_T^f$ once at the end of the process, and we do not need to compute the values at $1 \ldots T-1$.

However, for exponential averaging with decay parameter $\beta < 1$, it is helpful to compute these values iteratively and maintain a running average over the course of the algorithm:

$$\overline{W}_t^f = \frac{\sum_{i=1}^t \beta^{t-i} \overline{W}_i}{\sum_{i=1}^t \beta^{t-i}} \qquad \text{[Explicit Formula]}$$

$$\overline{W}_t^f = (1-\beta)\overline{W}_t + \beta \overline{W}_{t-1}^f \quad \text{[Recursive Formula]}$$

The two formulas above are approximately equivalent at large values of $t$. The second formula is convenient because it enables maintenance over the course of the algorithm, and one does not need to maintain the entire history of parameters. Exponentially decaying averages are more useful than simple averages to avoid the effect of stale points. In simple averaging, the final result may be too heavily influenced by the early points, which are poor approximations to the correct solution.

## 4.6 Second-Order Derivatives: The Newton Method

A number of methods have been proposed in recent years for using second-order derivatives for optimization. Such methods can partially alleviate some of the problems caused by the high curvature of the loss function.

Consider the parameter vector $\overline{W} = [w_1 \ldots w_d]^T$, which is expressed as a column vector. The second-order derivatives of the loss function $L(\overline{W})$ are of the following form:

$$H_{ij} = \frac{\partial^2 L(\overline{W})}{\partial w_i \partial w_j}$$

Note that the partial derivatives use all pairwise parameters in the denominator. Therefore, for a neural network with $d$ parameters, we have a $d \times d$ *Hessian matrix* $H$, for which the $(i,j)$th entry is $H_{ij}$.

One can write a quadratic approximation of the loss function in the vicinity of parameter vector $\overline{W}_0$ by using the following Taylor expansion:

$$L(\overline{W}) \approx L(\overline{W}_0) + (\overline{W} - \overline{W}_0)^T [\nabla L(\overline{W}_0)] + \frac{1}{2}(\overline{W} - \overline{W}_0)^T H (\overline{W} - \overline{W}_0) \tag{4.11}$$

The accuracy of this approximation falls off with increasing value of $||\overline{W} - \overline{W}_0||$. which is the Euclidean distance between $\overline{W}$ and $\overline{W}_0$. Note that the Hessian $H$ is computed at $\overline{W}_0$. Here, the parameter vectors $\overline{W}$ and $\overline{W}_0$ are $d$-dimensional column vectors, as is the gradient

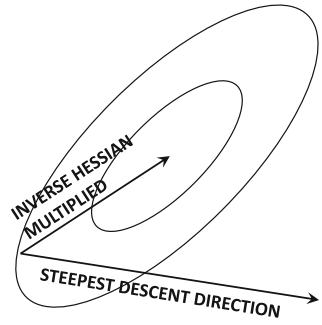



Figure 4.12: The effect of pre-multiplication of steepest-descent direction with the inverse Hessian

of the loss function. This is a quadratic approximation, and one can simply set the gradient to 0, which results in the following optimality condition for the quadratic approximation:

$$\nabla L(\overline{W}) = 0 \quad \text{[Gradient of Loss Function]}$$
$$\nabla L(\overline{W}_0) + H(\overline{W} - \overline{W}_0) = 0 \quad \text{[Gradient of Taylor approximation]}$$

One can rearrange the above optimality condition to obtain the following Newton update:

$$\overline{W}^* \Leftarrow \overline{W}_0 - H^{-1}[\nabla L(\overline{W}_0)] \tag{4.12}$$

The main difference of Equation 4.12 from the update of steepest-gradient descent is pre-multiplication of the steepest direction (which is $[\nabla L(\overline{W}_0)]$) with the inverse of the Hessian. This multiplication with the inverse Hessian plays a key role in changing the direction of the steepest-gradient descent, so that one can take larger steps in that direction (resulting in better improvement of the objective function) even if the *instantaneous* rate of change in that direction is not as large as the steepest-descent direction. This is because the Hessian encodes how fast the gradient is changing in each direction. Changing gradients are bad for larger updates because one might inadvertently worsen the objective function, if the signs of many components of the gradient change during the step. It is profitable to move in directions where the ratio of the gradient to the rate of change of the gradient is large, so that one can take larger steps without causing harm to the optimization. Pre-multiplication with the inverse of the Hessian achieves this goal. The effect of the pre-multiplication of the steepest-descent direction with the inverse Hessian is shown in Figure 4.12. It is helpful to reconcile this figure with the example of the quadratic bowl in Figure 4.3. In a sense, pre-multiplication with the inverse Hessian biases the learning steps towards low-curvature directions. In one dimension, the Newton step is simply the ratio of the first derivative (rate of change) to the second derivative (curvature). In multiple dimensions, the low-curvature directions tend to win out because of multiplication by the inverse Hessian.

One interesting characteristic of the update of Equation 4.12 is that it is directly obtained from an optimality condition, and therefore there is no learning rate. In other words, this update is approximating the loss function with a quadratic bowl and moving *exactly* to the bottom of the bowl *in a single step*; the learning rate is already incorporated implicitly. However, when dealing with non-quadratic functions, it is useful to incorporate a learning rate and set it to minimize the objective function (and not just the quadratic

approximation). Therefore, for non-quadratic functions, the following iterative update is used:

$$\overline{W}_{t+1} \Leftarrow \overline{W}_t - \alpha_t H^{-1}[\nabla L(\overline{W}_t)] \tag{4.13}$$

Here, $\alpha_t$ is chosen to minimize $L(\overline{W}_{t+1})$. This is done using a procedure called *line search*.

Assume that $\overline{q}_t = -H^{-1}\nabla L(\overline{W}_t)$ denotes the direction of movement in the Newton method. Then, Equation 4.13 an be written as follows:

$$\overline{W}_{t+1} \Leftarrow \overline{W}_t + \alpha_t \overline{q}_t$$

In line search, the step-size $\alpha_t$ is computed as follows:

$$\alpha_t = \text{argmin}_\alpha L(\overline{W}_t + \alpha \overline{q}_t) \tag{4.14}$$

There are several methods for performing line search, such as *binary search* and *golden section search*. In binary search, one starts by assuming that $\alpha_t$ belongs to the range $[a, b] = [0, \alpha_{max}]$, where $\alpha_{max}$ can be obtained by doubling successive intervals. Subsequently, one can reduce the size of the interval by a factor of 2 in each iteration by testing two points $c < d$ near the midpoint of the interval, and narrowing to the interval $[a, d]$ if the evaluation at $\alpha_t = d$ is larger. Otherwise the interval $[c, b]$ is used.

### 4.6.1 Example: Newton Method in the Quadratic Bowl

We will revisit how the Newton method behaves in the quadratic bowl of Figure 4.3. Consider the following elliptical objective function, which is the same as the one discussed in Figure 4.3(b):

$$J(w_1, w_2) = w_1^2 + 4w_2^2$$

This is a very simple convex quadratic, whose optimal point is the origin. Applying straightforward gradient descent starting at any point like $[w_1, w_2] = [1, 1]$ will result in the type of bouncing behavior shown in Figure 4.3(b). On the other hand, consider the Newton method, starting at the point $[w_1, w_2] = [1, 1]$. The gradient may be computed as $\nabla J(1, 1) = [2w_1, 8w_2]^T = [2, 8]^T$. Furthermore, the Hessian of this function is a constant that is independent of $[w_1, w_2]^T$:

$$H = \begin{bmatrix} 2 & 0 \\ 0 & 8 \end{bmatrix}$$

Applying the Newton update results in the following:

$$\begin{bmatrix} w_1 \\ w_2 \end{bmatrix} \Leftarrow \begin{bmatrix} 1 \\ 1 \end{bmatrix} - \begin{bmatrix} 2 & 0 \\ 0 & 8 \end{bmatrix}^{-1} \begin{bmatrix} 2 \\ 8 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

In other words, a single step suffices to reach the optimum point of this quadratic function. This is because the second-order Taylor "approximation" of a quadratic function is exact, and the Newton method solves this approximation in each iteration. Of course, real-world functions are not quadratic, and therefore multiple steps are typically needed.

### 4.6.2 Example: Newton Method in a Non-Quadratic Function

In this section, we will modify the objective function of the previous section to make it non-quadratic. The corresponding function is as follows:

$$J(w_1, w_2) = w_1^2 + 4w_2^2 - \cos(w_1 + w_2)$$

It is assumed that $w_1$ and $w_2$ are expressed[2] in radians. Note that the optimum of this objective function is still $[w_1, w_2] = [0, 0]$, since the value of $J(0, 0)$ is $-1$ at this point, where each additive term of the above expression takes on its minimum value. We will again start at $[w_1, w_2] = [1, 1]$, and show that one iteration no longer suffices in this case. In this case, we can show that the gradients and Hessian are as follows:

$$\nabla J(1,1) = \begin{bmatrix} 2 + \sin(2) \\ 8 + \sin(2) \end{bmatrix} = \begin{bmatrix} 2.91 \\ 8.91 \end{bmatrix}$$

$$H = \begin{bmatrix} 2 + \cos(2) & \cos(2) \\ \cos(2) & 8 + \cos(2) \end{bmatrix} = \begin{bmatrix} 1.584 & -0.416 \\ -0.416 & 7.584 \end{bmatrix}$$

The inverse of the Hessian is as follows:

$$H^{-1} = \begin{bmatrix} 0.64 & 0.035 \\ 0.035 & 0.134 \end{bmatrix}$$

Therefore, we obtain the following Newton update:

$$\begin{bmatrix} w_1 \\ w_2 \end{bmatrix} \Leftarrow \begin{bmatrix} 1 \\ 1 \end{bmatrix} - \begin{bmatrix} 0.64 & 0.035 \\ 0.035 & 0.134 \end{bmatrix} \begin{bmatrix} 2.91 \\ 8.91 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} - \begin{bmatrix} 2.1745 \\ 1.296 \end{bmatrix} = \begin{bmatrix} -1.1745 \\ -0.2958 \end{bmatrix}$$

Note that we do reach closer to an optimal solution, although we certainly do not reach the optimum point. This is because the objective function is not quadratic in this case, and one is only reaching the bottom of the *approximate* quadratic bowl of the objective function. However, Newton's method does find a better point in terms of the true objective function value. The approximate nature of the Hessian is why one must use either exact or approximate line search to control the step size. Note that if we used a step-size of 0.6 instead of the default value of 1, one would obtain the following solution:

$$\begin{bmatrix} w_1 \\ w_2 \end{bmatrix} \Leftarrow \begin{bmatrix} 1 \\ 1 \end{bmatrix} - 0.6 \begin{bmatrix} 2.1745 \\ 1.296 \end{bmatrix} = \begin{bmatrix} -0.30 \\ 0.22 \end{bmatrix}$$

Although this is only a very rough approximation to the optimal step size, it still reaches much closer to the true optimal value of $[w_1, w_2] = [0, 0]$. It is also relatively easy to show that this set of parameters yields a much better objective function value. This step would need to be repeated in order to reach closer and closer to an optimal solution.

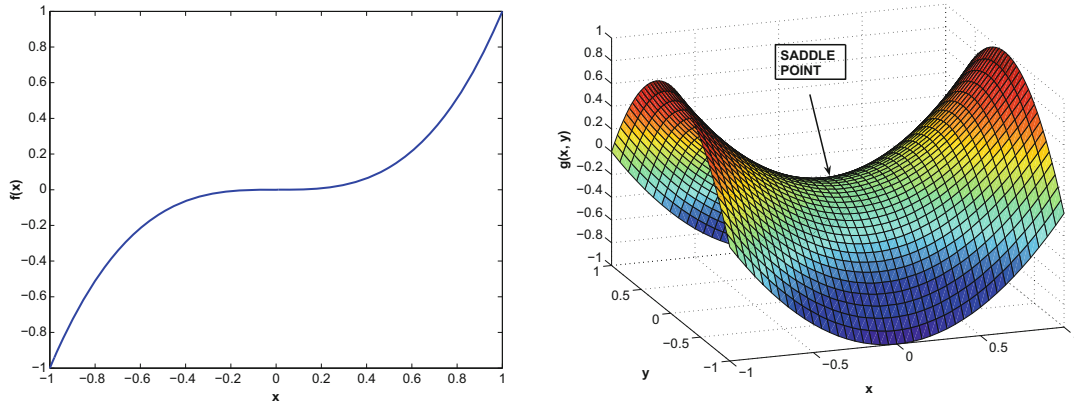
### 4.6.3 The Saddle-Point Problem with Second-Order Methods

Second-order methods are susceptible to the presence of *saddle points*. A saddle point is a stationary point of a gradient-descent method because its gradient is zero, but it is not a minimum (or maximum). A saddle point is an *inflection point*, which appears to be either a minimum or a maximum depending on which direction we approach it from. Therefore, the quadratic approximation of the Newton method will give vastly different shapes depending on the direction that one approaches the saddle point from. A 1-dimensional function with a saddle point is the following:

$$f(x) = x^3$$

This function is shown in Figure 4.13(a), and it has an inflection point at $x = 0$. Note that a quadratic approximation at $x > 0$ will look like an upright bowl, whereas a quadratic

[2]This ensures simplicity, as all calculus operations assume that angles are expressed in radians.




(a) 1-dimensional saddle point (b) 2-dimensional saddle point

Figure 4.13: Illustration of saddle points

approximation at $x < 0$ will look like an inverted bowl. Furthermore, even if one reaches $x = 0$ in the optimization process, both the second derivative and the first derivative will be zero. Therefore, a Newton update will take the 0/0 form and become indefinite. Such a point is a degenerate point from the perspective of numerical optimization. Not all saddle points are degenerate points and vice versa. For multivariate problems, such degenerate points are often wide and flat regions that are not minima of the objective function. They do present a significant problem for numerical optimization. An example of such a function is $h(x, y) = x^3 + y^3$, which is degenerate at $(0, 0)$. Furthermore, the region near $(0, 0)$ will appear like a flat plateau. These types of plateaus create problems for learning algorithms, because first-order algorithms slow down in these regions and second-order algorithms also cannot recognize them as unfruitful regions for exploration. It is noteworthy that such saddle points arise only in highly nonlinear functions, which are common in neural network optimization.

It is also instructive to examine the case of a saddle point that is not a degenerate point. An example of a 2-dimensional function with a saddle point is as follows:

$$g(x, y) = x^2 - y^2$$

This function is shown in Figure 4.13(b). The saddle point is $(0, 0)$. It is easy to see that the shape of this function resembles a riding saddle. In this case, approaching from the $x$ direction or from the $y$ direction will result in very different quadratic approximations. In one case, the function will appear to be a minimum, and in another case the function will appear to be a maximum. Furthermore, the saddle point $(0, 0)$ will be a stationary point from the perspective of a Newton update, even though it is not an extremum. Saddle points occur frequently in regions between two hills of the loss function, and they present a problematic topography for second-order methods. Interestingly, first-order methods are often able to escape from saddle points [153], because the trajectory of first-order methods is simply not attracted by such points. On the other hand, Newton's method will jump directly to the saddle point.

Unfortunately, some neural-network loss functions seem to contain a large number of saddle points. Second-order methods therefore are not always preferable to first-order methods; the specific topography of a particular loss function may have an important role to play.

Second-order methods are advantageous in situations with complex curvatures of the loss function or in the presence of cliffs. In other functions with saddle points, first-order methods are advantageous. Note that the pairing of computational algorithms (like Adam) with first-order methods can at least partially address many of the complexities in the loss surface such as plateaus or varying curvature. Therefore, real-world practitioners often prefer first-order methods in combination with computational algorithms like Adam. Recently, some methods have been proposed [91] to address saddle points in second-order methods.

## 4.7 Fast Approximations of Newton Method

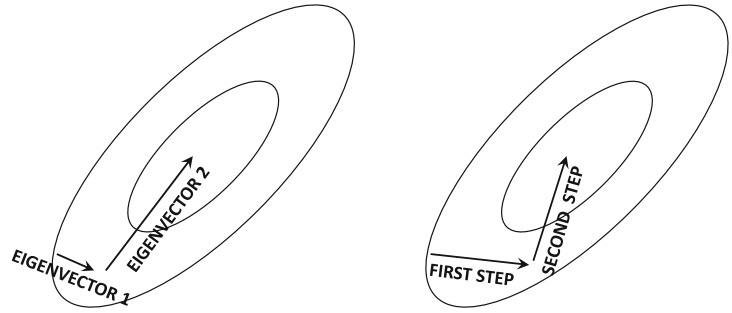
In most large-scale settings, the Hessian is too large to store or compute explicitly. It is not uncommon to have neural networks with millions of parameters. Trying to compute the inverse of a $10^6 \times 10^6$ Hessian matrix is a practically impossible task with the computational power available today. In fact, it is difficult to even compute the Hessian, let alone invert it! The following sections discuss efficient variations of the Newton method, which do not explicitly compute the Hessian.

### 4.7.1 Conjugate Gradient Method

The *conjugate gradient method* [199] requires $d$ steps to reach the optimal solution of a quadratic loss function (instead of a single Newton step). The basic idea is that any quadratic function can be transformed to a linearly additive function by using an appropriate basis transformation of variables. These variables represent directions in the data that do not interact with one another. Such noninteracting directions are extremely convenient for optimization because they can be independently optimized with line search. Since it is possible to find such directions only for quadratic loss functions, we will first discuss the conjugate gradient method under the assumption that the loss function $L(\overline{W})$ is quadratic. Later, we will discuss the generalization to non-quadratic functions. This approach is well known in the classical literature on neural networks [41, 463], and a variant has recently been reborn under the title of "Hessian-free optimization." This name is motivated by the fact that the search direction can be computed without the explicit computation of the Hessian.

A quadratic and convex loss function $L(\overline{W})$ has an ellipsoidal contour plot of the type shown in Figure 4.14, and has a constant Hessian over all regions of the optimization space. The orthonormal eigenvectors $\overline{q}_0 \ldots \overline{q}_{d-1}$ of the symmetric Hessian represent the axes directions of the ellipsoidal contour plot. One can rewrite the loss function in a new coordinate space defined by the eigenvectors as the basis vectors *to create a linearly additive quadratic function in the different variables.* This is because the new coordinate system creates an basis-aligned ellipse, which does not have interacting quadratic terms of the type $x_i x_j$. Therefore, each transformed variable can be optimized independently of the others. Alternatively, one can work with the original variables (without transformation), and simply perform line search along each eigenvector of the Hessian to select the step size. The nature of the movement is illustrated in Figure 4.14(a). Note that movement along the $j$th eigenvector does not disturb the work done along other eigenvectors, and therefore $d$ steps are sufficient to reach the optimal solution in quadratic loss functions.

Although it is impractical to compute the eigenvectors of the Hessian, there are other efficiently computable directions satisfying similar properties; this key property is referred to as *mutual conjugacy* of vectors. Note that the two eigenvectors $\overline{q}_i$ and $\overline{q}_j$ of the Hessian satisfy $\overline{q}_i^T \overline{q}_j = 0$ because of orthogonality of the eigenvectors of a symmetric matrix.




(a) Eigenvectors of Hessian Mutually Orthogonal: $\overline{q}_i^T \overline{q}_j = 0$

(b) Arbitrary conjugate pair Non-orthogonal: $\overline{q}_i^T H \overline{q}_j = 0$

Figure 4.14: The eigenvectors of the Hessian of a quadratic function represent the orthogonal axes of the quadratic ellipsoid and are also mutually orthogonal. The eigenvectors of the Hessian are orthogonal conjugate directions. The generalized definition of conjugacy may result in non-orthogonal directions.

Furthermore, since $\overline{q}_j$ is an eigenvector of $H$, we have $H\overline{q}_j = \lambda_j \overline{q}_j$ for some scalar eigenvalue $\lambda_j$. Multiplying both sides with $\overline{q}_i^T$, we can easily show that the eigenvectors of the Hessian satisfy $\overline{q}_i^T H \overline{q}_j = 0$ in pairwise fashion. The condition $\overline{q}_i^T H \overline{q}_j = 0$ is referred to as $H$-orthogonality in linear algebra, and is also referred to as the *mutual conjugacy condition* in optimization. It is this mutual conjugacy condition that results in linearly separated variables. However, the eigenvectors are not the only set of mutually conjugate conditions. Just as there are an infinite number of orthonormal basis sets, there are an infinite number of $H$-orthogonal basis sets in $d$-dimensional space. If we re-write the quadratic loss function in terms of coordinates in a *any* (possibly non-orthogonal) axis system of conjugate directions, the objective function will contain linearly additive components each of which is a univariate function. One can separately optimize along each of these $d$ $H$-orthogonal directions (in terms of the original variables) to solve the quadratic optimization problem in $d$ steps. Each of these optimization steps can be performed using line search along an $H$-orthogonal direction. Hessian eigenvectors represent a rather special set of $H$-orthogonal directions that are also orthogonal; conjugate directions other than Hessian eigenvectors, such as those shown in Figure 4.14(b), are not mutually orthogonal. Therefore, conjugate gradient descent optimizes a quadratic objective function by *implicitly* transforming the loss function into a *non-orthogonal* basis with a linearly additive representation of the objective function. One can state this observation as follows:

**Observation 4.7.1 (Properties of H-Orthogonal Directions)** *Let $H$ be the Hessian of a quadratic objective function. If any set of $d$ $H$-orthogonal directions are selected for movement, then one is implicitly moving along additively separable variables in a transformed representation of the function. Therefore, at most $d$ steps are required for quadratic optimization.*

The independent optimization along each separable direction (with line search) ensures that the component of the gradient along each conjugate direction will be 0. Strictly convex loss functions have linearly independent conjugate directions (see Exercise 5). In other words, the final gradient will have zero dot product with $d$ linearly independent directions; this is possible only when the final gradient is the zero vector (see Exercise 6), which implies

optimality for a convex function. In fact, one can often reach a near-optimal solution in far fewer than $d$ updates.

How can one identify conjugate directions? The simplest approach is to use *generalized Gram-Schmidt orthogonalization* on the Hessian of the quadratic function in order to generate $H$-orthogonal directions (see Exercise 7). Such an orthogonalization is easy to achieve using arbitrary vectors as starting points. However, this process can still be quite expensive because each direction $\overline{q}_t$ needs to use *all* the previous directions $\overline{q}_0 \ldots \overline{q}_{t-1}$ for iterative generation in the Gram-Schmidt method. Since each direction is a $d$-dimensional vector, and there are $O(d)$ such directions towards the end of the process, it follows that each step will require $O(d^2)$ time. Is there a way to do this using only the previous direction in order to reduce this time from $O(d^2)$ to $O(d)$? Surprisingly, only the most recent conjugate direction is needed to generate the next direction [370, 463], when steepest descent directions are used for iterative generation. In other words, one should not use Gram-Schmidt orthogonalization with arbitrary vectors, but should use steepest descent directions as the raw vectors to be orthogonalized. This choice makes all the difference in ensuring a more efficient form of orthogonalization. This is not an obvious result (see Exercise 8). The direction $\overline{q}_{t+1}$ is, therefore, defined iteratively as a linear combination of *only* the previous conjugate direction $\overline{q}_t$ and the current steepest descent direction $\nabla L(\overline{W}_{t+1})$ with combination parameter $\beta_t$:

$$\overline{q}_{t+1} = -\nabla L(\overline{W}_{t+1}) + \beta_t \overline{q}_t \tag{4.15}$$

Premultiplying both sides with $\overline{q}_t^T H$ and using the conjugacy condition to set the left-hand side to 0, one can solve for $\beta_t$:

$$\beta_t = \frac{\overline{q}_t^T H[\nabla L(\overline{W}_{t+1})]}{\overline{q}_t^T H \overline{q}_t} \tag{4.16}$$

This leads to an iterative update process, which initializes $\overline{q}_0 = -\nabla L(\overline{W}_0)$, and computes $\overline{q}_{t+1}$ iteratively for $t = 0, 1, 2, \ldots T$:

1. Update $\overline{W}_{t+1} \Leftarrow \overline{W}_t + \alpha_t \overline{q}_t$. Here, the step size $\alpha_t$ is computed using line search to minimize the loss function.
2. Set $\overline{q}_{t+1} = -\nabla L(\overline{W}_{t+1}) + \left( \frac{\overline{q}_t^T H[\nabla L(\overline{W}_{t+1})]}{\overline{q}_t^T H \overline{q}_t} \right) \overline{q}_t$. Increment $t$ by 1.

It can be shown [370, 463] that $\overline{q}_{t+1}$ satisfies conjugacy with respect to *all* previous $\overline{q}_i$. A systematic road-map of this proof is provided in Exercise 8.

The above updates do not *seem* to be Hessian-free, because the matrix $H$ is included in the above updates. However, the underlying computations only need the *projection* of the Hessian along particular directions; we will see that these can be computed indirectly using the method of finite differences without explicitly computing the individual elements of the Hessian. Let $\overline{v}$ be the vector direction for which the projection $H\overline{v}$ needs to be computed. The method of finite differences computes the loss gradient at the current parameter vector $\overline{W}$ and at $\overline{W} + \delta\overline{v}$ for some small value of $\delta$ in order to perform the approximation:

$$H\overline{v} \approx \frac{\nabla L(\overline{W} + \delta\overline{v}) - \nabla L(\overline{W})}{\delta} \propto \nabla L(\overline{W} + \delta\overline{v}) - \nabla L(\overline{W}) \tag{4.17}$$

The right-hand side is free of the Hessian. The condition is exact for quadratic functions. Other alternatives for Hessian-free updates are discussed in [41].

So far, we have discussed the simplified case of quadratic loss functions, in which the Hessian is a constant matrix (i.e., independent of the current parameter vector). However,

most loss functions in machine learning are not quadratic and, therefore, the Hessian matrix is dependent on the current value of the parameter vector $\overline{W}_t$. This leads to several choices in terms of how one can create a modified algorithm for non-quadratic functions. Do we first create a quadratic approximation at a point and then solve it for a few iterations with the Hessian (quadratic approximation) fixed at that point, or do we change the Hessian every iteration along with the change in parameter vector? The former is referred to as the *linear conjugate gradient method*, whereas the latter is referred to as the *nonlinear conjugate gradient method*.

In the nonlinear conjugate gradient method, the mutual conjugacy (i.e., H-orthogonality) of the directions will deteriorate over time, as the Hessian changes from one step to the next. This can have an unpredictable effect on the overall progress from one step to the next. Furthermore, the computation of conjugate directions needs to be restarted every few steps, as the mutual conjugacy deteriorates. If the deterioration occurs too fast, the restarts occur very frequently, and one does not gain much from conjugacy. On the other hand, each quadratic approximation in the linear conjugate gradient method can be solved exactly, and will typically be (almost) solved in much fewer than $d$ iterations. Therefore, one can make similar progress to the Newton method in each iteration. As long as the quadratic approximation is of high quality, the required number of approximations is often not too large. The nonlinear conjugate gradient method has been extensively used in traditional machine learning from a historical perspective [41], although recent work [324, 325] has advocated the use of linear conjugate methods. Experimental results in [324, 325] suggest that linear conjugate gradient methods have some advantages.

### 4.7.2 Quasi-Newton Methods and BFGS

The acronym BFGS stands for the Broyden–Fletcher–Goldfarb–Shanno algorithm, and it is derived as an approximation of the Newton method. Let us revisit the updates of the Newton method. A typical update of the Newton method is as follows:

$$\overline{W}^* \Leftarrow \overline{W}_0 - H^{-1}[\nabla J(\overline{W}_0)] \tag{4.18}$$

In quasi-Newton methods, a sequence of approximations of the inverse Hessian matrix are used in various steps. Let the approximation of the inverse Hessian matrix in the $t$th step be denoted by $G_t$. In the very first iteration, the value of $G_t$ is initialized to the identity matrix, which amounts to moving along the steepest-descent direction. This matrix is continuously updated from $G_t$ to $G_{t+1}$ with low-rank updates (derived from the matrix inversion lemma of Chapter 1). A direct restatement of the Newton update in terms of the inverse Hessian $G_t \approx H_t^{-1}$ is as follows:

$$\overline{W}_{t+1} \Leftarrow \overline{W}_t - G_t[\nabla J(\overline{W}_t)] \tag{4.19}$$

The above update can be improved with an optimized learning rate $\alpha_t$ for non-quadratic loss functions working with (inverse) Hessian approximations like $G_t$:

$$\overline{W}_{t+1} \Leftarrow \overline{W}_t - \alpha_t G_t[\nabla J(\overline{W}_t)] \tag{4.20}$$

The optimized learning rate $\alpha_t$ is identified with line search. The line search does not need to be performed exactly (like the conjugate gradient method), because maintenance of conjugacy is no longer critical. Nevertheless, approximate conjugacy of the early set of directions is maintained by the method when starting with the identity matrix. One can (optionally) reset $G_t$ to the identity matrix every $d$ iterations (although this is rarely done).

It remains to be discussed how the matrix $G_{t+1}$ is approximated from $G_t$. For this purpose, the *quasi-Newton condition*, also referred to as the *secant condition*, is needed:

$$\underbrace{\overline{W}_{t+1} - \overline{W}_t}_{\text{Parameter Change}} = G_{t+1} \underbrace{[\nabla J(\overline{W}_{t+1}) - \nabla J(\overline{W}_t)]}_{\text{First derivative change}} \tag{4.21}$$

The above formula is simply a finite-difference approximation. Intuitively, multiplication of the second-derivative matrix (i.e., Hessian) with the parameter change (vector) approximately provides the gradient change. Therefore, multiplication of the inverse Hessian approximation $G_{t+1}$ with the gradient change provides the parameter change. The goal is to find a symmetric matrix $G_{t+1}$ satisfying Equation 4.21, but it represents an underdetermined system of equations with an infinite number of solutions. Among these, BFGS chooses the closest symmetric $G_{t+1}$ to the current $G_t$, and achieves this goal by posing a minimization objective function $||G_{t+1} - G_t||_w$ in the form of a *weighted Frobenius norm*. In other words, we want to find $G_{t+1}$ satisfying the following:

$$\begin{aligned} \text{Minimize}_{[G_{t+1}]} \; & ||G_{t+1} - G_t||_w \\ & \text{subject to:} \\ & \overline{W}_{t+1} - \overline{W}_t = G_{t+1}[\nabla J(\overline{W}_{t+1}) - \nabla J(\overline{W}_t)] \\ & G_{t+1}^T = G_{t+1} \end{aligned}$$

The subscript of the norm is annotated by "$w$" to indicate that it is a weighted[3] form of the norm. This weight is an "averaged" form of the Hessian, and we refer the reader to [370] for details of how the averaging is done. Note that one is not constrained to using the weighted Frobenius norm, and different variations of how the norm is constructed lead to different variations of the quasi-Newton method. For example, one can pose the same objective function and secant condition in terms of the Hessian rather than the inverse Hessian, and the resulting method is referred to as the Davidson–Fletcher–Powell (DFP) method. In the following, we will stick to the use of the inverse Hessian, which is the BFGS method.

Since the weighted norm uses the Frobenius matrix norm (along with a weight matrix) the above is a quadratic optimization problem with linear constraints. In general, when there are linear equality constraints paired with a quadratic objective function, the structure of the optimization problem is quite simple, and closed-form solutions can sometimes be found. This is because the equality constraints can often be eliminated along with corresponding variables (using methods like Gaussian elimination), and an unconstrained, quadratic optimization problem can be defined in terms of the remaining variables. These problems sometimes turn out to have closed-form solutions like least-squared regression. In this case, the closed-form solution to the above optimization problem is as follows:

$$G_{t+1} \Leftarrow (I - \Delta_t \overline{q}_t \overline{v}_t^T) G_t (I - \Delta_t \overline{v}_t \overline{q}_t^T) + \Delta_t \overline{q}_t \overline{q}_t^T \tag{4.22}$$

Here, the (column) vectors $\overline{q}_t$ and $\overline{v}_t$ represent the parameter change and the gradient change; the scalar $\Delta_t = 1/(\overline{q}_t^T \overline{v}_t)$ is the inverse of the dot product of these two vectors.

$$\overline{q}_t = \overline{W}_{t+1} - \overline{W}_t; \quad \overline{v}_t = \nabla L(\overline{W}_{t+1}) - \nabla L(\overline{W}_t)$$

[3]The form of the objective function is $||A^{1/2}(G_{t+1} - G_t)A^{1/2}||_F$ norm, where $A$ is an averaged version of the Hessian matrix over various lengths of the step. We refer the reader to [370] for details.

The update in Equation 4.22 can be made more space efficient by expanding it, so that fewer temporary matrices need to be maintained. Interested readers are referred to [309, 370, 390] for implementation details and derivation of these updates.

Even though BFGS benefits from approximating the inverse Hessian, it does need to carry over a matrix $G_t$ of size $O(d^2)$ from one iteration to the next. The *limited memory BFGS* (L-BFGS) reduces the memory requirement drastically from $O(d^2)$ to $O(d)$ by not carrying over the matrix $G_t$ from the previous iteration. In the most basic version of the L-BFGS method, the matrix $G_t$ is replaced with the identity matrix in Equation 4.22 in order to derive $G_{t+1}$. A more refined choice is to store the $m \approx 30$ most recent vectors $\overline{q}_t$ and $\overline{v}_t$. Then, L-BFGS is equivalent to initializing $G_{t-m+1}$ to the identity matrix and recursively applying Equation 4.22 $m$ times to derive $G_{t+1}$. In practice, the implementation is optimized to directly compute the direction of movement from the vectors without explicitly storing large intermediate matrices from $G_{t-m+1}$ to $G_t$.

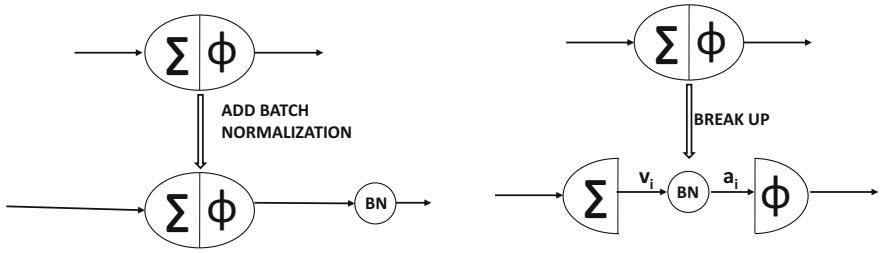
## 4.8 Batch Normalization

Batch normalization is a recent method to address the vanishing and exploding gradient problems, which cause activation gradients in successive layers to either reduce or increase in magnitude. Another important problem in training deep networks is that of *internal covariate shift*. The problem is that the parameters change during training, and therefore the hidden variable activations change as well. In other words, the hidden inputs from early layers to later layers keep changing. Changing inputs from early layers to later layers causes slower convergence during training because the training data for later layers is not stable. Batch normalization is able to reduce this effect.

In batch normalization, the idea is to add additional "normalization layers" between hidden layers that resist this type of behavior by creating features with somewhat similar variance. Furthermore, each unit in the normalization layers contains two additional parameters $\beta_i$ and $\gamma_i$ that regulate the precise level of normalization in the $i$th unit; these parameters are learned in a data-driven manner. The basic idea is that the output of the $i$th unit will have a mean of $\beta_i$ and a standard deviation of $\gamma_i$ *over each mini-batch of training instances*. One might wonder whether it might make sense to simply set each $\beta_i$ to 0 and each $\gamma_i$ to 1, but doing so reduces the representation power of the network. For example, if we make this transformation, then the sigmoid units will be operating within their linear regions, especially if the normalization is performed just before activation (see below for discussion of Figure 4.15). Recall from the discussion in Chapter 1 that multilayer networks do not gain power from depth without nonlinear activations. Therefore, allowing some "wiggle" with these parameters and learning them in a data-driven manner makes sense. Furthermore, the parameter $\beta_i$ plays the role of a learned bias variable, and therefore we do not need additional bias units in these layers.

We assume that the $i$th unit is connected to a special type of node $BN_i$, where $BN$ stands for batch normalization. This unit contains two parameters $\beta_i$ and $\gamma_i$ that need to be learned. Note that $BN_i$ has only one input, and its job is to perform the normalization and scaling. This node is then connected to the next layer of the network in the standard way in which a neural network is connected to future layers. Here, we mention that there are two choices for where the normalization layer can be connected:

1. The normalization can be performed just after applying the activation function to the linearly transformed inputs. This solution is shown in Figure 4.15(a). Therefore, the normalization is performed on *post-activation values*.




(a) Post-activation normalization (b) Pre-activation normalization

Figure 4.15: The different choices in batch normalization

2. The normalization can be performed after the linear transformation of the inputs, but before applying the activation function. This situation is shown in Figure 4.15(b). Therefore, the normalization is performed on *pre-activation values*.

It is argued in [225] that the second choice has more advantages. Therefore, we focus on this choice in this exposition. The BN node shown in Figure 4.15(b) is just like any other computational node (albeit with some special properties), and one can perform backpropagation through this node just like any other computational node.

What transformations does $BN_i$ apply? Consider the case in which its input is $v_i^{(r)}$, corresponding to the $r$th element of the batch feeding into the $i$th unit. Each $v_i^{(r)}$ is obtained by using the linear transformation defined by the coefficient vector $\overline{W}_i$ (and biases if any). For a particular batch of $m$ instances, let the values of the $m$ activations be denoted by $v_i^{(1)}, v_i^{(2)}, \ldots v_i^{(m)}$. The first step is to compute the mean $\mu_i$ and standard deviation $\sigma_i$ for the $i$th hidden unit. These are then scaled using the parameters $\beta_i$ and $\gamma_i$ to create the outputs for the next layer:

$$\mu_i = \frac{\sum_{r=1}^{m} v_i^{(r)}}{m} \quad \forall i \tag{4.23}$$

$$\sigma_i^2 = \frac{\sum_{r=1}^{m} (v_i^{(r)} - \mu_i)^2}{m} + \epsilon \quad \forall i \tag{4.24}$$

$$\hat{v}_i^{(r)} = \frac{v_i^{(r)} - \mu_i}{\sigma_i} \quad \forall i, r \tag{4.25}$$

$$a_i^{(r)} = \gamma_i \cdot \hat{v}_i^{(r)} + \beta_i \quad \forall i, r \tag{4.26}$$

A small value of $\epsilon$ is added to $\sigma_i^2$ to regularize cases in which all activations are the same, which results in zero variance. Note that $a_i^{(r)}$ is the pre-activation output of the $i$th node, when the $r$th batch instance passes through it. This value would otherwise have been set to $v_i^{(r)}$, if we had not applied batch normalization. We conceptually represent this node with a special node $BN_i$ that performs this additional processing. This node is shown in Figure 4.15(b). Therefore, the backpropagation algorithm has to account for this additional node and ensure that the loss derivative of layers earlier than the batch normalization layer accounts for the transformation implied by these new nodes. It is important to note that the function applied at each of these special BN nodes is specific to the *batch* at hand. This type of computation is unusual for a neural network in which the gradients are linearly additive sums of the gradients with respect to individual training examples. This is not quite true in

this case because the batch normalization layer computes nonlinear metrics from the batch (such as its standard deviation). Therefore, the activations depend on how the examples in a batch are related to one another, which is not common in most neural computations. However, this special property of the BN node does not prevent us from backpropagating through the computations performed in it.

The following will describe the changes in the backpropagation algorithm caused by the normalization layer. The main point of this change is to show how to backpropagate through the newly added layer of normalization nodes. Another point to be aware of is that we want to optimize the parameters $\beta_i$ and $\gamma_i$. For the gradient-descent steps with respect to each $\beta_i$ and $\gamma_i$, we need the gradients with respect to these parameters. Assume that we have already backpropagated up to the output of the BN node, and therefore we have each $\frac{\partial L}{\partial a_i^{(r)}}$ available. Then, the derivatives with respect to the two parameters can be computed as follows:

$$\frac{\partial L}{\partial \beta_i} = \sum_{r=1}^{m} \frac{\partial L}{\partial a_i^{(r)}} \cdot \frac{\partial a_i^{(r)}}{\partial \beta_i} = \sum_{r=1}^{m} \frac{\partial L}{\partial a_i^{(r)}}$$

$$\frac{\partial L}{\partial \gamma_i} = \sum_{r=1}^{m} \frac{\partial L}{\partial a_i^{(r)}} \cdot \frac{\partial a_i^{(r)}}{\partial \gamma_i} = \sum_{r=1}^{m} \frac{\partial L}{\partial a_i^{(r)}} \cdot \hat{v}_i^{(r)}$$

We also need a way to compute $\frac{\partial L}{\partial v_i^r}$. Once this value is computed, the backpropagation to the pre-activation values $\frac{\partial L}{\partial a_j^r}$ for all nodes $j$ in the previous layer uses the straightforward backpropagation update introduced earlier in this chapter. Therefore, the dynamic programming recursion will be complete because one can then use these values of $\frac{\partial L}{\partial a_j^r}$. One can compute the value of $\frac{\partial L}{\partial v_i^r}$ in terms of $\hat{v}_i^{(r)}$, $\mu_i$, and $\sigma_i$, by observing that $v_i^{(r)}$ can be written as a (normalization) function of only $\hat{v}_i^{(r)}$, mean $\mu_i$, and variance $\sigma_i^2$. Observe that $\mu_i$ and $\sigma_i$ are not treated as constants, but as variables because they depend on the batch at hand. Therefore, we have the following:

$$\frac{\partial L}{\partial v_i^{(r)}} = \frac{\partial L}{\partial \hat{v}_i^{(r)}} \frac{\partial \hat{v}_i^{(r)}}{\partial v_i^{(r)}} + \frac{\partial L}{\partial \mu_i} \frac{\partial \mu_i}{\partial v_i^{(r)}} + \frac{\partial L}{\partial \sigma_i^2} \frac{\partial \sigma_i^2}{\partial v_i^{(r)}} \tag{4.27}$$

$$= \frac{\partial L}{\partial \hat{v}_i^{(r)}} \left(\frac{1}{\sigma_i}\right) + \frac{\partial L}{\partial \mu_i} \left(\frac{1}{m}\right) + \frac{\partial L}{\partial \sigma_i^2} \left(\frac{2(v_i^{(r)} - \mu_i)}{m}\right) \tag{4.28}$$

We need to evaluate each of the three partial derivatives on the right-hand side of the above equation in terms of the quantities that have been computed using the already-executed dynamic programming updates of backpropagation. This allows the creation of the recurrence equation for the batch normalization layer. Among these, the first expression, which is $\frac{\partial L}{\partial \hat{v}_i^{(r)}}$, can be substituted in terms of the loss derivatives of the next layer by observing that $a_i^{(r)}$ is related to $\hat{v}_i^{(r)}$ by a constant of proportionality $\gamma_i$:

$$\frac{\partial L}{\partial \hat{v}_i^{(r)}} = \gamma_i \frac{\partial L}{\partial a_i^{(r)}} \quad [\text{Since } a_i^{(r)} = \gamma_i \cdot \hat{v}_i^{(r)} + \beta_i] \tag{4.29}$$

Therefore, by substituting this value of $\frac{\partial L}{\partial \hat{v}_i^{(r)}}$ in Equation 4.28, we have the following:

$$\frac{\partial L}{\partial v_i^{(r)}} = \frac{\partial L}{\partial a_i^{(r)}} \left(\frac{\gamma_i}{\sigma_i}\right) + \frac{\partial L}{\partial \mu_i} \left(\frac{1}{m}\right) + \frac{\partial L}{\partial \sigma_i^2} \left(\frac{2(v_i^{(r)} - \mu_i)}{m}\right) \tag{4.30}$$

It now remains to compute the partial derivative of the loss with respect to the mean and the variance. The partial derivative of the loss with respect to the variance is computed as follows:

$$\frac{\partial L}{\partial \sigma_i^2} = \underbrace{\sum_{q=1}^{m} \frac{\partial L}{\partial \hat{v}_i^{(q)}} \cdot \frac{\partial \hat{v}_i^{(q)}}{\partial \sigma_i^2}}_{\text{Chain rule}} = \underbrace{-\frac{1}{2\sigma_i^3} \sum_{q=1}^{m} \frac{\partial L}{\partial \hat{v}_i^{(q)}} (v_i^{(q)} - \mu_i)}_{\text{Use Equation 4.25}} = \underbrace{-\frac{1}{2\sigma_i^3} \sum_{q=1}^{m} \frac{\partial L}{\partial a_i^{(q)}} \gamma_i \cdot (v_i^{(q)} - \mu_i)}_{\text{Substitution from Equation 4.29}}$$

The partial derivatives of the loss with respect to the mean can be computed as follows:

$$\frac{\partial L}{\partial \mu_i} = \underbrace{\sum_{q=1}^{m} \frac{\partial L}{\partial \hat{v}_i^{(q)}} \cdot \frac{\partial \hat{v}_i^{(q)}}{\partial \mu_i} + \frac{\partial L}{\partial \sigma_i^2} \cdot \frac{\partial \sigma_i^2}{\partial \mu_i}}_{\text{Chain rule}} = \underbrace{-\frac{1}{\sigma_i} \sum_{q=1}^{m} \frac{\partial L}{\partial \hat{v}_i^{(q)}} - 2\frac{\partial L}{\partial \sigma_i^2} \cdot \frac{\sum_{q=1}^{m} (v_i^{(q)} - \mu_i)}{m}}_{\text{Use Equations 4.24 and 4.25}}$$

$$= \underbrace{-\frac{\gamma_i}{\sigma_i} \sum_{q=1}^{m} \frac{\partial L}{\partial a_i^{(q)}}}_{\text{Eq. 4.29}} + \underbrace{\left(\frac{1}{\sigma_i^3}\right) \cdot \left(\sum_{q=1}^{m} \frac{\partial L}{\partial a_i^{(q)}} \gamma_i \cdot (v_i^{(q)} - \mu_i)\right) \cdot \left(\frac{\sum_{q=1}^{m} (v_i^{(q)} - \mu_i)}{m}\right)}_{\text{Substitution for } \frac{\partial L}{\partial \sigma_i^2}}$$

By plugging in the partial derivatives of the loss with respect to the mean and variance in Equation 4.30, we get a full recursion for $\frac{\partial L}{\partial v_i^{(r)}}$ (value before batch-normalization layer) in terms of $\frac{\partial L}{\partial a_i^{(r)}}$ (value *after* the batch normalization layer). This provides a full view of the backpropagation of the loss through the batch-normalization layer corresponding to the BN node. The other aspects of backpropagation remain similar to the traditional case. Batch normalization enables faster inference because it prevents problems such as the exploding and vanishing gradient (which cause slow learning).

A natural question about batch normalization arises during inference (prediction) time. Since the transformation parameters $\mu_i$ and $\sigma_i$ depend on the batch, how should one compute them during testing when a *single* test instance is available? In this case, the values of $\mu_i$ are $\sigma_i$ are computed up front using the *entire* population (of training data), and then treated as constants during testing time. One can also keep an exponentially weighted average of these values during training. Therefore, the normalization is a simple linear transformation during inference.

An interesting property of batch normalization is that *it also acts as a regularizer*. Note that the same data point can cause somewhat different updates depending on which batch it is included in. One can view this effect as a kind of noise added to the update process. Regularization is often achieved by adding a small amount of noise to the training data. It has been experimentally observed that regularization methods like *Dropout* (cf. section 5.5.4 of Chapter 5) do not seem to improve performance when batch normalization is used [194], although there is not a complete agreement on this point. A variant of batch normalization, known as *layer normalization*, is known to work well with recurrent networks. This approach is discussed in section 8.3.1 of Chapter 8.

## 4.9 Practical Tricks for Acceleration and Compression

Neural network learning algorithms can be extremely expensive, both in terms of the number of parameters in the model and the amount of data that needs to be processed. There are

several strategies that are used to accelerate and compress the underlying implementations. Some of the common strategies are as follows:

1. *GPU-acceleration:* Graphics Processor Units (GPUs) have historically been used for rendering video games with intensive graphics because of their efficiency in settings where repeated matrix operations (e.g., on graphics pixels) are required. It was eventually realized by the machine learning community (and GPU hardware companies) that such repetitive matrix operations are also used in deep learning. Even the use of a single GPU can significantly speed up implementation because of its high memory bandwidth and multithreading within its multicore architecture.

2. *Parallel implementations:* One can parallelize the implementations of neural networks by using multiple GPUs or CPUs. Either the neural network model or the data can be partitioned across different processors. These implementations are referred to as *model-parallel* and *data-parallel* implementations.

3. *Algorithmic tricks for model compression:* A key point about the practical use of neural networks is that they have different computational requirements during training and deployment (prediction). While it is acceptable to train a model for a week on state-of-the-art hardware, the final deployment might be performed on a mobile phone with limited memory and computational power. Therefore, numerous tricks are used for model compression during testing time. This type of compression often results in better cache performance and efficiency as well.

In the following, we will discuss some of these acceleration and compression techniques.

### 4.9.1 GPU Acceleration

GPUs were originally developed for rendering graphics on screens with the use of lists of 3-dimensional coordinates. Therefore, graphics cards were inherently designed to perform many matrix multiplications in parallel to render the graphics rapidly. GPU processors have evolved significantly, moving well beyond their original functionality of graphics rendering. Like graphics applications, neural-network implementations require large matrix multiplications, which is inherently suited to the GPU setting. In a traditional neural network, each forward propagation is a multiplication of a matrix and vector, whereas in a convolutional neural network, two matrices are multiplied. When a mini-batch approach is used, activations become matrices (instead of vectors) in a traditional neural network. Therefore, forward propagations require matrix multiplications. A similar result is true for backpropagation, during which two matrices are multiplied frequently to propagate the derivatives backwards. In other words, most of the intensive computations involve vector, matrix, and tensor operations. Even a single GPU is good at parallelizing these operations in its different cores with multithreading [213], in which some groups of threads sharing the same code are executed concurrently. This principle is referred to as *Single Instruction Multiple Threads (SIMT)*. Although CPUs also support short-vector data parallelization via *Single Instruction Multiple Data (SIMD)* instructions, the degree of parallelism is much lower as compared to the GPU. There are different trade-offs when using GPUs as compared to traditional CPUs. GPUs are very good at repetitive operations, but they have difficulty at performing branching operations like *if-then* statements. Most of the intensive operations in neural network learning are repetitive matrix multiplications across different training instances, and therefore this setting is suited to the GPU. Although the clock speed of a

single instruction in the GPU is slower than the traditional CPU, the parallelization is so much greater in the GPU that huge advantages are gained.

GPU threads are grouped into small units called *warps*. Each thread in the warp shares the same code in each cycle, and this restriction enables a concurrent execution of the threads. The implementation needs to be carefully tailored to reduce the use of memory bandwidth. This is done by *coalescing* the memory reads and writes from different threads, so that a single memory transaction can be used to read and write values from different threads. Consider a common operation like matrix multiplication in neural network settings. The matrices are multiplied by making each thread responsible for computing a single entry in the product matrix. For example, consider a situation in which a $100 \times 50$ matrix is multiplied with a $50 \times 200$ matrix. In such a case, a total of $100 \times 200 = 20000$ threads would be launched in order to compute the entries of the matrix. These threads will typically be partitioned into multiple warps, each of which is highly parallelized. Therefore, speedups are achieved. A discussion of matrix multiplication on GPUs is provided in [213].

With high amounts of parallelization, memory bandwidth is often the primary limiting factor. Memory bandwidth refers to the speed at which the processor can access the relevant parameters from their stored locations in memory. GPUs have a high degree of parallelism and high memory bandwidth as compared to traditional CPUs. Note that if one cannot access the relevant parameters from memory fast enough, then faster execution does not help the speed of computation. In such cases, the memory transfer cannot keep up with the speed of the processor whether working with the CPU or the GPU, and the CPU/GPU cores will idle. GPUs have different trade-offs between cache access, computation, and memory access. CPUs have much larger caches than GPUs and they rely on the caches to store an intermediate result, such as the result of multiplying two numbers. Accessing a computed value from a cache is much faster than multiplying them again, which is where the CPU has an advantage over the GPU. However, this advantage is neutralized in neural network settings, where the sizes of the parameter matrices and activations are often too large to fit in the CPU cache. Even though the CPU cache is larger than that of the GPU, it is not large enough to handle the scale at which neural-network operations are performed. In such cases, one has to rely on high memory bandwidth, which is where the GPU has an advantage over the CPU. Furthermore, it is often faster to perform the same computation again rather than accessing it from memory, when working with the GPU (assuming that the result is unavailable in a cache). Therefore, GPU implementations are done somewhat differently from traditional CPU implementations. Furthermore, the advantage gained can be sensitive to the choice of neural network architecture, as the memory bandwidth requirements and multi-threading gains of different architectures can be different.

At first sight, it might seem that a lot of challenging and customized low-level programming is required for each GPU-based neural architecture. With this problem in mind, companies like NVIDIA have modularized the programmer interface with the GPU implementation. The key point is that the speeding of primitives like matrix multiplication and convolution can be hidden from the user by providing a library of neural network operations that perform these faster operations behind the scenes. The GPU library is tightly integrated with deep learning frameworks like Caffe or Torch to take advantage of accelerated GPU operations. A specific example of such a library is the *NVIDIA CUDA Deep Neural Network Library* [664], which is referred to in short as *cuDNN*. CUDA is a parallel computing platform and programming model that works with CUDA-enabled GPU processors. However, it provides an abstraction and a programming interface that is easy to use with relatively limited rewriting of code. The cuDNN library can be integrated with multiple deep learning frameworks such as Caffe, TensorFlow, Theano, and Torch. The changes re-

quired to convert the training code of a particular neural network from its CPU version to a GPU version are often small. For example, in Torch, the CUDA Torch package is incorporated at the beginning of the code, and various data structures (like tensors) are initialized as CUDA tensors (instead of regular tensors). With these types of modest modifications, virtually the same code can run on a GPU instead of a CPU in Torch. A similar situation holds true in other deep learning frameworks. Such an approach shields developers from the low-level performance tuning required in GPU frameworks, because the implementations of the library primitives take care of all the low-level details of GPU parallelization.

### 4.9.2 Parallel and Distributed Implementations

It is possible to make training even faster by using multiple CPUs or GPUs. Since it is more common to use multiple GPUs, we focus on this setting. Parallelism is not a simple matter when working with GPUs because there are overheads associated with the communication between different processors. The delay caused by these overheads has recently been reduced with specialized network cards for GPU-to-GPU transfer. Furthermore, algorithmic tricks like using 8-bit approximations of the gradients [100] can help in speeding up the communication. There are several ways in which one can partition the work across different processors, namely hyperparameter parallelism, model parallelism, and data parallelism. These methods are discussed below.

#### Hyperparameter Parallelism

The simplest possible way to achieve parallelism in the training process without much overhead is to train neural networks with different parameter settings on different processors. No communication is required across different executions, and therefore wasteful overhead is avoided. As discussed earlier in this chapter, runs with suboptimal hyperparameters are often terminated long before running them to completion. Nevertheless, a small number of different runs with optimized parameters are often used in order to create an ensemble of models. The training of different ensemble components can be performed independently on different processors.

#### Model Parallelism

Model parallelism is particularly useful when a single model is too large to fit on a GPU. In such a case, the hidden layer is divided across the different GPUs. The different GPUs work on exactly the same batch of training points, although different GPUs compute different parts of the activations and the gradients. Each GPU only contains the portion of the weight matrix that are multiplied with the hidden activations present in the GPU. However, it would still need to communicate the results of its activations to the other GPUs. Similarly, it would need to receive the derivatives with respect to the hidden units in other GPUs in order to compute the gradients of the weights between its hidden units and those of other GPUs. This is achieved with the use of inter-connections across GPUs, and the computations across these interconnections add to the overhead. In some cases, these interconnections are dropped in a subset of the layers in order to reduce the communication overhead (although the resulting model would not quite be the same as the sequential version). Model parallelism is not helpful in cases where the number of parameters in the neural network is small, and should only be used for large networks. A good practical example of model parallelism is the design of *AlexNet*, which is a convolutional neural network (cf. section 9.4.1 of Chapter 9). A sequential version of *AlexNet* and a GPU-partitioned version of *AlexNet* are both shown

in Figure 9.9 of Chapter 9. Note that the sequential version in Figure 9.9 is not exactly equivalent to the GPU-partitioned version because the interconnections between GPUs have been dropped in some of the layers. A discussion of model parallelism may be found in [77].

**Data Parallelism**

Data parallelism works best when the model is small enough to fit on each GPU, but the amount of training data is large. In these cases, the parameters are shared across the different GPUs and the goal of the updates is to use the different processors with different training points for faster updates. The problem is that perfect synchronization of the updates can slow down the process, because locking mechanisms would need to be used to synchronize the updates. The key point is that each processor would have to wait for the others to make their updates. As a result, the slowest processor creates a bottleneck. A method that uses *asynchronous* stochastic gradient descent was proposed in [94]. The basic idea is to use a parameter server in order to share the parameters across different GPU processors. The updates are performed without using any locking mechanism. In other words, each GPU can read the shared parameters at any time, perform the computation, and write the parameters to the parameter server without worrying about locks. In this case, inefficiency would still be caused by one GPU processor overwriting the progress made by another, but there would be no waiting times for writes. As a result, the overall progress would still be faster than with a synchronized mechanism. Distributed asynchronous gradient descent is quite popular as a strategy for parallelism in large-scale industrial settings.

**Exploiting the Trade-Offs for Hybrid Parallelism**

It is evident from the above discussion that model parallelism is well suited to models with a large parameter footprint, whereas data parallelism is well suited to smaller models. It turns out that one can combine the two types of parallelism over different parts of the network. In certain types of convolutional neural networks that have fully connected layers, the vast majority of parameters occur in the fully connected layers, whereas more computations are performed in the earlier layers. In these cases, it makes sense to use data parallelism for the early part of the network, and model parallelism for the later part of the network. This type of approach is referred to as *hybrid parallelism*. A discussion of this type of approach may be found in [262].

### 4.9.3 Algorithmic Tricks for Model Compression

Training a neural network and deploying it typically have different requirements in terms of memory and efficiency requirements. While it may be acceptable to require a week to train a neural network to recognize faces in images, the end user might wish to use the trained neural network to recognize a face within a a few seconds. Furthermore, the model might be deployed on a mobile device with limited memory and computational availability. Computational efficiency is generally not a problem at deployment time, because the prediction of a test instance often requires straightforward matrix multiplications over a few layers. On the other hand, the storage requirements are often a problem because of the large number of parameters in multilayer networks. There are several tricks that are used for model compression in such cases. In most of the cases, a larger trained neural network is modified so that it requires less space by approximating some parts of the model. In addition, some efficiency improvements can also be realized at prediction time by model

compression because of better cache performance and fewer operations, although this is not the primary goal. Interestingly, this approximation might occasionally *improve* accuracy on out-of-sample predictions because of regularization effects, especially if the original model is unnecessarily large compared to the training data size.

### Pruning Network Weights

The links in a neural network are associated with weights. If the absolute value of a particular weight is small, then the model is not strongly influenced by that weight. Such weights can be dropped, and the neural network can be fine-tuned starting with the current weights on links that have not yet been dropped. By repeating the process of network pruning and fine-tuning iteratively, it is possible to improve compression effects. One can also encourage the dropping of links by using $L_1$-regularization, because it causes many learned weights to have zero values (see Chapter 5). However, $L_2$-regularization (with pruning of low weights) seems to be preferred because of higher accuracy [179]. An interesting recent result, referred to as the *lottery ticket hypothesis* [126], shows that instead of fine-tuning the learned weights, using the *initialization* weights from the previous round of training to retrain the network provides excellent results (while random initialization on the pruned network does poorly). The basic idea is that neural networks need to have massively redundant structures for achieving high accuracy and appropriate subnetworks are implicitly chosen by specific initializations. For modest levels of pruning, accuracy might *improve* because of regularization effects.

Further enhancements were reported in [178], where the approach was combined with Huffman coding and quantization for compression. The goal of quantization is to reduce the number of bits representing each connection. This approach reduced the storage required by *AlexNet* [263] by a factor of 35, or from about 240MB to 6.9MB, with no loss of accuracy. It is now possible as a result of this reduction to fit the model into an on-chip SRAM cache rather than off-chip DRAM memory; this also provide a beneficial effect on prediction times.

### Compressing Network Weights

It was shown in [96] that the vast majority of the weights in a neural network are redundant. In other words, for any $m \times n$ weight matrix $W$ between a pair of layers with $m_1$ and $m_2$ units respectively, one can express this weight matrix as $W \approx UV^T$, where $U$ and $V$ are of sizes $m_1 \times k$ and $m_2 \times k$, respectively. Furthermore, it is assumed that $k \ll \min\{m_1, m_2\}$. This phenomenon occurs because of several peculiarities in the training process. For example, the features and weights in a neural network tend to *co-adapt* because of different parts of the network training at different rates. Therefore, the faster parts of the network often adapt to the slower parts. As a result, there is a lot of redundancy in the network both in terms of the features and the weights, and the full expressivity of the network is never utilized. In such a case, one can replace the pair of layers (containing weight matrix $W$) with three layers of size $m_1$, $k$, and $m_2$. The weight matrices between the first pair of layers is $U$ and the weight matrix between the second pair of layers is $V^T$. Even though the new matrix is deeper, it is better regularized as long as $W - UV^T$ only contains noise. Furthermore, the matrices $U$ and $V$ require $(m_1 + m_2) \cdot k$ parameters, which is less than the number of parameters in $W$ as long as $k$ is less than half the harmonic mean of $m_1$ and $m_2$:

$$\frac{\text{Parameters in } W}{\text{Parameters in } U, V} = \frac{m_1 \cdot m_2}{k(m_1 + m_2)} = \frac{\text{HARMONIC-MEAN}(m_1, m_2)}{2k}$$

As shown in [96], more than 95% of the parameters in the neural network are redundant, and therefore a low value of the rank $k$ suffices for approximation.

One can also fine-tune $U$ and $V$ with back-propagation. An important point is that fine-tuning $U$ and $V$ must be done *after* completion of the learning of $W$ and initializing $U$ and $V$ so that $UV^T \approx W$. For example, if we replaced the pair of layers corresponding to $W$ with the three layers containing the two weight matrices of the same shape as $U$ and $V^T$ and trained from scratch, good results may not be obtained. This is because co-adaptation will occur again during training, and the resulting matrices $U$ and $V$ will have a rank even lower than $k$. As a result, under-fitting might occur. Matrix factorization can also be applied after weight pruning.

### Hash-Based Compression

One can reduce the number of parameters to be stored by forcing randomly chosen entries of the weight matrix to take on shared values of the parameters. The random choice is achieved with the application of a hash function on the entry position $(i, j)$ in the matrix. For example, imagine a situation where we have a weight matrix of size $100 \times 100$ with $10^4$ entries. In such a case, one can hash each weight to a value in the range $\{1, \ldots 1000\}$ to create 1000 groups. Each of these groups will contain an average of 10 connections that will share weights. Backpropagation can handle shared weights using the approach discussed in section 2.6.6. This approach requires a space requirement of only 1000 for the matrix, which is 10% of the original space requirement. Note that one could instead use a matrix of size $100 \times 10$ to achieve the same compression, but the key point is that using shared weights does not hurt the expressivity of the model as much as would reducing the size of the weight matrix *a priori*. More details of this approach are discussed in [68].

### Leveraging Mimic Models

Some interesting results in [14, 55] show that it is possible to significantly compress a model by creating a new training data set from a trained model, which is easier to model. This "easier" training data can be used to train a much smaller network without significant loss of accuracy. This smaller model is referred to as a *mimic model*. The following steps are used to create the mimic model:

1. A model is created on the original training data. This model might be very large and would not be appropriate to use in space-constrained settings. It is assumed that the model outputs softmax probabilities of the different classes. This model is also referred to as the teacher model.

2. New training data is created by passing unlabeled examples through the trained network. The targets in the newly created training data are set to the softmax probability outputs of the trained model on the unlabeled examples. Since unlabeled data is often copious, it is possible to create a lot of training data in this way. The new training data contains soft (probabilistic) targets rather than discrete labels, the crucial effects of which will be discussed later.

3. A much smaller and shallower network, referred to as the mimic or *student* model, is trained using this synthesized training data. This model can be easily deployed in space-constrained settings. The accuracy of the mimic model often does not substantially degrade with respect to the original neural model.

A remarkable observation is that a shallow neural model might perform poorly with training on the original training data, whereas the mimic model of similar size performs very well. A number of possible reasons have been hypothesized for this phenomenon [14]:

1. Mislabeled examples in the original training data cause unnecessary complexity in the trained model. The new training data will often contain soft probabilities "disagreeing" with the original labels.

2. If there are complex regions of the decision space, the teacher model has the sophistication needed to simplify them in the most accurate way. Simplifying complexity selectively improves the performance of shallow models.

3. The original targets might depend on inputs that are not available in the training data. On the other hand, the teacher-created labels depend only on the available inputs. This makes the model simpler to learn and washes away unexplained complexity.

4. The original training data contains targets with 0/1 values, whereas the newly created training contains (more informative) soft targets. This is particularly useful in one-hot encoded multilabel targets with correlations across classes.

One can view some of the above benefits as a kind of regularization effect. The results in [14] are stimulating, because they show that deep networks are not *theoretically* necessary, although the regularization effect of depth is practically necessary when working with the original training data. The mimic model enjoys the benefits of this regularization effect by using the artificially created targets instead of depth.

## 4.10 Summary

This chapter discusses the problems of training deep neural networks. The vanishing and the exploding gradient problems are introduced along with the challenges associated with varying sensitivity of the loss function to different optimization variables. Certain types of activation functions like ReLU are less sensitive to this problem. However, the use of the ReLU can sometimes lead to dead neurons, if one is not careful about the learning rate. The type of gradient descent used to accelerate learning is also important for more efficient executions. Modified stochastic gradient-descent methods include the use of Nesterov momentum, AdaGrad, AdaDelta, RMSProp, and Adam. All these methods encourage gradient-steps that accelerate the learning process.

Numerous methods have been introduced for addressing the problem of cliffs with the use of second-order optimization methods. In particular, Hessian-free optimization is seen as an effective approach for handling many of the underlying optimization issues. An exciting method that has been used recently to improve learning rates is the use of batch normalization. Batch normalization transforms the data layer by layer in order to ensure that the scaling of different variables is done in an optimum way. The use of batch normalization has become extremely common in different types of deep networks. Numerous methods have been proposed for accelerating and compressing neural network algorithms. Acceleration is often achieved via hardware improvements, whereas compression is achieved with algorithmic tricks.

## 4.11 Bibliographic Notes and Software Resources

Nesterov's algorithm for gradient descent may be found in [364]. The delta-bar-delta method was proposed by [228]. The AdaGrad algorithm was proposed in [110]. The RMSProp algo-

rithm is discussed in [204]. Another adaptive algorithm using stochastic gradient descent, which is *AdaDelta*, is discussed in [578]. This algorithms shares some similarities with second-order methods, and in particular to the method in [447]. The Adam algorithm, which is a further enhancement along this line of ideas, is discussed in [251]. The practical importance of initialization and momentum in deep learning is discussed in [494]. Beyond the use of the stochastic gradient method, the use of coordinate descent has been proposed [279]. The strategy of *Polyak averaging* is discussed in [394].

Several of the challenges associated with the vanishing and exploding gradient problems are discussed in [147, 215, 381]. Ideas for parameter initialization that avoid some of these problems are discussed in [147]. The gradient clipping rule was discussed by Mikolov in his PhD thesis [336]. A discussion of the gradient clipping method in the context of recurrent neural networks is provided in [381]. The ReLU activation function was introduced in [174], and several of its interesting properties are explored in [148, 232].

A description of several second-order gradient optimization methods (such as the Newton method) is provided in [41, 309, 570]. The basic principles of the conjugate gradient method have been described in several classical books and papers [41, 199, 463], and the work in [324, 325] discusses applications to neural networks. The work in [327] leverages a Kronecker-factored curvature matrix for fast gradient descent. Another way of approximating the Newton method is the quasi-Newton method [279, 309], with the simplest approximation being a diagonal Hessian [25]. The acronym BFGS stands for the Broyden-Fletcher-Goldfarb-Shanno algorithm. A variant known as limited memory BFGS or L-BFGS [279, 309] does not require as much memory. Another popular second-order method is the Levenberg–Marquardt algorithm. This approach is, however, defined for squared loss functions and cannot be used with many forms of cross-entropy or log-losses that are common in neural networks. Overviews of the approach may be found in [139, 309]. General discussions of different types of nonlinear programming methods are provided in [24, 39].

The stability of neural networks to local minima is discussed in [91, 443]. Batch normalization methods were introduced recently in [225]. A method that uses whitening for batch normalization is discussed in [98], although the approach seems not to be practical. Batch normalization requires some minor adjustments for recurrent networks [84], although a more effective approach for recurrent networks is that of *layer normalization* [15]. In this method (cf. section 8.3.1), a single training case is used for normalizing all units in a layer, rather than using mini-batch normalization of a single unit. The approach is useful for recurrent networks. An analogous notion to batch normalization is that of weight normalization [436], in which the magnitudes and directions of the weight vectors are decoupled during the learning process. Related training tricks are discussed in [373].

A broader discussion of accelerating machine learning algorithms with GPUs may be found in [665]. Various types of parallelization tricks for GPUs are discussed in [77, 94, 262], and specific discussions on convolutional neural networks are provided in [563]. Model compression with regularization is discussed in [178, 179]. A related model compression method is proposed in [224]. The use of mimic models for compression is discussed in [14, 55]. A related approach is discussed in [212]. The leveraging of parameter redundancy for compressing neural networks is discussed in [96]. The compression of neural networks with the hashing trick is discussed in [68].

## Software Resources

All the training algorithms discussed in this chapter, including batch normalization options, are supported by numerous deep learning frameworks like *Caffe* [596], *Torch* [597],

*Theano* [598], and *TensorFlow* [599]. Several software libraries are available for Bhyperparameter optimization, such as *Hyperopt* [637], *Spearmint* [639], and *SMAC* [638]. Pointers to the NVIDIA cuDNN and its supported deep learning frameworks discussed in [664, 666].

## 4.12 Exercises

1. Some networks, such as recurrent neural networks, use shared weights across multiple layers. Discuss why vanishing and exploding gradients are more likely in this case.

2. Consider a 2-feature linear regression problem in which $x_1$ always lies in the range $[8, 9]$ and $x_2$ always lies in the range $[8000, 9000]$. Comment on why gradient descent is likely to perform more efficiently after feature normalization by inspecting the objective function over a toy training set with three examples.

3. Suppose that you have the quadratic function $f(x) = ax^2 + bx + c$ with optimum value st $x = -b/2a$. Show that a single Newton step starting at any point $x = x_0$ will always lead to $x = -b/2a$ irrespective of the value of $x_0$. How does this result show that a Newton update to reach a maximum rather than a minimum?

4. Consider the objective function $f(x) = [x(x-2)]^2 + x^2$. Write its Newton update starting at $x = 1$. Discuss why line search is important in this case.

5. The Hessian $H$ of a strongly convex quadratic function always satisfies $\overline{x}^T H \overline{x} > 0$ for any nonzero vector $\overline{x}$. For such problems, show that all conjugate directions are linearly independent.

6. Show that if the dot product of a $d$-dimensional vector $\overline{v}$ with $d$ linearly independent vectors is 0, then $\overline{v}$ must be the zero vector.

7. The chapter uses steepest descent directions to iteratively generate conjugate directions. Suppose we pick $d$ *arbitrary* directions $\overline{v}_0 \ldots \overline{v}_{d-1}$ that are linearly independent. Show that (with appropriate choice of $\beta_{ti}$) we can start with $\overline{q}_0 = \overline{v}_0$ and generate successive conjugate directions in the following form:

$$\overline{q}_{t+1} = \overline{v}_{t+1} + \sum_{i=0}^{t} \beta_{ti} \overline{q}_i$$

   Discuss why this approach is more expensive than the one discussed in the chapter.

8. The definition of $\beta_t$ in section 4.7.1 ensures that $\overline{q}_t$ is conjugate to $\overline{q}_{t+1}$. This exercise systematically shows that *any* direction $\overline{q}_i$ for $i \leq t$ satisfies $\overline{q}_i^T H \overline{q}_{t+1} = 0$.
   [Hint: Prove (b), (c), and (d) *jointly* with induction on $t$ while staring at (a).]

   (a) Recall from Equation 4.17 that $H\overline{q}_i = [\nabla L(\overline{W}_{i+1}) - \nabla L(\overline{W}_i)]/\delta_i$ for quadratic loss functions, where $\delta_i$ depends on $i$th step-size. Combine this condition with Equation 4.15 to show the following for all $i \leq t$:

$$\delta_i [\overline{q}_i^T H \overline{q}_{t+1}] = -[\nabla L(\overline{W}_{i+1}) - \nabla L(\overline{W}_i)]^T [\nabla L(\overline{W}_{t+1})] + \delta_i \beta_t (\overline{q}_i^T H \overline{q}_t)$$

   Also show that $[\nabla L(\overline{W}_{t+1}) - \nabla L(\overline{W}_t)] \cdot \overline{q}_i = \delta_t \overline{q}_i^T H \overline{q}_t$.

(b) Show that $\nabla L(\overline{W}_{t+1})$ is orthogonal to each $\overline{q}_i$ for $i \leq t$. [The proof for the case when $i = t$ is trivial because the gradient at line-search termination is always orthogonal to the search direction.]

(c) Show that the loss gradients at $\overline{W}_0 \ldots \overline{W}_{t+1}$ are mutually orthogonal.

(d) Show that $\overline{q}_i^T H \overline{q}_{t+1} = 0$ for $i \leq t$. [The case for $i = t$ is trivial.]

**9.** Revisit Exercise 11 of Chapter 2 and discuss why the tanh activation is less likely to cause gradient descent to zigzag than sigmoid activation. Also discuss why the vanishing gradient is less likely with tanh activation.

**10.** Weight pruning, matrix factorization, and mimic models are all compression techniques. If you had to use them in combination, then in what order would you apply them and why?


# Chapter 5

# Teaching Deep Learners to Generalize

"All generalizations are dangerous, even this one."–Alexandre Dumas

## 5.1 Introduction

Neural networks are powerful learners that have repeatedly proven to be capable of learning complex functions in many domains. However, the great power of neural networks is also their greatest weakness; neural networks often simply overfit the training data if care is not taken to design the learning process carefully. In practical terms, what overfitting means is that a neural network will provide excellent prediction performance on the training data that it is built on, but will perform poorly on unseen test instances. This is caused by the fact that the learning process often remembers random artifacts of the training data that do not generalize well to the test data. Extreme forms of overfitting are referred to as *memorization*. A helpful analogy is to think of a child who can solve all the analytical problems for which he or she has seen the solutions, but is unable to provide useful solutions to a new problem. However, if the child is exposed to the solutions of more and more different types of problems, he or she will be more likely to solve a new problem by abstracting out the essence of the patterns that are repeated across different problems and their solutions. Machine learning proceeds in a similar way by identifying patterns that are useful for prediction. For example, in a spam detection application, if the pattern "*Free Money!!*" occurs thousands of times in spam emails, the machine learner generalizes this rule to identify spam email instances it has not seen before. On the other hand, a prediction that is based on the patterns seen in a tiny training data set of two emails will lead to good performance on the two emails present in the training data but (usually) not on new emails. The ability of a learner to provide useful predictions for instances it has not seen before is referred to as *generalization*.

Generalization is a useful practical property, and is therefore the holy grail in all machine learning applications. After all, if the training examples are already labeled, there is no practical use of predicting such examples again. For example, in an image-captioning



C. C. Aggarwal, *Neural Networks and Deep Learning*,
https://doi.org/10.1007/978-3-031-29642-0_5

application, one is always looking to use the labeled images in order to learn captions for images that the learner has not seen before.

### 5.1.1 Example: Linear Regression

In order to understand the problem of generalization, consider a simple single-layer neural network on a data set with five attributes, where we use the identity activation to learn a real-valued target variable. This architecture is almost identical to that of Figure 1.3, except that the identity activation function is used in order to predict a real-valued target. Therefore, the network tries to learn the following function:

$$\hat{y} = \sum_{i=1}^{5} w_i \cdot x_i \tag{5.1}$$

Consider a situation in which the observed target value is real and is always twice the value of the first attribute, whereas other attributes are completely unrelated to the target. However, we have only four training instances, which is one less than the number of features (free parameters). For example, the training instances could be as follows:

| $x_1$ | $x_2$ | $x_3$ | $x_4$ | $x_5$ | $y$ |
|---|---|---|---|---|---|
| 1 | 1 | 0 | 0 | 0 | 2 |
| 2 | 0 | 1 | 0 | 0 | 4 |
| 3 | 0 | 0 | 1 | 0 | 6 |
| 4 | 0 | 0 | 0 | 1 | 8 |

The correct parameter vector in this case is $\overline{W} = [2, 0, 0, 0, 0]$ based on the known relationship between the first feature and target. The training data also provides zero error with this solution, although the relationship needs to be *learned* from the given instances since it is not given to us a priori. However, the problem is that the number of training points is fewer than the number of parameters and it is possible to find an infinite number of solutions with zero error. For example, the parameter set $[0, 2, 4, 6, 8]$ also provides zero error *on the training data*. However, if we used this solution on unseen test data, it is likely to provide very poor performance because the learned parameters are spuriously inferred and are unlikely to *generalize* well to new points in which the target is twice the first attribute (and other attributes are random). This type of spurious inference is caused by the paucity of training data, where random nuances are encoded into the model. As a result, the solution does not generalize well to unseen test data. This situation is almost similar to learning by rote, which is highly predictive for training data but not predictive for unseen test data. Increasing the number of training instances improves the generalization power of the model, whereas increasing the complexity of the model reduces its generalization power. At the same time, when a lot of training data is available, an overly simple model is unlikely to capture complex relationships between the features and target. A good rule of thumb is that the total number of training data points should be at least 2 to 3 times larger than the number of parameters in the neural network, although the precise number of data instances depends on the specific model at hand. In general, models with a larger number of parameters are said to have *high capacity*, and they require a larger amount of data in order to gain generalization power to unseen test data.

For example, if we change the training data in the table above to a different set of four points, we are likely to learn a completely different set of parameters (from the random

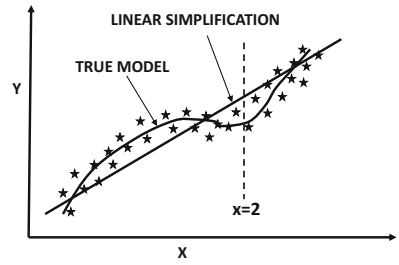



Figure 5.1: An example of a nonlinear distribution in which one would expect a model with $d = 3$ to work better than a linear model with $d = 1$.

nuances of those points). This new model is likely to yield a completely different prediction *on the same test instance* as compared to the predictions using the first training data set. Therefore, models that overfit the training data are said of have high *variance*.

### 5.1.2 Example: Polynomial Regression

Imagine a situation in which we attempt to predict the variable $y$ from $x$ using the following formula for polynomial regression:

$$\hat{y} = \sum_{i=0}^{d} w_i x^i \tag{5.2}$$

This is a model that uses $(d + 1)$ parameters $w_0 \ldots w_d$ in order to explain pairs $(x, y)$ available to us. One could implement this model by using a neural network with $d$ inputs corresponding to $x$, $x^2 \ldots x^d$, and a single bias neuron whose coefficient is $w_0$. The loss function uses the squared difference between the observed value $y$ and predicted value $\hat{y}$. In general, larger values of $d$ can capture better nonlinearity. For example, in the case of Figure 5.1, a nonlinear model with $d = 4$ should be able to fit the data better than a linear model with $d = 1$, *given an infinite amount (or a lot) of data*. However, when working with a small, finite data set, this does not always turn out to be the case.

If we have $(d + 1)$ or less training pairs $(x, y)$, it is possible to fit the data exactly with zero error *irrespective of how poorly these training pairs reflect the true distribution*. For example, consider a situation in which we have five particularly bad (unrepresentative) training points available. One can show that it is possible to fit the training points exactly with zero error using a polynomial of degree 4. This does not, however, mean that zero error will be achieved on unseen test data. An example of this situation is illustrated in Figure 5.2, where both the linear and polynomial models on three sets of five randomly chosen data points are shown. It is clear that the linear model is stable, although it is unable to exactly model the curved nature of the true data distribution. On the other hand, even though the polynomial model is capable of modeling the true data distribution more closely, it varies wildly over the different training data sets. Therefore, the same test instance at $x = 2$ (shown in Figure 5.2) would receive similar predictions from the linear model, but would receive very different predictions from the polynomial model over different choices of training data sets. The behavior of the polynomial model is, of course, undesirable from a practitioner's point of view, who would expect similar predictions for a particular test instance, even when different samples of the training data set are used. Since all the different predictions of the polynomial model cannot be correct, it is evident that the increased power of the

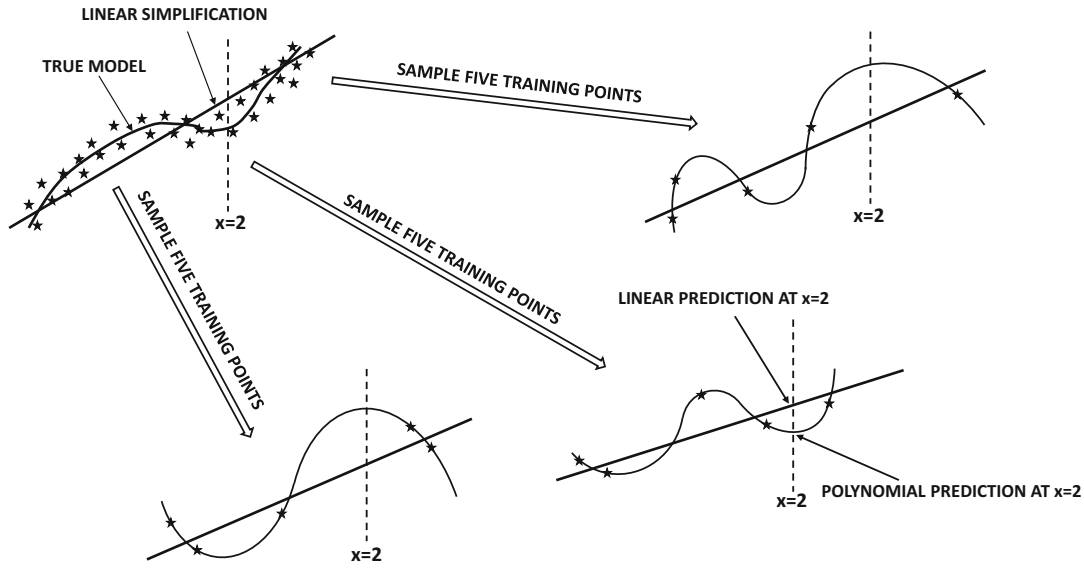



Figure 5.2: **Overfitting with increased model complexity:** The linear model does not change much with the training data, whereas the polynomial model changes drastically. As a result, the inconsistent predictions of the polynomial model at $x = 2$ are often more inaccurate than those of the linear model. The polynomial model does have the ability to outperform the linear model *if enough training data is provided*.

polynomial model over the linear model actually increases the error rather than reducing it. This difference in predictions for the same test instance (but different training data sets) is manifested as the *variance* of a model. As evident from Figure 5.2, models with high variance tend to memorize random artifacts of the training data, causing inconsistency and inaccuracy in the prediction of unseen test instances. It is noteworthy that a polynomial model with higher degree is inherently more powerful than a linear model because the higher-order coefficients could always be set to 0; however, it is unable to achieve its full potential when the amount of data is limited. Simply speaking, the variance inherent in the finiteness of the data set causes increased complexity to be counterproductive. This trade-off between the power of a model and its performance on limited data is captured with the *bias-variance trade-off*.

There are several tell-tale signs of overfitting:

1. When a model is trained on different data sets, the same test instance might obtain very different predictions. This is a sign that the training process is memorizing the nuances of the specific training data set, rather than learning patterns that generalize to unseen test instances. Note that the three predictions at $x = 2$ in Figure 5.2 are quite different for the polynomial model. This is not quite the case for the linear model.

2. The gap between the error of predicting training instances and unseen test instances is rather large. Note that in Figure 5.2, the predictions at the unseen test point $x = 2$ are often more inaccurate in the polynomial model than in the linear model. On the other hand, the training error is always zero for the polynomial model, whereas the training error is usually nonzero for the linear model.

Because of the large gaps between training and test error, models are often tested on unseen portions of the training data. These unseen portions of the training data are often held out early on, and then used in order to make different types of algorithmic decisions such as parameter tuning. This set of points is referred to as the *validation set*. The final accuracy is tested on a fully out-of-sample set of points that was not used for either model building or for parameter tuning. The error on out-of-sample test data is also referred to as the *generalization error*.

Neural networks are large, and they might have millions of parameters in complex applications. In spite of these challenges, there are a number of tricks that one can use in order to ensure that overfitting is not a problem. The key methods for avoiding overfitting in a neural network are as follows:

1. *Penalty-based regularization:* Penalty-based regularization is the most common technique used by neural networks in order to avoid overfitting. The idea in regularization is to create a penalty or other types of constraints on the parameters in order to favor simpler models. For example, in the case of polynomial regression, a possible constraint on the parameters would be to ensure that at most $k$ different values of $w_i$ are non-zero. This will ensure simpler models. However, since it is hard to impose such constraints explicitly, a simpler approach is to impose a softer penalty like $\lambda \sum_{i=0}^{d} w_i^2$ and add it to the loss function. Such an approach roughly amounts to multiplying each parameter $w_i$ with a multiplicative decay factor of $(1 - \alpha\lambda)$ during each update at learning rate $\alpha$.

2. *Generic and tailored ensemble methods:* Many ensemble methods are not specific to neural networks. We will discuss bagging and subsampling, which are two of the simplest ensemble methods that can be implemented for virtually any model or learning problem. There are also several ensemble methods that are specifically designed for neural networks. A straightforward approach is to average the predictions of different neural architectures obtained by quick and dirty hyper-parameter optimization. *Dropout* is another ensemble technique that is designed for neural networks. This chapter will discuss some of these methods.

3. *Early stopping:* In early stopping, the iterative optimization method is terminated early without converging to the optimal solution on the training data. The stopping point is determined using a portion of the training data that is not used for model building. One terminates when the error on the held-out data begins to rise. Even though this approach is not optimal for the training data, it seems to perform well on the test data because the stopping point is determined using held-out data.

4. *Pretraining:* Pretraining is a form of learning in which a greedy algorithm is used to find a good initialization. The weights in different layers of the neural network are trained sequentially in greedy fashion. These trained weights are used as a good starting point for the overall process of learning. Pretraining can be shown to be an indirect form of regularization.

5. *Continuation and curriculum methods:* These methods perform more effectively by first training simple models, and then making them more complex. The idea is that it is easy to train simpler models without overfitting. Furthermore, starting with the optimum point of the simpler model provides a good initialization for a complex model that is closely related to the simpler model. It is noteworthy that some of these methods can be considered similar to pretraining, which transforms solutions from

the simple to the complex by decomposing a deep neural network into a set of shallow layers for training.

6. *Sharing parameters with domain-specific insights:* In some data-domains like text and images, one often has some insight about the structure of the parameter space. In such cases, some of the parameters in different parts of the network can be set to the same value. This reduces the number of degrees of freedom of the model. Such an approach is used in recurrent neural networks (for sequence data) and convolutional neural networks (for image data).

This chapter will first discuss the issue of model generalization in a generic way by introducing some theoretical results associated with the bias-variance trade-off. Subsequently, the different ways of reducing overfitting will be discussed.

An interesting observation is that several forms of regularization can be shown to be roughly equivalent to the injection of noise in either the input data or the hidden variables. For example, it can be shown that many penalty-based regularizers are equivalent to the addition of noise [43]. Furthermore, even the use of *stochastic* gradient descent instead of gradient descent can be viewed as a kind of noise addition to the steps of the algorithm. As a result, stochastic gradient descent often shows good accuracy on the test data, even though its performance on the training data might not be as good as that of gradient descent. Furthermore, some ensemble techniques like *Dropout* and data perturbation are equivalent to injecting noise. Throughout this chapter, the similarities between noise injection and regularization will be discussed where needed.

Even though a natural way of avoiding overfitting is to simply build smaller networks (with fewer units and parameters), it has often been observed that it is better to build large networks and then regularize them in order to avoid overfitting. This is because large networks retain the *option* of building a more complex model if it is truly warranted. At the same time, the regularization process can smooth out the random artifacts that are not supported by sufficient data. By using this approach, we are giving the model the choice to decide what complexity it needs, rather than making a rigid decision for the model up front (which might even underfit the data).

Supervised settings tend to be more prone to overfitting than unsupervised settings, and supervised problems are therefore the main focus of the literature on generalization. To understand this point, consider that a supervised application tries to learn a single target variable and might have hundreds of input (explanatory) variables. It is easy to overfit the process of learning a very focused goal because a limited degree of supervision (e.g., binary label) is available for each training example. On the other hand, an unsupervised application has the same number of target variables as the explanatory variables. In the latter case, overfitting is less likely because a single training example has a larger number of bits of information. Nevertheless, regularization is still used in unsupervised applications, especially when the intent is to impose a desired structure on the learned representations.

### Chapter Organization

This chapter is organized as follows. The next section introduces the bias-variance trade-off. The practical implications of the bias-variance trade-off for model training are discussed in section 5.3. The use of penalty-based regularization to reduce overfitting is presented in section 5.4. Ensemble methods are introduced in section 5.5. Early stopping methods are discussed in section 5.6. Methods for unsupervised pretraining are discussed in section 5.7.

Continuation and curriculum learning methods are presented in section 5.8. Parameter sharing methods are discussed in section 5.9. Unsupervised forms of regularization are discussed in section 5.10. A summary is given in section 5.11.

## 5.2 The Bias-Variance Trade-Off

The introduction section provides an example of how a polynomial model fits a smaller training data set, leading to the predictions on unseen test data being more erroneous than are the predictions of a (simpler) linear model. This is because a polynomial model requires more data in order to not be misled by random artifacts of the training data set. The fact that more powerful models do not always win in terms of prediction accuracy with a finite data set is the key take-away from the bias-variance trade-off.

The bias-variance trade-off states that the squared error of a learning algorithm can be partitioned into three components:

1. *Bias:* The bias is the error caused by the simplifying assumptions in the model, which causes certain test instances to have consistent errors across different choices of training data sets. Even if the model has access to an infinite source of training data, the bias cannot be removed. For example, in the case of Figure 5.2, the linear model has a higher model bias than the polynomial model, because it can never fit the (slightly curved) data distribution exactly, no matter how much data is available. The prediction of a particular out-of-sample test instance at $x = 2$ will always have an error in a particular direction when using a linear model for any choice of training sample. If we assume that the linear and curved lines in the top left of Figure 5.2 were estimated using an infinite amount of data, then the difference between the two at any particular values of $x$ is the bias. An example of the bias at $x = 2$ is shown in Figure 5.2.

2. *Variance:* Variance is caused by the inability to learn all the parameters of the model in a statistically robust way, especially when the data is limited and the model tends to have a larger number of parameters. The presence of higher variance is manifested by overfitting to the specific training data set at hand. Therefore, if different choices of training data sets are used, different predictions will be provided for the same test instance. Note that the linear prediction provides similar predictions at $x = 2$ in Figure 5.2, whereas the predictions of the polynomial model vary widely over different choices of training instances. In many cases, the widely inconsistent predictions at $x = 2$ are wildly incorrect predictions, which is a manifestation of model variance. Therefore, the polynomial predictor has a higher variance than the linear predictor in Figure 5.2.

3. *Noise:* The noise is caused by the inherent error in the data. For example, all data points in the scatter plot vary from the true model in the upper-left corner of Figure 5.2. If there had been no noise, all points in the scatter plot would overlap with the curved line representing the true model.

The above description provides a qualitative view of the bias-variance trade-off. In the following, we will provide a more formal and mathematical view.

## Formal View

We assume that the base distribution from which the training data set is generated is denoted by $\mathcal{B}$. One can generate a data set $\mathcal{D}$ from this base distribution:

$$\mathcal{D} \sim \mathcal{B} \tag{5.3}$$

One could draw the training data in many different ways, such as selecting only data sets of a particular size. For now, assume that we have some well-defined generative process according to which training data sets are drawn from $\mathcal{B}$. The analysis below does not rely on the specific mechanism with which training data sets are drawn from $\mathcal{B}$.

Access to the base distribution $\mathcal{B}$ is equivalent to having access to an infinite resource of training data, because one can use the base distribution an unlimited number of times to generate training data sets. In practice, such base distributions (i.e., infinite resources of data) are not available. As a practical matter, an analyst uses some data collection mechanism to collect only *one finite instance* of $\mathcal{D}$. However, the conceptual existence of a base distribution from which other training data sets can be generated is useful in theoretically quantifying the sources of error in training on this finite data set.

Now imagine that the analyst had a set of $t$ test instances in $d$ dimensions, denoted by $\overline{Z}_1 \ldots \overline{Z}_t$. The dependent variables of these test instances are denoted by $y_1 \ldots y_t$. For clarity in discussion, let us assume that the test instances and their dependent variables were also generated from the same base distribution $\mathcal{B}$ by a third party, but the analyst was provided access only to the feature representations $\overline{Z}_1 \ldots \overline{Z}_t$, and no access to the dependent variables $y_1 \ldots y_t$. Therefore, the analyst is tasked with job of using the single finite instance of the training data set $\mathcal{D}$ in order to predict the dependent variables of $\overline{Z}_1 \ldots \overline{Z}_t$.

Now assume that the relationship between the dependent variable $y_i$ and its feature representation $\overline{Z}_i$ is defined by the *unknown* function $f(\cdot)$ as follows:

$$y_i = f(\overline{Z}_i) + \epsilon_i \tag{5.4}$$

Here, the notation $\epsilon_i$ denotes the intrinsic noise, which is independent of the model being used. The value of $\epsilon_i$ might be positive or negative, although it is assumed that $E[\epsilon_i] = 0$. If the analyst knew what the function $f(\cdot)$ corresponding to this relationship was, then they could simply apply the function to each test point $\overline{Z}_i$ in order to approximate the dependent variable $y_i$, with the only remaining uncertainty being caused by the intrinsic noise.

The problem is that the analyst does not know what the function $f(\cdot)$ is in practice. Note that this function is used within the generative process of the base distribution $\mathcal{B}$, and the entire generating process is like an oracle that is unavailable to the analyst. The analyst only has examples of the input and output of this function. Clearly, the analyst would need to develop some type of *model* $g(\overline{Z}_i, \mathcal{D})$ using the training data in order to *approximate* this function in a data-driven way.

$$\hat{y}_i = g(\overline{Z}_i, \mathcal{D}) \tag{5.5}$$

Note the use of the circumflex (i.e., the symbol '^') on the variable $\hat{y}_i$ to indicate that it is a *predicted* value by a specific algorithm rather than the observed (true) value of $y_i$.

All prediction functions of learning models (including neural networks) are examples of the estimated function $g(\cdot, \cdot)$. Some algorithms (such as linear regression and perceptrons)

can even be expressed in a concise and understandable way:

$$g(\overline{Z}_i, \mathcal{D}) = \underbrace{\overline{W} \cdot \overline{Z}_i}_{\text{Learn } \overline{W} \text{ with } \mathcal{D}} \quad \text{[Linear Regression]}$$

$$g(\overline{Z}_i, \mathcal{D}) = \underbrace{\text{sign}\{\overline{W} \cdot \overline{Z}_i\}}_{\text{Learn } \overline{W} \text{ with } \mathcal{D}} \quad \text{[Perceptron]}$$

Most neural networks are expressed algorithmically as compositions of multiple functions computed at different nodes. The choice of computational function includes the effect of its specific parameter setting, such as the coefficient vector $\overline{W}$ in a perceptron. Neural networks with a larger number of units will require more parameters to fully learn the function. This is where the variance in predictions arises on the same test instance; a model with a large parameter set $\overline{W}$ will learn very different values of these parameters, when a different choice of the training data set is used. Consequently, the prediction of the same test instance will also be very different for different training data sets. These inconsistencies add to the error, as illustrated in Figure 5.2.

The goal of the bias-variance trade-off is to quantify the expected error of the learning algorithm in terms of its bias, variance, and the (data-specific) noise. For generality in discussion, we assume a numeric form of the target variable, so that the error can be intuitively quantified by the *mean-squared error* between the predicted values $\hat{y}_i$ and the observed values $y_i$. This is a natural form of error quantification in regression, although one can also use it in classification in terms of probabilistic predictions of test instances. The mean squared error, $MSE$, of the learning algorithm $g(\cdot, \mathcal{D})$ is defined over the set of test instances $\overline{Z}_1 \ldots \overline{Z}_t$ as follows:

$$MSE = \frac{1}{t} \sum_{i=1}^{t} (\hat{y}_i - y_i)^2 = \frac{1}{t} \sum_{i=1}^{t} (g(\overline{Z}_i, \mathcal{D}) - f(\overline{Z}_i) - \epsilon_i)^2$$

The best way to estimate the error in a way that is independent of the specific choice of training data set is to compute the *expected* error over different choices of training data sets:

$$\begin{aligned} E[MSE] &= \frac{1}{t} \sum_{i=1}^{t} E[(g(\overline{Z}_i, \mathcal{D}) - f(\overline{Z}_i) - \epsilon_i)^2] \\ &= \frac{1}{t} \sum_{i=1}^{t} E[(g(\overline{Z}_i, \mathcal{D}) - f(\overline{Z}_i))]^2 + \frac{\sum_{i=1}^{t} E[\epsilon_i^2]}{t} \end{aligned}$$

The second relationship is obtained by expanding the quadratic expression on the right-hand side of the first equation, and then using the fact that the average value of $\epsilon_i$ over a large number of test instances is 0.

The right-hand side of the above expression can be further decomposed by adding and subtracting $E[g(\overline{Z}_i, \mathcal{D})]$ within the squared term on the right-hand side:

$$E[MSE] = \frac{1}{t} \sum_{i=1}^{t} E[\{(f(\overline{Z}_i) - E[g(\overline{Z}_i, \mathcal{D})]) + (E[g(\overline{Z}_i, \mathcal{D})] - g(\overline{Z}_i, \mathcal{D}))\}^2] + \frac{\sum_{i=1}^{t} E[\epsilon_i^2]}{t}$$

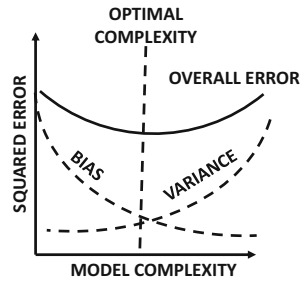



Figure 5.3: The trade-off between bias and variance usually causes a point of optimal model complexity.

One can expand the quadratic polynomial on the right-hand side to obtain the following:

$$\begin{aligned} E[MSE] =& \frac{1}{t}\sum_{i=1}^{t} E[\{f(\overline{Z}_i) - E[g(\overline{Z}_i, \mathcal{D})]\}^2] \\ &+ \frac{2}{t}\sum_{i=1}^{t} \{f(\overline{Z}_i) - E[g(\overline{Z}_i, \mathcal{D})]\}\{E[g(\overline{Z}_i, \mathcal{D})] - E[g(\overline{Z}_i, \mathcal{D})]\} \\ &+ \frac{1}{t}\sum_{i=1}^{t} E[\{E[g(\overline{Z}_i, \mathcal{D})] - g(\overline{Z}_i, \mathcal{D})\}^2] + \frac{\sum_{i=1}^{t} E[\epsilon_i^2]}{t} \end{aligned}$$

The second term on the right-hand side of the aforementioned expression evaluates to 0 because one of the multiplicative factors is $E[g(\overline{Z}_i, \mathcal{D})] - E[g(\overline{Z}_i, \mathcal{D})]$. On simplification, we obtain the following:

$$E[MSE] = \underbrace{\frac{1}{t}\sum_{i=1}^{t} \{f(\overline{Z}_i) - E[g(\overline{Z}_i, \mathcal{D})]\}^2}_{\text{Bias}^2} + \underbrace{\frac{1}{t}\sum_{i=1}^{t} E[\{g(\overline{Z}_i, \mathcal{D}) - E[g(\overline{Z}_i, \mathcal{D})]\}^2]}_{\text{Variance}} + \underbrace{\frac{\sum_{i=1}^{t} E[\epsilon_i^2]}{t}}_{\text{Noise}}$$

In other words, the squared error can be decomposed into the (squared) bias, variance, and noise. The variance is the key term that prevents neural networks from generalizing. In general, the variance will be higher for neural networks that have a large number of parameters. On the other hand, too few model parameters can cause bias because there are not sufficient degrees of freedom to model the complexities of the data distribution. This trade-off between bias and variance with increasing model complexity is illustrated in Figure 5.3. Clearly, there is a point of optimal model complexity where the performance is optimized. Furthermore, paucity of training data will increase variance. However, careful choice of design can reduce overfitting. This chapter will discuss several such choices.

## 5.3 Generalization Issues in Model Tuning and Evaluation

There are several practical issues in the training of neural network models that one must be careful of because of the bias-variance trade-off. The first of these issues is associated with model tuning and hyperparameter choice. For example, if one tuned the neural network with

the same data that were used to train it, one would not obtain very good results because of overfitting. Therefore, the hyperparameters (e.g., regularization parameter) are tuned on a separate held-out set than the one on which the weight parameters on the neural network are learned.

Given a labeled data set, one needs to use this resource for training, tuning, and testing the accuracy of the model. Clearly, one cannot use the entire resource of labeled data for model building (i.e., learning the weight parameters). For example, using the same data set for both model building and testing grossly overestimates the accuracy. This is because the main goal of classification is to *generalize* a model of labeled data to unseen test instances. Furthermore, the portion of the data set used for *model selection* and *parameter tuning* also needs to be different from that used for model building. A common mistake is to use the same data set for both parameter tuning and final evaluation (testing). Such an approach partially mixes the training and test data, and the resulting accuracy is overly optimistic. A given data set should always be divided into three parts defined according to the way in which the data are used:

1. *Training data:* This part of the data is used to build the training model (i.e., during the process of learning the weights of the neural network). Multiple models may be constructed using different hyperparameters and architectures. This process sets the stage for *model selection*, in which the best algorithm is selected out of these different models. However, the actual *evaluation* of these algorithms for selecting the best model is not done on the training data, but on a separate validation data set to avoid favoring overfitted models.

2. *Validation data:* This part of the data is used for model selection and parameter tuning. The accuracy of various models constructed on the training data is tested using the validation set. As discussed in section 2.7.1 of Chapter 2, different combinations of parameters are sampled within a range and tested for accuracy on the validation set. The best combination of parameters is determined by using this accuracy. In a sense, validation data should be viewed as a kind of test data set to tune the parameters of the algorithm (e.g., learning rate, number of layers or units in each layer), or to select the best design choice (e.g., sigmoid versus tanh activation).

3. *Testing data:* This part of the data is used to test the accuracy of the final (tuned) model. It is important that the testing data are not even looked at during the process of parameter tuning and model selection to prevent overfitting. The testing data are *used only once at the very end of the process.* Furthermore, if the analyst uses the results on the test data to adjust the model in some way, then the results will be contaminated with knowledge from the testing data. The idea that one is allowed to look at a test data set only once is an extraordinarily strict requirement (and an important one). Yet, it is frequently violated in real-life benchmarks. The temptation to use what one has learned from the final accuracy evaluation is simply too high.

The division of the labeled data set into training data, validation data, and test data is shown in Figure 5.4. Strictly speaking, the validation data is also a part of the training data, because it influences the final model (although only the model building portion is often referred to as the training data). The division in the ratio of 2:1:1 is a conventional rule of thumb that has been followed since the nineties. However, it should not be viewed as a strict rule. For very large labeled data sets, one needs only a modest number of examples to estimate accuracy. When a very large data set is available, it makes sense to use as much

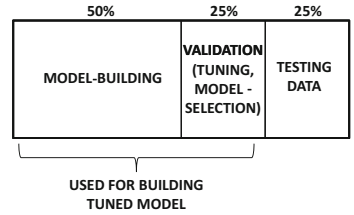



Figure 5.4: Partitioning a labeled data set for evaluation design

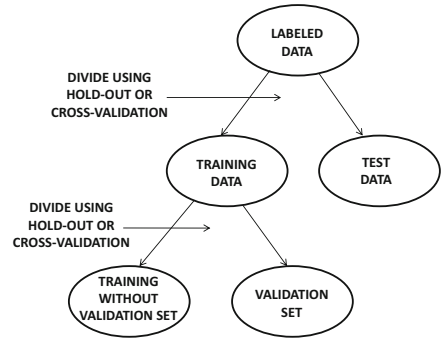



Figure 5.5: Hierarchical division into training, validation, and testing portions

of it for model building as possible, because the variance induced by the validation and evaluation stage is often quite low. A constant number of examples (e.g., less than a few thousand) in the validation and test data sets are sufficient to provide accurate estimates. The 2:1:1 division is a rule of thumb that has now become defunct, because it is inherited from an era in which data sets were small. In the modern era, where data sets are large, almost all of the points are used for training, and a modest (constant) number are used for validation and testing. It is not uncommon to have divisions such as 98:1:1.

### 5.3.1 Evaluating with Hold-Out and Cross-Validation

The aforementioned description of partitioning the labeled data into three segments is an implicit description of a method referred to as *hold-out* for segmenting the labeled data into various portions. However, the division into *three* parts is not done in one shot. Rather, the training data is first divided into *two* parts for training and testing. The testing part is then carefully hidden away from any further analysis *until the very end where it can be used only once*. The remainder of the data set is then divided again into the training and validation portions. This type of recursive division is shown in Figure 5.5.

The types of division at both levels of the hierarchy are conceptually identical. In the following, we will consistently use the terminology of the first level of division in Figure 5.5 into "training" and "testing" data, even though the same approach can also be used for the second-level division into model building and validation portions. This allows us to provide a unified description of evaluation processes at both levels of the division.

#### Hold-Out

In the hold-out method, a fraction of the instances are used to build the training model. The remaining instances, which are also referred to as the *held-out* instances, are used for testing. The accuracy of predicting the labels of the held-out instances is then reported as the overall accuracy. Such an approach ensures that the reported accuracy is not a result of overfitting to the specific data set, because different instances are used for training and testing. The approach, however, underestimates the true accuracy. Consider the case where the held-out examples have a higher presence of a particular class than the labeled data set. This means that the held-in examples have a lower average presence of the same class, which will cause a mismatch between the training and test data. Furthermore, the class-wise frequency of the held-in examples will always be inversely related to that of the held-out examples. This will lead to a consistent pessimistic bias in the evaluation. In spite of these weaknesses, the hold-out method has the advantage of being simple and efficient, which makes it a popular choice in large-scale settings. From a deep-learning perspective, this is an important observation because large data sets are common.

#### Cross-Validation

In the cross-validation method, the labeled data is divided into $q$ equal segments. One of the $q$ segments is used for testing, and the remaining $(q-1)$ segments are used for training. This process is repeated $q$ times by using each of the $q$ segments as the test set. The average accuracy over the $q$ different test sets is reported. Note that this approach can closely estimate the true accuracy when the value of $q$ is large. A special case is one where $q$ is chosen to be equal to the number of labeled data points and therefore a single point is used for testing. Since this single point is left out from the training data, this approach is referred to as *leave-one-out cross-validation*. Although such an approach can closely approximate the accuracy, it is usually too expensive to train the model a large number of times. In fact, cross-validation is sparingly used in neural networks because of efficiency issues.

### 5.3.2 Issues with Training at Scale

One practical issue that arises in the specific case of neural networks is when the sizes of the training data sets are large. Therefore, while methods like cross-validation are well established to be superior choices to hold-out in traditional machine learning, their technical soundness is often sacrificed in favor of efficiency. In general, training time is such an important consideration in neural network modeling that many compromises have to be made to enable practical implementation.

A computational problem often arises in the context of grid search of hyperparameters (cf. section 2.7.1 of Chapter 2). Even a single hyperparameter choice can sometimes require a few days to evaluate, and a grid search requires the testing of a large number of possibilities. Therefore, a common strategy is to run the training process of each setting for a fixed number of epochs. Multiple runs are executed over different choices of hyperparameters in different threads of execution. Those choices of hyperparameters in which good progress is not made after a fixed number of epochs are terminated. In the end, only a few ensemble members are allowed to run to completion. One reason that such an approach works well is because the vast majority of the progress is often made in the early phases of the training. This process is also described in section 2.7.1 of Chapter 2.

### 5.3.3 How to Detect Need to Collect More Data

The high generalization error in a neural network may be caused by several reasons. First, the data itself might have a lot of noise, in which case there is little one can do in order to improve accuracy. Second, neural networks are hard to train, and the large error might be caused by the poor convergence behavior of the algorithm. The error might also be caused by high bias, which is referred to as *underfitting*. Finally, overfitting (i.e., high variance) may cause a large part of the generalization error. In most cases, the error is a combination of more than one of these different factors. However, one can detect overfitting in a specific training data set by examining the gap between the training and test accuracy. Overfitting is manifested *by a large gap between training and test accuracy*. It is not uncommon to have close to 100% training accuracy on a small training set, even when the test error is quite low. The first solution to this problem is to collect more data. With increased training data, the training accuracy will reduce, whereas the test/validation accuracy will increase. However, if more data is not available, one would need to use other techniques such as regularization in order to improve generalization performance.

## 5.4 Penalty-Based Regularization

Penalty-based regularization is the most common approach for reducing overfitting. In order to understand this point, let us revisit the example of the polynomial with degree $d$. In this case, the prediction $\hat{y}$ for a given value of $x$ is as follows:

$$\hat{y} = \sum_{i=0}^{d} w_i x^i \tag{5.6}$$

It is possible to use a single-layer network with $d$ inputs and a single bias neuron with weight $w_0$ in order to model this prediction. The $i$th input is $x^i$. This neural network uses linear activations, and the squared loss function for a set of training instances $(x, y)$ from data set $\mathcal{D}$ can be defined as follows:

$$L = \sum_{(x,y)\in\mathcal{D}} (y - \hat{y})^2$$

As discussed in the example of Figure 5.2, a large value of $d$ tends to increase overfitting. One possible solution to this problem is to reduce the value of $d$. In other words, using a model with *economy in parameters* leads to a simpler model. For example, reducing $d$ to 1 creates a linear model that has fewer degrees of freedom and tends to fit the data in a similar way over different training samples. However, doing so does lose some expressivity when the data patterns are indeed complex. In other words, oversimplification reduces the expressive power of a neural network, so that it is unable to adjust sufficiently to the needs of different types of data sets.

How can one retain some of this expressiveness without causing too much overfitting? Instead of reducing the number of parameters in a hard way, one can use a *soft* penalty on the use of parameters. Furthermore, large (absolute) values of the parameters are penalized more than small values, because small values do not affect the prediction significantly. What kind of penalty can one use? The most common choice is $L_2$-regularization, which is also referred to as *Tikhonov regularization*. In such a case, the additional penalty is defined by the sum of squares of the values of the parameters. Then, for the regularization parameter $\lambda > 0$, one can define the regularized objective function $J$ (including loss $L$) as follows:

$$J = \underbrace{\sum_{(x,y)\in\mathcal{D}} (y - \hat{y})^2}_{L} + \lambda \cdot \sum_{i=0}^{d} w_i^2$$

Increasing or decreasing the value of $\lambda$ reduces the softness of the penalty. One can tune this parameter for optimum performance on a held-out portion of the data set. Using this type of approach provides greater flexibility than fixing the economy of the model up front. Consider the case of polynomial regression discussed above. Restricting the number of parameters up front severely constrains the learned polynomial to a specific shape (e.g., a linear model), whereas a soft penalty is able to control the shape of the learned polynomial in a more data-driven manner. In general, it has been experimentally observed that it is more desirable to use complex models (e.g., larger neural networks) with regularization rather than simple models without regularization. The former also provides greater flexibility by providing a tunable knob (i.e., regularization parameter), which can be chosen in a data-driven manner, rather than making a rigid decision on the size of the model up front.

How does regularization affect the updates in a neural network? For any given weight $w_i$ in the neural network, the updates are defined by gradient descent (or the batched version of it):

$$w_i \Leftarrow w_i - \alpha \frac{\partial J}{\partial w_i}$$

Here, $\alpha$ is the learning rate. The use of $L_2$-regularization is roughly equivalent to the use of decay imposition after each parameter update:

$$w_i \Leftarrow w_i(1 - \alpha\lambda) - \alpha \frac{\partial L}{\partial w_i}$$

Note that the update above first multiplies the weight with the decay factor $(1 - \alpha\lambda)$, and then uses the gradient-based update. The decay of the weights can also be understood in terms of a biological interpretation, if we assume that the initial values of the weights are close to 0. One can view weight decay as a kind of forgetting mechanism, which brings the weights closer to their initial values. This ensures that only the repeated updates have a significant effect on the absolute magnitude of the weights. A forgetting mechanism prevents a model from *memorizing* the training data, because only significant and repeated updates will be reflected in the weights.

### 5.4.1 Connections with Noise Injection

The addition of noise to the input has connections with penalty-based regularization. It can be shown that the addition of an equal amount of Gaussian noise to each input is equivalent to Tikhonov regularization of a single-layer neural network with an identity activation function (for linear regression).

One way of showing this result is by examining a single training case $(\overline{X}, y)$, which becomes $(\overline{X} + \sqrt{\lambda}\overline{\epsilon}, y)$ after noise with variance $\lambda$ is added to each feature. Here, $\overline{\epsilon}$ is a random vector, in which each entry $\epsilon_i$ is independently drawn from the standard normal distribution with zero mean and unit variance. Then, the noisy prediction $\hat{y}$, which is based on $\overline{X} + \sqrt{\lambda}\overline{\epsilon}$, is as follows:

$$\hat{y} = \overline{W} \cdot (\overline{X} + \sqrt{\lambda}\epsilon) = \overline{W} \cdot \overline{X} + \sqrt{\lambda}\overline{W} \cdot \overline{\epsilon} \tag{5.7}$$

Now, let us examine the squared loss function $L = (y - \hat{y})^2$ contributed by a single training case. We will compute the *expected* value of the loss function. It is easy to show the following in expectation:

$$\begin{aligned} E[L] &= E[(y - \hat{y})^2] \\ &= E[(y - \overline{W} \cdot \overline{X} - \sqrt{\lambda}\overline{W} \cdot \overline{\epsilon})^2] \end{aligned}$$

One can then expand the expression on the right-hand side as follows:

$$\begin{aligned} E[L] &= (y - \overline{W} \cdot \overline{X})^2 - 2\sqrt{\lambda}(y - \overline{W} \cdot \overline{X}) \underbrace{E[\overline{W} \cdot \overline{\epsilon}]}_{0} + \lambda E[(\overline{W} \cdot \overline{\epsilon})^2] \\ &= (y - \overline{W} \cdot \overline{X})^2 + \lambda E[(\overline{W} \cdot \overline{\epsilon})^2] \end{aligned}$$

The second expression can be expanded using $\overline{\epsilon} = (\epsilon_1 \ldots \epsilon_d)$ and $\overline{W} = (w_1 \ldots w_d)$. Furthermore, one can set any term of the form $E[\epsilon_i \epsilon_j]$ to $E[\epsilon_i] \cdot E[\epsilon_j] = 0$ because of independence of the random variables $\epsilon_i$ and $\epsilon_j$. Any term of the form $E[\epsilon_i^2]$ is set to 1, because each $\epsilon_i$ is drawn from a standard normal distribution. On expanding $E[(\overline{W} \cdot \overline{\epsilon})^2]$ and making the above substitutions, one finds the following:

$$E[L] = (y - \overline{W} \cdot \overline{X})^2 + \lambda(\sum_{i=1}^{d} w_i^2) \tag{5.8}$$

It is noteworthy that *this loss function is exactly the same as $L_2$-regularization* of a single instance.

Although the equivalence between weight decay and noise addition holds for the case of linear regression, the analysis is not quite exact in the case of neural networks with nonlinear activations. Nevertheless, penalty-based regularization continues to be intuitively similar to noise addition even in these cases. Because of these similarities one sometimes tries to perform regularization by direct noise addition. One such approach is referred to as *data perturbation*, in which noise is added to the training input, and the test data points are predicted with the added noise. This approach is described in section 5.5.5.

### 5.4.2 $L_1$-Regularization

The use of the squared norm penalty, which is also referred to as $L_2$-regularization, is the most common approach for regularization. However, it is possible to use other types of penalties on the parameters. A common approach is $L_1$-regularization in which the squared penalty is replaced with a penalty on the sum of the absolute magnitudes of the coefficients. Therefore, the new objective function is as follows:

$$J = L + \lambda \cdot \sum_{i=0}^{d} |w_i|_1 = \sum_{(x,y) \in \mathcal{D}} (y - \hat{y})^2 + \lambda \cdot \sum_{i=0}^{d} |w_i|_1$$

The main problem with this objective function is that it contains the term $|w_i|$, which is not differentiable when $w_i$ is exactly equal to 0. This requires some modifications to the gradient-descent method when $w_i$ is 0. For the case when $w_i$ is non-zero, one can use the straightforward update obtained by computing the partial derivative. By differentiating the above objective function, we can define the update equation at least for the case when $w_i$ is different than 0:

$$w_i \Leftarrow w_i - \alpha\lambda s_i - \alpha \frac{\partial L}{\partial w_i}$$

The value of $s_i$, which is the partial derivative of $|w_i|$ (with respect to $w_i$), is as follows:

$$s_i = \begin{cases} -1 & w_i < 0 \\ +1 & w_i > 0 \end{cases}$$

However, we also need to set the partial derivative of $|w_i|$ for cases in which the value of $w_i$ is exactly 0. One possibility is to use the *subgradient* method in which the value of $w_i$ is set stochastically to a value in $\{-1, +1\}$. However, this is not necessary in practice. Computers are of finite-precision, and the computational errors will rarely cause $w_i$ to be *exactly* 0. Therefore, the computational errors will often perform the task that would otherwise be achieved by stochastic sampling. Furthermore, for the rare cases in which the value $w_i$ is exactly 0, one can omit the regularization and simply set $s_i$ to 0. This type of approximation to the subgradient method works reasonably well in many settings.

One difference between the update equations for $L_1$-regularization and those in $L_2$-regularization is that $L_2$-regularization uses multiplicative decay as a forgetting mechanism, whereas $L_1$-regularization uses additive updates as a forgetting mechanism. In both cases, the regularization portions of the updates tend to move the coefficients closer to 0. However, they have different properties, which are discussed in the next section.

### 5.4.3 $L_1$- or $L_2$-Regularization?

A question arises as to whether $L_1$- or $L_2$-regularization is desirable. From an accuracy point of view, $L_2$-regularization usually outperforms $L_1$-regularization. This is the reason that $L_2$-regularization is almost always preferred over $L_1$-regularization is most implementations. The performance gap is small when the number of inputs and units is large.

However, $L_1$-regularization does have specific applications from an interpretability point of view. An interesting property of $L_1$-regularization is that it creates *sparse* solutions in which the vast majority of the values of $w_i$ are 0s (after ignoring[1] computational errors). If the value of $w_i$ is zero for a connection incident on the input layer, then that particular input has no effect on the final prediction. In other words, such an input can be *dropped*, and the $L_1$-regularizer acts as a feature selector. Therefore, one can use $L_1$-regularization to estimate which features are predictive to the application at hand.

What about the connections in the hidden layers whose weights are set to 0? These connections can be dropped, which results in a sparse neural network. Such sparse neural networks can be useful in cases where one repeatedly performs training on the same type of data set, but the nature and broader characteristics of the data set do not change significantly with time. Since the sparse neural network will contain only a small fraction of the connections in the original neural network, it can be retrained much more efficiently whenever more training data is received.

### 5.4.4 Penalizing Hidden Units: Learning Sparse Representations

The penalty-based methods, which have been discussed so far, penalize the *parameters* of the neural network. A different approach is to penalize the *activations* of the neural network, so that only a small subset of the neurons are activated for any given data instance. In other words, even though the neural network might be large and complex only a small part of it is used for predicting any given data instance.

[1]Computational errors can be ignored by requiring that $|w_i|$ should be at least $10^{-6}$ in order for $w_i$ to be considered truly non-zero.

The simplest way to achieve sparsity is to impose an $L_1$-penalty on the hidden units. In other words, the loss on the $i$th hidden unit is $L_i^h = \lambda |h(i)|$, where $\lambda$ is the regularization parameter. Therefore, the original loss function $L$ is modified to the regularized loss function $L^+$ as follows:

$$L^+ = L + \sum_{i=1}^{M} L_i^h = L + \lambda \sum_{i=1}^{M} |h(i)| \tag{5.9}$$

Here, $M$ is the total number of units in the network, and $h(i)$ is the value of the $i$th hidden unit.

As discussed in section 2.6.5 of Chapter 2, this change in loss function does not affect the overall dynamics and principles of backpropagation. The backpropagation algorithm needs to be modified so that the regularization penalty contributed by a hidden unit is incorporated into the backwards gradient flow at that node. Let $\Delta(i)$ be the derivative of the loss with respect to the pre-activation value in node $i$, and $A(i)$ be the set of nodes connected to node $i$ using outgoing edges from node $i$. Furthermore, let $\Phi_i'$ be the numerical value of the local derivative of the activation function at node $i$, and $w_{ij}$ be the weight of edges from node $i$ to node $j$. We repeat Equation 2.35 from section 2.6.5, which shows how one can incorporate the effect of node penalties with an extra flow term to account for these penalties:

$$\Delta(i) \Leftarrow [\Phi_i' \sum_{j \in A(i)} w_{ij} \Delta(j)] + \underbrace{\Phi_i' \frac{\partial L_i^h}{\partial h(i)}}_{\text{Extra flow}} \tag{5.10}$$

The above update is specific to node $i$, since it computes the partial derivative of the loss with respect to the pre-activation variable at node $i$. The second term accounts for the node penalties. One can simplify this update to obtain the following:

$$\Delta(i) \Leftarrow [\Phi_i' \sum_{j \in A(i)} w_{ij} \Delta(j)] + \underbrace{\lambda \, \Phi_i' \, \text{sign}\{h(i)\}}_{\text{Extra flow}} \tag{5.11}$$

In other words, the only change that is required to a node-specific gradient update is to add the derivative of the regularization penalty at that node.

## 5.5 Ensemble Methods

Ensemble methods derive their inspiration from the bias-variance trade-off. One way of reducing the error of a classifier is to find a way to design the classifier to reduce the sum of the squared bias and the variance. Most ensemble methods in neural networks are focused on variance reduction. This is because neural networks are valued for their ability to build arbitrarily complex models in which the bias is relatively low. However, operating at the complex end of the bias-variance trade-off almost always leads to higher variance, which is manifested as overfitting. Therefore, this section will focus on such variance reduction methods.

### 5.5.1 Bagging and Subsampling

Imagine that you had an infinite resource of training data available to you, where you could generate as many training points as you wanted from a base distribution. A natural approach for reducing the variance in this case would be to repeatedly create different training data

sets and predict the same test instance using these data sets. The prediction across different data sets can then be averaged to yield the final prediction. If a sufficient number of training data sets is used, the variance of the prediction will be reduced to 0, although the bias will still remain depending on the choice of model.

The approach described above can be used only when an infinite resource of data is available. However, in practice, we only have a single finite instance of the data available to us. In such cases, one obviously cannot implement the above methodology. However, it turns out that an imperfect simulation of the above methodology still has better variance characteristics than a single execution of the model on the entire training data set. The basic idea is to generate new training data sets from the single instance of the base data by sampling. The sampling can be performed with or without replacement. The predictions on a particular test instance, which are obtained from the models built with different training sets, are then averaged to create the final prediction. One can average either the real-valued predictions (e.g., probability estimates of class labels) or the discrete predictions. In the case of real-valued predictions, better results are sometimes obtained by using the median of the values.

It is common to use the softmax to yield probabilistic predictions of discrete outputs. If probabilistic predictions are averaged, it is common to average the *logarithms* of these values. This is the equivalent of using the *geometric* means of the probabilities. For discrete predictions, arithmetically averaged voting is used. This distinction between the handling of discrete and probabilistic predictions is carried over to other types of ensemble methods that require averaging of the predictions. This is because the logarithms of the probabilities have a log-likelihood interpretation, and log-likelihoods are inherently additive.

The main difference between bagging and subsampling is in terms of whether or not replacement is used in the creation of the sampled training data sets. We summarize these methods as follows:

1. *Bagging:* In bagging, the training data is sampled with replacement. The sample size $s$ may be different from the size of the training data size $n$, although the classical approach is to set $s$ to $n$. In the latter case, the resampled data will contain duplicates, and about a fraction $(1 - 1/n)^n \approx 1/e$ of the original data set will not be included at all. Here, the notation $e$ denotes the base of the natural logarithm. A model is constructed on the resampled training data set, and each test instance is predicted with the resampled data. The entire process of resampling and model building is repeated $m$ times. For a given test instance, each of these $m$ models is applied to the test data. The predictions from different models are then averaged to yield a single robust prediction. Although classical bagging uses $s = n$, the best results are often obtained by choosing values of $s$ much less than $n$.

2. Subsampling is similar to bagging, except that the different models are constructed on the samples of the data created *without* replacement. The predictions from the different models are averaged. In this case, it is essential to choose $s < n$, because choosing $s = n$ yields the same training data set and identical results across different ensemble components.

When a sufficient training data are available, subsampling is often preferable to bagging. However, using bagging makes sense when the amount of available data is limited.

It is noteworthy that all the variance cannot be removed by using bagging or subsampling, because the different training samples will have overlaps in the included points. Therefore, the predictions of test instances from different samples will be positively correlated. The average of a set of random variables that are positively correlated will always

have a variance that increases with positive correlation. As a result, there will always be a residual variance in the predictions. This residual variance is a consequence of the fact that bagging and subsampling are imperfect simulations of drawing the training data from a base distribution. Nevertheless, the variance of this approach is still lower than that of constructing a single model on the entire training data set. The main challenge in directly using bagging for neural networks is that one must construct multiple training models, which is highly inefficient. However, the construction of different models can be fully parallelized, which makes the approach highly scalable.

### 5.5.2 Parametric Model Selection and Averaging

One challenge in the case of neural network construction is the selection of a large number of hyperparameters like the depth of the network and the number of neurons in each layer. Furthermore, the choice of the activation function also has an effect on performance, depending on the application at hand. The presence of a large number of parameters creates problems in model construction, because the performance might be sensitive to the particular configuration used. One possibility is to hold out a portion of the training data and try different combinations of parameters and model choices. The selection that provides the highest accuracy on the held-out portion of the training data is then used for prediction. This is, of course, the standard approach used for parameter tuning in all machine learning models, and is also referred to as *model selection*. In a sense, model selection is inherently an ensemble-centric approach, where the best out of bucket of models is selected. Therefore, the approach is also sometimes referred to as the *bucket-of-models* technique.

The main problem in deep learning settings is that the number of possible configurations is rather large. For example, one might need to select the number of layers, the number of units in each layer, and the activation function. The combination of these possibilities is rather large. Therefore, one is often forced to try only a limited number of possibilities to choose the configuration. An additional approach that can be used to reduce the variance, is to select the $k$ best configurations and then average the predictions of these configurations. Such an approach leads to more robust predictions, especially if the configurations are very different from one another. Even though each individual configuration might be suboptimal, the overall prediction will still be quite robust. As in the case of bagging, the training of multiple configurations can be parallelized.

### 5.5.3 Randomized Connection Dropping

The random dropping of connections between different layers in a multilayer neural network often leads to diverse models in which different combinations of features are used to construct the hidden variables. The dropping of connections between layers does tend to create less powerful models because of the addition of constraints to the model-building process. However, since different random connections are dropped from different models, the predictions from different models are very diverse. The averaged prediction from these different models is often highly accurate. It is noteworthy that the weights of different models are not shared in this approach, which is different from another technique called *Dropout*.

Randomized connection dropping can be used for any type of predictive problem and not just classification. For example, the approach has been used for outlier detection with autoencoder ensembles [64]. As discussed in section 3.4.4 of Chapter 3, autoencoders can be used for outlier detection by estimating the reconstruction error of each data point. The work in [64] uses multiple autoencoders with randomized connections, and then aggregates

the outlier scores from these different components in order to create the score of a single data point. However, the use of the median is preferred to the mean in [64]. It has been shown in [64] that such an approach improves the overall accuracy of outlier detection. It is noteworthy that this approach might seem superficially similar to *Dropout* and *DropConnect*, although it is quite different. This is because methods like *Dropout* and *DropConnect* share weights between different ensemble components, whereas this approach does not share any weights between ensemble components.

### 5.5.4 Dropout

*Dropout* is a method that uses node sampling instead of edge sampling in order to create a neural network ensemble. If a node is dropped, then all incoming and outgoing connections from that node need to be dropped as well. The nodes are sampled only from the input and hidden layers of the network. Note that sampling the output node(s) would make it impossible to provide a prediction and compute the loss function. In some cases, the input nodes are sampled with a different probability from that of the hidden nodes. Therefore, if the full neural network contains $M$ nodes, then the total number of possible sampled networks is $2^M$.

A key point that is different from the connection sampling approach discussed in the previous section is that *weights of the different sampled networks are shared*. Therefore, *Dropout* combines node sampling with weight sharing. The training process then uses a single sampled example in order to update the weights of the sampled network using backpropagation. The training process proceeds using the following steps, which are repeated again and again in order to cycle through all of the training points in the network:

1. Sample a neural network from the base network. The input nodes are each sampled with probability $p_i$, and the hidden nodes are each sampled with probability $p_h$. Furthermore, all samples are independent of one another. When a node is removed from the network, all its incident edges are removed as well.

2. Sample a single training instance or a mini-batch of training instances.

3. Update the weights of the retained edges in the network using backpropagation on the sampled training instance or the mini-batch of training instances.

It is common to drop out nodes with probability between 20% and 50%. Large learning rates are often used with momentum, which are tempered with a max-norm constraint on the weights. In other words, the $L_2$-norm of the weights entering each node is constrained to be no larger than a small constant such as 3 or 4.

It is noteworthy that a different neural network is used for every small mini-batch of training examples. Therefore, the number of neural networks sampled is rather large, and depends on the size of the training data set. This is different from most other ensemble methods like bagging in which the number of ensemble components is rarely larger than 25. In the *Dropout* method, thousands of neural networks are sampled with shared weights, and a tiny training data set is used to update the weights in each case. Even though a large number of neural networks is sampled, the *fraction* of neural networks sampled out of the base number of possibilities is still minuscule. Another assumption that is used in this class of neural networks is that the output is in the form of a probability. This assumption has a bearing on the way in which the predictions of the different neural networks are combined.

How can one use the ensemble of neural networks to create a prediction for an unseen test instance? One possibility is to predict the test instance using all the neural networks

that were sampled, and then use the geometric mean of the probabilities that are predicted by the different networks. The geometric mean is used rather than the arithmetic mean, because the assumption is that the output of the network is a probability and the geometric mean is equivalent to averaging log-likelihoods. For example, if the neural network has $k$ probabilistic outputs corresponding to the $k$ classes, and the $j$th ensemble yields an output of $p_i^{(j)}$ for the $i$th class, then the ensemble estimate for the $i$th class is computed as follows:

$$p_i^{Ens} = \left[\prod_{j=1}^{m} p_i^{(j)}\right]^{1/m} \tag{5.12}$$

Here, $m$ is the total number of ensemble components, which can be rather large in the case of the *Dropout* method. One problem with this estimation is that the use of geometric means results in a situation where the probabilities over the different classes do not sum to 1. Therefore, the values of the probabilities are re-normalized so that they sum to 1:

$$p_i^{Ens} \Leftarrow \frac{p_i^{Ens}}{\sum_{i=1}^{k} p_i^{Ens}} \tag{5.13}$$

The main problem with this approach is that the number of ensemble components is too large, which makes the approach inefficient.

A key insight of the *Dropout* method is that it is not necessary to evaluate the prediction on all ensemble components. Rather, one can perform forward propagation on only the base network (with no dropping) after re-scaling the weights. The basic idea is to multiply the weights going out of each unit with the probability of sampling that unit. By using this approach, the expected output of that unit from a sampled network is captured. This rule is referred to as the *weight scaling inference rule*. Using this rule also ensures that the input going into a unit is also the same as the expected input that would occur in a sampled network.

The weight scaling inference rule is exact for many types of networks with linear activations, although the rule is not exactly true for networks with nonlinearities. In practice, the rule tends to work well across a broad variety of networks. Since most practical neural networks have nonlinear activations, the weight scaling inference rule of *Dropout* should be viewed as a heuristic rather than a theoretically justified result.

The main effect of *Dropout* is to incorporate regularization into the learning procedure. By dropping both input units and hidden units, *Dropout* effectively incorporates noise into both the input data and the hidden representations. The nature of this noise can be viewed as a kind of masking noise in which some inputs and hidden units are set to 0. Noise addition is a form of regularization. It has been shown in the original paper [483] on *Dropout* that this approach works better than other regularizers such as weight decay. *Dropout* prevents a phenomenon referred to as *feature co-adaptation* from occurring between hidden units. Since the effect of *Dropout* is a masking noise that removes some of the hidden units, this approach forces a certain level of redundancy between the features learned at the different hidden units. This type of redundancy leads to increased robustness.

*Dropout* is efficient because each of the sampled subnetworks is trained with a small set of sampled instances. Therefore, only the work of sampling the hidden units needs to be done additionally. However, since *Dropout* is a regularization method, it reduces the expressive power of the network. Therefore, one needs to use larger models and more units in order to gain the full advantages of *Dropout*. This results in a hidden computational

overhead. Furthermore, if the original training data set is already large enough to reduce the likelihood of overfitting, the additional computational advantages of *Dropout* may be small but still perceptible. For example, many of the convolutional neural networks trained on large data repositories like *ImageNet* [263] report consistently improved results of about 2% with *Dropout*. A variation of *Dropout* is *DropConnect*, which applies a similar approach to the weights rather than to the neural network nodes [531].

**A Note on Feature Co-Adaptation**

In order to understand why *Dropout* works, it is useful to understand the notion of feature co-adaptation. Ideally, it is useful for the hidden layers of the neural network to create features that reflect important classification characteristics of the input without having complex dependencies on other features, unless these other features are truly useful. To understand this point, consider a situation in which all edges incident on 50% of the nodes in each layer are fixed at their initial random values, and are not *updated* during backpropagation (even though all gradients are *computed* in the normal fashion). Interestingly, even in this case, it will often be possible for the neural network to provide reasonably good results by adapting the other weights and features to the effect of these randomly fixed subsets of weights (and corresponding activations). Of course, this is not a desirable situation because the goal of features working together is to combine the powers held by each essential feature rather than merely having some features adjust to the detrimental effects of others. Even in the normal training of a neural network (where all weights are updated), this type of co-adaptation can occur. For example, if the updates in some parts of the neural network are not fast enough, some of the features will not be useful and other features will adapt to these less-than-useful features. This situation is very likely in neural network training, because different parts of the neural network do tend to learn at different rates. An even more troubling scenario arises when the co-adapted features work well in predicting training points by picking up on complex dependencies in the training points, which do not generalize well to out-of-sample test points. *Dropout* prevents this type of co-adaptation by forcing the neural network to make predictions using only a subset of the inputs and activations. This forces the network to be able to make predictions with a certain level of redundancy while also encouraging smaller subsets of learned features to have predictive power. In other words, co-adaptation occurs only when it is truly essential for modeling instead of learning random nuances of the training data. This is, of course, a form of regularization. Furthermore, by learning redundant features, *Dropout* averages over the predictions of redundant features, which is similar to what is done in bagging.

### 5.5.5 Data Perturbation Ensembles

Many variance reduction methods like penalty-based regularization and *Dropout* are equivalent to noise injection. Data perturbation *explicitly* injects noise to the training data. In the simplest case, a small amount of noise can be added to the input data, and the weights can be learned on the perturbed data. This process can be repeated with multiple such additions to the training data, and it does not need to be added to the test data. When explicitly adding noise, it is important to average the prediction of the same test instance over multiple runs in order to ensure that the solution properly represents the *expected* value of the loss (without added variance caused by the noise). This type of approach is a generic ensemble method, which is not specific to neural networks. As discussed in section 5.10, this approach is used commonly in the unsupervised setting with *de-noising autoencoders*. It is

also possible to add noise to the hidden layer. However, in this case, the noise has to be carefully calibrated [396]. It is noteworthy that the *Dropout* method indirectly adds noise to the hidden layer by dropping nodes randomly. A dropped node is similar to masking noise in which the activation of that node is set to 0.

One can also perform other types of data set augmentation. For example, an image instance can be rotated or translated in order to add to the data set. Carefully designed data augmentation schemes can often greatly improve the accuracy of a learner by increasing its generalization power. However, strictly speaking such schemes are not perturbation schemes because the augmented examples are created with a calibrated procedure and an understanding of the domain at hand. Such methods are used commonly in convolutional neural networks (cf. section 9.3.4 of Chapter 9).

## 5.6 Early Stopping

Neural networks are trained using variations of gradient-descent methods. In most optimization models, gradient-descent methods are executed to convergence. However, executing gradient descent to convergence optimizes the loss on the training data, but not necessarily on the out-of-sample test data. This is because the final few steps often overfit to the specific nuances of the training data, which might not generalize well to the test data.

A natural solution to this dilemma is to use *early stopping*. In this method, a portion of the training data is held out as a validation set. The backpropagation-based training is only applied to the portion of the training data that does not include the validation set. At the same time, the error of the model on the validation set is continuously monitored. At some point, this error begins to rise on the validation set, even though it continues to reduce on the training set. This is the point at which further training causes overfitting. Therefore, this point can be chosen for termination. It is important to keep track of the best solution achieved so far in the learning process (as computed on the validation data). This is because one does not perform early stopping after tiny increases in the out-of-sample error (which might be caused by noisy variations), but it is advisable to continue to train to check if the error continues to rise. In other words, the termination point is chosen in hindsight after the error on the validation set continues to rise, and all hope is lost of improving the error performance on the validation set.

One advantage of early stopping is that it can be easily added to neural network training without significantly changing the training procedure. Furthermore, methods like weight decay require us to try different values of the regularization parameter, $\lambda$, which can be expensive. Because of the ease in combining it with existing algorithms, early stopping can be used in combination with other regularizers in a relatively straightforward way. Therefore, early stopping is almost always used, because one does not lose much by adding it to the learning procedure.

One can view early stopping as a kind of constraint on the optimization process. By restricting the number of steps in the gradient descent, one is effectively restricting the distance of the final solution from the initialization point. Adding constraints to the model of a machine learning problem is often a form of regularization.

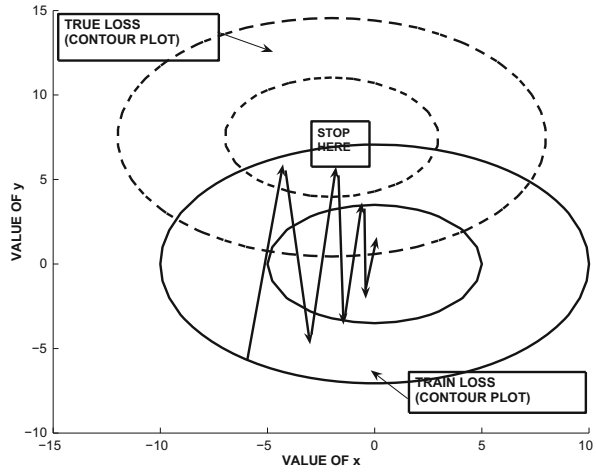



Figure 5.6: Shift in loss function caused by variance effects and the effect of early stopping. Because of the differences in the true loss function and that on the training data, the error will begin to rise if gradient descent is continued beyond a certain point. Here, we have shown a similar shape of the true and training loss functions for simplicity, although this might not be the case in practice.

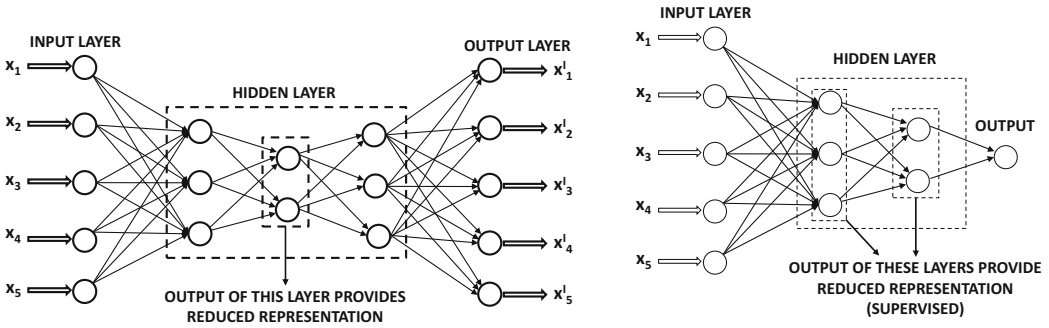
### 5.6.1 Understanding Early Stopping from the Variance Perspective

One way of understanding the bias-variance trade-off is that the true loss function of an optimization problem can only be constructed if we have infinite data. If we have a finite amount of data, the loss function constructed from the training data does not reflect the true loss function. Illustrative examples of the contours of the true loss function and its shifted counterpart on the training data are illustrated in Figure 5.6. This shifting is an indirect manifestation of the variance in prediction created by a particular training data set. Different training data sets will shift the loss function in different and unpredictable ways.

Unfortunately, the learning procedure can perform the gradient-descent only on the loss function defined on the training data set, because the true loss function is unknown. However, if the training data is representative of the true loss function, the optimum solutions in the two cases will be reasonably close as shown in Figure 5.6. As discussed in Chapter 4, most gradient-descent procedures take a circuitous and oscillatory route to the optimal solution. During the final stage of convergence to the optimal solution (on the training data), the gradient descent will often encounter better solutions with respect to the true loss function before it converges to the best solution with respect to the training data. These solutions will be detected by the improved accuracy on the validation set, and therefore provide good termination points. An example of a good early stopping point is shown in Figure 5.6.

## 5.7 Unsupervised Pretraining

Deep networks are inherently hard to train because of a number of different characteristics discussed in the previous chapter. One issue is the exploding and vanishing gradient problem,




(a) Multilayer autoencoder (b) Multilayer classifier

Figure 5.7: Both the multilayer classifier and the multilayer autoencoder use a similar pretraining procedure.

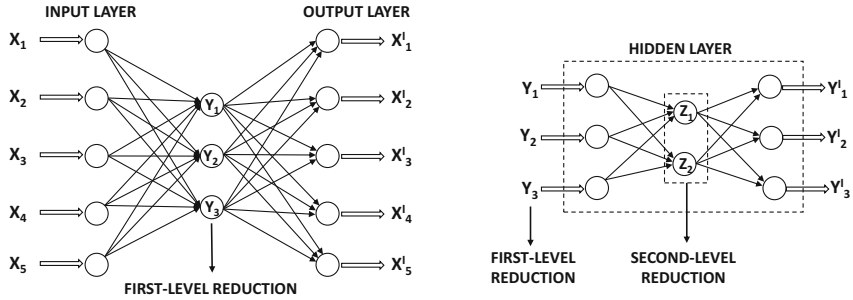
because of which the different layers of the neural network do not get trained at the same rate. The multiple layers of the neural network cause distortions in the gradient, which make them hard to train.

A ground-breaking break-through in this context was the use of unsupervised pretraining in order to provide robust initializations [206]. This initialization is achieved by training the network greedily in layer-wise fashion. The approach was originally proposed in the context of deep belief networks, but it was later extended to other types of models such as autoencoders [402, 526]. In this chapter, we will study the autoencoder approach because of its simplicity. First, we will start with the dimensionality reduction application, because the application is unsupervised and it is easy to show how to use unsupervised pretraining in this case. However, unsupervised pretraining can also be used for supervised applications like classification with minor modifications.

In pretraining, a greedy approach is used to train the network one layer at a time by learning the weights of the outer hidden layers first and then learning the weights of the inner hidden layers. The resulting weights are used as starting points for a final phase of traditional neural network backpropagation in order to fine-tune them.

Consider the autoencoder and classifier architectures shown in Figure 5.7. Since these architectures have multiple layers, randomized initialization can sometimes cause challenges. However, it is possible to create a good initialization by setting the initial weights layer by layer in a greedy fashion. First, we describe the process in the context of the autoencoder shown in Figure 5.7(a), although an almost identical procedure is relevant to the classifier of Figure 5.7(b). We have intentionally chosen neural architectures in the two cases so that the hidden layers have similar numbers of nodes.

The pretraining process is shown in Figure 5.8. The basic idea is to assume that the two (symmetric) outer hidden layers contain a first-level reduced representation of larger dimensionality, and the inner hidden layer contains a second-level reduced representation of smaller dimensionality. Therefore, the first step is to learn the first-level reduced representation and the corresponding weights associated with the outer hidden layers using the simplified network of Figure 5.8(a). In this network, the middle hidden layer is missing and the two outer hidden layers are collapsed into a single hidden layer. The assumption is that the two outer hidden layers are related to one another in a symmetric way like a smaller autoencoder. In the second step, the reduced representation in the first step is used to learn the second-level reduced representation (and weights) of the inner hidden layers. Therefore,




(a) Pretraining first-level reduction and outer weights

(b) Pretraining second-level reduction and inner weights

Figure 5.8: Pretraining a neural network

the inner portion of the neural network is treated as a smaller autoencoder in its own right. Since each of these pretrained subnetworks is much smaller, the weights can be learned more easily. This initial set of weights is then used to train the entire neural network with backpropagation. Note that this process can be performed in layerwise fashion for a deep neural network containing any number of hidden layers.

So far, we have only discussed how we can use unsupervised pretraining for unsupervised applications. A natural question arises as to how one can use pretraining for supervised applications. Consider a multilayer classification architecture with a single output layer and $k$ hidden layers. During the pretraining stage, the output layer is removed, and the representation of the final hidden layer is learned in an unsupervised way. This is achieved by creating an autoencoder with $2 \cdot k - 1$ hidden layers, where the middle layer is the final hidden layer of the supervised setting. For example, the relevant autoencoder for Figure 5.7(b) is shown in Figure 5.7(a). Therefore, an additional $(k-1)$ hidden layers are added, each of which has a symmetric counterpart in the original network. This network is trained in exactly the same layer-wise fashion as discussed above for the autoencoder architecture. The weights of only the encoder portion of this autoencoder are used for initialization of the weights entering into all hidden layers. The weights between the final hidden layer and the output layer can also be initialized by treating the final hidden layer and output nodes as a single-layer network. This single-layer network is fed with the reduced representations of the final hidden layer (based on the autoencoder learned in pretraining). After the weights of all the layers have been learned, the output nodes are re-attached to the final hidden layer. The backpropagation algorithm is applied to this initialized network in order to fine-tune the weights from the pretrained stage. Note that this approach learns all the initial hidden representations in an unsupervised way, and only the weights entering into the output layer are initialized using the labels. Therefore, the pretraining can still be considered to be largely unsupervised.

During the early years, pretraining was often seen as a more stable way to train a deep network in which the different layers have a better chance of being initialized in an equally effective way. Although this issue does play a role in explaining the improvements of pretraining, the problem is often manifested as overfitting. As discussed in Chapter 4, the (finally converged) weights in the early layers may not change much from their random initializations, when the network exhibits the vanishing gradient problem. Even when the connection weights in the first few layers are random (as a result of poor training), it is possible for the later layers to adapt their weights sufficiently so as to give zero error on the

*training* data. In this case, the random connections in the early layers provide near-random transformations to the later layers, but the later layers are still able to overfit to these features in order to provide very low training error. In other words, the features in later layers *adapt* to those in early layers as a result of training inefficiencies. Any kind of feature co-adaptation caused by training inefficiencies almost always leads to overfitting. Therefore, when the approach is applied to unseen test data, the overfitting becomes apparent because the various layers are not specifically adapted to these unseen test instances. In this sense, pretraining is an unusual form of regularization.

Incidentally, unsupervised pretraining helps even in cases where the amount of training data is very large. It is likely that this behavior is caused by the fact that pretraining helps in issues beyond model generalization. One evidence of this fact is that in larger data sets, even the error on the training data seems to be high, when methods like pretraining are not used. In these cases, the weights of the early layers often do not change much from their initializations, and one is using only a small number of later layers on a random transformation of the data (defined by the random initialization of the early layers). As a result, the trained portion of the network is rather shallow, with some additional loss caused by the random transformation. In such cases, pretraining also helps a model realize the full benefits of depth, thereby facilitating the improvement of prediction accuracy on larger data sets.

Another way of understanding pretraining is that it provides insights into the repeated patterns in the data, which are the features learned from the training data points. For example, an autoencoder might learn that many digits have loops in them, and certain digits have strokes that are curved in a particular way. The decoder reconstructs the digits by putting together these frequent shapes. However, these shapes also have discriminative power with respect to recognizing digits. Expressing the data in terms of a few features then helps in recognizing how these features are related to the class labels. This principle is summarized by Geoff Hinton [202] in the context of image classification as follows: "*To recognize shapes, first learn to generate images.*" This type of regularization preconditions the training process in a semantically relevant region of the parameter space, where several important features have already been learned, and further training can fine-tune and combine them for prediction.

### 5.7.1 Variations of Unsupervised Pretraining

There are many different ways in which one can introduce variations to the procedure of unsupervised pretraining. For example, multiple layers can be trained at one time instead of performing pretraining only one layer at a time. A particular case in point is *VGG* (cf. section 9.4.3 of Chapter 9) in which as many as eleven layers of an even deeper architecture were trained together. Indeed, there are some advantages in grouping as many layers as possible within the pretraining because a (successful) training procedure with larger pieces of the neural network leads to more powerful initializations. On the other hand, grouping too many layers together within each pretraining component can lead to problems (such as the vanishing and exploding gradient problems) within each component.

A second point is that the pretraining procedure of Figure 5.8 assumes that the autoencoder works in a completely symmetric way in which the reduction in the $k$th layer of the encoder is approximately similar to the reduction in its mirror layer in the decoder. This might be a restrictive assumption in practice, if different types of activation functions are used in different layers. For example, a sigmoid activation function in a particular layer of the encoder will create only nonnegative values, whereas a tanh activation in the matching

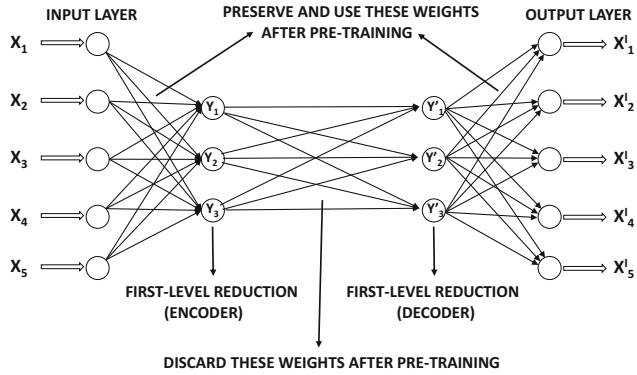



Figure 5.9: This architecture allows the first-level representations in the encoder and decoder to be significantly different. It is helpful to compare this architecture with that in Figure 5.8(a).

layer of the decoder might create both positive and negative values. Another approach is to use a relaxed pretraining architecture in which we learn separate reductions for the $k$th level reduction in the encoder and its mirror image in the decoder. This allows the corresponding reductions in the encoder and the decoder to be different. An additional layer of weights must be added between the two layers to allow for the differences between the two reductions. This additional layer of weights is discarded after the reduction, and only the encoder-decoder weights are preserved. The only location at which an additional set of weights is not used for pretraining is in the innermost reduction, which proceeds in a similar manner to that discussed in the earlier section (cf. Figure 5.8(b)). An example of such an architecture for the first-level reduction of Figure 5.8(a) is shown in Figure 5.9. Note that the first-level representations for the encoder and decoder layers can be quite different in this case, which provides some flexibility during the pretraining process. When the approach is used for classification, only the weights in the encoder can be used, and the final reduced code can be capped with a classification layer for learning.

### 5.7.2 What About Supervised Pretraining?

So far, we have only discussed *unsupervised* pretraining, whether the base application is supervised or unsupervised. Even in the case where the base application is supervised, the initialization was done using an unsupervised autoencoder architecture. Although it is possible to perform supervised pretraining as well, an interesting and surprising result is that supervised pretraining does not seem to give as good results as unsupervised pretraining in at least some settings [31, 116]. This does not mean that supervised pretraining is *never* helpful. Indeed, there are cases of networks in which it is hard to train the network itself because of its depth. For example, networks with hundreds of layers are extremely hard to train because of issues associated with convergence and other problems. In such cases, even the error *on the training data* is high, which means that one is unable to make the training algorithm work. *This is a different problem from that of model generalization.* Aside from supervised pretraining, many techniques such as the construction of *highway networks* [168, 486], *gating networks* [214], and *residual networks* [194], can address many of these problems. However, these solutions do not specifically address overfitting, whereas unsupervised pretraining seems to hedge its bets in addressing both issues in at least some types of networks.

In supervised pretraining [31], the autoencoder architecture is not used for learning the weights of connections incident on the hidden layer. In the first iteration, the constructed network contains only the first hidden layer, which is connected to all nodes in the output layer. This step learns the weights of the connections from the input to hidden layer, although the weights of the output layer are discarded. Subsequently, the outputs of the first hidden layer are used as the new representations of the training points. Then, we create another neural network containing the first and second hidden layers and the output layer. The first hidden layer is now treated as an input layer with its inputs as the transformed representations of the training points learned in the previous iteration. These are then used to learn the next layer of weights and their hidden representations. This approach is repeated all the way to the final layer. Although this approach does provide improvements over an approach that does not use pretraining, it does not seem to work as well as unsupervised pretraining in at least some settings. The main difference in performance is on the *generalization error* on unseen test data, whereas the errors on the training data are often similar [31]. This is a near-certain sign of differential levels of overfitting of different methods.

Why does supervised pretraining not help as much as unsupervised pretraining in many settings? A key problem of supervised pretraining is that it is a bit too greedy and the early layers are initialized to representations that are very directly related to the outputs. As a result, the full advantages of depth are not exploited. This is a different type of overfitting. An important explanation for the success of unsupervised pretraining is that the learned representations are often related to the class labels in a gentle way; as a result, further learning is able to isolate and fine-tune the important characteristics of these representations. Therefore, one can view pretraining as an unusual form of *semi-supervised learning* as well, which forces the initial representations of the hidden layers to lie on the low-dimensional manifolds of data instances. The secret to the success of pretraining is that more features on these manifolds are predictive of classification accuracy than the features corresponding to random regions of the data space. After all, class distributions vary smoothly over the underlying data manifolds. The locations of data points on these manifolds are therefore good features in predicting class distributions. Therefore, the final phase of learning only has to fine-tune and enhance these features.

## 5.8 Continuation and Curriculum Learning

The discussions in the previous and current chapter show that the learning of neural network parameters is inherently a complex optimization problem, in which the loss function has a complex topological shape. Furthermore, the loss function on the training data is not exactly the same as the true loss function, which leads to spurious minima. These minima are spurious because they might be near optimal minima on the training data, but they might not be minima at all on unseen test instances. In many cases, optimizing a complex loss function tends to lead to such solutions with little generalization power.

The experience with pretraining shows that simplifying the optimization problem (or providing simple greedy solutions without too much optimization) can often precondition the solution towards the basins of better optima on the test data. In other words, instead of trying to solve a complex problem in one shot, one should first try to solve simplifications, and gradually work one's way towards complex solutions. Two such notions are those of *continuation* and *curriculum* learning:

1. *Continuation learning:* In continuation learning, one starts with a simplified version of the optimization problem and solves it. Starting with this solution, one continues to a more complex refinement of the optimization problem and updates the solution. This process is repeated until the complex optimization problem is solved. Thus, continuation learning leverages a model-centric view of working from simpler to complex problems. For example, if one has a loss function with many local optima, one can smooth it to a loss function with a single global optimum and find the optimal solution. Then, one can gradually work with better and better approximations (with increased complexity) until the exact loss function is used.

2. *Curriculum learning:* In curriculum learning, one starts by training the model on simpler data instances, and then gradually adds more difficult instances to the training data. Therefore, curriculum learning leverages a data-centric view of working from the simple to the complex, whereas continuation methods leverage a model-centric view.

A different view of curriculum and continuation learning may be obtained by examining how humans naturally learn tasks. Humans often learn simple concepts first and then move to the complex. The training of a child is often created using such a *curriculum* in order to accelerate learning. This principle also seems to work well in machine learning. In the following, we will examine both continuation and curriculum learning.

## Continuation Learning

In continuation learning, one designs a series of loss functions $L_1 \ldots L_r$, in which the difficulty in optimizing this sequence of loss functions grows from the easy to the difficult. In other words, each $L_{i+1}$ is more difficult to optimize than $L_i$. All the optimization problems are defined on the same set of parameters, because they are defined on the same neural network. The smoothing of a loss function is a form of regularization. One can view each $L_i$ as a smoothed version of $L_{i+1}$. Solving each $L_i$ brings the solution closer to the basin of optimal solutions from the point of view of generalization error.

Continuation loss functions are often constructed by using *blurring*. The idea is to compute the loss function at sampled points in the vicinity of a given point, and then average these values in order to create the new loss function. For example, one could use a normal distribution with standard deviation $\sigma_i$ for computing the $i$th loss function $L_i$. One can view this approach as a type of noise addition to the loss function, which is also a form of regularization. The amount of blurring depends on the size of the locality used for blurring, which is defined by $\sigma_i$. If the value of $\sigma_i$ is set to be too large, then the cost will be very similar at all points, and the loss function will not retain sufficient details about the objective. However, it will often be very simple to optimize. On the other hand, setting $\sigma_i$ to 0 will retain all the details in the loss function. Therefore, the natural solution is to start with large values of $\sigma_i$ and then reduce the value over successive loss functions. One can view this approach as that of using an increased amount of noise for regularization in the early iterations, and then reducing the level of regularization as the algorithm nears an attractive solution. Such tricks of adding a varying amount of calibrated noise to enable the avoidance of local optima is a recurring theme in many optimization techniques such as *simulated annealing* [254]. The main problem with continuation methods is that they are expensive due to the need to optimize a series of loss functions.

### Curriculum Learning

Curriculum learning methods take a *data-centric* view of the goals that are achieved by the *model-centric* continuation learning methods. The main hypothesis is that different training data sets present different levels of difficulty to a learner. In curriculum methods, easy examples are first presented to the learner. One possible way of defining a difficult example is as one that falls on the wrong side of a decision boundary with a perceptron or an SVM. There are other possibilities, such as the use of a Bayes classifier. The basic idea is that the difficult examples are often noisy or they represent exceptional patterns that confuse the learner. Therefore, it is inadvisable to start training with such examples.

In other words, the initial iterations of stochastic gradient descent use only the easy examples to "pretrain" the learner towards a reasonable parameter setting. Subsequently, difficult examples are included with the easy examples in later iterations. It is important to include both easy and difficult examples in the later phases, or else the learner will overfit to only the difficult examples. In many cases, the difficult examples might be exceptional patterns in particular regions of the space, or they might even be noise. If only the difficult examples are presented to the learner in later phases, the overall accuracy will not be good. The best results are often obtained by using a random mixture of simple and difficult examples in later phases. The proportion of difficult examples are increased over the course of the curriculum until the input represents the true data distribution. This type of *stochastic curriculum* has been shown to be an effective approach.

## 5.9 Parameter Sharing

A natural form of regularization that reduces the parameter footprint of the model is the sharing of parameters across different connections. Often, this type of parameter sharing is enabled by domain-specific insights. The main insight required to share parameters is that the function computed at two nodes should be related in some way. This type of insight can be obtained when one has a good idea of how a particular computational node relates to the input data. Examples of such parameter-sharing methods are as follows:

1. *Sharing weights in autoencoders:* The symmetric weights in the encoder and decoder portion of the autoencoder are often shared. Although an autoencoder will work whether or not the weights are shared, doing so improves the generalization properties of the algorithm to out-of-sample data.

2. *Recurrent neural networks:* These networks are often used for modeling sequential data, such as time-series, biological sequences, and text. In recurrent neural networks, a time-layered representation of the network is created in which the neural network is replicated across layers associated with time stamps. Since each time stamp is assumed to use the same model, the parameters are shared between different layers. Recurrent neural networks are discussed in detail in Chapter 8.

3. *Convolutional neural networks:* Convolutional neural networks are used for image recognition and prediction. Correspondingly, the inputs of the network are arranged into a rectangular grid pattern, along with all the layers of the network. Furthermore, the weights across contiguous patches of the network are typically shared. The basic idea is that a rectangular patch of the image corresponds to a portion of the visual field, and it should be interpreted in the same way no matter where it is located. In other words, a carrot means the same thing whether it is at the left or the right of

the image. In essence, these methods use semantic insights about the data to reduce the parameter footprint, share weights, and sparsify the connections. Convolutional neural networks are discussed in Chapter 9.

In many of these cases, it is evident that parameter sharing is enabled by the use of domain-specific insights about the training data as well as a good understanding of how the computed function at a node relates to the training data. The modifications to the backpropagation algorithm required for enabling weight sharing are discussed in section 2.6.6 of Chapter 2.

An additional type of weight sharing is *soft weight sharing* [371]. In soft weight sharing, the parameters are not completely tied, but a penalty is associated with them being different. For example, if one expects the weights $w_i$ and $w_j$ to be similar, the penalty $\lambda(w_i - w_j)^2/2$ might be added to the loss function. In such a case, the quantity $\alpha\lambda(w_j - w_i)$ might be added to the update of $w_i$, and the quantity $\alpha\lambda(w_i - w_j)$ might be added to the update of $w_j$. Here, $\alpha$ is the learning rate. These types of changes to the updates tend to pull the weights towards each other.

## 5.10 Regularization in Unsupervised Applications

Although overfitting does occur in unsupervised applications, it is often less of a problem. In classification, one is trying to learn a single bit of information associated with each example, and therefore using more parameters than the number of examples can cause overfitting. This is not quite the case in unsupervised applications in which a single training example may contain many more bits of information corresponding to the different dimensions. In general, the number of bits of information will depend on the intrinsic dimensionality of the data set. Therefore, one tends to hear fewer complaints about overfitting in unsupervised applications. Nevertheless, there are many unsupervised settings in which it is beneficial to use regularization. These settings will be discussed in this section.

### 5.10.1 When the Hidden Layer is Broader than the Input Layer

The discussion of autoencoders in Chapter 3 assumes that the hidden layer has fewer units than the input layer. It makes sense for the hidden layer to have fewer units than the input layer when one is looking for a compressed representation of the data. A constricted hidden layer forces dimensionality reduction, and the loss function is designed to avoid information loss. Such representations are referred to as *undercomplete representations*, and they correspond to the traditional use-case of autoencoders.

What about the case when the number of hidden units is greater than the input dimensionality? This situation corresponds to the case of *over-complete representations*. Increasing the number of hidden units beyond the number of input units makes it possible for the hidden layer to simply learn the identity function (with zero loss). Simply copying the input across the layers does not seem to be particularly useful. However, this does not occur in practice (while learning weights), especially if certain types of regularization and *sparsity constraints* are imposed on the hidden layer. Even if no sparsity constraints are imposed, and stochastic gradient descent is used for learning, the probabilistic regularization caused by stochastic gradient descent is sufficient to ensure that the hidden representation will always scramble the input before reconstructing it at the output. This is because stochastic

gradient descent is a type of noise addition to the learning process, and therefore it will not be possible to learn weights that simply copy input to output as identity functions across layers. Furthermore, because of some peculiarities of the training process, a neural network almost never uses its full modeling ability, which leads to dependencies among the weights [96]. Rather, an over-complete representation may be created, although it may not have the property of sparsity (which needs to be explicitly encouraged). The next section will discuss ways of encouraging sparsity.

#### 5.10.1.1 Sparse Feature Learning

When explicit sparsity constraints are imposed, the resulting autoencoder is referred to as a *sparse autoencoder*. A sparse representation of a $d$-dimensional point is a $k$-dimensional point in which $k \gg d$ and most of the values in the sparse representation are 0s. Sparse feature learning has tremendous applicability to many settings like image data, where the learned features are often intuitively more interpretable from an application-specific perspective. Sparse representations also enable the effective use of particular types of efficient algorithms that are highly dependent on sparsity. There are many ways in which constraints might be enforced on the hidden layer to create sparsity. Some examples are as follows:

1. One can impose an $L_1$-penalty on the activations in the hidden layer to force sparse activations. The notion of $L_1$-penalties for creating sparse solutions (in terms of either weights or hidden units) is discussed in section 5.4.2 and 5.4.4 of Chapter 5. In such a case, backpropagation must also propagate the gradient of this penalty in the backwards direction. Surprisingly, this natural alternative is rarely used.

2. One can allow only the top-$r$ activations in the hidden layer to be nonzero for $r \leq k$. In such a case, backpropagation only backpropagates through the activated units. This approach is referred to as the $r$-sparse autoencoder [321].

3. Another approach is the *winner-take-all* autoencoder [322], in which only a fraction $f$ of the activations of *each* hidden unit are allowed over the *whole training data*. In this case, the top activations are computed across training examples, whereas in the previous case the top activations are computed across a hidden layer for a single training example. Therefore node-specific thresholds need to be estimated using the statistics of a minibatch. The backpropagation algorithm needs to propagate the gradient only through the activated units.

Note that the implementations of the competitive mechanisms are almost like ReLU activations with adaptive thresholds. Refer to the bibliographic notes for pointers and more details of these algorithms.

### 5.10.2 Noise Injection: De-noising Autoencoders

As discussed in section 5.4.1, noise injection is a form of penalty-based regularization of the weights. The use of Gaussian noise in the input is roughly equal to $L_2$-regularization in single-layer networks with linear activation. The de-noising autoencoder is based on noise injection rather than penalization of the weights or hidden units. However, the goal of the de-noising autoencoder is to reconstruct good examples from corrupted training data. Therefore, the type of noise should be calibrated to the nature of the input. Several different types of noise can be added:

1. *Gaussian noise:* This type of noise is appropriate for real-valued inputs. The added noise has zero mean and variance $\lambda > 0$ for each input. Here, $\lambda$ is the regularization parameter.

2. *Masking noise:* The basic idea is to set a fraction $f$ of the inputs to zeros in order to corrupt the inputs. This type of approach is particularly useful when working with binary inputs.

3. *Salt-and-pepper noise:* In this case, a fraction $f$ of the inputs are set to either their minimum or maximum possible values according to a fair coin flip. The approach is typically used for binary inputs, for which the minimum and maximum values are 0 and 1, respectively.

De-noising autoencoders are useful when dealing with data that is corrupted. Therefore, the main application of such autoencoders is to reconstruct corrupted data. The inputs to the autoencoder are corrupted training records, and the outputs are the uncorrupted data records. As a result, the autoencoder learns to recognize the fact that the input is corrupted, and the true representation of the input needs to be reconstructed. Therefore, even if there is corruption in the test data (as a result of application-specific reasons), the approach is able to reconstruct clean versions of the test data. Note that the noise in the training data is explicitly added, whereas that in the test data is already present as a result of various application-specific reasons. For example, as shown in the top portion of Figure 5.10, one can use the approach to removing blurring or other noise from images. The nature of the noise added to the input training data should be based on insights about the type of corruption present in the test data. Therefore, one does require uncorrupted examples of the training data for best performance. In most domains, this is not very difficult to achieve. For example, if the goal is to remove noise from images, the training data might contain high-quality images as the output and artificially blurred images as the input. It is common for the de-noising autoencoder to be overcomplete, when it is used for reconstruction from corrupted data. However, this choice also depends on the nature of the input and the amount of noise added. Aside from its use for reconstructing inputs, the addition of noise is also an excellent regularizer that tends to make the approach work better for out-of-sample inputs even when the autoencoder is undercomplete.

The way in which the de-noising autoencoder works is that it uses the noise in the input data to learn the true manifold on which the data is embedded. Each corrupted point is projected to its "closest" matching point on the true manifold of the data distribution. The closest matching point is the expected position on the manifold from which the model predicts that the noisy point has originated. This projection is shown in the bottom portion of Figure 5.10. The true manifold is a more concise representation of the data as compared to the noisy data, and this conciseness is a result of the regularization inherent in the addition of noise to the input. All forms of regularization tend to increase the conciseness of the underlying model.

### 5.10.3 Gradient-Based Penalization: Contractive Autoencoders

As in the case of the de-noising autoencoder, the hidden representation of the contractive autoencoder is often overcomplete, because the number of hidden units is greater than the number of input units. A contractive autoencoder is a heavily regularized encoder in which we do not want the hidden representation to change very significantly with small changes in input values. Obviously, this will also result in an output that is less sensitive to the input.

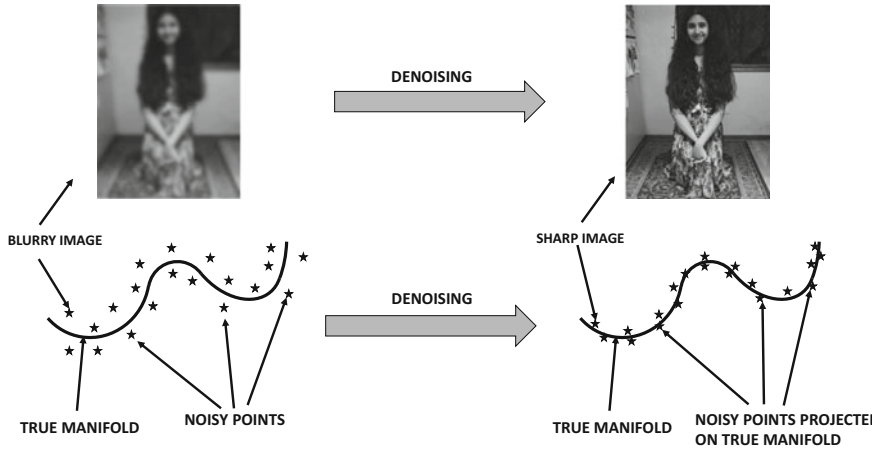



Figure 5.10: The de-noising autoencoder

Trying to create an autoencoder in which the output is less sensitive to changes in the input seems like an odd goal at first sight. After all, an autoencoder is supposed to reconstruct the data exactly. Therefore, the goals of regularization seem to be completely at odds with those of the contractive regularization portion of the loss function.

A key point is that contractive encoders are designed to be robust only to *small* changes in the input data. Furthermore, they tend to be insensitive to those changes that are inconsistent with the manifold structure of the data. In other words, if one makes a small change to the input that does not lie on the manifold structure of the input data, the contractive autoencoder will tend to damp the change in the reconstructed representation. Here, it is important to understand that the vast majority of (randomly chosen) directions in high-dimensional input data (with a much lower-dimensional manifold) tend to be approximately orthogonal to the manifold structure, which has the effect of changing the components of the change on the manifold structure. The damping of the changes in the reconstructive representation based on the local manifold structure is also referred to as the *contractive* property of the autoencoder. As a result, contractive autoencoders tend to remove noise from the input data (like de-noising autoencoders), although the mechanism for doing this is different from that of de-noising autoencoders. As we will see later, contractive autoencoders penalize the gradients of the hidden values with respect to the inputs. When the hidden values have low gradients with respect to the inputs, it means that they are not very sensitive to small changes in the inputs (although larger changes or changes parallel to manifold structure will tend to change the gradients).

For ease in discussion, we will discuss the case where the contractive autoencoder has a single hidden layer. The generalization to multiple hidden layers is straightforward. Let $h_1 \ldots h_k$ be the values of the $k$ hidden units for the input variables $x_1 \ldots x_d$. Let the reconstructed values in the output layer be given by $\hat{x}_1 \ldots \hat{x}_d$. Then, the objective function is given by the weighted sum of the reconstruction loss and the regularization term. The loss $L$ for a single training instance is given by the following:

$$L = \sum_{i=1}^{d} (x_i - \hat{x}_i)^2 \tag{5.14}$$

The regularization term is constructed by using the sum of the squares of the partial derivatives of all hidden variables with respect to all input dimensions. For a problem with $k$ hidden units denoted by $h_1 \ldots h_k$, the regularization term $R$ can be written as follows:

$$R = \frac{1}{2} \sum_{i=1}^{d} \sum_{j=1}^{k} \left( \frac{\partial h_j}{\partial x_i} \right)^2 \tag{5.15}$$

In the original paper [414], the sigmoid nonlinearity is used in the hidden layer, in which case the following can be shown (cf. section 2.4.2 of Chapter 2):

$$\frac{\partial h_j}{\partial x_i} = w_{ij} h_j (1 - h_j) \ \forall i, j \tag{5.16}$$

Here, $w_{ij}$ is the weight of the input unit $i$ to the hidden unit $j$.

The overall objective function for a single training instance is given by a weighted sum of the loss and the regularization terms.

$$\begin{aligned} J &= L + \lambda \cdot R \\ &= \sum_{i=1}^{d} (x_i - \hat{x}_i)^2 + \frac{\lambda}{2} \sum_{j=1}^{k} h_j^2 (1 - h_j)^2 \sum_{i=1}^{d} w_{ij}^2 \end{aligned}$$

This objective function contains a combination of weight and hidden unit regularization. Penalties on hidden units can be handled in the same way as discussed in section 2.6.5 of Chapter 2. Let $a_{h_j}$ be the pre-activation value for the node $h_j$. The backpropagation updates are traditionally defined in terms of the preactivation values, where the value of $\frac{\partial J}{\partial a_{h_j}}$ is propagated backwards. After $\frac{\partial J}{\partial a_{h_j}}$ is computed using the dynamic programming update of backpropagation from the output layer, one can further update it to incorporate the effect of hidden-layer regularization of $h_j$:

$$\begin{aligned} \frac{\partial J}{\partial a_{h_j}} &\Leftarrow \frac{\partial J}{\partial a_{h_j}} + \frac{\lambda}{2} \frac{\partial [h_j^2 (1 - h_j)^2]}{\partial a_{h_j}} \sum_{i=1}^{d} w_{ij}^2 \\ &= \frac{\partial J}{\partial a_{h_j}} + \lambda h_j (1 - h_j)(1 - 2h_j) \underbrace{\frac{\partial h_j}{\partial a_{h_j}}}_{h_j (1 - h_j)} \sum_{i=1}^{d} w_{ij}^2 \\ &= \frac{\partial J}{\partial a_{h_j}} + \lambda h_j^2 (1 - h_j)^2 (1 - 2h_j) \sum_{i=1}^{d} w_{ij}^2 \end{aligned}$$

The value of $\frac{\partial h_j}{\partial a_{h_j}}$ is set to $h_j(1 - h_j)$ because the sigmoid activation is assumed, although it would be different for other activations. According to the chain rule, the value of $\frac{\partial J}{\partial a_{h_j}}$ should be multiplied with the value of $\frac{\partial a_{h_j}}{\partial w_{ij}} = x_i$ to obtain the gradient of the loss with respect to $w_{ij}$. However, according to the *multivariable* chain rule, we also need to directly add the derivative of the regularizer with respect to $w_{ij}$ in order to obtain the full gradient. Therefore, the partial derivative of the hidden-layer regularizer $R$ with respect to the weight is added as follows:

$$\frac{\partial J}{\partial w_{ij}} \Leftarrow \frac{\partial J}{\partial a_{h_j}} \frac{\partial a_{h_j}}{\partial w_{ij}} + \lambda \frac{\partial R}{\partial w_{ij}}$$
$$= x_i \frac{\partial J}{\partial a_{h_j}} + \lambda w_{ij} h_j^2 (1 - h_j)^2$$

Interestingly, if a linear hidden unit is used instead of the sigmoid, it is easy to see that the objective function will become identical to that of an $L_2$-regularized autoencoder. Therefore, it makes sense to use this approach only with a nonlinear hidden layer, because a linear hidden layer can be handled in a much simpler way. The weights in the encoder and decoder can be either tied or independent. If the weights are tied then the gradients over both copies of a weight need to be added. The above discussion assumes a single hidden layer, although it is easy to generalize to more hidden layers. The work in [414] showed that better compression can be achieved with the use of deeper variants of the approach.

Some interesting relationships exist between the de-noising autoencoder and the contractive autoencoder. The de-noising autoencoder achieves its goals of robustness stochastically by explicitly adding noise, whereas a contractive autoencoder achieves its goals analytically by adding a regularization term. Adding a small amount of Gaussian noise in a de-noising autoencoder achieves roughly similar goals as a contractive autoencoder, when the hidden layer uses linear activation. When the hidden layer uses linear activation, the partial derivative of the hidden unit with respect to an input is simply the connecting weight, and therefore the objective function of the contractive autoencoder becomes the following:

$$J_{linear} = \sum_{i=1}^{d} (x_i - \hat{x}_i)^2 + \frac{\lambda}{2} \sum_{i=1}^{d} \sum_{j=1}^{k} w_{ij}^2 \tag{5.17}$$

In that case, both the contractive and the de-noising autoencoders become similar to regularized singular value decomposition with $L_2$-regularization. The difference between the de-noising autoencoder and the contractive autoencoder is visually illustrated in Figure 5.11. In the case of the de-noising autoencoder on the left, the autoencoder learns the directions along the true manifold of uncorrupted data by using the relationship between the corrupted data in the output and the true data in the input. This goal is achieved analytically in the contractive autoencoder, because the vast majority of random perturbations are roughly orthogonal to the manifold when the dimensionality of the manifold is much smaller than the input data dimensionality. In such a case, perturbing the data point slightly does not change the hidden representation along the manifold very much. Penalizing the partial derivative of the hidden layer equally along all directions ensures that the partial derivative is significant only along the small number of directions along the true manifold, and the partial derivatives along the vast majority of orthogonal directions are close to 0. In other words, the variations that are not meaningful to the distribution of the specific training data set at hand are damped, and only the meaningful variations are kept.

Another difference between the two methods is that the de-noising autoencoder shares the responsibility for regularization between the encoder and decoder, whereas the contractive autoencoder places this responsibility only on the encoder. Only the encoder portion is used in feature extraction; therefore, contractive autoencoders are more useful for feature engineering.

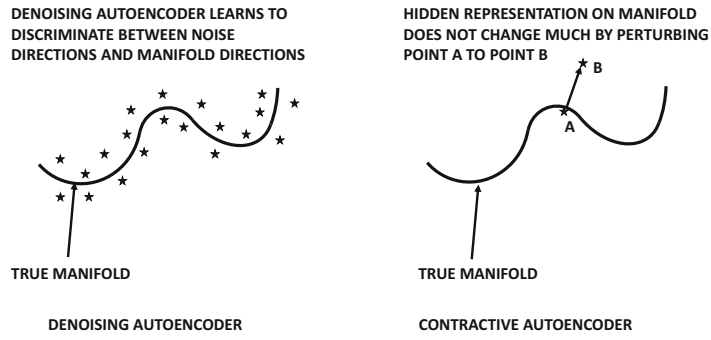
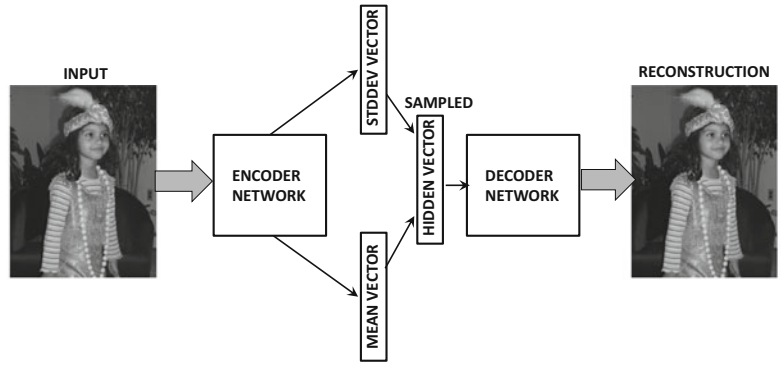



Figure 5.11: The difference between the de-noising and the contractive autoencoder

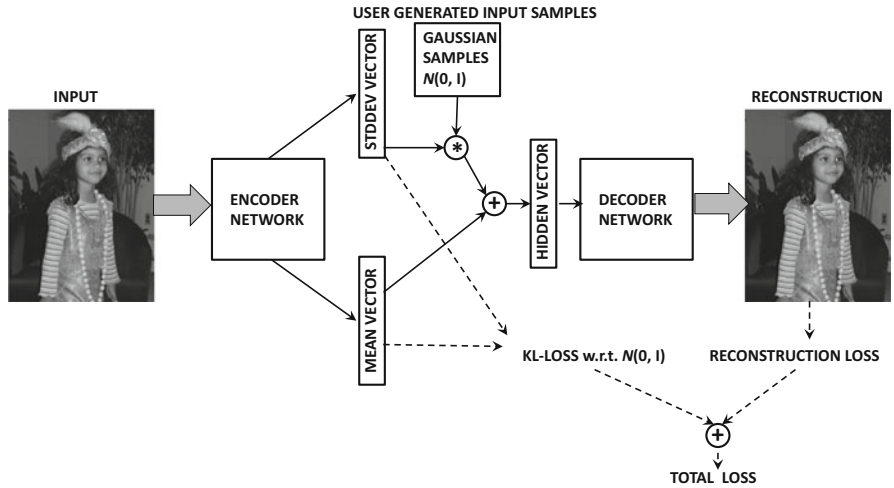
### 5.10.4 Hidden Probabilistic Structure: Variational Autoencoders

Just as sparse encoders impose a sparsity constraint on the hidden units, variational encoders impose a specific probabilistic structure on the hidden representation. Specifically, it is assumed that the hidden representation over the whole data should be drawn from the standard Gaussian distribution with zero mean and unit variance in each direction. By imposing this type of probabilistic structure on the hidden representation, the decoder becomes particularly useful for data generation — one can simply feed samples from the standard normal distribution to the decoder in order to generate very realistic samples of the training data. This is not possible in a traditional autoencoder, where the hidden space has arbitrary structure and a carelessly chosen sample from an unpopulated region of the hidden space might not decode into a realistic sample of the original data. The normal distribution assumption can be viewed as a *probabilistic prior*, although the *conditional* (or *posterior*) distribution of the activations in the hidden layer (after having seen a specific input object) would be a Gaussian with a different-from-zero mean and smaller-than-unit standard deviation (owing to greater knowledge of the object).

How does one control the hidden distribution with an appropriate loss function? A regularization term is used to pull the conditional Gaussian distribution (with respect to a single object) towards the standard normal distribution, which also has the effect of pulling the hidden representation of the whole data towards the standard normal distribution. The key is to use a re-parametrization approach in which the encoder creates the $k$-dimensional mean and standard deviations vector of the conditional Gaussian distribution and penalizes the KL-divergence of this conditional distribution from the standard normal distribution — the hidden vector for the specific object is sampled from this conditional Gaussian distribution as shown in Figure 5.12(a) in order to create its reconstruction using the decoder (whose loss is also penalized). Unfortunately, this network has a sampling component. The weights of such a network cannot be learned by backpropagation because the stochastic portions of the computations are not differentiable, and therefore backpropagation cannot be not used. Therefore, the stochastic part of it can be addressed by the user explicitly generating $k$-dimensional samples in which each component is drawn from the standard normal distribution. The mean and standard deviation output by the encoder are used to scale and translate the input sample from the Gaussian distribution. This architecture is shown in Figure 5.12(b). By generating the stochastic potion explicitly as a part of the input, the resulting architecture is now fully deterministic, and its weights can be learned by backpropagation. It is noteworthy that the backpropagation updates will depend on the




(a) Point-specific Gaussian distribution (stochastic and non-differentiable loss)




(b) Point-specific Gaussian distribution (deterministic and differentiable loss)

Figure 5.12: Re-parameterizing a variational autoencoder

stochastically sampled noise (although the effects of this noise will be muted over a large number of samples).

For each object $\overline{X}$, separate hidden activations for the mean and standard deviation are created by the encoder. The $k$-dimensional activations for the mean and standard deviation are denoted by $\overline{\mu}(\overline{X})$ and $\overline{\sigma}(\overline{X})$, respectively. In addition, a $k$-dimensional sample $\overline{z}$ is generated from $\mathcal{N}(0, I)$, where $I$ is the identity matrix, and treated as an input into the hidden layer by the user. The hidden representation $\overline{h}(\overline{X})$ is created by scaling this random input vector $\overline{z}$ with the mean and standard deviation as follows:

$$\overline{h}(\overline{X}) = \overline{z} \odot \overline{\sigma}(\overline{X}) + \overline{\mu}(\overline{X}) \tag{5.18}$$

Here, $\odot$ indicates element-wise multiplication. These operations are shown in Figure 5.12(b) with the little circles containing the multiplication and addition operators. The elements of the vector $\overline{h}(\overline{X})$ for a particular object will obviously diverge from the standard normal distribution unless the vectors $\overline{\mu}(\overline{X})$ and $\overline{\sigma}(\overline{X})$ contain only 0s and 1s, respectively. This will not be the case because of the reconstruction component of the loss, which forces the

conditional distributions of the hidden representations of particular points to have different means and lower standard deviations than that of the standard normal distribution (which is like a prior distribution). The distribution of the hidden representation of a particular point is a posterior distribution (conditional on the specific training data point), and therefore it will differ from the Gaussian prior. The overall loss function is expressed as a weighted sum of the reconstruction loss and the regularization loss. One can use a variety of choices for the reconstruction error, and for simplicity we will use the squared loss, which is defined as follows:

$$L = ||\overline{X} - \overline{X}'||^2 \tag{5.19}$$

Here, $\overline{X}'$ is the reconstruction of the input point $\overline{X}$ from the decoder. The regularization loss $R$ is simply the Kullback-Leibler (KL)-divergence measure of the conditional hidden distribution with parameters $(\overline{\mu}(\overline{X}), \overline{\sigma}(\overline{X}))$ with respect to the $k$-dimensional spherical Gaussian distribution with unit variance in each direction. The KL-divergence term is defined as follows:

$$R = \frac{1}{2}\left( \underbrace{||\overline{\mu}(\overline{X})||^2}_{\overline{\mu}(\overline{X})_i \Rightarrow 0} + \underbrace{||\overline{\sigma}(\overline{X})||^2 - 2\sum_{i=1}^{k} \ln(\overline{\sigma}(\overline{X})_i) - k}_{\overline{\sigma}(\overline{X})_i \Rightarrow 1} \right) \tag{5.20}$$

The overall objective function $J$ for the data point $\overline{X}$ is defined as the weighted sum of the reconstruction loss and the regularization term:

$$J = L + \lambda R \tag{5.21}$$

Here, $\lambda > 0$ is the regularization parameter. Small values of $\lambda$ will favor exact reconstruction like a traditional autoencoder. The regularization term takes on its minimum value of 0, when $(\overline{\mu}(\overline{X}), \overline{\sigma}(\overline{X})) = (\overline{0}, \overline{1})$ corresponds to the standard Gaussian with zero mean and unit variance. However, the reconstruction portion of the objective function will push $\overline{\mu}(\overline{X})$ towards non-zero values favoring faithful reconstruction of $\overline{X}$ and the components of $\overline{\sigma}(\overline{X})$ to values smaller than 1 (favoring the addition of less point-specific noise to the hidden representation before reconstruction). Even though the *point-specific* posteriors represented by $(\overline{\mu}(\overline{X}), \overline{\sigma}(\overline{X}))$ will be far from the standard Gaussian, the distribution of the hidden representation *over all training points* will be much closer to the standard Gaussian because of the regularization term.

The regularization term forces the hidden representations to be stochastic, so that multiple hidden representations generate almost the same point. This increases generalization power because it is easier to model a new image that is like (but not an exact likeness of) an image in the training data within a stochastic range of hidden values. However, since there will be overlaps among the distributions of the hidden representations of similar points, it has some undesirable side effects. For example, the reconstructions of image data points tend to be blurry. This is caused by an averaging effect over somewhat similar images. In the extreme case, if the value of $\lambda$ is chosen to be exceedingly large, all points will have the same hidden (posterior) distribution — the isotropic Gaussian with zero mean and unit variance. The resulting reconstruction results will exhibit a gross averaging effect over the entire training data, which will obviously not be meaningful.

**Training the Variational Autoencoder**

The training of a variational autoencoder is relatively straightforward because the stochasticity has been pulled out as an additional input. One can backpropagate as in any traditional neural network. The only difference is that one needs to backpropagate across the unusual form of Equation 5.18. Furthermore, one needs to account for the penalties of the hidden layer during backpropagation.

First, one can backpropagate the loss $L$ up to the hidden state $\overline{h}(\overline{X}) = (h_1 \ldots h_k)$ using traditional methods. Let $\overline{z} = (z_1 \ldots z_k)$ be the $k$ random samples from $\mathcal{N}(0, 1)$, which are used in the current iteration. In order to backpropagate from $\overline{h}(\overline{X})$ to $\overline{\mu}(\overline{X}) = (\mu_1 \ldots \mu_k)$ and $\overline{\sigma}(\overline{X}) = (\sigma_1 \ldots \sigma_k)$, one can use the following relationship:

$$J = L + \lambda R \tag{5.22}$$

$$\frac{\partial J}{\partial \mu_i} = \frac{\partial L}{\partial h_i} \underbrace{\frac{\partial h_i}{\partial \mu_i}}_{=1} + \lambda \frac{\partial R}{\partial \mu_i} \tag{5.23}$$

$$\frac{\partial J}{\partial \sigma_i} = \frac{\partial L}{\partial h_i} \underbrace{\frac{\partial h_i}{\partial \sigma_i}}_{=z_i} + \lambda \frac{\partial R}{\partial \sigma_i} \tag{5.24}$$

The values below the under-braces show the evaluations of partial derivatives of $h_i$ with respect to $\mu_i$ and $\sigma_i$, respectively. Note that the values of $\frac{\partial h_i}{\partial \mu_i} = 1$ and $\frac{\partial h_i}{\partial \sigma_i} = z_i$ are obtained by differentiating Equation 5.18 with respect to $\mu_i$ and $\sigma_i$, respectively. The value of $\frac{\partial L}{\partial h_i}$ on the right-hand side is available from backpropagation. The values of $\frac{\partial R}{\partial \mu_i}$ and $\frac{\partial R}{\partial \sigma_i}$ are straightforward derivatives of the KL-divergence in Equation 5.20. Subsequent error propagation from the activations for $\overline{\mu}(\overline{X})$ and $\overline{\sigma}(\overline{X})$ can proceed in a similar way to the normal workings of the backpropagation algorithm.

The architecture of the variational autoencoder is considered fundamentally different from other types of autoencoders because it models the hidden variables in a stochastic way. However, there are still some interesting connections. In the de-noising autoencoder, one adds noise to the input; however, there is no constraint on the shape of the hidden distribution. In the variational autoencoder, one works with a stochastic hidden representation, although the stochasticity is pulled out by using it as an additional input during training. In other words, noise is added to the hidden representation rather than the input data. The variational approach improves generalization, because it encourages each input to map to its own stochastic region in the hidden space rather than mapping it to a single point. Small changes in the hidden representation, therefore, do not change the reconstruction too much. This assertion would also be true with a contractive autoencoder. However, constraining the shape of the hidden distribution to be Gaussian is a more fundamental difference of the variational autoencoder from other types of transformations.

#### 5.10.4.1 Reconstruction and Generative Sampling

The approach can be used for creating the reduced representations as well as generating samples. In the case of data reduction, a Gaussian distribution with mean $\overline{\mu}(\overline{X})$ and standard deviation $\overline{\sigma}(\overline{X})$ is obtained, which represents the distribution of the hidden representation.

However, a particularly interesting application of the variational autoencoder is to generate samples from the underlying data distribution. Just as feature engineering methods

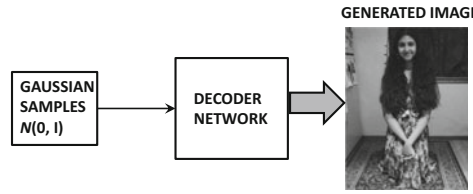



Figure 5.13: Generating samples from the variational autoencoder. The images are illustrative only.

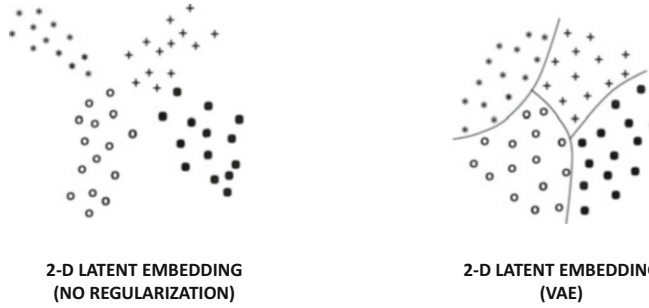



Figure 5.14: Illustrations of the embeddings created by a variational autoencoder in relation to the unregularized version. The unregularized version has large discontinuities in the latent space, which might not correspond to meaningful points. The Gaussian embedding of the points in the variational autoencoder makes sampling possible.

use only the encoder portion of the autoencoder (once training is done), variational autoencoders use only the decoder portion. The basic idea is to repeatedly draw a point from the Gaussian distribution and feed it to the hidden units in the decoder. The resulting "reconstruction" output of the decoder will be a point satisfying a similar distribution as the original data. As a result, the generated point will be a realistic sample from the original data. The architecture for sample generation is shown in Figure 5.13. The shown image is illustrative only, and does not reflect the actual output of a variational autoencoder (which is generally of somewhat lower quality). To understand why a variational autoencoder can generate images in this way, it is helpful to view the typical types of embeddings an unregularized autoencoder would create versus a method like the variational autoencoder. In the left side of Figure 5.14, we have shown an example of the 2-dimensional embeddings of the training data created by an unregularized autoencoder of a four-class distribution (e.g., four digits of MNIST). It is evident that there are large discontinuities in particular regions of the latent space, and that these sparse regions may not correspond to meaningful points. On the other hand, the regularization term in the variational autoencoder encourages the training points to be (roughly) distributed in a Gaussian distribution, and there are far fewer discontinuities in the embedding on the right-hand side of Figure 5.14. Consequently, sampling from any point in the latent space will yield meaningful reconstructions of one of the four classes (i.e., one of the digits of MNIST). Furthermore, "walking" from one point in the latent space to another along a straight line in the second case will result in a smooth transformation across classes. For example, walking from a region containing instances of '4' to a region containing instances of '7' in the latent space of the MNIST data set would result in a slow change in the style of the digit '4' until a transition point, where the handwritten digit could be interpreted either as a '4' or a '7'. This situation does occur in real

settings as well because such types of confusing handwritten digits do occur in the MNIST data set. Furthermore, the placement of different digits within the embedding would be such that digit pairs with smooth transitions at confusion points (e.g., [4, 7] or [5, 6]) are placed adjacent to one another in the latent space.

It is important to understand that the generated objects are often similar to but not exactly the same as those drawn from the training data. Because of its stochastic nature, the variational autoencoder has the ability to explore different modes of the generation process, which leads to a certain level of creativity in the face of ambiguity. This property can be put to good use by conditioning the approach on another object.

#### 5.10.4.2 Conditional Variational Autoencoders

One can apply conditioning to variational autoencoders in order to obtain some interesting results [479, 530]. The basic idea in conditional variational autoencoders is to add an additional conditional input, which typically provides a related context. For example, the context might be a damaged image with missing holes, and the job of the autoencoder is to reconstruct it. Predictive models will generally perform poorly in this type of setting because the level of ambiguity may be too large, and an averaged reconstruction across all images might not be useful. During the training phase, pairs of damaged and original images are needed, and therefore the encoder and decoder are able to learn how the context relates to the images being generated from the training data. The architecture of the training phase is illustrated in the upper part of Figure 5.15. The training is otherwise similar to the unconditional variational autoencoder. During the testing phase, the context is provided as an additional input, and the autoencoder reconstructs the missing portions in a reasonable way based on the model learned in the training phase. The architecture of the reconstruction phase is illustrated in the lower part of Figure 5.15. The simplicity of this architecture is particularly notable. The shown images are only illustrative; in actual executions on image data, the generated images are often blurry, especially in the missing portions. This is a type of image-to-image translation approach, which will be revisited in Chapter 12 under the context of a discussion on *generative adversarial networks*.

#### 5.10.4.3 Relationship with Generative Adversarial Networks

Variational autoencoders are closely related to another class of models, referred to as generative adversarial networks. However, there are some key differences as well. Like variational autoencoders, generative adversarial networks can be used to create images that are similar to a base training data set. Furthermore, conditional variants of both models are useful for completing missing data, especially in cases where the ambiguity is large enough to require a certain level of creativity from the generative process. However, the results of generative adversarial networks are often more realistic because the decoders are explicitly trained to create good counterfeits. This is achieved by having a discriminator as a judge of the quality of the generated objects. Furthermore, the objects are also generated in a more creative way because the generator is never shown the original objects in the training data set, but is only given guidance to fool the discriminator. As a result, generative adversarial networks learn to create creative counterfeits. In certain domains such as image and video data, this approach can have remarkable results; unlike variational autoencoders, the quality of the images is not blurry. One can create vivid images and videos with an artistic flavor, that give the impression of dreaming. These techniques can also be used in numerous applications like text-to-image or image-to-image translation. For example, one can specify a text

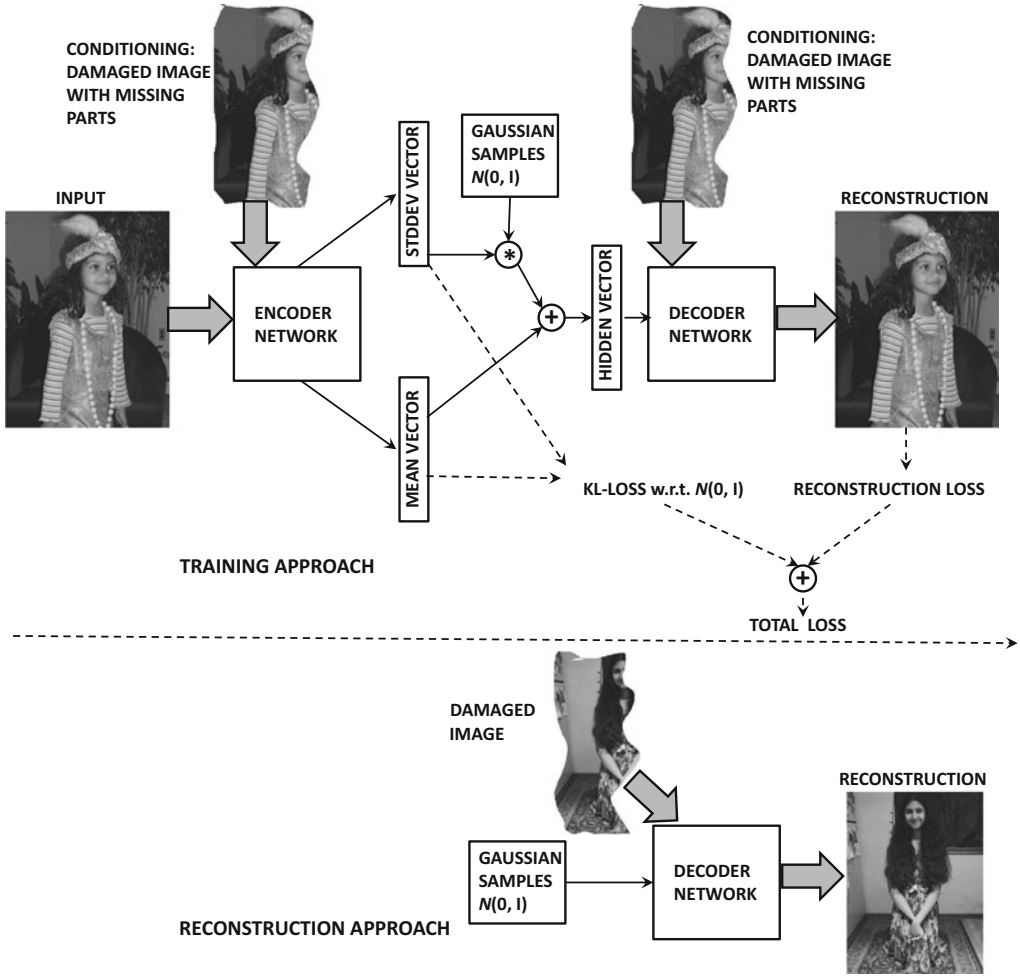



Figure 5.15: Reconstructing damaged images with the conditional variational autoencoder. The images are illustrative only.

description, and then obtain a fantasy image that matches the description [408]. Generative adversarial networks are discussed in section 12.5 of Chapter 12.

## 5.11 Summary

Large neural networks often face the challenge of overfitting. Reducing network size up front results in suboptimal solutions when the underlying model is complex. A more flexible approach is to use tunable regularization, in which a large number of parameters are allowed. In such cases, the regularization restricts the size of the parameter space in a soft way. The most common form of regularization is penalty-based regularization on either the parameters or the hidden units. Ensemble learning is a common approach to reduce variance, and some ensemble methods like *Dropout* are specifically designed for neural networks. Other common regularization methods include early stopping and pretraining. Pretraining acts as

a regularizer by acting as a form of semi-supervised learning, which works from the simple to the complex by initializing with a simple heuristic and using backpropagation to discover refined solutions. Other related techniques include curriculum and continuation methods, which also work from the simple to the complex in order to provide solutions with low generalization error. Although overfitting is a less serious problem in unsupervised settings, regularization is often used to impose structure on the learned representations.

## 5.12 Bibliographic Notes and Software Resources

A detailed discussion of the bias-variance trade-off may be found in [187]. The bias-variance trade-off originated in the field of statistics, where it was proposed in the context of the regression problem. The generalization to the case of binary loss functions in classification was proposed in [257, 260]. Early methods for reducing overfitting were proposed in [185, 288] in which unimportant weights were removed from a network to reduce its parameter footprint. It was shown that this type of pruning had significant benefits in terms of generalization. The early work also showed [470] that deep and narrow networks tended to generalize better than broad and shallow networks. This is primarily because the depth imposes a structure on the data, and can represent the data in a fewer number of parameters. A recent study of model generalization in neural networks is provided in [582].

The use of $L_2$-regularization in regression dates back to Tikhonov-Arsenin's seminal work [516]. The equivalence of Tikhonov regularization and training with noise was shown by Bishop [43]. The use of $L_1$-regularization is studied in detail in [189]. Several regularization methods have also been proposed that are specifically designed for neural architectures. For example, the work in [211] proposes a regularization technique that constrains the norm of each layer in a neural network. Sparse representations of the data are explored in [69, 279, 280, 290, 365].

Detailed discussions of ensemble methods for classification may be found in [458, 591]. The bagging and subsampling methods are discussed in [49, 56]. The work in [535] proposes an ensemble architecture that is inspired by a random forest. This architecture is illustrated in Figure 2.10 of Chapter 1. This type of ensemble is particularly well suited for problems with small data sets, where a random forest is known to work well. The approach for random edge dropping was introduced in the context of outlier detection [64], whereas the *Dropout* approach was presented in [483]. The work in [592] discusses the notion that it is better to combine the results of the top-performing ensemble components rather than combining all of them. Most ensemble methods are designed for variance reduction, although a few techniques like *boosting* [127] are also designed for bias reduction. Boosting has also been used in the context of neural network learning [455]. However, the use of boosting in neural networks is generally restricted to the incremental addition of hidden units based on error characteristics. A key point about boosting is that it tends to overfit the data, and is therefore suitable for high-bias learners but not high-variance learners. Neural networks are inherently high-variance learners. The relationship between boosting and certain types of neural architectures is pointed out in [32]. Combinations of data perturbation methods with ensembles are discussed in [5]. Ensemble methods for neural networks are proposed in [180].

Different types of pretraining have been explored in the context of neural networks [31, 116, 206, 402, 526]. The earliest methods for unsupervised pretraining were proposed in [206]. The original work of pretraining [206] was based on probabilistic graphical models (cf. section 7.7) and was later extended to conventional autoencoders [402, 526].

Compared to unsupervised pretraining, the effect of supervised pretraining is limited [31]. A detailed discussion of why unsupervised pretraining helps deep learning is provided in [116]. This work posits that unsupervised pretraining implicitly acts as a regularizer, and therefore it improves the generalization power to unseen test instances. This fact is also evidenced by the experimental results in [31], which show that supervised variations of pretraining do not help as much as unsupervised variations of pretraining. Pretraining also does not seem to help with certain types of tasks [312]. Another form of semi-supervised learning can be performed with *ladder networks* [404, 520], in which skip-connections are used in conjunction with an autoencoder-like architecture.

Curriculum and continuation learning are applications of the principle of moving from simple to complex models. Continuation learning methods are discussed in [350, 557]. A number of methods were proposed in the early years [114, 439, 480] that showed the advantages of curriculum learning. The basic principles of curriculum learning are discussed in [248]. The relationship between curriculum and continuation learning is explored in [33].

Sparse autoencoders are discussed in [69, 279, 280, 290, 365]. De-noising autoencoders are discussed in [526]. The contractive autoencoder is discussed in [414]. The use of de-noising autoencoders in recommender systems is discussed in [488, 555]. The ideas in the contractive autoencoder are reminiscent of *double backpropagation* [109] in which small changes in the input are not allowed to change the output. Related ideas are also discussed in the *tangent classifier* [415].

The variational autoencoder was introduced in [252, 416]. The use of importance weighting to improve over the representations learned by the variational autoencoder is discussed in [58]. Conditional variational autoencoders are discussed in [479, 530]. A tutorial on variational autoencoders is found in [108]. Generative variants of de-noising autoencoders are discussed in [34]. Variational autoencoders are closely related to generative adversarial networks, which are discussed in Chapter 12. Closely related methods for designing adversarial autoencoders are discussed in [323].

### Software Resources

Numerous ensemble methods are available from machine learning libraries like *scikit-learn* [611]. Most of the weight-decay and penalty-based methods are available as standardized options in the deep learning libraries. However, techniques like *Dropout* are application-specific and need to be implemented from scratch. Implementations of several different types of autoencoders may be found in [618]. Several implementations of the variational autoencoder may be found in [619, 620, 662].

## 5.13 Exercises

1. Consider two neural networks used for regression modeling, each with an input layer, 10 hidden layers containing 100 units each, and a single linear output unit. The only difference is that one of them uses linear activations in the hidden layers and the other uses sigmoid activations. Which model will have higher variance of prediction?

2. Consider a situation in which you have four attributes $x_1 \ldots x_4$, and the dependent variable $y$ is such that $y = 2x_1$. Create a tiny training data set of 5 distinct examples in which a linear regression model without regularization will have an infinite number of coefficient solutions with $w_1 = 0$. Discuss the performance of such a model on out-of-sample data. Why will regularization help?

**3.** Implement a perceptron with and without regularization. Test the accuracy of both variations on the training data and the out-of-sample data on the *Ionosphere* data set of the *UCI Machine Learning Repository* [625]. What is the effect of regularization in the two cases? Repeat the experiment with smaller samples of the *Ionosphere* training data, and report your observations.

**4.** Implement an autoencoder with a single hidden layer. Reconstruct inputs for the *Ionosphere* data set of the previous exercise with (a) no added noise and weight regularization, (b) added Gaussian noise and no weight regularization.

**5.** The discussion in the chapter uses an example of sigmoid activation for the contractive autoencoder. Consider a contractive autoencoder with a single hidden layer and ReLU activation. Discuss how the updates change when ReLU activation is used.

**6.** Suppose that you have a model that provides around 80% accuracy on the training as well as on the out-of-sample test data. Would you recommend increasing the amount of data or adjusting the model to improve accuracy?

**7.** In the chapter, we showed that adding Gaussian noise to the input features in linear regression is equivalent to $L_2$-regularization of linear regression. Discuss why adding of Gaussian noise to the input data in a de-noising single-hidden layer autoencoder with linear units is roughly equivalent to $L_2$-regularized singular value decomposition.

**8.** Consider a network with a single input layer, two hidden layers, and a single output predicting a binary label. All hidden layers use the sigmoid activation function and no regularization is used. The input layer contains $d$ units, and each hidden layer contains $p$ units. Suppose that you add an additional hidden layer between the two current hidden layers, and this additional hidden layer contains $q$ linear units.

(a) Even though the number of parameters have increased by adding the hidden layer, discuss why the capacity of this model will decrease when $q < p$.

(b) Does the capacity of the model increase when $q > p$?

**9.** Bob divided the labeled classification data into a portion used for model construction and another portion for validation. Bob then tested 1000 neural architectures by learning parameters (backpropagating) on the model-construction portion and testing its accuracy on the validation portion. Discuss why the resulting model is likely to yield poorer accuracy on the out-of-sample test data as compared to the validation data, even though the validation data was not used for learning parameters. Do you have any recommendations for Bob on using the results of his 1000 validations?

**10.** Does the classification accuracy on the training data generally improve with increasing training data size? How about the point-wise average of the loss on training instances? For what types of data sizes do training and testing accuracy become similar? Explain your answer.

**11.** What is the effect of increasing the regularization parameter on the training and testing accuracy? For what types of regularization parameters do training and testing accuracy become similar?

**12.** Consider an autoencoder with $k$ real-valued units $h_1 \ldots h_k$ in the hidden layer. In addition to the least-squares loss $L$, you apply the following regularization term $R$ in order to create the objective function $L + \lambda R$ for regularization parameter $\lambda > 0$:

$$R = \sum_{i=1}^{k} \max\{1 - h_i, 0\}$$

Discuss the effect of this regularization on the hidden representation. [Hint: Hinge Loss]

# Bibliography


[1] D. Ackley, G. Hinton, and T. Sejnowski. A learning algorithm for Boltzmann machines. *Cognitive Science*, 9(1), pp. 147–169, 1985.

[2] C. Aggarwal. Data classification: Algorithms and applications, *CRC Press*, 2014.

[3] C. Aggarwal. Data mining: The textbook. *Springer*, 2015.

[4] C. Aggarwal. Recommender systems: The textbook. *Springer*, 2016.

[5] C. Aggarwal. Outlier analysis. *Springer*, 2017.

[6] C. Aggarwal. Linear algebra and optimization for machine learning: A Textbook. *Springer*, 2020.

[7] C. Aggarwal. Machine learning for text. *Springer*, 2018.

[8] R. Ahuja, T. Magnanti, and J. Orlin. Network flows: Theory, algorithms, and applications. *Prentice Hall*, 1993.

[9] E. Aljalbout, V. Golkov, Y. Siddiqui, and D. Cremers. Clustering with deep learning: Taxonomy and new methods. *arXiv:1801.07648*, 2018. https://arxiv.org/abs/1801.07648

[10] R. Al-Rfou, B. Perozzi, and S. Skiena. Polyglot: Distributed word representations for multilingual nlp. *arXiv:1307.1662*, 2013. https://arxiv.org/abs/1307.1662

[11] D. Amodei *at al.* Concrete problems in AI safety. *arXiv:1606.06565*, 2016. https://arxiv.org/abs/1606.06565

[12] M. Arjovsky and L. Bottou. Towards principled methods for training generative adversarial networks. *arXiv:1701.04862*, 2017. https://arxiv.org/abs/1701.04862

[13] M. Arjovsky, S. Chintala, and L. Bottou. Wasserstein gan. *arXiv:1701.07875*, 2017. https://arxiv.org/abs/1701.07875

[14] J. Ba and R. Caruana. Do deep nets really need to be deep? *NeurIPS*, pp. 2654–2662, 2014.




C. C. Aggarwal, *Neural Networks and Deep Learning*,
https://doi.org/10.1007/978-3-031-29642-0

[15] J. Ba, J. Kiros, and G. Hinton. Layer normalization. *arXiv:1607.06450*, 2016. https://arxiv.org/abs/1607.06450

[16] J. Ba, V. Mnih, and K. Kavukcuoglu. Multiple object recognition with visual attention. *arXiv: 1412.7755*, 2014. https://arxiv.org/abs/1412.7755

[17] A. Babenko, A. Slesarev, A. Chigorin, and V. Lempitsky. Neural codes for image retrieval. *arXiv:1404.1777*, 2014. https://arxiv.org/abs/1404.1777

[18] M. Baccouche, F. Mamalet, C. Wolf, C. Garcia, and A. Baskurt. Sequential deep learning for human action recognition. *International Workshop on Human Behavior Understanding*, pp. 29–39, 2011.

[19] D. Bahdanau, K. Cho, and Y. Bengio. Neural machine translation by jointly learning to align and translate. *ICLR*, 2015. https://arxiv.org/abs/1409.0473

[20] B. Baker, O. Gupta, N. Naik, and R. Raskar. Designing neural network architectures using reinforcement learning. *arXiv:1611.02167*, 2016. https://arxiv.org/abs/1611.02167

[21] P. Baldi, S. Brunak, P. Frasconi, G. Soda, and G. Pollastri. Exploiting the past and the future in protein secondary structure prediction. *Bioinformatics*, 15(11), pp. 937–946, 1999.

[22] N. Ballas, L. Yao, C. Pal, and A. Courville. Delving deeper into convolutional networks for learning video representations. *arXiv:1511.06432*, 2015. https://arxiv.org/abs/1511.06432

[23] J. Baxter, A. Tridgell, and L. Weaver. Knightcap: a chess program that learns by combining td (lambda) with game-tree search. *arXiv cs/9901002*, 1999. https://arxiv.org/abs/cs/9901002

[24] M. Bazaraa, H. Sherali, and C. Shetty. Nonlinear programming: theory and algorithms. *John Wiley and Sons*, 2013.

[25] S. Becker, and Y. LeCun. Improving the convergence of back-propagation learning with second order methods. *Proceedings of the 1988 connectionist models summer school*, pp. 29–37, 1988.

[26] M. Bellemare, Y. Naddaf, J. Veness, and M. Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47, pp. 253–279, 2013.

[27] R. E. Bellman. Dynamic Programming. *Princeton University Press*, 1957.

[28] Y. Bengio. Learning deep architectures for AI. *Foundations and Trends in Machine Learning*, 2(1), pp. 1–127, 2009.

[29] Y. Bengio, A. Courville, and P. Vincent. Representation learning: A review and new perspectives. *IEEE TPAMI*, 35(8), pp. 1798–1828, 2013.

[30] Y. Bengio and O. Delalleau. Justifying and generalizing contrastive divergence. *Neural Computation*, 21(6), pp. 1601–1621, 2009.

[31] Y. Bengio, P. Lamblin, D. Popovici, and H. Larochelle. Greedy layer-wise training of deep networks. *NeurIPS*, 19, 153, 2007.

[32] Y. Bengio, N. Le Roux, P. Vincent, O. Delalleau, and P. Marcotte. Convex neural networks. *NeurIPS*, pp. 123–130, 2005.

[33] Y. Bengio, J. Louradour, R. Collobert, and J. Weston. Curriculum learning. *ICML*, 2009.

[34] Y. Bengio, L. Yao, G. Alain, and P. Vincent. Generalized denoising auto-encoders as generative models. *NeurIPS*, pp. 899–907, 2013.

[35] J. Bergstra *et al.* Theano: A CPU and GPU math compiler in Python. *Python in Science Conference*, 2010.

[36] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kegl. Algorithms for hyper-parameter optimization. *NeurIPS*, pp. 2546–2554, 2011.

[37] J. Bergstra and Y. Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13, pp. 281–305, 2012.

[38] J. Bergstra, D. Yamins, and D. Cox. Making a science of model search: Hyperparameter optimization in hundreds of dimensions for vision architectures. *ICML*, pp. 115–123, 2013.

[39] D. Bertsekas. Nonlinear programming *Athena Scientific*, 1999.

[40] C. M. Bishop. Pattern recognition and machine learning. *Springer*, 2007.

[41] C. M. Bishop. Neural networks for pattern recognition. *Oxford University Press*, 1995.

[42] C. M Bishop. Improving the generalization properties of radial basis function neural networks. *Neural Computation*, 3(4), pp. 579–588, 1991.

[43] C. M. Bishop. Training with noise is equivalent to Tikhonov regularization. *Neural computation*, 7(1),pp. 108–116, 1995.

[44] C. M. Bishop, M. Svensen, and C. K. Williams. GTM: A principled alternative to the self-organizing map. *NeurIPS*, pp. 354–360, 1997.

[45] M. Bojarski *et al.* End to end learning for self-driving cars. *arXiv:1604.07316*, 2016. https://arxiv.org/abs/1604.07316

[46] M. Bojarski *et al.* Explaining How a Deep Neural Network Trained with End-to-End Learning Steers a Car. *arXiv:1704.07911*, 2017. https://arxiv.org/abs/1704.07911

[47] H. Bourlard and Y. Kamp. Auto-association by multilayer perceptrons and singular value decomposition. *Biological Cybernetics*, 59(4), pp. 291–294, 1988.

[48] L. Breiman. Random forests. *Machine Learning*, 45(1), pp. 5–32, 2001.

[49] L. Breiman. Bagging predictors. *Machine Learning*, 24(2), pp. 123–140, 1996.

[50] T. Brown *et al.* Language models are few-shot learners. *arXiv:2005.14165*, 2020. https://arxiv.org/abs/2005.14165

[51] D. Broomhead and D. Lowe. Multivariable functional interpolation and adaptive networks. *Complex Systems*, 2, pp. 321–355, 1988.

[52] C. Browne *et al.* A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1), pp. 1–43, 2012.

[53] T. Brox and J. Malik. Large displacement optical flow: descriptor matching in variational motion estimation. *IEEE TPAMI*, 33(3), pp. 500–513, 2011.

[54] A. Bryson. A gradient method for optimizing multi-stage allocation processes. *Harvard University Symposium on Digital Computers and their Applications*, 1961.

[55] C. Bucilu, R. Caruana, and A. Niculescu-Mizil. Model compression. *KDD*, pp. 535–541, 2006.

[56] P. Bühlmann and B. Yu. Analyzing bagging. *Annals of Statistics*, pp. 927–961, 2002.

[57] M. Buhmann. Radial Basis Functions: Theory and implementations. *Cambridge University Press*, 2003.

[58] Y. Burda, R. Grosse, and R. Salakhutdinov. Importance weighted autoencoders. *arXiv:1509.00519*, 2015. https://arxiv.org/abs/1509.00519

[59] N. Butko and J. Movellan. I-POMDP: An infomax model of eye movement. *IEEE International Conference on Development and Learning*, pp. 139–144, 2008.

[60] Y. Cao, Y. Chen, and D. Khosla. Spiking deep convolutional neural networks for energy-efficient object recognition. *International Journal of Computer Vision*, 113(1), 54–66, 2015.

[61] M. Carreira-Perpinan and G. Hinton. On Contrastive Divergence Learning. *AISTATS*, 10, pp. 33–40, 2005.

[62] A. Chakraborty *et al.* Adversarial attacks and defences: A survey. *arXiv:1810.00069* 2018. https://arxiv.org/abs/1810.00069

[63] S. Chang, W. Han, J. Tang, G. Qi, C. Aggarwal, and T. Huang. Heterogeneous network embedding via deep architectures. *KDD*, pp. 119–128, 2015.

[64] J. Chen, S. Sathe, C. Aggarwal, and D. Turaga. Outlier detection with autoencoder ensembles. *SDM Conference*, 2017.

[65] J. Chen, J. Zhu, and L. Song. Stochastic training of graph convolutional networks with variance reduction. *arXiv:1710.10568*, 2017. https://arxiv.org/abs/1710.10568

[66] J. Chen, T. Ma, and C. Xiao. Fastgcn: fast learning with graph convolutional networks via importance sampling. *arXiv:1801.10247*, 2018. https://arxiv.org/abs/1801.10247

[67] S. Chen, C. Cowan, and P. Grant. Orthogonal least-squares learning algorithm for radial basis function networks. *IEEE TNNLS*, 2(2), pp. 302–309, 1991.

[68] W. Chen *et al.* Compressing neural networks with the hashing trick. *ICML*, 2015.

[69] Y. Chen and M. Zaki. KATE: K-Competitive Autoencoder for Text. *KDD*, 2017.

[70] Y. Chen, T. Krishna, J. Emer, and V. Sze. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. *IEEE Journal of Solid-State Circuits*, 52(1), pp. 127–138, 2017.

[71] R. Child *et al.* Generating long sequences with sparse transformers. *arXiv:1904.10509*, 2019.

[72] K. Cho, B. Merrienboer, C. Gulcehre, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. *EMNLP*, 2014. https://arxiv.org/abs/1406.1078

[73] J. Chorowski, D. Bahdanau, D. Serdyuk, K. Cho, and Y. Bengio. Attention-based models for speech recognition. *NeurIPS*, pp. 577–585, 2015.

[74] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv:1412.3555*, 2014. https://arxiv.org/abs/1412.3555

[75] D. Ciresan, U. Meier, L. Gambardella, and J. Schmidhuber. Deep, big, simple neural nets for handwritten digit recognition. *Neural Computation*, 22(12), pp. 3207–3220, 2010.

[76] C. Clark and A. Storkey. Training deep convolutional neural networks to play go. *ICML*, pp. 1766–1774, 2015.

[77] A. Coates, B. Huval, T. Wang, D. Wu, A. Ng, and B. Catanzaro. Deep learning with COTS HPC systems. *ICML*, pp. 1337–1345, 2013.

[78] A. Coates and A. Ng. The importance of encoding versus training with sparse coding and vector quantization. *ICML*, pp. 921–928, 2011.

[79] A. Coates and A. Ng. Learning feature representations with k-means. *Neural networks: Tricks of the Trade*, Springer, pp. 561–580, 2012.

[80] A. Coates, A. Ng, and H. Lee. An analysis of single-layer networks in unsupervised feature learning. *AAAI*, pp. 215–223, 2011.

[81] R. Collobert *et al.* Natural language processing (almost) from scratch. *Journal of Machine Learning Research*, 12, pp. 2493–2537, 2011.

[82] R. Collobert and J. Weston. A unified architecture for natural language processing: Deep neural networks with multitask learning. *ICML*, pp. 160–167, 2008.

[83] J. Connor, R. Martin, and L. Atlas. Recurrent neural networks and robust time series prediction. *IEEE TNNLS*, 5(2), pp. 240–254, 1994.

[84] T. Cooijmans, N. Ballas, C. Laurent, C. Gulcehre, and A. Courville. Recurrent batch normalization. *arXiv:1603.09025*, 2016. https://arxiv.org/abs/1603.09025

[85] C. Cortes and V. Vapnik. Support-vector networks. *Machine Learning*, 20(3), pp. 273–297, 1995.

[86] M. Courbariaux, Y. Bengio, and J.-P. David. BinaryConnect: Training deep neural networks with binary weights during propagations. *arXiv:1511.00363*, 2015. https://arxiv.org/abs/1511.00363

[87] T. Cover. Geometrical and statistical properties of systems of linear inequalities with applications to pattern recognition. *IEEE Transactions on Electronic Computers*, pp. 326–334, 1965.

[88] D. Cox and N. Pinto. Beyond simple features: A large-scale feature search approach to unconstrained face recognition. *Face and Gesture Recognition*, pp. 8–15, 2011.

[89] G. Dahl, R. Adams, and H. Larochelle. Training restricted Boltzmann machines on word observations. *arXiv:1202.5695*, 2012. https://arxiv.org/abs/1202.5695

[90] N. Dalal and B. Triggs. Histograms of oriented gradients for human detection. *CVPR*, pp. 886–893, 2005.

[91] Y. Dauphin, R. Pascanu, C. Gulcehre, K. Cho, S. Ganguli, and Y. Bengio. Identifying and attacking the saddle point problem in high-dimensional non-convex optimization. *NeurIPS*, pp. 2933–2941, 2014.

[92] N. de Freitas. Machine Learning, University of Oxford (Course Video), 2013. https://www.youtube.com/watch?v=w2OtwL5T1ow&list=PLE6Wd9FR--EdyJ5lbFl8UuGjecvVw66F6

[93] N. de Freitas. Deep Learning, University of Oxford (Course Video), 2015. https://www.youtube.com/watch?v=PlhFWT7vAEw&list=PLjK8ddCbDMphIMSXn-w1IjyYpHU3DaUYw

[94] J. Dean *et al.* Large scale distributed deep networks. *NeurIPS*, 2012.

[95] M. Defferrard, X. Bresson, and P. Vandergheynst. Convolutional neural networks on graphs with fast localized spectral filtering. *NeurIPS*, pp. 3844–3852, 2016.

[96] M. Denil, B. Shakibi, L. Dinh, M. A. Ranzato, and N. de Freitas. Predicting parameters in deep learning. *NeurIPS*, pp. 2148–2156, 2013.

[97] E. Denton, S. Chintala, and R. Fergus. Deep Generative Image Models using a Laplacian Pyramid of Adversarial Networks. *NeurIPS*, pp. 1466–1494, 2015.

[98] G. Desjardins, K. Simonyan, and R. Pascanu. Natural neural networks. *NeurIPS*, pp. 2071–2079, 2015.

[99] F. Despagne and D. Massart. Neural networks in multivariate calibration. *Analyst*, 123(11), pp. 157R–178R, 1998.

[100] T. Dettmers. 8-bit approximations for parallelism in deep learning. *arXiv:1511.04561*, 2015. https://arxiv.org/abs/1511.04561

[101] J. Devlin *et al.* Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv:1810.04805*, 2018. https://arxiv.org/abs/1810.04805

[102] J. Donahue, L. Anne Hendricks, S. Guadarrama, M. Rohrbach, S. Venugopalan, K. Saenko, and T. Darrell. Long-term recurrent convolutional networks for visual recognition and description. *CVPR*, pp. 2625–2634, 2015.

[103] G. Dorffner. Neural networks for time series processing. *Neural Network World*, 1996.

[104] C. Dos Santos and M. Gatti. Deep Convolutional Neural Networks for Sentiment Analysis of Short Texts. *COLING*, pp. 69–78, 2014.

[105] A. Dosovitskiy and T. Brox. Inverting visual representations with convolutional networks. *CVPR Conference*, pp. 4829–4837, 2016.

[106] A. Dosovitsky *et al.* An image is worth $16 \times 16$ words: Transformers for image recognition at scale. *arXiv:2010.11929*, 2020. https://arxiv.org/abs/2010.11929

[107] K. Doya. Bifurcations of recurrent neural networks in gradient descent learning. *IEEE TNNLS*, 1, pp. 75–80, 1993.

[108] C. Doersch. Tutorial on variational autoencoders. *arXiv:1606.05908*, 2016. https://arxiv.org/abs/1606.05908

[109] H. Drucker and Y. LeCun. Improving generalization performance using double backpropagation. *IEEE TNNLS*, 3(6), pp. 991–997, 1992.

[110] J. Duchi, E. Hazan, and Y. Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12, pp. 2121–2159, 2011.

[111] V. Dumoulin and F. Visin. A guide to convolution arithmetic for deep learning. *arXiv:1603.07285*, 2016. https://arxiv.org/abs/1603.07285

[112] A. Elkahky, Y. Song, and X. He. A multi-view deep learning approach for cross domain user modeling in recommendation systems. *WWW Conference*, pp. 278–288, 2015.

[113] J. Elman. Finding structure in time. *Cognitive Science*, 14(2), pp. 179–211, 1990.

[114] J. Elman. Learning and development in neural networks: The importance of starting small. *Cognition*, 48, pp. 781–799, 1993.

[115] T. Elsken, J. Metzen, and F. Hutter. Neural architecture search: A survey. *Journal of Machine Learning Research*, 20(55), pp. 1–21, 2019.

[116] D. Erhan *et al.* Why does unsupervised pre-training help deep learning? *Journal of Machine Learning Research*, 11, pp. 625–660, 2010.

[117] S. Essar *et al.* Convolutional neural networks for fast, energy-efficient neuromorphic computing. *Proceedings of the National Academy of Science of the USA*, 113(41), pp. 11441–11446, 2016.

[118] A. Fader, L. Zettlemoyer, and O. Etzioni. Paraphrase-Driven Learning for Open Question Answering. *ACL*, pp. 1608–1618, 2013.

[119] W. Fan *et al.* Graph neural networks for social recommendation. *WWW*, 2019.

[120] W. Fedus, B. Zoph, and N. Shazeer. Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity. *arXiv:2101.03961*, 2021. https://arxiv.org/abs/2101.03961

[121] L. Fei-Fei, R. Fergus, and P. Perona. One-shot learning of object categories. *IEEE TPAMI*, 28(4), pp. 594–611, 2006.

[122] P. Felzenszwalb, R. Girshick, D. McAllester, and D. Ramanan. Object detection with discriminatively trained part-based models. *IEEE TPAMI*, 32(9), pp. 1627–1645, 2010.

[123] A. Fader, L. Zettlemoyer, and O. Etzioni. Open question answering over curated and extracted knowledge bases. *KDD*, 2014.

[124] A. Fischer and C. Igel. An introduction to restricted Boltzmann machines. *Progress in Pattern Recognition, Image Analysis, Computer Vision, and Applications*, pp. 14–36, 2012.

[125] R. Fisher. The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7: pp. 179–188, 1936.

[126] J. Frankle and M. Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. *arXiv:1803.03635*, 2018. https://arxiv.org/abs/1803.03635

[127] Y. Freund and R. Schapire. A decision-theoretic generalization of online learning and application to boosting. *Computational Learning Theory*, pp. 23–37, 1995.

[128] Y. Freund and R. Schapire. Large margin classification using the perceptron algorithm. *Machine Learning*, 37(3), pp. 277–296, 1999.

[129] Y. Freund and D. Haussler. Unsupervised learning of distributions on binary vectors using two layer networks. *Technical report*, Santa Cruz, CA, USA, 1994

[130] B. Fritzke. Fast learning with incremental RBF networks. *Neural Processing Letters*, 1(1), pp. 2–5, 1994.

[131] B. Fritzke. A growing neural gas network learns topologies. *NeurIPS*, pp. 625–632, 1995.

[132] K. Fukushima. Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biological Cybernetics*, 36(4), pp. 193–202, 1980.

[133] S. Gallant. Perceptron-based learning algorithms. *IEEE TNNLS*, 1(2), pp. 179–191, 1990.

[134] S. Gallant. Neural network learning and expert systems. *MIT Press*, 1993.

[135] H. Gao, H. Yuan, Z. Wang, and S. Ji. Pixel Deconvolutional Networks. *arXiv:1705.06820*, 2017. https://arxiv.org/abs/1705.06820

[136] H. Gao and S. Ji. Graph u-nets. *arXiv:1905.05178*, 2019. http://proceedings.mlr.press/v97/gao19a/gao19a.pdf
Code: https://github.com/divelab/gunet

[137] L. Gatys, A. S. Ecker, and M. Bethge. Texture synthesis using convolutional neural networks. *NeurIPS*, pp. 262–270, 2015.

[138] L. Gatys, A. Ecker, and M. Bethge. Image style transfer using convolutional neural networks. *CVPR*, pp. 2414–2423, 2015.

[139] H. Gavin. The Levenberg-Marquardt method for nonlinear least squares curve-fitting problems, 2011. http://people.duke.edu/~hpgavin/ce281/lm.pdf

[140] P. Gehler, A. Holub, and M. Welling. The Rate Adapting Poisson (RAP) model for information retrieval and object recognition. *ICML*, 2006.

[141] S. Gelly *et al.* The grand challenge of computer Go: Monte Carlo tree search and extensions. *Communications of the ACM*, 55, pp. 106–113, 2012.

[142] A. Gersho and R. M. Gray. Vector quantization and signal compression. *Springer Science and Business Media*, 2012.

[143] A. Ghodsi. STAT 946: Topics in Probability and Statistics: Deep Learning, *University of Waterloo*, Fall 2015. https://www.youtube.com/watch?v=fyAZszlPphs&list=PLehuLRPyt1Hyi78UOkMPWCGRxGcA9NVOE

[144] W. Gilks, S. Richardson, and D. Spiegelhalter. Markov chain Monte Carlo in practice.*CRC Press*, 1995.

[145] J. Gilmer *et al.* Neural message passing for quantum chemistry. *ICML*, 2017.

[146] F. Girosi and T. Poggio. Networks and the best approximation property. *Biological Cybernetics*, 63(3), pp. 169–176, 1990.

[147] X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. *AISTATS*, pp. 249–256, 2010.

[148] X. Glorot, A. Bordes, and Y. Bengio. Deep Sparse Rectifier Neural Networks. *AISTATS*, 15(106), 2011.

[149] P. Glynn. Likelihood ratio gradient estimation: an overview, *Proceedings of the 1987 Winter Simulation Conference*, pp. 366–375, 1987.

[150] Y. Goldberg. A primer on neural network models for natural language processing. *Journal of Artificial Intelligence Research (JAIR)*, 57, pp. 345–420, 2016.

[151] I. Goodfellow, J. Shlens, and C. Szegedy. Explaining and harnessing adversarial examples. *arXiv:1412.6572*, 2014. https://arxiv.org/abs/1412.6572

[152] I. Goodfellow. NIPS 2016 tutorial: Generative adversarial networks. *arXiv:1701.00160*, 2016. https://arxiv.org/abs/1701.00160

[153] I. Goodfellow, O. Vinyals, and A. Saxe. Qualitatively characterizing neural network optimization problems. *ICLR*, 2015. https://arxiv.org/abs/1412.6544

[154] I. Goodfellow, Y. Bengio, and A. Courville. Deep learning. *MIT Press*, 2016.

[155] I. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, and Y. Bengio. Maxout networks. *arXiv:1302.4389*, 2013.

[156] I. Goodfellow *et al.* Generative adversarial nets. *NeurIPS*, 2014.

[157] A. Graves, A. Mohamed, and G. Hinton. Speech recognition with deep recurrent neural networks. *Acoustics, Speech and Signal Processing (ICASSP)*, pp. 6645–6649, 2013.

[158] A. Graves. Generating sequences with recurrent neural networks. *arXiv:1308.0850*, 2013. https://arxiv.org/abs/1308.0850

[159] A. Graves. Supervised sequence labelling with recurrent neural networks *Springer*, 2012.

[160] A. Graves, S. Fernandez, F. Gomez, and J. Schmidhuber. Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. *ICML*, pp. 369–376, 2006.

[161] A. Graves, M. Liwicki, S. Fernandez, R. Bertolami, H. Bunke, and J. Schmidhuber. A novel connectionist system for unconstrained handwriting recognition. *IEEE TPAMI*, 31(5), pp. 855–868, 2009.

[162] A. Graves and J. Schmidhuber. Framewise Phoneme Classification with Bidirectional LSTM and Other Neural Network Architectures. *Neural Networks*, 18(5-6), pp. 602–610, 2005.

[163] A. Graves and J. Schmidhuber. Offline handwriting recognition with multidimensional recurrent neural networks. *NeurIPS*, pp. 545–552, 2009.

[164] A. Graves and N. Jaitly. Towards End-To-End Speech Recognition with Recurrent Neural Networks. *ICML*, pp. 1764–1772, 2014.

[165] A. Graves, G. Wayne, and I. Danihelka. Neural turing machines. *arXiv:1410.5401*, 2014. https://arxiv.org/abs/1410.5401

[166] A. Graves *et al.* Hybrid computing using a neural network with dynamic external memory. *Nature*, 538.7626, pp. 471–476, 2016.

[167] K. Greff, R. K. Srivastava, J. Koutnik, B. Steunebrink, and J. Schmidhuber. LSTM: A search space odyssey. *IEEE TNNLS*, 2016.

[168] K. Greff, R. K. Srivastava, and J. Schmidhuber. Highway and residual networks learn unrolled iterative estimation. *arXiv:1612.07771*, 2016. https://arxiv.org/abs/1612.07771

[169] I. Grondman, L. Busoniu, G. A. Lopes, and R. Babuska. A survey of actor-critic reinforcement learning: Standard and natural policy gradients. *IEEE Transactions on Systems, Man, and Cybernetics*, 42(6), pp. 1291–1307, 2012.

[170] R. Girshick, F. Iandola, T. Darrell, and J. Malik. Deformable part models are convolutional neural networks. *CVPR*, pp. 437–446, 2015.

[171] A. Grover and J. Leskovec. node2vec: Scalable feature learning for networks. *KDD*, pp. 855–864, 2016.

[172] X. Guo, S. Singh, H. Lee, R. Lewis, and X. Wang. Deep learning for real-time Atari game play using offline Monte-Carlo tree search planning. *NeurIPS*, pp. 3338–3346, 2014.

[173] M. Gutmann and A. Hyvarinen. Noise-contrastive estimation: A new estimation principle for unnormalized statistical models. *AISTATS*, 1(2), pp. 6, 2010.

[174] R. Hahnloser and H. S. Seung. Permitted and forbidden sets in symmetric threshold-linear networks. *NeurIPS*, pp. 217–223, 2001.

[175] W. Hamilton, R. Ying, and J. Leskovec. Representation learning on graphs: methods and applications. *arXiv:1709.05584*, 2017. https://arxiv.org/abs/1709.05584

[176] W. Hamilton, R. Ying, and J. Leskovec. Inductive representation learning on large graphs. *NeurIPS*, pp. 1024–1034, 2017.

[177] W. Hamilton, J. Leskovec, R. Ying, and R. Sosic. Representation learning on networks, *WWW Tutorial*, 2018. http://snap.stanford.edu/proj/embeddings-www/

[178] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. Horowitz, and W. Dally. EIE: Efficient Inference Engine for Compressed Neural Network. *ACM SIGARCH Computer Architecture News*, 44(3), pp. 243–254, 2016.

[179] S. Han, J. Pool, J. Tran, and W. Dally. Learning both weights and connections for efficient neural networks. *NeurIPS*, pp. 1135–1143, 2015.

[180] L. K. Hansen and P. Salamon. Neural network ensembles. *IEEE TPAMI*, 12(10), pp. 993–1001, 1990.

[181] M. Hardt, B. Recht, and Y. Singer. Train faster, generalize better: Stability of stochastic gradient descent. *ICML*, pp. 1225–1234, 2006.

[182] B. Hariharan, P. Arbelaez, R. Girshick, and J. Malik. Simultaneous detection and segmentation. *arXiv:1407.1808*, 2014. https://arxiv.org/abs/1407.1808

[183] E. Hartman, J. Keeler, and J. Kowalski. Layered neural networks with Gaussian hidden units as universal approximations. *Neural Computation*, 2(2), pp. 210–215, 1990.

[184] H. van Hasselt, A. Guez, and D. Silver. Deep Reinforcement Learning with Double Q-Learning. *AAAI*, 2016.

[185] B. Hassibi and D. Stork. Second order derivatives for network pruning: Optimal brain surgeon. *NeurIPS*, 1993.

[186] D. Hassabis, D. Kumaran, C. Summerfield, and M. Botvinick. Neuroscience-inspired artificial intelligence. *Neuron*, 95(2), pp. 245–258, 2017.

[187] T. Hastie, R. Tibshirani, and J. Friedman. The elements of statistical learning. *Springer*, 2009.

[188] T. Hastie and R. Tibshirani. Generalized additive models. *CRC Press*, 1990.

[189] T. Hastie, R. Tibshirani, and M. Wainwright. Statistical learning with sparsity: the lasso and generalizations. *CRC Press*, 2015.

[190] M. Havaei *et al.* Brain tumor segmentation with deep neural networks. *Medical Image Analysis*, 35, pp. 18–31, 2017.

[191] S. Hawkins, H. He, G. Williams, and R. Baxter. Outlier detection using replicator neural networks. *International Conference on Data Warehousing and Knowledge Discovery*, pp. 170–180, 2002.

[192] S. Haykin. Neural networks and learning machines. *Pearson*, 2008.

[193] K. He, X. Zhang, S. Ren, and J. Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. *ICCV*, pp. 1026–1034, 2015.

[194] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. *CVPR*, pp. 770–778, 2016.

[195] K. He, X. Zhang, S. Ren, and J. Sun. Identity mappings in deep residual networks. *ECCV*, pp. 630–645, 2016.

[196] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T. S. Chua. Neural collaborative filtering. *WWW*, pp. 173–182, 2017.

[197] N. Heess *et al.* Emergence of Locomotion Behaviours in Rich Environments. *arXiv:1707.02286*, 2017. https://arxiv.org/abs/1707.02286
**Video 1 at:** https://www.youtube.com/watch?v=hx_bgoTF7bs
**Video 2 at:** https://www.youtube.com/watch?v=gn4nRCC9TwQ&feature=youtu.be

[198] M. Henaff, J. Bruna, and Y. LeCun. Deep convolutional networks on graph-structured data. *arXiv:1506.05163*, 2015. https://arxiv.org/abs/1506.05163

[199] M. Hestenes and E. Stiefel. Methods of conjugate gradients for solving linear systems. *Journal of Research of the National Bureau of Standards*, 49(6), 1952.

[200] G. Hinton. Connectionist learning procedures. *Artificial Intelligence*, 40(1–3), pp. 185–234, 1989.

[201] G. Hinton. Training products of experts by minimizing contrastive divergence. *Neural Computation*, 14(8), pp. 1771–1800, 2002.

[202] G. Hinton. To recognize shapes, first learn to generate images. *Progress in Brain Research*, 165, pp. 535–547, 2007.

[203] G. Hinton. A practical guide to training restricted Boltzmann machines. *Momentum*, 9(1), 926, 2010.

[204] G. Hinton. Neural networks for machine learning, *Coursera Video*, 2012.

[205] G. Hinton, P. Dayan, B. Frey, and R. Neal. The wake–sleep algorithm for unsupervised neural networks. *Science*, 268(5214), pp. 1158–1162, 1995.

[206] G. Hinton, S. Osindero, and Y. Teh. A fast learning algorithm for deep belief nets. *Neural Computation*, 18(7), pp. 1527–1554, 2006.

[207] G. Hinton and T. Sejnowski. Learning and relearning in Boltzmann machines. *Parallel Distributed Processing: Explorations in the Microstructure of Cognition*, MIT Press, 1986.

[208] G. Hinton and R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 313, (5766), pp. 504–507, 2006.

[209] G. Hinton and R. Salakhutdinov. Replicated softmax: an undirected topic model. *NeurIPS*, pp. 1607–1614, 2009.

[210] G. Hinton and R. Salakhutdinov. A better way to pretrain deep Boltzmann machines. *NeurIPS*, pp. 2447–2455, 2012.

[211] G. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. *arXiv:1207.0580*, 2012. https://arxiv.org/abs/1207.0580

[212] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. *NeurIPS Workshop*, 2014.

[213] R. Hochberg. Matrix Multiplication with CUDA: A basic introduction to the CUDA programming model. *Unpublished manuscript*, 2012. http://www.shodor.org/media/content/petascale/materials/UPModules/matrixMultiplication/moduleDocument.pdf

[214] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8), pp. 1735–1785, 1997.

[215] S. Hochreiter, Y. Bengio, P. Frasconi, and J. Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies, *A Field Guide to Dynamical Recurrent Neural Networks*, IEEE Press, 2001.

[216] T. Hofmann. Probabilistic latent semantic indexing. *ACM SIGIR Conference*, pp. 50–57, 1999.

[217] J. J. Hopfield. Neural networks and physical systems with emergent collective computational abilities. *National Academy of Sciences of the USA*, 79(8), pp. 2554–2558, 1982.

[218] K. Hornik, M. Stinchcombe, and H. White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5), pp. 359–366, 1989.

[219] J. Hu, L. Shen, and G. Sun. Squeeze-and-excitation networks. *CVPR*, pp. 7132–7141, 2018.

[220] Y. Hu, Y. Koren, and C. Volinsky. Collaborative filtering for implicit feedback datasets. *IEEE International Conference on Data Mining*, pp. 263–272, 2008.

[221] G. Huang, Y. Sun, Z. Liu, D. Sedra, and K. Weinberger. Deep networks with stochastic depth. *ECCV*, pp. 646–661, 2016.

[222] G. Huang, Z. Liu, K. Weinberger, and L. van der Maaten. Densely connected convolutional networks. *arXiv:1608.06993*, 2016. https://arxiv.org/abs/1608.06993

[223] D. Hubel and T. Wiesel. Receptive fields of single neurones in the cat's striate cortex. *The Journal of Physiology*, 124(3), pp. 574–591, 1959.

[224] F. Iandola *et al.* SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and $<$ 0.5 MB model size. *arXiv:1602.07360*, 2016. https://arxiv.org/abs/1602.07360

[225] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv:1502.03167*, 2015. https://arxiv.org/abs/1502.03167

[226] P. Isola, J. Zhu, T. Zhou, and A. Efros. Image-to-image translation with conditional adversarial networks. *arXiv:1611.07004*, 2016. https://arxiv.org/abs/1611.07004

[227] M. Iyyer, J. Boyd-Graber, L. Claudino, R. Socher, and H. Daume III. A Neural Network for Factoid Question Answering over Paragraphs. *EMNLP*, 2014.

[228] R. Jacobs. Increased rates of convergence through learning rate adaptation. *Neural Networks*, 1(4), pp. 295–307, 1988.

[229] M. Jaderberg *et al.* Spatial transformer networks. *NeurIPS*, pp. 2017–2025, 2015.

[230] H. Jaeger. The "echo state" approach to analysing and training recurrent neural networks – with an erratum note. *German National Research Center for Information Technology GMD Technical Report*, 148(34), 13, 2001.

[231] H. Jaeger and H. Haas. Harnessing nonlinearity: Predicting chaotic systems and saving energy in wireless communication. *Science*, 304, pp. 78–80, 2004.

[232] K. Jarrett, K. Kavukcuoglu, M. Ranzato, and Y. LeCun. What is the best multi-stage architecture for object recognition? *ICCV*, 2009.

[233] S. Ji, W. Xu, M. Yang, and K. Yu. 3D convolutional neural networks for human action recognition. *IEEE TPAMI*, 35(1), pp. 221–231, 2013.

[234] Y. Jia *et al.* Caffe: Convolutional architecture for fast feature embedding. *ACM International Conference on Multimedia*, 2014.

[235] C. Johnson. Logistic matrix factorization for implicit feedback data. *NeurIPS*, 2014.

[236] J. Johnson, A. Karpathy, and L. Fei-Fei. Densecap: Fully convolutional localization networks for dense captioning. *CVPR*, pp. 4565–4574, 2015.

[237] J. Johnson, A. Alahi, and L. Fei-Fei. Perceptual losses for real-time style transfer and super-resolution. *ECCV*, pp. 694–711, 2015.

[238] R. Johnson and T. Zhang. Effective use of word order for text categorization with convolutional neural networks. *arXiv:1412.1058*, 2014. https://arxiv.org/abs/1412.1058

[239] R. Jozefowicz, W. Zaremba, and I. Sutskever. An empirical exploration of recurrent network architectures. *ICML*, pp. 2342–2350, 2015.

[240] S. Kakade. A natural policy gradient. *NeurIPS*, pp. 1057–1063, 2002.

[241] N. Kalchbrenner and P. Blunsom. Recurrent continuous translation models. *EMNLP*, 3, 39, pp. 413, 2013.

[242] H. Kandel, J. Schwartz, T. Jessell, S. Siegelbaum, and A. Hudspeth. Principles of neural science. *McGraw Hill*, 2012.

[243] A. Karpathy, J. Johnson, and L. Fei-Fei. Visualizing and understanding recurrent networks. *arXiv:1506.02078*, 2015. https://arxiv.org/abs/1506.02078

[244] A. Karpathy, G. Toderici, S. Shetty, T. Leung, R. Sukthankar, and L. Fei-Fei. Large-scale video classification with convolutional neural networks. *CVPR*, pp. 725–1732, 2014.

[245] A. Karpathy. The unreasonable effectiveness of recurrent neural networks, *Blog post*, 2015. http://karpathy.github.io/2015/05/21/rnn-effectiveness/

[246] A. Karpathy, J. Johnson, and L. Fei-Fei. Stanford University Class CS321n: Convolutional neural networks for visual recognition, 2016. http://cs231n.github.io/

[247] H. J. Kelley. Gradient theory of optimal flight paths. *Ars Journal*, 30(10), pp. 947–954, 1960.

[248] F. Khan, B. Mutlu, and X. Zhu. How do humans teach: On curriculum learning and teaching dimension. *NeurIPS*, pp. 1449–1457, 2011.

[249] T. Kietzmann, P. McClure, and N. Kriegeskorte. Deep Neural Networks In Computational Neuroscience. *bioRxiv, 133504*, 2017. https://www.biorxiv.org/content/early/2017/05/04/133504

[250] Y. Kim. Convolutional neural networks for sentence classification. *arXiv:1408.5882*, 2014. https://arxiv.org/abs/1408.5882

[251] D. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv:1412.6980*, 2014. https://arxiv.org/abs/1412.6980

[252] D. Kingma and M. Welling. Auto-encoding variational bayes. *arXiv:1312.6114*, 2013. https://arxiv.org/abs/1312.6114

[253] T. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. *arXiv:1609.02907*, 2016. https://arxiv.org/abs/1609.02907
Code: https://github.com/tkipf/gcn

[254] S. Kirkpatrick, C. Gelatt, and M. Vecchi. Optimization by simulated annealing. *Science*, 220, pp. 671–680, 1983.

[255] J. Kivinen and M. Warmuth. The perceptron algorithm vs. winnow: linear vs. logarithmic mistake bounds when few input variables are relevant. *Computational Learning Theory*, pp. 289–296, 1995.

[256] L. Kocsis and C. Szepesvari. Bandit based monte-carlo planning. *ECML*, pp. 282–293, 2006.

[257] R. Kohavi and D. Wolpert. Bias plus variance decomposition for zero-one loss functions. *ICML*, 1996.

[258] T. Kohonen. Self-organizing maps, *Springer*, 2001.

[259] D. Koller and N. Friedman. Probabilistic graphical models: principles and techniques. *MIT Press*, 2009.

[260] E. Kong and T. Dietterich. Error-correcting output coding corrects bias and variance. *ICML*, pp. 313–321, 1995.

[261] Y. Koren. Factor in the neighbors: Scalable and accurate collaborative filtering. *ACM TKDD Journal*, 4(1), 1, 2010.

[262] A. Krizhevsky. One weird trick for parallelizing convolutional neural networks. *arXiv:1404.5997*, 2014. https://arxiv.org/abs/1404.5997

[263] A. Krizhevsky, I. Sutskever, and G. Hinton. Imagenet classification with deep convolutional neural networks. *NeurIPS*, pp. 1097–1105. 2012.

[264] M. Kubat. Decision trees can initialize radial-basis function networks. *IEEE TNNLS*, 9(5), pp. 813–821, 1998.

[265] M. Lai. Giraffe: Using deep reinforcement learning to play chess. *arXiv:1509.01549*, 2015. https://arxiv.org/abs/1509.01549

[266] S. Lai, L. Xu, K. Liu, and J. Zhao. Recurrent Convolutional Neural Networks for Text Classification. *AAAI*, pp. 2267–2273, 2015.

[267] B. Lake, T. Ullman, J. Tenenbaum, and S. Gershman. Building machines that learn and think like people. *Behavioral and Brain Sciences*, pp. 1–101, 2016.

[268] H. Larochelle. Neural Networks (Course). Universite de Sherbrooke, 2013. https://www.youtube.com/watch?v=SGZ6BttHMPw&list=PL6Xpj9I5qXYEcOhn7TqghAJ6NAPrNmUBH

[269] H. Larochelle and Y. Bengio. Classification using discriminative restricted Boltzmann machines. *ICML*, pp. 536–543, 2008.

[270] H. Larochelle, M. Mandel, R. Pascanu, and Y. Bengio. Learning algorithms for the classification restricted Boltzmann machine. *Journal of Machine Learning Research*, 13, pp. 643–669, 2012.

[271] H. Larochelle and I. Murray. The neural autoregressive distribution estimator. *International Conference on Artificial Intelligence and Statistics*, pp. 29–37, 2011.

[272] H. Larochelle and G. E. Hinton. Learning to combine foveal glimpses with a third-order Boltzmann machine. *NeurIPS*, 2010.

[273] H. Larochelle, D. Erhan, A. Courville, J. Bergstra, and Y. Bengio. An empirical evaluation of deep architectures on problems with many factors of variation. *ICML*, pp. 473–480, 2007.

[274] G. Larsson, M. Maire, and G. Shakhnarovich. Fractalnet: Ultra-deep neural networks without residuals. *arXiv:1605.07648*, 2016. https://arxiv.org/abs/1605.07648

[275] S. Lawrence, C. L. Giles, A. C. Tsoi, and A. D. Back. Face recognition: A convolutional neural-network approach. *IEEE TNNLS*, 8(1), pp. 98–113, 1997.

[276] Q. Le *et al.* Building high-level features using large scale unsupervised learning. *ICASSP*, 2013.

[277] Q. Le, N. Jaitly, and G. Hinton. A simple way to initialize recurrent networks of rectified linear units. *arXiv:1504.00941*, 2015. https://arxiv.org/abs/1504.00941

[278] Q. Le and T. Mikolov. Distributed representations of sentences and documents. *ICML*, pp. 1188–196, 2014.

[279] Q. Le, J. Ngiam, A. Coates, A. Lahiri, B. Prochnow, and A. Ng, On optimization methods for deep learning. *ICML*, pp. 265–272, 2011.

[280] Q. Le, W. Zou, S. Yeung, and A. Ng. Learning hierarchical spatio-temporal features for action recognition with independent subspace analysis. *CVPR*, 2011.

[281] Y. LeCun. Modeles connexionnistes de l'apprentissage. *Doctoral Dissertation*, Universite Paris, 1987.

[282] Y. LeCun and Y. Bengio. Convolutional networks for images, speech, and time series. *The Handbook of Brain Theory and Neural Networks*, 3361(10), 1995.

[283] Y. LeCun, Y. Bengio, and G. Hinton. Deep learning. *Nature*, 521(7553), pp. 436–444, 2015.

[284] Y. LeCun, L. Bottou, G. Orr, and K. Muller. Efficient backprop. in G. Orr and K. Muller (eds.) *Neural Networks: Tricks of the Trade*, Springer, 1998.

[285] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), pp. 2278–2324, 1998.

[286] Y. LeCun, S. Chopra, R. M. Hadsell, M. A. Ranzato, and F.-J. Huang. A tutorial on energy-based learning. *Predicting Structured Data*, MIT Press, pp. 191–246,, 2006.

[287] Y. LeCun, C. Cortes, and C. Burges. The MNIST database of handwritten digits, 1998. http://yann.lecun.com/exdb/mnist/

[288] Y. LeCun, J. Denker, and S. Solla. Optimal brain damage. *NeurIPS*, pp. 598–605, 1990.

[289] Y. LeCun, K. Kavukcuoglu, and C. Farabet. Convolutional networks and applications in vision. *IEEE International Symposium on Circuits and Systems*, pp. 253–256, 2010.

[290] H. Lee, C. Ekanadham, and A. Ng. Sparse deep belief net model for visual area V2. *NeurIPS*, 2008.

[291] H. Lee, R. Grosse, B. Ranganath, and A. Y. Ng. Convolutional deep belief networks for scalable unsupervised learning of hierarchical representations. *ICML*, pp. 609–616, 2009.

[292] S. Levine, C. Finn, T. Darrell, and P. Abbeel. End-to-end training of deep visuomotor policies. *Journal of Machine Learning Research*, 17(39), pp. 1–40, 2016.
**Video at:** https://sites.google.com/site/visuomotorpolicy/

[293] O. Levy and Y. Goldberg. Neural word embedding as implicit matrix factorization. *NeurIPS*, pp. 2177–2185, 2014.

[294] O. Levy, Y. Goldberg, and I. Dagan. Improving distributional similarity with lessons learned from word embeddings. *Transactions of the Association for Computational Linguistics*, 3, pp. 211–225, 2015.

[295] W. Levy and R. Baxter. Energy efficient neural codes. *Neural Computation*, 8(3), pp. 531–543, 1996.

[296] M. Lewis *et al.* Deal or No Deal? End-to-End Learning for Negotiation Dialogues. *arXiv:1706.05125*, 2017. https://arxiv.org/abs/1706.05125

[297] G. Li, M. Muller, A. Thabet, and B. Ghanem. Deepgcns: Can gcns go as deep as cnns? *CVPR*, pp. 9267-9276, 2019.

[298] J. Li, W. Monroe, A. Ritter, M. Galley,, J. Gao, and D. Jurafsky. Deep reinforcement learning for dialogue generation. *arXiv:1606.01541*, 2016. https://arxiv.org/abs/1606.01541

[299] L. Li, W. Chu, J. Langford, and R. Schapire. A contextual-bandit approach to personalized news article recommendation. *WWW Conference*, pp. 661–670, 2010.

[300] Q. Li, Z. Han, and X.-M. Wu. Deeper insights into graph convolutional networks for semi-supervised learning. *AAAI*, 2018.

[301] Y. Li. Deep reinforcement learning: An overview. *arXiv:1701.07274*, 2017. https://arxiv.org/abs/1701.07274

[302] Y. Li, D. Tarlow, M. Brockschmidt, and R. Zemel. Gated graph sequence neural networks. *arXiv:1511.05493*, 2015. https://arxiv.org/abs/1511.05493

[303] Q. Liao, K. Kawaguchi, and T. Poggio. Streaming normalization: Towards simpler and more biologically-plausible normalizations for online and recurrent learning. *arXiv:1610.06160*, 2016. https://arxiv.org/abs/1610.06160

[304] D. Liben-Nowell, and J. Kleinberg. The link-prediction problem for social networks. *Journal of the American Society for Information Science and Technology*, 58(7), pp. 1019–1031, 2007.

[305] L.-J. Lin. Reinforcement learning for robots using neural networks. *Technical Report*, DTIC Document, 1993.

[306] M. Lin, Q. Chen, and S. Yan. Network in network. *arXiv:1312.4400*, 2013. https://arxiv.org/abs/1312.4400

[307] Z. Lipton, J. Berkowitz, and C. Elkan. A critical review of recurrent neural networks for sequence learning. *arXiv:1506.00019*, 2015. https://arxiv.org/abs/1506.00019

[308] J. Lu, J. Yang, D. Batra, and D. Parikh. Hierarchical question-image co-attention for visual question answering. *NeurIPS*, pp. 289–297, 2016.

[309] D. Luenberger and Y. Ye. Linear and nonlinear programming, *Addison-Wesley*, 1984.

[310] M. Lukosevicius and H. Jaeger. Reservoir computing approaches to recurrent neural network training. *Computer Science Review*, 3(3), pp. 127–149, 2009.

[311] M. Luong, H. Pham, and C. Manning. Effective approaches to attention-based neural machine translation. *arXiv:1508.04025*, 2015. https://arxiv.org/abs/1508.04025

[312] J. Ma, R. P. Sheridan, A. Liaw, G. E. Dahl, and V. Svetnik. Deep neural nets as a method for quantitative structure-activity relationships. *Journal of Chemical Information and Modeling*, 55(2), pp. 263–274, 2015.

[313] Y. Ma, S. Wang, C. Aggarwal, and J. Tang. Graph convolutional networks with eigenpooling. *KDD*, 2019. Code: https://github.com/alge24/eigenpooling

[314] Y. Ma, S. Wang, C. Aggarwal, D. Yin, and J. Tang. Multi-dimensional graph convolutional networks. *SDM Conference*, 2019.

[315] W. Maass, T. Natschlager, and H. Markram. Real-time computing without stable states: A new framework for neural computation based on perturbations. *Neural Computation*, 14(11), pp. 2351–2560, 2002.

[316] L. Maaten and G. E. Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9, pp. 2579–2605, 2008.

[317] D. J. MacKay. A practical Bayesian framework for backpropagation networks. *Neural Computation*, 4(3), pp. 448–472, 1992.

[318] C. Maddison, A. Huang, I. Sutskever, and D. Silver. Move evaluation in Go using deep convolutional neural networks. *ICLR*, 2015.

[319] A. Madry *et al.* Towards deep learning models resistant to adversarial attacks. *arXiv:1706.06083*, 2017. https://arxiv.org/abs/1706.06083

[320] A. Mahendran and A. Vedaldi. Understanding deep image representations by inverting them. *CVPR*, pp. 5188–5196, 2015.

[321] A. Makhzani and B. Frey. K-sparse autoencoders. *arXiv:1312.5663*, 2013. https://arxiv.org/abs/1312.5663

[322] A. Makhzani and B. Frey. Winner-take-all autoencoders. *NeurIPS*, pp. 2791–2799, 2015.

[323] A. Makhzani, J. Shlens, N. Jaitly, I. Goodfellow, and B. Frey. Adversarial autoencoders. *arXiv:1511.05644*, 2015. https://arxiv.org/abs/1511.05644

[324] J. Martens. Deep learning via Hessian-free optimization. *ICML*, pp. 735–742, 2010.

[325] J. Martens and I. Sutskever. Learning recurrent neural networks with hessian-free optimization. *ICML*, pp. 1033–1040, 2011.

[326] J. Martens, I. Sutskever, and K. Swersky. Estimating the hessian by back-propagating curvature. *arXiv:1206.6464*, 2016. https://arxiv.org/abs/1206.6464

[327] J. Martens and R. Grosse. Optimizing Neural Networks with Kronecker-factored Approximate Curvature. *ICML*, 2015.

[328] T. Martinetz, S. Berkovich, and K. Schulten. 'Neural-gas' network for vector quantization and its application to time-series prediction. *IEEE TNNLS*, 4(4), pp. 558–569, 1993.

[329] J. Masci, U. Meier, D. Ciresan, and J. Schmidhuber. Stacked convolutional auto-encoders for hierarchical feature extraction. *Artificial Neural Networks and Machine Learning*, pp. 52–59, 2011.

[330] M. Mathieu, C. Couprie, and Y. LeCun. Deep multi-scale video prediction beyond mean square error. *arXiv:1511.054*, 2015. https://arxiv.org/abs/1511.05440

[331] P. McCullagh and J. Nelder. Generalized linear models *CRC Press*, 1989.

[332] W. S. McCulloch and W. H. Pitts. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4), pp. 115–133, 1943.

[333] G. McLachlan. Discriminant analysis and statistical pattern recognition *John Wiley & Sons*, 2004.

[334] C. Micchelli. Interpolation of scattered data: distance matrices and conditionally positive definite functions. *Constructive Approximations*, 2, pp. 11–22, 1986.

[335] P. Michel, O. Levy, and G. Neubig. Are sixteen heads really better than one? *arXiv:1905.10650*, 2019. https://arxiv.org/abs/1905.10650

[336] T. Mikolov. Statistical language models based on neural networks. *Ph.D. thesis, Brno University of Technology*, 2012.

[337] T. Mikolov, K. Chen, G. Corrado, and J. Dean. Efficient estimation of word representations in vector space. *arXiv:1301.3781*, 2013. https://arxiv.org/abs/1301.3781

[338] T. Mikolov, A. Joulin, S. Chopra, M. Mathieu, and M. Ranzato. Learning longer memory in recurrent neural networks. *arXiv:1412.7753*, 2014. https://arxiv.org/abs/1412.7753

[339] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean. Distributed representations of words and phrases and their compositionality. *NeurIPS*, pp. 3111–3119, 2013.

[340] T. Mikolov, M. Karafiat, L. Burget, J. Cernocky, and S. Khudanpur. Recurrent neural network based language model. *Interspeech*, Vol 2, 2010.

[341] G. Miller, R. Beckwith, C. Fellbaum, D. Gross, and K. J. Miller. Introduction to WordNet: An on-line lexical database. *International Journal of Lexicography*, 3(4), pp. 235–312, 1990. https://wordnet.princeton.edu/

[342] M. Minsky and S. Papert. Perceptrons. An Introduction to Computational Geometry, *MIT Press*, 1969.

[343] M. Mirza and S. Osindero. Conditional generative adversarial nets. *arXiv:1411.1784*, 2014. https://arxiv.org/abs/1411.1784

[344] A. Mnih and K. Kavukcuoglu. Learning word embeddings efficiently with noise-contrastive estimation. *NeurIPS*, pp. 2265–2273, 2013.

[345] A. Mnih and Y. Teh. A fast and simple algorithm for training neural probabilistic language models. *arXiv:1206.6426*, 2012. https://arxiv.org/abs/1206.6426

[346] V. Mnih *et al.* Human-level control through deep reinforcement learning. *Nature*, 518 (7540), pp. 529–533, 2015.

[347] V. Mnih *et al.* Playing atari with deep reinforcement learning. *arXiv:1312.5602.*, 2013. https://arxiv.org/abs/1312.5602

[348] V. Mnih *et al.* Asynchronous methods for deep reinforcement learning. *ICML*, pp. 1928–1937, 2016.

[349] V. Mnih, N. Heess, and A. Graves. Recurrent models of visual attention. *NeurIPS*, pp. 2204–2212, 2014.

[350] H. Mobahi and J. Fisher. A theoretical analysis of optimization by Gaussian continuation. *AAAI*, 2015.

[351] G. Montufar. Universal approximation depth and errors of narrow belief networks with discrete units. *Neural Computation*, 26(7), pp. 1386–1407, 2014.

[352] G. Montufar and N. Ay. Refinements of universal approximation results for deep belief networks and restricted Boltzmann machines. *Neural Computation*, 23(5), pp. 1306–1319, 2011.

[353] J. Moody and C. Darken. Fast learning in networks of locally-tuned processing units. *Neural Computation*, 1(2), pp. 281–294, 1989.

[354] A. Moore and C. Atkeson. Prioritized sweeping: Reinforcement learning with less data and less time. *Machine Learning*, 13(1), pp. 103–130, 1993.

[355] F. Morin and Y. Bengio. Hierarchical Probabilistic Neural Network Language Model. *AISTATS*, pp. 246–252, 2005.

[356] R. Miotto, F. Wang, S. Wang, X. Jiang, and J. T. Dudley. Deep learning for healthcare: review, opportunities and challenges. *Briefings in Bioinformatics*, pp. 1–11, 2017.

[357] M. Müller, M. Enzenberger, B. Arneson, and R. Segal. Fuego - an open-source framework for board games and Go engine based on Monte-Carlo tree search. *IEEE Transactions on Computational Intelligence and AI in Games*, 2, pp. 259–270, 2010.

[358] M. Musavi, W. Ahmed, K. Chan, K. Faris, and D. Hummels. On the training of radial basis function classifiers. *Neural Networks*, 5(4), pp. 595–603, 1992.

[359] V. Nair and G. Hinton. Rectified linear units improve restricted Boltzmann machines. *ICML*, pp. 807–814, 2010.

[360] K. S. Narendra and K. Parthasarathy. Identification and control of dynamical systems using neural networks. *IEEE TNNLS*, 1(1), pp. 4–27, 1990.

[361] R. Neal. Connectionist learning of belief networks. *Artificial intelligence*, 1992.

[362] R. Neal. Probabilistic inference using Markov chain Monte Carlo methods. *Technical Report CRG-TR-93-1*, 1993.

[363] R. Neal. Annealed importance sampling. *Statistics and Computing*, 11(2), pp. 125–139, 2001.

[364] Y. Nesterov. A method of solving a convex programming problem with convergence rate $O(1/k^2)$. *Soviet Mathematics Doklady*, 27, pp. 372–376, 1983.

[365] A. Ng. Sparse autoencoder. *CS294A Lecture notes*, 2011.
https://nlp.stanford.edu/~socherr/sparseAutoencoder_2011new.pdf
https://web.stanford.edu/class/cs294a/sparseAutoencoder_2011new.pdf

[366] A. Ng and M. Jordan. PEGASUS: A policy search method for large MDPs and POMDPs. *Uncertainity in Artificial Intelligence*, pp. 406–415, 2000.

[367] J. Y.-H. Ng, M. Hausknecht, S. Vijayanarasimhan, O. Vinyals, R. Monga, and G. Toderici. Beyond short snippets: Deep networks for video classification. *CVPR*, pp. 4694–4702, 2015.

[368] J. Ngiam, A. Khosla, M. Kim, J. Nam, H. Lee, and A. Ng. Multimodal deep learning. *ICML*, pp. 689–696, 2011.

[369] A. Nguyen *et al.* Synthesizing the preferred inputs for neurons in neural networks via deep generator networks. *NeurIPS*, pp. 3387–3395, 2016.

[370] J. Nocedal and S. Wright. Numerical optimization. *Springer*, 2006.

[371] S. Nowlan and G. Hinton. Simplifying neural networks by soft weight-sharing. *Neural Computation*, 4(4), pp. 473–493, 1992.

[372] M. Oquab, L. Bottou, I. Laptev, and J. Sivic. Learning and transferring mid-level image representations using convolutional neural networks. *CVPR*, pp. 1717–1724, 2014.

[373] G. Orr and K.-R. Müller (editors). Neural Networks: Tricks of the Trade, *Springer*, 1998.

[374] M. J. L. Orr. Introduction to radial basis function networks, *University of Edinburgh Technical Report, Centre of Cognitive Science*, 1996. ftp://ftp.cogsci.ed.ac.uk/pub/mjo/intro.ps.Z

[375] L. Ouyang *et al.* Training language models to follow instructions with human feedback. *arXiv*:2203.02155, 2022. https://arxiv.org/abs/2203.02155

[376] N. Papernot *et al.* Practical black-box attacks against machine learning. *ACM CCS Conference*, 2017. https://dl.acm.org/doi/pdf/10.1145/3052973.3053009

[377] N. Papernot *et al.* Distillation as a Defense to Adversarial Perturbations against Deep Neural Networks. *arXiv:1511.04508*, 2015. https://arxiv.org/abs/1511.04508

[378] J. Park and I. Sandberg. Universal approximation using radial-basis-function networks. *Neural Computation*, 3(1), pp. 246–257, 1991.

[379] J. Park and I. Sandberg. Approximation and radial-basis-function networks. *Neural Computation*, 5(2), pp. 305–316, 1993.

[380] O. Parkhi, A. Vedaldi, and A. Zisserman. Deep Face Recognition. *BMVC*, 1(3), pp. 6, 2015.

[381] R. Pascanu, T. Mikolov, and Y. Bengio. On the difficulty of training recurrent neural networks. *ICML*, 28, pp. 1310–1318, 2013.

[382] R. Pascanu, T. Mikolov, and Y. Bengio. Understanding the exploding gradient problem. *CoRR, abs/1211.5063*, 2012.

[383] D. Pathak, P. Krahenbuhl, J. Donahue, T. Darrell, and A. A. Efros. Context encoders: Feature learning by inpainting. *CVPR Conference*, 2016.

[384] J. Pennington, R. Socher, and C. Manning. Glove: Global Vectors for Word Representation. *EMNLP*, pp. 1532–1543, 2014.

[385] B. Perozzi, R. Al-Rfou, and S. Skiena. Deepwalk: Online learning of social representations. *KDD*, pp. 701-710, 2014.

[386] M. Peters *et al.* Deep contextualized word representations. *arXiv:1802.05365*, 2018. https://arxiv.org/abs/1802.05365

[387] J. Peters and S. Schaal. Reinforcement learning of motor skills with policy gradients. *Neural Networks*, 21(4), pp. 682–697, 2008.

[388] C. Peterson and J. Anderson. A mean field theory learning algorithm for neural networks. *Complex Systems*, 1(5), pp. 995-1019, 1987.

[389] F. Pineda. Generalization of back-propagation to recurrent neural networks. *Physical Review Letters*, 59(19), 2229, 1987.

[390] E. Polak. Computational methods in optimization: a unified approach. *Academic Press*, 1971.

[391] L. Polanyi and A. Zaenen. Contextual valence shifters. *Computing Attitude and Affect in Text: Theory and Applications*, pp. 1–10, Springer, 2006.

[392] G. Pollastri, D. Przybylski, B. Rost, and P. Baldi. Improving the prediction of protein secondary structure in three and eight classes using recurrent neural networks and profiles. *Proteins: Structure, Function, and Bioinformatics*, 47(2), pp. 228–235, 2002.

[393] J. Pollack. Recursive distributed representations. *Artificial Intelligence*, 46(1), pp. 77–105, 1990.

[394] B. Polyak and A. Juditsky. Acceleration of stochastic approximation by averaging. *SIAM Journal on Control and Optimization*, 30(4), pp. 838–855, 1992.

[395] D. Pomerleau. ALVINN: An autonomous land vehicle in a neural network. *Technical Report*, Carnegie Mellon University, 1989.

[396] B. Poole, J. Sohl-Dickstein, and S. Ganguli. Analyzing noise in autoencoders and deep networks. *arXiv:1406.1831*, 2014. https://arxiv.org/abs/1406.1831

[397] T. Plotz and S. Roth. Neural nearest neighbors networks. *NeurIPS*, pp. 1087–1098, 2018.

[398] A. Radford, L. Metz, and S. Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv:1511.06434*, 2015. https://arxiv.org/abs/1511.06434

[399] A. Radford *et al.* Language models are unsupervised multitask learners. *OpenAI blog*, 1(8), 2019.

[400] C. Raffel *et al.* Exploring the limits of transfer learning with a unified text-to-text transformer. *arXiv:1910.10683*, 2019. https://arxiv.org/abs/1910.10683

[401] A. Rahimi and B. Recht. Random features for large-scale kernel machines. *NeurIPS*, pp. 1177–1184, 2008.

[402] M.' A. Ranzato, Y-L. Boureau, and Y. LeCun. Sparse feature learning for deep belief networks. *NeurIPS*, pp. 1185–1192, 2008.

[403] M.' A. Ranzato, F. J. Huang, Y-L. Boureau, and Y. LeCun. Unsupervised learning of invariant feature hierarchies with applications to object recognition. *CVPR*, pp. 1–8, 2007.

[404] A. Rasmus, M. Berglund, M. Honkala, H. Valpola, and T. Raiko. Semi-supervised learning with ladder networks. *NeurIPS*, pp. 3546–3554, 2015.

[405] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi. Xnor-net: Imagenet classification using binary convolutional neural networks. *ECCV*, pp. 525–542, 2016.

[406] A. Razavian, H. Azizpour, J. Sullivan, and S. Carlsson. CNN features off-the-shelf: an astounding baseline for recognition. *CVPR Workshops*, pp. 806–813, 2014.

[407] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. You only look once: Unified, real-time object detection. *CVPR*, pp. 779–788, 2016.

[408] S. Reed, Z. Akata, X. Yan, L. Logeswaran, B. Schiele, and H. Lee. Generative adversarial text to image synthesis. *ICML Conference*, pp. 1060–1069, 2016.

[409] S. Reed and N. de Freitas. Neural programmer-interpreters. *arXiv:1511.06279*, 2015.

[410] R. Rehurek and P. Sojka. Software framework for topic modelling with large corpora. *LREC 2010 Workshop on New Challenges for NLP Frameworks*, pp. 45–50, 2010. https://radimrehurek.com/gensim/index.html

[411] M. Ren, R. Kiros, and R. Zemel. Exploring models and data for image question answering. *NeurIPS*, pp. 2953–2961, 2015.

[412] S. Ren, K. He, R. Girshick, and J. Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. *NeurIPS*, 2015.

[413] S. Rendle. Factorization machines. *IEEE ICDM Conference*, pp. 995–100, 2010.

[414] S. Rifai, P. Vincent, X. Muller, X. Glorot, and Y. Bengio. Contractive autoencoders: Explicit invariance during feature extraction. *ICML*, pp. 833–840, 2011.

[415] S. Rifai, Y. Dauphin, P. Vincent, Y. Bengio, and X. Muller. The manifold tangent classifier. *NeurIPS*, pp. 2294–2302, 2011.

[416] D. Rezende, S. Mohamed, and D. Wierstra. Stochastic backpropagation and approximate inference in deep generative models. *arXiv:1401.4082*, 2014. https://arxiv.org/abs/1401.4082

[417] R. Rifkin. Everything old is new again: a fresh look at historical approaches in machine learning. *Ph.D. Thesis*, Massachusetts Institute of Technology, 2002.

[418] R. Rifkin and A. Klautau. In defense of one-vs-all classification. *Journal of Machine Learning Research*, 5, pp. 101–141, 2004.

[419] V. Romanuke. Parallel Computing Center (Khmelnitskiy, Ukraine) represents an ensemble of 5 convolutional neural networks which performs on MNIST at 0.21 percent error rate. Retrieved 24 November 2016.

[420] X. Rong. Word2vec parameter learning explained. *arXiv:1411.2738*, 2014. https://arxiv.org/abs/1411.2738

[421] F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386, 1958.

[422] D. Ruck, S. Rogers, and M. Kabrisky. Feature selection using a multilayer perceptron. *Journal of Neural Network Computing*, 2(2), pp. 40–88, 1990.

[423] H. A. Rowley, S. Baluja, and T. Kanade. Neural network-based face detection. *IEEE TPAMI*, 20(1), pp. 23–38, 1998.

[424] D. Rumelhart, G. Hinton, and R. Williams. Learning representations by back-propagating errors. *Nature*, 323 (6088), pp. 533–536, 1986.

[425] D. Rumelhart, G. Hinton, and R. Williams. Learning internal representations by back-propagating errors. In *Parallel Distributed Processing: Explorations in the Microstructure of Cognition*, pp. 318–362, 1986.

[426] D. Rumelhart, D. Zipser, and J. McClelland. Parallel Distributed Processing, *MIT Press*, pp. 151–193, 1986.

[427] D. Rumelhart and D. Zipser. Feature discovery by competitive learning. *Cognitive science*, 9(1), pp. 75–112, 1985.

[428] G. Rummery, and M. Niranjan. Online Q-learning using connectionist systems (Vol. 37). *University of Cambridge, Department of Engineering*, 1994.

[429] A. M. Rush, S. Chopra, and J. Weston. A Neural Attention Model for Abstractive Sentence Summarization. *arXiv:1509.00685*, 2015. https://arxiv.org/abs/1509.00685

[430] S. Sabour, N. Frosst, and G. Hinton. Dynamic routing between capsules. *arXiv:1710.09829*, 2017. https://arxiv.org/abs/1710.09829

[431] R. Salakhutdinov, A. Mnih, and G. Hinton. Restricted Boltzmann machines for collaborative filtering. *ICML*, pp. 791–798, 2007.

[432] R. Salakhutdinov and G. Hinton. Semantic Hashing. *SIGIR Workshop on Information Retrieval and Applications of Graphical Models*, 2007.

[433] A. Santoro *et al.* One shot learning with memory-augmented neural networks. *arXiv:1605:06065*, 2016. https://arxiv.org/abs/1605.06065

[434] R. Salakhutdinov and G. Hinton. Deep Boltzmann machines. *AISTATS*, pp. 448–455, 2009.

[435] R. Salakhutdinov and H. Larochelle. Efficient Learning of Deep Boltzmann Machines. *AISTATS*, pp. 693–700, 2010.

[436] T. Salimans and D. Kingma. Weight normalization: A simple reparameterization to accelerate training of deep neural networks. *NeurIPS*, pp. 901–909, 2016.

[437] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen. Improved techniques for training gans. *NeurIPS*, pp. 2234–2242, 2016.

[438] A. Samuel. Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3, pp. 210–229, 1959.

[439] T Sanger. Neural network learning control of robot manipulators using gradually increasing task difficulty. *IEEE Transactions on Robotics and Automation*, 10(3), 1994.

[440] H. Sarimveis, A. Alexandridis, and G. Bafas. A fast training algorithm for RBF networks based on subtractive clustering. *Neurocomputing*, 51, pp. 501–505, 2003.

[441] W. Saunders, G. Sastry, A. Stuhlmueller, and O. Evans. Trial without Error: Towards Safe Reinforcement Learning via Human Intervention. *arXiv:1707.05173*, 2017. https://arxiv.org/abs/1707.05173

[442] A. Saxe, P. Koh, Z. Chen, M. Bhand, B. Suresh, and A. Ng. On random weights and unsupervised feature learning. *ICML*, pp. 1089–1096, 2011.

[443] A. Saxe, J. McClelland, and S. Ganguli. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. *arXiv:1312.6120*, 2013. https://arxiv.org/abs/1312.6120

[444] F. Scarselli *et al.* The graph neural network model. *IEEE TNNLS*, 20(1), pp. 61–80, 2008.

[445] S. Schaal. Is imitation learning the route to humanoid robots? *Trends in Cognitive Sciences*, 3(6), pp. 233–242, 1999.

[446] T. Schaul, J. Quan, I. Antonoglou, and D. Silver. Prioritized experience replay. *arXiv:1511.05952*, 2015. https://arxiv.org/abs/1511.05952

[447] T. Schaul, S. Zhang, and Y. LeCun. No more pesky learning rates. *ICML*, pp. 343–351, 2013.

[448] M. Schlichtkrull *et al.* Modeling relational data with graph convolutional networks. *European Semantic Web Conference*, Springer, 2018.

[449] B. Schölkopf *et al.* Comparing support vector machines with Gaussian kernels to radial basis function classifiers. *IEEE Transactions on Signal Processing*, 45(11), pp. 2758–2765, 1997.

[450] J. Schmidhuber. Deep learning in neural networks: An overview. *Neural Networks*, 61, pp. 85–117, 2015.

[451] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz. Trust region policy optimization. *ICML*, 2015.

[452] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. High-dimensional continuous control using generalized advantage estimation. *ICLR*, 2016.

[453] J. Schulman *et al.* Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. https://arxiv.org/abs/1707.06347

[454] M. Schuster and K. Paliwal. Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11), pp. 2673–2681, 1997.

[455] H. Schwenk and Y. Bengio. Boosting neural networks. *Neural Computation*, 12(8), pp. 1869–1887, 2000.

[456] S. Sedhain, A. K. Menon, S. Sanner, and L. Xie. Autorec: Autoencoders meet collaborative filtering. *WWW*, pp. 111–112, 2015.

[457] T. J. Sejnowski. Higher-order Boltzmann machines. *AIP Conference Proceedings*, 15(1), pp. 298–403, 1986.

[458] G. Seni and J. Elder. Ensemble methods in data mining: Improving accuracy through combining predictions. *Morgan and Claypool*, 2010.

[459] B. Sennrich, B. Haddow, and A. Birch. Neural machine translation of rare words with subword units. *arXiv:1508.07909*, 2015. https://arxiv.org/abs/1508.07909

[460] I. Serban, A. Sordoni, R. Lowe, L. Charlin, J. Pineau, A. Courville, and Y. Bengio. A hierarchical latent variable encoder-decoder model for generating dialogues. *AAAI*, pp. 3295–3301, 2017.

[461] I. Serban, A. Sordoni, Y. Bengio, A. Courville, and J. Pineau. Building end-to-end dialogue systems using generative hierarchical neural network models. *AAAI*, pp. 3776–3784, 2016.

[462] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus, and Y. LeCun. Overfeat: Integrated recognition, localization and detection using convolutional networks. *arXiv:1312.6229*, 2013. https://arxiv.org/abs/1312.6229

[463] J. Shewchuk. An introduction to the conjugate gradient method without the agonizing pain. *Technical Report, CMU-CS-94-125*, Carnegie-Mellon University, 1994.

[464] H. Siegelmann and E. Sontag. On the computational power of neural nets. *Journal of Computer and System Sciences*, 50(1), pp. 132–150, 1995.

[465] D. Silver *et al.* Mastering the game of Go with deep neural networks and tree search. *Nature*, 529.7587, pp. 484–489, 2016.

[466] D. Silver *et al.* Mastering the game of go without human knowledge. *Nature*, 550.7676, pp. 354–359, 2017.

[467] D. Silver *et al.* Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv*, 2017. https://arxiv.org/abs/1712.01815

[468] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter. Pegasos: Primal estimated sub-gradient solver for SVM. *Mathematical Programming*, 127(1), pp. 3–30, 2011.

[469] E. Shelhamer, J., Long, and T. Darrell. Fully convolutional networks for semantic segmentation. *IEEE TPAMI*, 39(4), pp. 640–651, 2017.

[470] J. Sietsma and R. Dow. Creating artificial neural networks that generalize. *Neural Networks*, 4(1), pp. 67–79, 1991.

[471] B. W. Silverman. Density Estimation for Statistics and Data Analysis. *Chapman and Hall*, 1986.

[472] P. Simard, D. Steinkraus, and J. C. Platt. Best practices for convolutional neural networks applied to visual document analysis. *ICDAR*, pp. 958–962, 2003.

[473] H. Simon. The Sciences of the Artificial. *MIT Press*, 1996.

[474] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv:1409.1556*, 2014. https://arxiv.org/abs/1409.1556

[475] K. Simonyan and A. Zisserman. Two-stream convolutional networks for action recognition in videos. *NeurIPS*, pp. 568–584, 2014.

[476] K. Simonyan, A. Vedaldi, and A. Zisserman. Deep inside convolutional networks: Visualising image classification models and saliency maps. *arXiv:1312.6034*, 2013. https://arxiv.org/abs/1312.6034

[477] P. Smolensky. Information processing in dynamical systems: Foundations of harmony theory. *Parallel Distributed Processing: Explorations in the Microstructure of Cognition*, Volume 1: Foundations. pp. 194–281, 1986.

[478] J. Snoek, H. Larochelle, and R. Adams. Practical bayesian optimization of machine learning algorithms. *NeurIPS*, pp. 2951–2959, 2013.

[479] K. Sohn, H. Lee, and X. Yan. Learning structured output representation using deep conditional generative models. *NeurIPS*, 2015.

[480] R. Solomonoff. A system for incremental learning based on algorithmic probability. *Sixth Israeli Conference on Artificial Intelligence, CVPR*, pp. 515–527, 1994.

[481] Y. Song, A. Elkahky, and X. He. Multi-rate deep learning for temporal recommendation. *ACM SIGIR Conference*, pp. 909–912, 2016.

[482] J. Springenberg, A. Dosovitskiy, T. Brox, and M. Riedmiller. Striving for simplicity: The all convolutional net. *arXiv:1412.6806*, 2014. https://arxiv.org/abs/1412.6806

[483] N. Srivastava *et al.* Dropout: A simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research*, 15(1), pp. 1929–1958, 2014.

[484] N. Srivastava and R. Salakhutdinov. Multimodal learning with deep Boltzmann machines. *NeurIPS*, pp. 2222–2230, 2012.

[485] N. Srivastava, R. Salakhutdinov, and G. Hinton. Modeling documents with deep Boltzmann machines. *Uncertainty in Artificial Intelligence*, 2013.

[486] R. K. Srivastava, K. Greff, and J. Schmidhuber. Highway networks. *arXiv:1505.00387*, 2015. https://arxiv.org/abs/1505.00387

[487] A. Storkey. Increasing the capacity of a Hopfield network without sacrificing functionality. *Artificial Neural Networks*, pp. 451–456, 1997.

[488] F. Strub and J. Mary. Collaborative filtering with stacked denoising autoencoders and sparse inputs. *NeurIPS Workshop on Machine Learning for eCommerce*, 2015.

[489] S. Sukhbaatar, J. Weston, and R. Fergus. End-to-end memory networks. *NeurIPS*, pp. 2440–2448, 2015.

[490] Y. Sun, D. Liang, X. Wang, and X. Tang. Deepid3: Face recognition with very deep neural networks. *arXiv:1502.00873*, 2013. https://arxiv.org/abs/1502.00873

[491] Y. Sun, X. Wang, and X. Tang. Deep learning face representation from predicting 10,000 classes. *CVPR*, pp. 1891–1898, 2014.

[492] M. Sundermeyer, R. Schluter, and H. Ney. LSTM neural networks for language modeling. *Interspeech*, 2010.

[493] M. Sundermeyer, T. Alkhouli, J. Wuebker, and H. Ney. Translation modeling with bidirectional recurrent neural networks. *EMNLP*, pp. 14–25, 2014.

[494] I. Sutskever, J. Martens, G. Dahl, and G. Hinton. On the importance of initialization and momentum in deep learning. *ICML*, pp. 1139–1147, 2013.

[495] I. Sutskever and T. Tieleman. On the convergence properties of contrastive divergence. *International Conference on Artificial Intelligence and Statistics*, pp. 789–795, 2010.

[496] I. Sutskever, O. Vinyals, and Q. V. Le. Sequence to sequence learning with neural networks. *NeurIPS*, pp. 3104–3112, 2014.

[497] I. Sutskever and V. Nair. Mimicking Go experts with convolutional neural networks. *International Conference on Artificial Neural Networks*, pp. 101–110, 2008.

[498] R. Sutton. Learning to Predict by the Method of Temporal Differences, *Machine Learning*, 3, pp. 9–44, 1988.

[499] R. Sutton and A. Barto. Reinforcement Learning: An Introduction. *MIT Press*, 1998.

[500] R. Sutton, D. McAllester, S. Singh, and Y. Mansour. Policy gradient methods for reinforcement learning with function approximation. *NeurIPS*, pp. 1057–1063, 2000.

[501] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. *CVPR*, pp. 1–9, 2015.

[502] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna. Rethinking the inception architecture for computer vision. *CVPR*, pp. 2818–2826, 2016.

[503] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. Alemi. Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning. *AAAI*, pp. 4278–4284, 2017.

[504] X. Tang, H. Yao, Y. Sun, Y. Wang, J. Tang, C. Aggarwal, P. Mitra, and S. Wang. Investigating and mitigating degree-related biases in graph convolutional networks. *CIKM Conference*, 2020.

[505] G. Taylor, R. Fergus, Y. LeCun, and C. Bregler. Convolutional learning of spatio-temporal features. *ECCV*, pp. 140–153, 2010.

[506] G. Taylor, G. Hinton, and S. Roweis. Modeling human motion using binary latent variables. *NeurIPS*, 2006.

[507] C. Thornton, F. Hutter, H. H. Hoos, and K. Leyton-Brown. Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms. *KDD*, pp. 847–855, 2013.

[508] T. Tieleman. Training restricted Boltzmann machines using approximations to the likelihood gradient. *ICML*, pp. 1064–1071, 2008.

[509] G. Tesauro. Practical issues in temporal difference learning. *NeurIPS*, pp. 259–266, 1992.

[510] G. Tesauro. Td-gammon: A self-teaching backgammon program. *Applications of Neural Networks*, Springer, pp. 267–285, 1992.

[511] G. Tesauro. Temporal difference learning and TD-Gammon. *Communications of the ACM*, 38(3), pp. 58–68, 1995.

[512] Y. Teh and G. Hinton. Rate-coded restricted Boltzmann machines for face recognition. *NeurIPS*, 2001.

[513] S. Thrun. Learning to play the game of chess *NeurIPS*, pp. 1069–1076, 1995.

[514] S. Thrun and L. Platt. Learning to learn. *Springer*, 2012.

[515] Y. Tian, Q. Gong, W. Shang, Y. Wu, and L. Zitnick. ELF: An extensive, lightweight and flexible research platform for real-time strategy games. *arXiv:1707.01067*, 2017. https://arxiv.org/abs/1707.01067

[516] A. Tikhonov and V. Arsenin. Solution of ill-posed problems. *Winston and Sons*, 1977.

[517] D. Tran *et al.* Learning spatiotemporal features with 3d convolutional networks. *ICCV*, 2015.

[518] D. Tsipras *et al.* Robustness may be at odds with accuracy. *ICLR Conference*, 2019.

[519] R. Uijlings, A. van de Sande, T. Gevers, and M. Smeulders. Selective search for object recognition. *International Journal of Computer Vision*, 104(2), 2013.

[520] H. Valpola. From neural PCA to deep unsupervised learning. *Advances in Independent Component Analysis and Learning Machines*, pp. 143–171, Elsevier, 2015.

[521] A. Vaswani *et al.* Attention is All you Need. *NeurIPS*, 2017. https://arxiv.org/abs/1706.03762

[522] A. Vedaldi and K. Lenc. Matconvnet: Convolutional neural networks for matlab. *ACM International Conference on Multimedia*, pp. 689–692, 2005. http://www.vlfeat.org/matconvnet/

[523] V. Veeriah, N. Zhuang, and G. Qi. Differential recurrent neural networks for action recognition. *ICCV*, pp. 4041–4049, 2015.

[524] A. Veit, M. Wilber, and S. Belongie. Residual networks behave like ensembles of relatively shallow networks. *NeurIPS*, pp. 550–558, 2016.

[525] P. Velickovic *at al.* Graph attention networks. *arXiv:1710.10903*, 2017. https://arxiv.org/abs/1710.10903

[526] P. Vincent, H. Larochelle, Y. Bengio, and P. Manzagol. Extracting and composing robust features with denoising autoencoders. *ICML*, pp. 1096–1103, 2008.

[527] O. Vinyals, C. Blundell, T. Lillicrap, and D. Wierstra. Matching networks for one-shot learning. *NeurIPS*, pp. 3530–3638, 2016.

[528] O. Vinyals and Q. Le. A Neural Conversational Model. *arXiv:1506.05869*, 2015. https://arxiv.org/abs/1506.05869

[529] O. Vinyals, A. Toshev, S. Bengio, and D. Erhan. Show and tell: A neural image caption generator. *CVPR*, pp. 3156–3164, 2015.

[530] J. Walker, C. Doersch, A. Gupta, and M. Hebert. An uncertain future: Forecasting from static images using variational autoencoders. *ECCV*, pp. 835–851, 2016.

[531] L. Wan, M. Zeiler, S. Zhang, Y. LeCun, and R. Fergus. Regularization of neural networks using dropconnect. *ICML*, pp. 1058–1066, 2013.

[532] B. Wang *et al.* On position embeddings in BERT. *ICLR*, 2021.

[533] H. Wang, N. Wang, and D. Yeung. Collaborative deep learning for recommender systems. *KDD*, pp. 1235–1244, 2015.

[534] L. Wang, Y. Qiao, and X. Tang. Action recognition with trajectory-pooled deep-convolutional descriptors. *CVPR*, pp. 4305–4314, 2015.

[535] S. Wang, C. Aggarwal, and H. Liu. Using a random forest to inspire a neural network and improving on it. *SDM Conference*, 2017.

[536] S. Wang, C. Aggarwal, and H. Liu. Randomized feature engineering as a fast and accurate alternative to kernel methods. *KDD*, 2017.

[537] T. Wang, D. Wu, A. Coates, and A. Ng. End-to-end text recognition with convolutional neural networks. *International Conference on Pattern Recognition*, pp. 3304–3308, 2012.

[538] X. Wang and A. Gupta. Generative image modeling using style and structure adversarial networks. *ECCV*, 2016.

[539] C. J. H. Watkins. Learning from delayed rewards. *PhD Thesis*, King's College, Cambridge, 1989.

[540] C. J. H. Watkins and P. Dayan. Q-learning. *Machine Learning*, 8(3–4), pp. 279–292, 1992.

[541] M. Welling, M. Rosen-Zvi, and G. Hinton. Exponential family harmoniums with an application to information retrieval. *NeurIPS*, pp. 1481–1488, 2005.

[542] A. Wendemuth. Learning the unlearnable. *Journal of Physics A: Math. Gen.*, 28, pp. 5423–5436, 1995.

[543] P. Werbos. Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences. *PhD thesis, Harvard University*, 1974.

[544] P. Werbos. The roots of backpropagation: from ordered derivatives to neural networks and political forecasting (Vol. 1). *John Wiley and Sons*, 1994.

[545] P. Werbos. Backpropagation through time: what it does and how to do it. *Proceedings of the IEEE*, 78(10), pp. 1550–1560, 1990.

[546] J. Weston, A. Bordes, S. Chopra, A. Rush, B. van Merrienboer, A. Joulin, and T. Mikolov. Towards ai-complete question answering: A set of pre-requisite toy tasks. *arXiv:1502.05698*, 2015. https://arxiv.org/abs/1502.05698

[547] J. Weston, S. Chopra, and A. Bordes. Memory networks. *ICLR*, 2015.

[548] J. Weston and C. Watkins. Multi-class support vector machines. *Technical Report CSD-TR-98-04*, Department of Computer Science, Royal Holloway, University of London, May, 1998.

[549] D. Wettschereck and T. Dietterich. Improving the performance of radial basis function networks by learning center locations. *NeurIPS*, pp. 1133–1140, 1992.

[550] B. Widrow and M. Hoff. Adaptive switching circuits. *IRE WESCON Convention Record*, 4(1), pp. 96–104, 1960.

[551] S. Wieseler and H. Ney. A convergence analysis of log-linear training. *NeurIPS*, pp. 657–665, 2011.

[552] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4), pp. 229–256, 1992.

[553] F. Wu *et al.* Simplifying graph convolutional networks. *arXiv:1902.07153* 2019. https://arxiv.org/abs/1902.07153

[554] C. Wu, A. Ahmed, A. Beutel, A. Smola, and H. Jing. Recurrent recommender networks. *WSDM Conference*, pp. 495–503, 2017.

[555] Y. Wu, C. DuBois, A. Zheng, and M. Ester. Collaborative denoising auto-encoders for top-n recommender systems. *WSDM Conference*, pp. 153–162, 2016.

[556] Y. Wu *et al.* Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv:1609.08144*, 2016. https://arxiv.org/abs/1609.08144

[557] Z. Wu. Global continuation for distance geometry problems. *SIAM Journal of Optimization*, 7, pp. 814–836, 1997.

[558] Z. Wu *et al.* A comprehensive survey on graph neural networks. *IEEE TNNLS*, 2020.

[559] S. Xie, R. Girshick, P. Dollar, Z. Tu, and K. He. Aggregated residual transformations for deep neural networks. *arXiv:1611.05431*, 2016. https://arxiv.org/abs/1611.05431

[560] E. Xing, R. Yan, and A. Hauptmann. Mining associated text and images with dual-wing harmoniums. *Uncertainty in Artificial Intelligence*, 2005.

[561] C. Xiong, S. Merity, and R. Socher. Dynamic memory networks for visual and textual question answering. *ICML*, pp. 2397–2406, 2016.

[562] K. Xu *et al.* Show, attend, and tell: Neural image caption generation with visual attention. *ICML*, 2015.

[563] O. Yadan, K. Adams, Y. Taigman, and M. Ranzato. Multi-gpu training of convnets. *arXiv:1312.5853*, 2013. https://arxiv.org/abs/1312.5853

[564] Z. Yang, X. He, J. Gao, L. Deng, and A. Smola. Stacked attention networks for image question answering. *CVPR*, pp. 21–29, 2016.

[565] Z. Yang *et al.* Xlnet: Generalized autoregressive pretraining for language understanding. *arXiv:1906.08237*, 2019. https://arxiv.org/abs/1906.08237

[566] X. Yao. Evolving artificial neural networks. *Proceedings of the IEEE*, 87(9), pp. 1423–1447, 1999.

[567] Z. Ying *et al.* Graph convolutional neural networks for web-scale recommender systems. *KDD*, 2018.

[568] Z. Ying *et al.* Hierarchical graph representation learning with differentiable pooling. *NeurIPS*, 2018.
Code: https://github.com/RexYing/diffpool

[569] F. Yu and V. Koltun. Multi-scale context aggregation by dilated convolutions. *arXiv:1511.07122*, 2015. https://arxiv.org/abs/1511.07122

[570] H. Yu and B. Wilamowski. Levenberg–Marquardt training. *Industrial Electronics Handbook*, 5(12), 1, 2011.

[571] L. Yu, W. Zhang, J. Wang, and Y. Yu. SeqGAN: Sequence Generative Adversarial Nets with Policy Gradient. *AAAI*, pp. 2852–2858, 2017.

[572] W. Yu, W. Cheng, C. Aggarwal, K. Zhang, H. Chen, and Wei Wang. NetWalk: A flexible deep embedding approach for anomaly Detection in dynamic networks, *KDD*, 2018.

[573] W. Yu, C. Zheng, W. Cheng, C. Aggarwal, D. Song, B. Zong, H. Chen, and W. Wang. Learning deep network representations with adversarially regularized autoencoders. *KDD*, 2018.

[574] S. Zagoruyko and N. Komodakis. Wide residual networks. *arXiv:1605.07146*, 2016. https://arxiv.org/abs/1605.07146

[575] W. Zaremba and I. Sutskever. Reinforcement learning neural turing machines. *arXiv:1505.00521*, 2015. https://arxiv.org/abs/1505.00521

[576] W. Zaremba, T. Mikolov, A. Joulin, and R. Fergus. Learning simple algorithms from examples. *ICML*, pp. 421–429, 2016.

[577] W. Zaremba, I. Sutskever, and O. Vinyals. Recurrent neural network regularization. *arXiv:1409.2329*, 2014. https://arxiv.org/abs/1409.2329

[578] M. Zeiler. ADADELTA: an adaptive learning rate method. *arXiv:1212.5701*, 2012. https://arxiv.org/abs/1212.5701

[579] M. Zeiler, D. Krishnan, G. Taylor, and R. Fergus. Deconvolutional networks. *CVPR*, pp. 2528–2535, 2010.

[580] M. Zeiler, G. Taylor, and R. Fergus. Adaptive deconvolutional networks for mid and high level feature learning. *ICCV*, pp. 2018-2025, 2011.

[581] M. Zeiler and R. Fergus. Visualizing and understanding convolutional networks. *ECCV*, Springer, pp. 818–833, 2013.

[582] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals. Understanding deep learning requires rethinking generalization. *arXiv:1611.03530*, 2016. https://arxiv.org/abs/1611.03530

[583] L. Zhang, C. Aggarwal, and G.-J. Qi. Stock Price Prediction via Discovering Multi-Frequency Trading Patterns. *KDD*, 2017.

[584] S. Zhang, L. Yao, and A. Sun. Deep learning based recommender system: A survey and new perspectives. *arXiv:1707.07435*, 2017. https://arxiv.org/abs/1707.07435

[585] X. Zhang, J. Zhao, and Y. LeCun. Character-level convolutional networks for text classification. *NeurIPS*, pp. 649–657, 2015.

[586] J. Zhao, M. Mathieu, and Y. LeCun. Energy-based generative adversarial network. *arXiv:1609.03126*, 2016. https://arxiv.org/abs/1609.03126

[587] D. Zheng, M. Wang, Q. Gan, Z. Zhang, and G. Karypis. Learning Graph Neural Networks with Deep Graph Library, *WWW Tutorial*, 2020. https://youtu.be/bD6S3xUXNds

[588] V. Zhong, C. Xiong, and R. Socher. Seq2SQL: Generating structured queries from natural language using reinforcement learning. *arXiv:1709.00103*, 2017. https://arxiv.org/abs/1709.00103

[589] C. Zhou and R. Paffenroth. Anomaly detection with robust deep autoencoders. *KDD*, pp. 665–674, 2017.

[590] M. Zhou, Z. Ding, J. Tang, and D. Yin. Micro Behaviors: A new perspective in e-commerce recommender systems. *WSDM Conference*, 2018.

[591] Z.-H. Zhou. Ensemble methods: Foundations and algorithms. *CRC Press*, 2012.

[592] Z.-H. Zhou, J. Wu, and W. Tang. Ensembling neural networks: many could be better than all. *Artificial Intelligence*, 137(1–2), pp. 239–263, 2002.

[593] Y. Zhu *et al.* Aligning books and movies: Towards story-like visual explanations by watching movies and reading books. *CVPR*, 2019.

[594] C. Zitnick and P. Dollar. Edge Boxes: Locating object proposals from edges. *ECCV*, pp. 391–405, 2014.

[595] B. Zoph and Q. V. Le. Neural architecture search with reinforcement learning. *arXiv:1611.01578*, 2016. https://arxiv.org/abs/1611.01578

[596] http://caffe.berkeleyvision.org/

[597] http://torch.ch/

[598] https://pypi.org/project/Theano/

[599] https://www.tensorflow.org/

[600] https://keras.io/

[601] https://lasagne.readthedocs.io/en/latest/

[602] http://www.netflixprize.com/community/topic_1537.html

[603] https://arxiv.org/abs/1609.08144

[604] https://github.com/karpathy/char-rnn

[605] http://www.image-net.org/

[606] http://www.image-net.org/challenges/LSVRC/

[607] https://www.cs.toronto.edu/~kriz/cifar.html

[608] http://code.google.com/p/cuda-convnet/

[609] http://caffe.berkeleyvision.org/gathered/examples/feature_extraction.html

[610] https://github.com/caffe2/caffe2/wiki/Model-Zoo

[611] http://scikit-learn.org/

[612] http://clic.cimec.unitn.it/composes/toolkit/

[613] https://github.com/stanfordnlp/GloVe

[614] https://deeplearning4j.org/

[615] https://code.google.com/archive/p/word2vec/

[616] https://www.tensorflow.org/tutorials/word2vec/

[617] https://github.com/aditya-grover/node2vec

[618] https://github.com/caglar/autoencoders

[619] https://github.com/y0ast

[620] https://github.com/fastforwardlabs/vae-tf/tree/master

[621] https://science.education.nih.gov/supplements/webversions/BrainAddiction/guide/lesson2-1.html

[622] https://www.ibm.com/us-en/marketplace/deep-learning-platform

[623] https://www.youtube.com/watch?v=2fRnHVVLf1Y&list=PLiPvV5TNogxKKwvKb-1RKwkq2hm7ZvpHz0

[624] https://www.coursera.org/learn/neural-networks-deep-learning

[625] https://archive.ics.uci.edu/ml/datasets.html

[626] http://www.bbc.com/news/technology-35785875

[627] https://deepmind.com/blog/exploring-mysteries-alphago/

[628] http://deeplearning.mit.edu/

[629] http://karpathy.github.io/2016/05/31/rl/

[630] https://github.com/hughperkins/kgsgo-dataset-preprocessor

[631] https://www.wired.com/2016/03/two-moves-alphago-lee-sedol-redefined- future/

[632] https://qz.com/639952/googles-ai-won-the-game-go-by-defying-millennia-of-basic-human-instinct/

[633] http://www.mujoco.org/

[634] https://sites.google.com/site/gaepapersupp/home

[635] https://www.youtube.com/watch?v=1L0TKZQcUtA&list=PLrAXtmErZgOeiKm4-sgNOknGvNjby9efdf

[636] https://openai.com/

[637] http://jaberg.github.io/hyperopt/

[638] http://www.cs.ubc.ca/labs/beta/Projects/SMAC/

[639] https://github.com/JasperSnoek/spearmint

[640] http://colah.github.io/posts/2015-08-Understanding-LSTMs/

[641] https://www.youtube.com/watch?v=2pWv7GOvuf0

[642] https://gym.openai.com

[643] https://universe.openai.com

[644] https://github.com/facebookresearch/ParlAI

[645] https://github.com/openai/baselines

[646] https://github.com/carpedm20/deep-rl-tensorflow

[647] https://github.com/matthiasplappert/keras-rl

[648] http://apollo.auto/

[649] https://github.com/Element-Research/rnn/blob/master/examples/

[650] https://github.com/lmthang/nmt.matlab

[651] https://github.com/carpedm20/NTM-tensorflow

[652] https://github.com/camigord/Neural-Turing-Machine

[653] https://github.com/SigmaQuan/NTM-Keras

[654] https://github.com/snipsco/ntm-lasagne

[655] https://github.com/kaishengtai/torch-ntm

[656] https://github.com/facebook/MemNN

[657] https://github.com/carpedm20/MemN2N-tensorflow

[658] https://github.com/YerevaNN/Dynamic-memory-networks-in-Theano

[659] https://github.com/carpedm20/DCGAN-tensorflow

[660] https://github.com/carpedm20

[661] https://github.com/jacobgil/keras-dcgan

[662] https://github.com/wiseodd/generative-models

[663] https://github.com/paarthneekhara/text-to-image

[664] https://developer.nvidia.com/cudnn

[665] http://www.nvidia.com/object/machine-learning.html

[666] https://developer.nvidia.com/deep-learning-frameworks

[667] http://snap.stanford.edu

[668] http://snap.stanford.edu/graphsage/

[669] https://github.com/microsoft/tf-gnn-samples

[670] https://www.dgl.ai/

[671] https://github.com/rusty1s/pytorch_geometric

[672] https://msturing.org/

[673] https://www.tensorflow.org/tutorials/text/transformer

[674] https://github.com/google-research/bert

[675] https://github.com/google-research/text-to-text-transfer-transformer

[676] https://github.com/tensorflow/mesh/blob/master/mesh_tensorflow/transformer/moe.py

[677] https://github.com/zihangdai/xlnet

[678] https://github.com/allenai/bilm-tf

[679] https://adversarial-ml-tutorial.org/

[680] https://github.com/huggingface/transformers

[681] https://github.com/google/trax

[682] https://chat.openai.com/chatgpt

# Index

**Symbols**
$t$-SNE, 95
$\epsilon$-Greedy Algorithm, 392
$L_1$-Regularization, 180
$L_2$-Regularization, 178

**A**
Activation, 6
AdaDelta Algorithm, 137
AdaGrad, 136
Adaline, 78
Adam Algorithm, 138
Adaptive Linear Neuron, 78
Adversarial Learning, 463
AlexNet, 327
AlphaGo, 415
ALVINN Self-Driving System, 426
Annealed Importance Sampling, 263
Ant Hypothesis, 389
Apollo Self-Driving, 431
Associative Memory, 234
Associative Recall, 234
Attention Layer, 443
Attention Mechanisms, 431, 436
Autoencoders, 88, 347
Automatic Differentiation, 68
Autoregressive Model, 298
Average-Pooling, 316

**B**
Backpropagation, 38
Backpropagation through Time (BPTT), 273, 274
Bagging, 182
Batch Normalization, 150
BERT, 451, 482, 484
BFGS, 148, 161
Bidirectional Recurrent Networks, 275, 297
Black-Box Attack, 463
Bucket-of-Models, 184
Byte-Pair Encoding, 452

**C**
Caffe, 25, 68, 161, 303
Chain Rule for Vectored Derivatives, 52
Chatbots, 423
ChatGPT, 453
CIFAR-10, 308, 359
Competitive Learning, 476
Computational Graph, 15
Conditional Generative Adversarial Network (CGAN), 471
Conditional Variational Autoencoders, 208
Conjugate Gradient Method, 145
Connectionist Temporal Classification, 301
Content-Addressable Memory, 234, 461
Continuation Learning, 194
Continuous Action Spaces, 413
Continuous Bag-of-Words Model (CBOW Model), 100
Contractive Autoencoder, 199
Contrastive Divergence, 245
Conversational Systems, 423
Convolutional Neural Networks, 20, 291, 305
Convolution Operation, 307
Covariate Shift, 150


C. C. Aggarwal, *Neural Networks and Deep Learning*,
https://doi.org/10.1007/978-3-031-29642-0

Cross-Entropy Loss, 13
Cross-Validation, 177
cuDNN, 155
Curriculum Learning, 194

**D**
Data Augmentation, 326
Data Parallelism, 157
Davidson–Fletcher–Powell (DFP), 149
DCGAN, 470
Deconvolution, 348
Deep Belief Network, 262
Deep Boltzmann Machine, 261
DeepLearning4j, 114
Defensive Distillation, 466
Deformable Parts Model, 357
Delta Rule, 78
De-noising Autoencoder, 198
DenseNet, 337
Dialog Systems, 423
Dilated Convolution, 351, 357
Distributional Shift, 429
Doc2vec, 114
Double Backpropagation, 211
DropConnect, 185, 187
Dropout, 185

**E**
Early Stopping, 188
Echo-State Networks, 282, 297, 303
EdgeBoxes, 354
Elman Network, 302
ELMo, 290, 303
Energy-Efficient Computing, 482
Ensembles, 182
Exploding Gradient Problem, 124
External Memory, 459

**F**
Facial Recognition, 358
FC7 Features, 328, 340
Feature Co-Adaptation, 187
Feature Preprocessing, 62
Few-Shot Learning, 481
Filters for Convolution, 308
Finite Difference Methods, 408
Fisher's Linear Discriminant, 78
FractalNet, 357
Fractional Convolution, 324
Fractionally Strided Convolution, 351
Full-Padding, 313
Fully Convolutional Networks, 317

**G**
Gated Graph Neural Networks, 374
Gated Recurrent Unit (GRU), 287
Generalization Error, 169
Generative Adversarial Networks (GANs), 208, 467
Gibbs Sampling, 240
Glorot Initialization, 66, 132
GloVe, 114
GoogLeNet, 333, 357
GPT-$n$, 451, 484
Gradient-Based Visualization, 342
Gradient Clipping, 139, 280
Graph Convolutional Function, 368
Graph Neural Networks, 361
GraphSAGE, 369
Graph U-Net, 380
Graphics Processor Units (GPUs), 154
Guided Backpropagation, 343

**H**
Half-Padding, 312
Handwriting Recognition, 301
Hard Tanh Activation, 11
Harmonium, 242
Hash-Based Compression, 159
Hebbian Learning Rule, 235
Helmholtz Machine, 264
Hessian, 140
Hierarchical Feature Engineering, 320
Hierarchical Softmax, 114
Hold-Out, 177
Hopfield Networks, 232
Hubel and Wiesel, 306
Hybrid Parallelism, 157
Hyperbolic Tangent Activation, 10
Hyperopt, 64, 162
Hyperparameter Parallelism, 156

**I**
Identity Activation, 10
ILSVRC, 23, 357
Image Captioning, 291
ImageNet, 22, 306
ImageNet Competition, 23, 306

Image Retrieval, 352
Imitation Learning, 426
Inception Architecture, 333
Information Extraction, 266
InstructGPT, 453
Interpolation, 226

**J**
Jacobian, 51

**K**
Keras, 25
Kernels for Convolution, 308
Kohonen Self-Organizing Map, 478

**L**
Ladder Networks, 211
Lasagne, 25
Layer Normalization, 153, 281
L-BFGS, 148, 150, 161
Leaky ReLU, 130
Learning Rate Decay, 60
Learning-to-Learn, 481
Least Squares Regression, 76
Leave-One-Out Cross-Validation, 177
LeNet-5, 306
Levenberg–Marquardt Algorithm, 161
Linear Activation, 10
Linear Conjugate Gradient Method, 148
Liquid-State Machines, 283, 303
Logistic Regression, 81
Logistic Loss, 13
Logistic Matrix Factorization, 93
Logistic Regression, 13
Lottery Ticket Hypothesis, 158

**M**
Machine Translation, 292, 442
Mark I Perceptron, 9
Markov Chain Monte Carlo (MCMC), 240
Markov Decision Process, 394
MatConvNet, 359
Matrix Calculus Notation, 51
Matrix Factorization, 88
Maximum-Likelihood Estimation, 82
Maxout Networks, 131
Max-Pooling, 315
McCulloch-Pitts Model, 9
MCG, 354
Mean-Field Boltzmann Machine, 263
Memory Networks, 459
Meta-Learning, 481
Mimic Models, 159
MNIST Database, 21
Model Compression, 157, 482
Model Parallelism, 156
Momentum-based Learning, 133
Monte Carlo Tree Search, 413
Multi-Armed Bandits, 391
Multiclass Models, 84
Multiclass Perceptron, 84
Multiclass SVM, 85
Multilayer Neural Networks, 13
Multimodal Learning, 95, 256
Multinomial Logistic Regression, 13, 86

**N**
Nash Equilibrium, 469
Neocognitron, 306
Nesterov Momentum, 134
Neural Architecture Search, 64, 428
Neural Gas, 484
Neural Turing Machines, 459
Neuromorphic Computing, 483
Newton Update, 141
Nonlinear Conjugate Gradient Method, 148
NVIDIA CUDA Deep Neural Network Library, 155

**O**
Object Localization, 352
One-hot Encoding, 20
Off-Policy Reinforcement Learning, 404
On-Policy Reinforcement Learning, 404
OpenAI, 429
Overfeat, 354, 357

**P**
Parameter Sharing, 196
ParlAI, 431
Partition Function, 239
Perceptron, 5
Persistent Contrastive Divergence, 263
Pocket Algorithm, 10
Policy Gradient Methods, 407
Policy Network, 407
Polyak Averaging, 139
Pooling, 308, 315
PowerAI, 25

Pretraining, 189, 262
Probabilistic Latent Semantic Analysis (PLSA), 254
Protein Structure Prediction, 301

**Q**
Q-Network, 401
Quasi-Newton Methods, 148
Question Answering, 294

**R**
Radial Basis Function Network (RBF Network), 215
Receptive Field, 312
Recommender Systems, 96, 249, 299
Recurrent Models of Visual Attention, 437
Recurrent Neural Networks, 20, 265
Region Proposal Method, 354
Regularization, 178
REINFORCE, 430
Reinforcement Learning, 389
ReLU Activation, 11
Replicator Neural Network, 88
Reservoir Computing, 303
ResNet, 335, 357
ResNext, 337
Restricted Boltzmann Machines (RBM), 20, 231, 242
RMSProp, 136
RMSProp with Nesterov Momentum, 138

**S**
Saddle Points, 143
Safety Issues in AI, 429
Saliency Map, 342
SARSA, 404
Sayre's Paradox, 301
Scikit-Learn, 114
SelectiveSearch, 354
Self-Driving Cars, 390, 425
Self-Learning Robots, 420
Self-Organizing Map, 478
Semantic Hashing, 263
Sentiment Analysis, 266
Sequence-to-Sequence Learning, 292
SGNS, 107
Sigmoid Activation, 10
Sigmoid Belief Nets, 262
Sign Activation, 10
Simulated Annealing, 195
Singular Value Decomposition, 91
Skip Connections, 131
SMAC, 64, 162
Softmax Activation Function, 12
Softmax Classifier, 86
Soft Weight Sharing, 197
Sparse Autoencoders, 198
Spatial Transformer, 440
Spearmint, 64, 162
Speech Recognition, 301
Spiking Neurons, 482
Squeeze-And-Excitation Networks (SENet), 338
Stochastic Curriculum, 196
Stochastic Depth in ResNets, 338
Storkey Learning Rule, 236
Strides, 313
Subsampling, 182
Support Vector Machines, 79

**T**
Tangent Classifier, 211
TD($\lambda$) Algorithm, 406
TD-Gammon, 430
TD-Leaf, 415
Teacher Forcing Methods, 303
Temporal Difference Learning, 404
Temporal Recommender Systems, 299
TensorFlow, 25, 68, 162, 303
TextCNN, 355
Theano, 25, 68, 162, 303
Tikhonov Regularization, 178
Time-Series Data, 265
Time-Series Forecasting, 297
Topic Models, 254
Torch, 25, 68, 162, 303
Transfer Learning, 340
Transformer Networks, 446
Transposed Convolution, 324, 348
Tuning Hyperparameters, 63
Turing Complete, 267, 462

**U**
Universal Function Approximators, 16, 18
Unpooling, 348
Unsupervised Pretraining, 189
Upper Bounding for Bandit Algorithms, 392

**V**
Valid Padding, 312
Value Function Models, 398
Value Networks, 417
Vanishing Gradient Problem, 124
Variational Autoencoder, 203, 470
Vector Quantization, 477
VGG, 330, 357
Video Classification, 355
Vision Transformer, 457
Visual Attention, 437
Visualization, 95

**W**
Weight Scaling Inference Rule, 186
Weston-Watkins SVM, 85
White-Box Attack, 463
Whitening, 65
Widrow-Hoff Learning, 78
Winnow Algorithm, 24
WordNet, 23

**X**
Xavier Initialization, 66, 132

**Y**
Yolo, 358

**Z**
ZFNet, 329, 357